

.NET Microservices – Azure DevOps and AKS

Section 1: Introduction to Microservices - Notes

Introduction to ASP.NET Core Microservices

Microservices is an architectural style that structures an application as a collection of small, loosely coupled services, each responsible for a specific piece of functionality. In ASP.NET Core, building microservices involves creating a suite of small, independent services that communicate with each other to form a complete application.

1. What Are Microservices?

Microservices are a way to design and develop software applications as a set of independently deployable services. Each microservice focuses on a specific business function and can be developed, deployed, and scaled independently.

Characteristics of Microservices:

- **Independence:** Each microservice operates independently of the others, allowing for isolated development, deployment, and scaling.
- **Single Responsibility:** Each microservice is responsible for a single business capability or domain.
- **Decentralized Data Management:** Each service manages its own data store, avoiding a centralized database.
- **Inter-Service Communication:** Services communicate with each other over network protocols, typically HTTP or messaging queues.

2. Key Concepts in ASP.NET Core Microservices

a. Service Composition

- **API Gateway:** An API Gateway acts as a reverse proxy to route requests to the appropriate microservice. It can handle tasks like authentication, request routing, and response aggregation.
- **Service Registry and Discovery:** Microservices often use a service registry (like Consul or Eureka) to keep track of available services and their locations.

b. Data Management

- **Database Per Service:** Each microservice usually has its own database, ensuring that services are loosely coupled and can be updated independently.
- **Data Synchronization:** For services that need to share data, event-driven architecture (using message brokers like RabbitMQ or Kafka) is commonly used to synchronize data across services.

c. Inter-Service Communication

- **Synchronous Communication:** Typically done via HTTP/REST or gRPC, where one service directly calls another.
- **Asynchronous Communication:** Done using message queues or event streams, allowing services to communicate without waiting for a response.

d. Deployment

- **Containerization:** Microservices are often deployed using containers (like Docker), which package the service and its dependencies into a single unit.
- **Orchestration:** Tools like Kubernetes manage the deployment, scaling, and operation of containerized applications.

Example of ASP.NET Core Microservices Setup

1. **Create Multiple ASP.NET Core Projects:**
 - Each project represents a different microservice, such as OrderService, PaymentService, and InventoryService.
2. **Configure API Gateway:**
 - Use a tool like Ocelot or YARP (Yet Another Reverse Proxy) to route requests to the appropriate microservices.
3. **Implement Inter-Service Communication:**
 - Use HTTP clients or gRPC for synchronous calls, and message brokers like RabbitMQ for asynchronous messaging.
4. **Data Management:**
 - Each microservice manages its own database schema, and data synchronization is handled through events or APIs.

5. Deploy Using Containers:

- Package each microservice into Docker containers and deploy them using Kubernetes or another orchestration tool.

Explanation

- **Independence:** Each microservice can be updated and deployed independently, reducing the impact of changes on other services.
- **Single Responsibility:** This principle helps keep each microservice focused on a specific business function, making it easier to understand and maintain.
- **Decentralized Data Management:** Ensures that services are not tightly coupled through a shared database, which can improve performance and scalability.
- **Inter-Service Communication:** Ensures that services can interact with each other while remaining independent.
- **Containerization:** Provides a consistent environment for running microservices, simplifying deployment and scaling.

Key Points to Remember

1. **Microservices:** Structure applications as a collection of loosely coupled services, each handling a specific function.
2. **Independence:** Microservices operate independently, allowing for isolated updates and scaling.
3. **Database Per Service:** Each microservice manages its own data store.
4. **Inter-Service Communication:** Use HTTP/REST, gRPC, or message brokers for communication between services.
5. **Containerization and Orchestration:** Use Docker for containerization and Kubernetes for managing deployments.

Drawbacks of Monolithic Architecture

A **monolithic architecture** refers to a traditional design where an application is built as a single, unified unit. While monolithic applications can be simpler to develop initially, they come with several drawbacks, especially as the application grows.

1. Scalability Issues

- **Single Deployment Unit:** Scaling a monolithic application usually means scaling the entire application, even if only a part of it requires more resources. This can lead to inefficiencies and increased costs.
- **Resource Utilization:** The whole application must be scaled together, which can be wasteful if only specific functionalities are experiencing high demand.

2. Complexity and Maintainability

- **Codebase Size:** As the application grows, the codebase can become large and complex, making it difficult for developers to understand and manage.
- **Coupled Components:** Components within a monolithic application are often tightly coupled, making it challenging to modify one part of the application without affecting others.

3. Deployment Challenges

- **Entire Application Deployment:** Even small changes require redeploying the entire application, which can lead to longer deployment cycles and increased risk of downtime.
- **Testing Difficulties:** Testing a monolithic application can be complex, as it involves ensuring that changes don't inadvertently break other parts of the application.

4. Technology Stack Limitations

- **Limited Flexibility:** The technology stack is typically unified for the entire application, which can limit the use of newer or different technologies that might be more suitable for certain parts of the application.
- **Dependency Management:** Managing dependencies across a monolithic codebase can become cumbersome, especially as the application grows.

5.

Development Speed

- **Team Collaboration:** As teams grow, coordinating development efforts within a monolithic application can become challenging. Different teams working on different parts of the application may face integration issues.
- **Code Conflicts:** Developers working on the same codebase may experience frequent merge conflicts and integration issues.

Example of Monolithic Architecture Issues

Imagine an e-commerce application with a monolithic architecture. This application handles user management, product catalog, order processing, and payment processing all within a single codebase.

- **Scaling Issue:** If the payment processing module experiences high load, scaling the entire application to handle the load can be inefficient.
- **Deployment Challenge:** A bug fix in the order processing module requires redeploying the entire application, risking downtime for all functionalities.

Explanation

- **Scalability:** Scaling a monolithic application often involves scaling the whole application, leading to inefficient use of resources.
- **Complexity:** A large, tightly coupled codebase can be difficult to manage and maintain.
- **Deployment:** Frequent, small changes require full redeployment, which can increase risk and downtime.
- **Technology Flexibility:** A monolithic application is constrained to a single technology stack, limiting the ability to use new or different technologies.

Key Points to Remember

1. **Monolithic Architecture:** An application is built as a single, unified unit, leading to challenges in scalability, complexity, and deployment.
2. **Scalability Issues:** Scaling a monolithic application means scaling the entire application, even if only specific parts require more resources.
3. **Maintainability:** A large, tightly coupled codebase can be difficult to manage and update.
4. **Deployment:** Changes require redeploying the entire application, increasing the risk of downtime.
5. **Technology Constraints:** Limited flexibility in choosing or updating technology stacks.

Microservices Architecture

Microservices architecture is an approach to designing applications as a collection of small, loosely coupled services that can be developed, deployed, and scaled independently. This contrasts with monolithic architecture, where an application is a single, unified unit.

1. What is Microservices Architecture?

Microservices architecture structures an application as a set of distinct services, each responsible for a specific business capability. These services are designed to be:

- **Independently Deployable:** Each service can be updated and deployed independently of others.
- **Focused on Business Capabilities:** Each service addresses a specific business function or domain.
- **Communicating Over Networks:** Services interact with each other over HTTP, gRPC, or messaging systems.

2. Key Components of Microservices Architecture

a. Services

- **Business Capability:** Each microservice focuses on a specific business function (e.g., user management, order processing).
- **Autonomous:** Services are developed, deployed, and scaled independently.

b. API Gateway

- **Routing:** An API Gateway routes requests to the appropriate microservice.
- **Cross-Cutting Concerns:** Handles concerns such as authentication, logging, and request aggregation.

c. Service Discovery

- **Registry:** Services register themselves with a service registry (e.g., Consul, Eureka) so that other services can discover them.
- **Dynamic Lookup:** Allows services to dynamically discover and interact with each other.

d.

Inter-Service Communication

- **HTTP/REST:** For synchronous communication, where services make direct HTTP requests to each other.
- **gRPC:** A high-performance RPC framework for communication between services.
- **Message Brokers:** For asynchronous communication using message queues or event streams (e.g., RabbitMQ, Kafka).

e. Data Management

- **Database Per Service:** Each service manages its own database, promoting data encapsulation.
- **Event-Driven Architecture:** Uses events to synchronize data across services.

3. Design Principles

a. Single Responsibility Principle

- **Focused Functionality:** Each microservice is designed to perform a specific function, ensuring clarity and separation of concerns.

b. Loose Coupling

- **Independent Changes:** Services are loosely coupled, meaning changes in one service have minimal impact on others.

c. Decentralized Data Management

- **Autonomous Data Storage:** Each service has its own data store, reducing dependencies and enhancing scalability.

d. API-First Design

- **Contract-Driven Development:** Define service APIs before implementation to ensure clear communication and integration between services.

e. Fault Isolation

- **Resilience:** If one service fails, it doesn't necessarily bring down the entire system. Failures are isolated to individual services.

4.

Benefits of Microservices Architecture

a. Scalability

- **Independent Scaling:** Services can be scaled individually based on their load, improving resource utilization and performance.

b. Flexibility

- **Technology Diversity:** Different services can use different technologies or languages, allowing teams to choose the best tools for each service.

c. Faster Time to Market

- **Parallel Development:** Teams can work on different services simultaneously, accelerating development and deployment cycles.

d. Improved Maintainability

- **Modular Design:** Smaller codebases are easier to understand, test, and maintain.

e. Enhanced Resilience

- **Fault Tolerance:** Failures in one service do not impact the entire application, improving overall system resilience.

Example of Microservices Architecture

Consider an e-commerce application with the following microservices:

- **User Service:** Manages user accounts and authentication.
- **Product Service:** Manages product catalog and inventory.
- **Order Service:** Handles order processing and payment.
- **Notification Service:** Sends notifications (e.g., emails, SMS).

These services communicate through HTTP/REST or messaging queues, with an API Gateway routing incoming requests to the appropriate service. Each service has its own database and can be independently scaled and deployed.

Explanation

- **Independent Deployability:** Microservices can be updated and deployed without affecting other services, facilitating continuous delivery and integration.
- **Focused on Business Capabilities:** Each service addresses a specific domain, making it easier to develop and manage.
- **API Gateway:** Centralizes cross-cutting concerns like authentication and request routing.
- **Service Discovery:** Enables dynamic communication between services by maintaining a registry of available services.
- **Data Management:** Promotes encapsulation and scalability through decentralized data management and event-driven synchronization.

Key Points to Remember

1. **Microservices Architecture:** Structures an application as a collection of small, independent services focusing on specific business functions.
2. **Key Components:** Include services, API Gateway, service discovery, inter-service communication, and data management.
3. **Design Principles:** Emphasize single responsibility, loose coupling, decentralized data management, API-first design, and fault isolation.
4. **Benefits:** Include scalability, flexibility, faster time to market, improved maintainability, and enhanced resilience.

Design Principles and Benefits of Microservices

Microservices architecture is built on several design principles that ensure the system is efficient, maintainable, and scalable. Understanding these principles and their associated benefits is crucial for effectively implementing and leveraging a microservices-based approach.

1. Design Principles

a. Single Responsibility Principle (SRP)

- **Definition:** Each microservice should have a single, well-defined responsibility or business capability.
- **Purpose:** Helps in keeping the service focused and manageable. For instance, a `UserService` handles user-related operations, while a `ProductService` manages product-related tasks.

b. Loose Coupling

- **Definition:** Microservices should be loosely coupled, meaning that changes in one service should not require changes in other services.
- **Purpose:** Promotes independence among services. For example, updating the OrderService should not impact the PaymentService.

c. Decentralized Data Management

- **Definition:** Each microservice manages its own data and database schema.
- **Purpose:** Avoids the problem of a single, monolithic database, which can become a bottleneck. For example, the InventoryService might use a NoSQL database, while the OrderService uses a relational database.

d. API-First Design

- **Definition:** Design the API contracts before implementation to ensure that services can interact smoothly.
- **Purpose:** Ensures clear communication protocols between services. For instance, define the API endpoints for the UserService before coding it, to enable the OrderService to integrate with it effectively.

e. Fault Isolation

- **Definition:** Services should be designed to handle failures gracefully without affecting other services.
- **Purpose:** Enhances system resilience. For example, if the NotificationService fails, the OrderService should continue to process orders without interruption.

f. Continuous Delivery and Deployment

- **Definition:** Implement continuous integration and delivery pipelines to automate testing and deployment.
- **Purpose:** Allows frequent, reliable releases of services. For example, new features in the ProductService can be deployed without affecting other services.

g. Independent Scalability

- **Definition:** Each service can be scaled independently based on its load and performance requirements.
- **Purpose:** Optimizes resource usage and performance. For example, during high traffic, only the OrderService might need additional instances, rather than the entire application.

h. Technology Diversity

- **Definition:** Different services can use different technologies, programming languages, and frameworks.
- **Purpose:** Allows teams to choose the best tools for each service. For instance, the UserService could be built using .NET Core, while the NotificationService uses Node.js.

2. Benefits of Microservices Architecture

a. Scalability

- **Independent Scaling:** Services can be scaled individually, allowing more efficient use of resources. For example, if the ProductService needs more capacity, it can be scaled without impacting the UserService.

b. Flexibility

- **Technology Agnostic:** Different services can use different technology stacks, enabling the use of the best tools for each job. For example, use Python for data analytics services and Java for high-performance services.

c. Faster Time to Market

- **Parallel Development:** Teams can work on different services concurrently, speeding up development cycles. For instance, while the OrderService team works on order management, the InventoryService team can develop inventory management features.

d. Improved Maintainability

- **Modular Design:** Smaller, focused services are easier to understand, test, and maintain. For example, the ShippingService is simpler to maintain and upgrade compared to a monolithic shipping module.

e. Enhanced Resilience

- **Fault Tolerance:** If one service fails, it does not bring down the entire system. For example, if the PaymentService fails, the OrderService can continue to operate and queue orders for later processing.

f. Better Alignment with Business Capabilities

- **Domain-Driven Design:** Microservices can be aligned with business domains, making it easier to model complex business processes. For example, a service dedicated to managing customer loyalty programs aligns with the business capability of customer engagement.

Example of Applying Design Principles

Single Responsibility Principle:

- **Service:** OrderService manages order creation and processing.
- **Benefit:** Simplifies the codebase and makes it easier to understand and maintain.

Loose Coupling:

- **Scenario:** InventoryService updates stock levels independently of the OrderService.
- **Benefit:** Changes in stock management do not affect order processing.

API-First Design:

- **Service:** Define RESTful APIs for PaymentService before implementation.
- **Benefit:** Ensures other services (like OrderService) can integrate seamlessly.

Fault Isolation:

- **Service:** ReviewService can fail without impacting ProductService.
- **Benefit:** Improves overall system reliability.

Key Points to Remember

1. **Single Responsibility Principle:** Each microservice should handle a specific business function.
2. **Loose Coupling:** Services should be independent to facilitate changes without impacting others.
3. **Decentralized Data Management:** Each service manages its own data store.
4. **API-First Design:** Design APIs before implementation for better integration.
5. **Fault Isolation:** Ensure that failures in one service do not affect others.
6. **Continuous Delivery and Deployment:** Automate testing and deployment to speed up releases.
7. **Independent Scalability:** Scale services individually based on needs.
8. **Technology Diversity:** Use the best technology for each service.

Drawbacks of Microservices

While microservices offer numerous benefits, they also come with challenges and drawbacks that need to be managed effectively. Understanding these drawbacks helps in planning and mitigating potential issues when adopting a microservices architecture.

1. Complexity

a. Increased System Complexity

- **Multiple Services:** Managing and orchestrating multiple services adds complexity compared to a monolithic system.
- **Inter-Service Communication:** Ensuring reliable communication and data consistency between services can be challenging.

b. Deployment Overhead

- **Service Management:** Each service needs to be deployed, monitored, and maintained separately, which can increase operational overhead.
- **Configuration Management:** Managing configurations for numerous services can be cumbersome.

c. Data Management Complexity

- **Data Consistency:** Maintaining data consistency across distributed services requires careful handling, especially with eventual consistency models.
- **Distributed Transactions:** Handling transactions that span multiple services is more complex than in a monolithic architecture.

2. Performance Overheads

a. Network Latency

- **Communication Overhead:** Microservices often communicate over a network (e.g., HTTP, gRPC), which can introduce latency compared to in-process calls in monolithic systems.

b. Serialization/Deserialization

- **Data Transformation:** Data must be serialized and deserialized when transmitted between services, adding processing overhead.

c. Service Discovery and Load Balancing

- **Performance Costs:** Service discovery and load balancing introduce additional overhead that can impact performance.

3. Testing Challenges

a. Integration Testing

- **Complex Test Scenarios:** Testing interactions between multiple services can be more complex compared to testing a single monolithic application.
- **Mocking Dependencies:** Mocking or stubbing service dependencies for tests can be challenging.

b. End-to-End Testing

- **Coordination:** Coordinating end-to-end tests across multiple services requires careful planning and setup.

4. Operational Challenges

a. Monitoring and Logging

- **Distributed Tracing:** Monitoring and tracing requests across multiple services requires sophisticated tools and setups.
- **Centralized Logging:** Aggregating logs from various services into a centralized system can be complex.

b. Deployment Complexity

- **Deployment Strategies:** Implementing deployment strategies like blue-green deployments or canary releases requires more sophisticated setups compared to monolithic systems.

c. Security Considerations

- **Service-to-Service Security:** Ensuring secure communication between services can be challenging and requires consistent security practices.

5. Data Management Issues

a. Data Duplication

- **Redundant Data:** Each service managing its own data store can lead to data duplication and synchronization issues.

b. Data Integrity

- **Consistency Challenges:** Ensuring data integrity and consistency across services is more complex compared to a single database model.

Example of Drawbacks in Microservices

Complexity:

- **Scenario:** An e-commerce system with separate UserService, OrderService, and InventoryService introduces complexity in managing interactions and configurations.
- **Challenge:** Ensuring that OrderService and InventoryService are synchronized with up-to-date inventory information.

Performance Overheads:

- **Scenario:** Microservices communicate over HTTP, leading to network latency and serialization overhead.
- **Challenge:** OrderService calling InventoryService over HTTP introduces latency compared to a monolithic system where these calls are in-process.

Testing Challenges:

- **Scenario:** Testing end-to-end scenarios involving PaymentService, OrderService, and NotificationService requires comprehensive integration tests.
- **Challenge:** Coordinating these tests and handling service dependencies can be complex.

Operational Challenges:

- **Scenario:** Aggregating logs from multiple microservices and setting up centralized monitoring.
- **Challenge:** Implementing and maintaining distributed tracing and centralized logging systems.

Explanation

- **Complexity:** Managing multiple services increases overall system complexity, affecting deployment, configuration, and data management.
- **Performance Overheads:** Network communication, data serialization, and service discovery can introduce performance issues.
- **Testing:** Integrating and testing interactions between services requires more effort compared to a monolithic system.
- **Operational:** Monitoring, logging, and deploying multiple services add operational overhead and complexity.
- **Data Management:** Managing data consistency and integrity across multiple services can be challenging.

Key Points to Remember

1. **Complexity:** Microservices increase system complexity and require more sophisticated management and deployment strategies.
2. **Performance Overheads:** Network latency and serialization can impact performance.
3. **Testing Challenges:** Testing interactions and end-to-end scenarios is more complex.
4. **Operational:** Requires advanced monitoring, logging, and deployment strategies.
5. **Data Management:** Managing data consistency and integrity across services is challenging.

Correlation Between "Sub-Domains of Business" and "Microservices"

Microservices architecture and the concept of sub-domains of business are closely related. Microservices are often designed around business sub-domains, aligning technical services with business functions to create a cohesive system.

Here's a detailed look at how these concepts intersect:

1. Understanding Sub-Domains of Business

a. Definition of Sub-Domains

- **Sub-Domain:** In the context of Domain-Driven Design (DDD), a sub-domain represents a distinct area of business functionality or concern. Each sub-domain addresses a specific aspect of the business.
- **Example:** In an e-commerce application, sub-domains might include User Management, Order Processing, Inventory Management, and Payment Processing.

b. Importance of Sub-Domains

- **Clarity and Focus:** Sub-domains help break down complex business requirements into manageable areas, each with its own specific focus.
- **Alignment with Business Goals:** Designing services around sub-domains ensures that technical solutions align closely with business objectives and processes.

2. Microservices Aligned with Sub-Domains

a. Service Design

- **Domain Alignment:** Microservices are often designed to align with business sub-domains. Each service encapsulates a specific sub-domain and its associated functionality.
- **Example:** The OrderService microservice in an e-commerce application might handle all aspects of order management, including order creation, updates, and tracking.

b. Benefits of Domain Alignment

- **Focused Development:** Teams can focus on a specific sub-domain, leading to clearer objectives and easier management.
- **Decoupled Services:** Services that correspond to distinct sub-domains are less likely to interfere with one another, reducing coupling and complexity.

c. Decentralized Data Management

- **Data Ownership:** Each microservice manages its own data related to its sub-domain. This aligns with the principle of decentralized data management in microservices.
- **Example:** The InventoryService manages data related to inventory levels, while the PaymentService manages transaction data.

3. Implementation Example

Consider a retail system with the following sub-domains and corresponding microservices:

- **User Management:** UserService
 - **Responsibilities:** User registration, authentication, and profile management.
- **Order Processing:** OrderService
 - **Responsibilities:** Order creation, status tracking, and order history.
- **Inventory Management:** InventoryService
 - **Responsibilities:** Stock levels, product availability, and inventory updates.
- **Payment Processing:** PaymentService
 - **Responsibilities:** Payment transactions, refunds, and payment methods.

Each microservice is designed to handle the functionality of its respective sub-domain, leading to a modular and scalable system. Services communicate with each other as needed, but are independently deployable and maintainable.

4. Benefits of Aligning Microservices with Sub-Domains

a. Business-Driven Development

- **Alignment with Business Objectives:** Services reflect real business functions, ensuring that technical solutions address actual business needs.

b. Enhanced Flexibility

- **Independent Development:** Teams can work on different sub-domains independently, speeding up development and deployment cycles.

c. Improved Maintainability

- **Focused Codebases:** Smaller, focused services are easier to understand, test, and maintain compared to a monolithic codebase.

d. Better Scalability

- **Service-Specific Scaling:** Services can be scaled independently based on the demands of their respective sub-domains.

5. Challenges and Considerations

a. Boundaries and Integration

- **Defining Boundaries:** Careful consideration is needed to define clear boundaries between sub-domains and corresponding services.
- **Integration:** Services need to communicate effectively and handle integration points between sub-domains.

b. Data Consistency

- **Handling Consistency:** Ensuring data consistency across services representing different sub-domains requires careful planning and potentially event-driven architectures.

c. Service Coordination

- **Inter-Service Communication:** Managing communication and data sharing between services can be complex and requires well-defined protocols.

Example of Domain Alignment

Scenario: In an e-commerce platform, the OrderService is responsible for managing orders while the InventoryService manages stock levels.

- **OrderService:** Manages order creation, updates, and order history.
- **InventoryService:** Updates stock levels and manages product availability.

Benefits: Each service focuses on its specific sub-domain, leading to better organization and easier management of business functions.

Key Points to Remember

1. **Sub-Domains of Business:** Represent distinct areas of functionality within a business, helping to break down complex requirements.
2. **Microservices Alignment:** Aligning microservices with business sub-domains ensures that technical services reflect actual business needs.
3. **Domain-Driven Design:** Using sub-domains to guide service design enhances focus, flexibility, and maintainability.
4. **Data Management:** Each microservice manages its own data, promoting decentralized data management.
5. **Integration and Consistency:** Properly managing boundaries, communication, and data consistency between services is crucial.

Microservices Best Practices

Implementing microservices effectively requires adhering to best practices that ensure scalability, maintainability, and overall system efficiency. Here are some key best practices to follow:

1. Define Clear Service Boundaries

a. Domain-Driven Design (DDD)

- **Bounded Contexts:** Use DDD principles to define clear boundaries for each microservice based on business sub-domains. Each service should be responsible for a specific part of the business domain.
- **Example:** In an e-commerce system, the UserService handles user management, while the OrderService handles order processing. Each has a distinct bounded context.

b. Service Contracts

- **API Contracts:** Define and document APIs and interactions between services clearly. Use API documentation tools like Swagger/OpenAPI.
- **Example:** Document the OrderService API endpoints so that other services, like InventoryService, can integrate seamlessly.

2. Implement Robust Communication

a. Synchronous Communication

- **HTTP/REST:** For real-time, request-response interactions between services. Ensure that service endpoints are well-defined and versioned.
- **gRPC:** Use gRPC for high-performance RPC communication, especially when low latency is critical.

b. Asynchronous Communication

- **Message Brokers:** Use message brokers (e.g., RabbitMQ, Kafka) for event-driven communication and to decouple services.
- **Event Streaming:** Implement event streaming to handle real-time data processing and notifications.

c. Handling Failures

- **Retries and Circuit Breakers:** Implement retry logic and circuit breakers to handle transient failures and prevent cascading failures.
- **Fallback Mechanisms:** Provide fallback responses when a service is unavailable to ensure system resilience.

3. Data Management Strategies

a. Database per Service

- **Decentralized Data Stores:** Each microservice should manage its own database to avoid tight coupling.
- **Example:** The OrderService might use a relational database, while the InventoryService uses a NoSQL database.

b. Data Consistency

- **Eventual Consistency:** Use eventual consistency models to handle data synchronization between services.
- **Sagas:** Implement the Saga pattern for managing distributed transactions and ensuring consistency across services.

c. Data Access Patterns

- **API Composition:** Aggregate data from multiple services using an API Gateway or a separate data aggregation service.
- **CQRS:** Use Command Query Responsibility Segregation (CQRS) to separate read and write operations.

4. DevOps and Continuous Delivery

a. Continuous Integration/Continuous Deployment (CI/CD)

- **Automated Pipelines:** Set up automated CI/CD pipelines to build, test, and deploy microservices independently.
- **Example:** Use tools like Jenkins, GitHub Actions, or Azure DevOps to automate deployments for each microservice.

b. Containerization and Orchestration

- **Containers:** Use Docker to containerize microservices for consistent deployment across different environments.
- **Kubernetes:** Use Kubernetes for container orchestration, scaling, and managing microservices deployments.

c. Monitoring and Logging

- **Centralized Logging:** Implement centralized logging solutions (e.g., ELK Stack, Fluentd) to aggregate logs from all services.
- **Distributed Tracing:** Use distributed tracing tools (e.g., Jaeger, Zipkin) to track requests across microservices and diagnose performance issues.

5. Security Best Practices

a. Authentication and Authorization

- **OAuth2 and JWT:** Implement OAuth2 for authentication and JWT for authorization across services.
- **API Gateways:** Use API Gateways to manage authentication and enforce security policies.

b. Secure Communication

- **TLS/SSL:** Ensure all communication between services is encrypted using TLS/SSL.
- **Service Mesh:** Use a service mesh (e.g., Istio) for managing service-to-service security and communication.

c. Data Protection

- **Encryption:** Encrypt sensitive data at rest and in transit.
- **Access Controls:** Implement strict access controls and audit logs to monitor access to data.

6. Best Practices for Development

a. Documentation

- **Service Documentation:** Maintain comprehensive documentation for each service, including API documentation, data models, and architectural decisions.
- **Code Comments:** Use inline comments and documentation to explain complex logic and design decisions.

b. Testing

- **Unit Tests:** Write unit tests for individual service components.
- **Integration Tests:** Test interactions between services to ensure they work together correctly.

- **Contract Tests:** Use contract testing to verify that services adhere to agreed-upon APIs and contracts.

c. Versioning

- **API Versioning:** Implement versioning for APIs to handle changes and backward compatibility.
- **Semantic Versioning:** Follow semantic versioning practices to clearly communicate changes and updates.

Example of Best Practices

Communication Strategy:

- **Synchronous:** The OrderService might call InventoryService over HTTP/REST to check stock levels.
- **Asynchronous:** The InventoryService could use RabbitMQ to publish events about stock changes.

Data Management:

- **Database per Service:** OrderService uses PostgreSQL, while InventoryService uses MongoDB.
- **Eventual Consistency:** Implement an event-driven approach to synchronize inventory levels with order processing.

CI/CD:

- **Pipeline:** Configure a CI/CD pipeline in GitHub Actions to automate testing and deployment of the UserService.

Key Points to Remember

1. **Clear Service Boundaries:** Define services based on business sub-domains and use well-defined API contracts.
2. **Robust Communication:** Implement both synchronous and asynchronous communication strategies, and handle failures gracefully.
3. **Data Management:** Use decentralized data stores, handle data consistency with eventual consistency models, and use CQRS when appropriate.
4. **DevOps Practices:** Implement CI/CD, use containerization and orchestration, and ensure robust monitoring and logging.
5. **Security:** Implement secure communication, authentication, authorization, and data protection practices.
6. **Development Best Practices:** Maintain comprehensive documentation, implement thorough testing, and manage API versioning effectively.

.NET Microservices – Azure DevOps and AKS

Section 2: Users Microservice - Notes

Clean Architecture

In the context of **Clean Architecture**, the **Core** layer contains business logic and entities that are completely independent of external services like databases, APIs, or UI frameworks. This layer serves as the heart of the application, containing the essential domain logic and models.

1. DTOs (Data Transfer Objects)

Conceptual Understanding:

- **DTO (Data Transfer Object):** DTOs are simple objects used to transfer data between layers or services in a system. They don't contain any business logic and are mainly used to carry data.
- **Purpose:** By using DTOs, you decouple your internal models from external clients or services, allowing for more flexible code and easier adaptation to changes. It also helps avoid exposing internal data models directly.
- **In Clean Architecture:** DTOs live in the **Core** layer because they are shared between different application layers (e.g., Application Layer, Web Layer). They facilitate communication between components without coupling them.

Now, let's break down the code file by file.

File 1: AuthenticationResponse Record

```
namespace eCommerce.Core.DTO;
```

```
public record AuthenticationResponse(  
    Guid UserID,  
    string? Email,  
    string? PersonName,  
    string? Gender,  
    string? Token,  
    bool Success  
)
```

```

{
// Parameterless constructor

public AuthenticationResponse() : this(default, default, default, default, default, default)
{
}
}

```

Conceptual Breakdown

- **Namespace (eCommerce.Core.DTO):** The namespace defines the logical grouping of your code. Here, the Core.DTO namespace represents that the file belongs to the core layer of your eCommerce application and contains DTOs.
- **record Keyword:** This defines a **record type** in C#. A record is a reference type that provides built-in functionality for **value equality**, meaning two records with the same data are considered equal, unlike classes which use reference equality.
 - **Benefits of Records:** Records are immutable by default. You can define a set of properties that don't change, making them perfect for DTOs since they only carry data. You also get automatic ToString(), Equals(), and GetHashCode() implementations.
- **Properties:**
 - Guid UserID: Stores the user's unique identifier.
 - string? Email: The user's email. The ? denotes it is a nullable string.
 - string? PersonName: The user's name, also nullable.
 - string? Gender: Represents the gender of the user, nullable string.
 - string? Token: Stores the authentication token.
 - bool Success: A flag to indicate whether the authentication was successful.
- **Parameterless Constructor:** The empty constructor is required for some deserialization frameworks or APIs that need to create objects without parameters.
 - The this(default, default, ...) call assigns default values (e.g., null for reference types, Guid.Empty for Guid, false for bool) to each parameter when no arguments are provided.

File 2: GenderOptions Enum

```
namespace eCommerce.Core.DTO;
```

```
public enum GenderOptions  
{  
    Male, Female, Others  
}
```

Conceptual Breakdown

- **Enum (enum):** An enum is a value type that defines a set of named constants. In this case, the GenderOptions enum provides specific, pre-defined options for the gender of a user.
- **Values:**
 - Male, Female, Others: These are the predefined gender options available. Using an enum ensures that only these values are used throughout the system, enforcing data consistency.
- **Benefits of Enums:**
 - **Type Safety:** When you use enums, you prevent invalid values (e.g., typos in strings) from being assigned.
 - **Readability:** Makes code more readable by giving meaningful names to constants.

File 3: LoginRequest Record

```
namespace eCommerce.Core.DTO;
```

```
public record LoginRequest(  
    string? Email,  
    string? Password);
```

Conceptual Breakdown

- **LoginRequest DTO:** This record serves as a simple DTO to encapsulate the data required for a login operation.
- **Properties:**
 - string? Email: The user's email for login. Nullable because it may not always be required at all layers.
 - string? Password: The password for authentication. Nullable as well, to support flexibility (though in practice, passwords are usually required).

- **Why use a DTO for Login?:**
 - Separation of concerns: Keeps the user model separate from the request model, avoiding unnecessary dependencies on internal details of the user entity.
 - Flexibility: It allows modifications (e.g., adding fields) without impacting the user entity or other parts of the system.

File 4: RegisterRequest Record

namespace eCommerce.Core.DTO;

```
public record RegisterRequest(
    string? Email,
    string? Password,
    string? PersonName,
    GenderOptions Gender);
```

Conceptual Breakdown

- **RegisterRequest DTO:** This DTO is used to capture the data needed for registering a new user in the system.
- **Properties:**
 - string? Email: User's email, typically required for registration.
 - string? Password: User's password, required for authentication.
 - string? PersonName: The user's name.
 - GenderOptions Gender: The GenderOptions enum ensures the gender is one of the predefined options (Male, Female, Others).
- **Why use a RegisterRequest DTO?:**
 - **Encapsulation:** It isolates the data required for registration from other parts of the system, focusing only on what is necessary for this operation.
 - **Input Validation:** Easier to validate the incoming registration request, as all necessary data is packaged in one object.

Summary of Concepts and Usage in Code:

1. DTOs in Clean Architecture:

- DTOs are used to transport data between layers without involving complex domain models. They keep your core business logic independent from infrastructure concerns.

2. Records:

- Records in C# are immutable by default and support value equality. They are perfect for DTOs because they encapsulate data and make working with immutable types easier.

3. Enums:

- Enums provide a type-safe way of working with fixed sets of values. Using enums like GenderOptions ensures that your application only works with valid gender types, reducing the chance of errors.

4. Parameterless Constructor:

- Required for scenarios where deserialization frameworks need to create objects without passing in parameters.

5. Separation of Concerns:

- DTOs help to cleanly separate the data model that interacts with external systems from your internal business logic, ensuring flexibility and maintainability.

Clean Architecture - Core Layer and Entities

In **Clean Architecture**, the **Entities** in the **Core** layer represent the fundamental business models that encapsulate your domain logic. These entities are pure C# classes that define the structure and behavior of the objects used within your domain. They are central to the business logic and are free from any dependency on external systems like databases, APIs, or frameworks.

Entity: ApplicationUser

Let's break down the ApplicationUser entity:

```
namespace eCommerce.Core.Entities;
```

```
/// <summary>
```

```
/// Define the ApplicationUser class which acts as entity model class to store user details in data store
```

```
/// </summary>
```

```
public class ApplicationUser
```

```
{
```

```
    public Guid UserID { get; set; }
```

```
    public string? Email { get; set; }
```

```
    public string? Password { get; set; }
```

```
    public string? PersonName { get; set; }
```

```
    public string? Gender { get; set; }
```

```
}
```

Conceptual Breakdown

- **Namespace (eCommerce.Core.Entities):**
 - The **Entities** folder under the **Core** layer contains classes that represent the core business objects. These objects usually map directly to concepts in your business domain and might be stored in a data store (e.g., a database). The eCommerce.Core.Entities namespace logically organizes these models within the core of the application.

Understanding Each Part of the Code

1. Class Definition: ApplicationUser

- **public class ApplicationUser:**
 - The ApplicationUser class defines the structure of a user entity within the system. This class is primarily concerned with the core information related to the user (ID, email, password, etc.).
 - In **Clean Architecture**, entities are at the heart of the domain and are usually pure data models. These models define the essential characteristics and behavior of the domain objects.

2. Properties

- **Guid UserID { get; set; }:**
 - This is the **primary identifier** for each user in the system. The Guid (Globally Unique Identifier) ensures that each user has a unique and non-sequential ID.
 - **Guid** is preferred over sequential integers because it reduces the likelihood of collision in a distributed system and adds a layer of security by making it harder to guess other IDs.
 - This is a property with both a **getter** and a **setter**, meaning that the value can be read and modified.
- **string? Email { get; set; }:**
 - Stores the **email address** of the user. The string? type indicates that this field is nullable, meaning it can store either a valid email string or null.
 - In the context of validation, it's important to ensure that emails follow a valid format, though this will usually be handled in the application or presentation layer.
- **string? Password { get; set; }:**
 - Stores the **password** of the user.
- **string? PersonName { get; set; }:**
 - Represents the **name** of the user (e.g., full name, first name, or display name).
- **string? Gender { get; set; }:**
 - Stores the **gender** of the user. It's represented as a string, and it's nullable, which means users might not provide this information.

Conceptual Understanding of Core Entities

- **Entities in Clean Architecture:**
 - Entities represent **core domain objects** that are meaningful to the business. They encapsulate data and sometimes behavior that is central to the domain.
 - In the **Core** layer, entities are designed to be **independent of frameworks**. This ensures that they can evolve without the constraints of infrastructure concerns such as databases or APIs.
- **Data Representation:**
 - The ApplicationUser entity only represents data. It doesn't include any behavior or business rules (such as password encryption or email validation). Those responsibilities belong to higher layers of the application (e.g., Application or Infrastructure layers).

- **Why Use Nullable Strings?:**
 - Nullable strings (like string?) allow flexibility in scenarios where certain fields (such as PersonName or Gender) may not be mandatory at every point in time.
 - In terms of validation, nullable fields require checks to ensure they meet application-specific constraints, but those checks should happen outside the entity (usually in the application layer).
- **GUID for IDs:**
 - Using **GUIDs** as identifiers allows for distributed systems to generate unique IDs without coordination between different parts of the system. This is useful in systems that may be horizontally scaled or that need to prevent ID collisions across distributed databases.

Best Practices and Common Considerations:

- **Entity Design:**
 - The ApplicationUser entity should contain only fields and properties that are directly relevant to the user. Other aspects (e.g., authentication methods, user preferences) could be represented by additional entities or value objects.
- **Separation of Concerns:**
 - The entity does not handle validation, business rules, or persistence logic. These responsibilities are delegated to other layers (e.g., Application Layer for validation, Infrastructure Layer for persistence).

Clean Architecture - Repository Contracts in Core Layer

In **Clean Architecture**, the **Core** layer defines the essential business logic and contracts, while the actual implementation details (e.g., how to interact with databases) are pushed to the **Infrastructure** layer. This approach follows the **Dependency Inversion Principle**, which means that high-level policies (business logic) should not depend on low-level details (data storage or retrieval).

The **Repository Pattern** is commonly used in this architecture to abstract data access logic. In the **Core** layer, we define repository **contracts** (interfaces), while the actual repository implementation (that interacts with the database) resides in the **Infrastructure** layer. This allows us to change the data source (e.g., switch databases or use an API) without changing the business logic.

Repository Contract: IUsersRepository

Let's break down the code:

```
using eCommerce.Core.Entities;
```

```
namespace eCommerce.Core.RepositoryContracts;
```

```
/// <summary>
```

```
/// Contract to be implemented by UsersRepository that contains data access logic of Users data store
```

```
/// </summary>
```

```
public interface IUsersRepository
```

```
{
```

```
    /// <summary>
```

```
    /// Method to add a user to the data store and return the added user
```

```
    /// </summary>
```

```
    /// <param name="user"></param>
```

```
    /// <returns></returns>
```

```
    Task<ApplicationUser?> AddUser(ApplicationUser user);
```

```
    /// <summary>
```

```
    /// Method to retrieve existing user by their email and password
```

```
    /// </summary>
```

```

/// <param name="email"></param>
/// <param name="password"></param>
/// <returns></returns>
Task<ApplicationUser?> GetUserByEmailAndPassword(string? email, string? password);
}

```

Conceptual Breakdown

1. Interface Definition: IUsersRepository

- **public interface IUsersRepository:**
 - This is the **repository contract** that defines how the application will interact with the user data. It ensures that any repository implementation (e.g., for a database or in-memory storage) will follow this contract.
 - **Purpose:** The interface is used to define the methods that the actual repository (which deals with database interaction) must implement. By depending on interfaces (abstractions) rather than concrete classes, we can easily swap out implementations.
- **Repository Pattern:**
 - The repository pattern abstracts the **data access logic**. Instead of directly interacting with the database or data source, the application interacts with a repository, which provides methods for adding, retrieving, updating, or deleting entities.
 - In Clean Architecture, this pattern helps decouple business logic from infrastructure concerns (like databases or external services).

2. AddUser Method

```
Task<ApplicationUser?> AddUser(ApplicationUser user);
```

- **Purpose:**
 - This method is used to **add a new user** to the data store (e.g., a database) and return the added user after it's stored.
- **Parameters:**
 - ApplicationUser user: This represents the **user entity** to be added to the data store. It's an instance of the ApplicationUser class, which contains information such as UserID, Email, Password, PersonName, and Gender.

- **Return Type:**
 - Task<ApplicationUser?>:
 - The method is **asynchronous** (hence the Task<> wrapper) because data access operations (like saving to a database) often involve I/O operations, which are typically time-consuming.
 - The method returns an ApplicationUser?, meaning it returns the user that was successfully added, or null if the operation failed (e.g., if the data store rejected the operation).
 - Using asynchronous methods improves application performance by not blocking the main thread during I/O-bound operations.

3. GetUserByEmailAndPassword Method

Task<ApplicationUser?> GetUserByEmailAndPassword(string? email, string? password);

- **Purpose:**
 - This method retrieves an existing user from the data store based on the provided **email** and **password**. It is commonly used in authentication processes.
- **Parameters:**
 - string? email: The email of the user to be retrieved. The ? indicates it's nullable, meaning this method can handle cases where the email is not provided.
 - string? password: The password of the user, also nullable. This is expected to be stored securely (usually hashed) in the actual data store.
- **Return Type:**
 - Task<ApplicationUser?>:
 - The method is asynchronous and returns an ApplicationUser?. If the user is found with the provided email and password, it returns that user. If no user matches the criteria, it returns null.
 - The use of asynchronous programming (Task<>) ensures that the method is non-blocking, which is crucial for applications that need to scale.

Summary of Repository Contract Code and Concepts

1. **Repository Contract (IUsersRepository):**
 - This contract defines two essential methods for managing user data: adding a user and retrieving a user by their email and password.
 - The contract helps decouple the business logic from the actual data store, as any implementation (SQL database, NoSQL, etc.) must conform to this interface.

2. Asynchronous Methods:

- Both methods are asynchronous (Task<>) to ensure that the application can handle time-consuming I/O operations, such as database access, without blocking the main execution thread.

3. Method Responsibilities:

- AddUser(ApplicationUser user): Adds a user to the data store and returns the added user or null if the operation fails.
- GetUserByEmailAndPassword(string? email, string? password): Retrieves a user based on email and password for scenarios like authentication, returning null if no match is found.

In this way, the **Core** layer stays **completely independent of infrastructure concerns** (like databases). By using **repository contracts**, you ensure that the business logic depends only on abstractions and not on concrete implementations of data access. This makes the application flexible and easier to test or extend.

Clean Architecture - Service Contracts in Core Layer

In **Clean Architecture**, the **Service Contracts** (typically interfaces) in the **Core** layer define the business use cases or operations that the application can perform. These interfaces contain business logic but are abstracted from the actual implementation. The implementations of these contracts are usually placed in the **Application** or **Infrastructure** layer.

The **Core** layer should only know about the interfaces, which provides flexibility and decouples the business logic from the underlying infrastructure or framework dependencies.

Service Contract: IUsersService

Let's break down the code:

```
using eCommerce.Core.DTO;
```

```
namespace eCommerce.Core.ServiceContracts;
```

```
/// <summary>
```

```
/// Contract for users service that contains use cases for users
```

```
/// </summary>
```

```
public interface IUsersService
```

```
{
```

```
    /// <summary>
```

```
    /// Method to handle user login use case and returns an AuthenticationResponse object that  
    contains status of login
```

```
    /// </summary>
```

```
    /// <param name="loginRequest"></param>
```

```
    /// <returns></returns>
```

```
    Task<AuthenticationResponse?> Login(LoginRequest loginRequest);
```

```
    /// <summary>
```

```
    /// Method to handle user registration use case and returns an object of AuthenticationResponse  
    type that represents status of user registration
```

```
    /// </summary>
```

```
    /// <param name="registerRequest"></param>
```

```
    /// <returns></returns>
```

```
    Task<AuthenticationResponse?> Register(RegisterRequest registerRequest);
```

```
}
```

Conceptual Breakdown

1. Interface Definition: IUsersService

- **public interface IUsersService:**
 - The IUsersService interface defines the **service contract** for handling **user-related business operations** such as login and registration.
 - It defines two essential methods (Login and Register) that represent **use cases** in the application. These methods are typically implemented in the **Application Layer** or an external service, but they are defined here in the **Core** layer as abstract contracts.
 - By defining this interface, we ensure that the business logic is decoupled from any framework-specific or implementation-specific details. The actual login and registration logic can be implemented in any way, and you can easily swap the implementation without changing the rest of the application.

2. The Login Method

Task<AuthenticationResponse?> Login(LoginRequest loginRequest);

- **Purpose:**
 - The Login method handles the **user login use case**. It takes a LoginRequest object as input and returns an AuthenticationResponse object that contains the status of the login attempt (e.g., success or failure, along with other details like a token).
- **Parameters:**
 - LoginRequest loginRequest:
 - The loginRequest is a **Data Transfer Object (DTO)** that carries the **email** and **password** for the login attempt.
 - Using DTOs ensures that the data being passed between layers or services is well-defined and encapsulates only the necessary information (in this case, the credentials).
- **Return Type:**
 - Task<AuthenticationResponse?>:
 - The method is **asynchronous** (Task<>) because operations like verifying credentials and generating authentication tokens often involve external systems like databases or identity servers. These I/O-bound operations should not block the execution thread.
 - It returns an AuthenticationResponse?, which means the method will return an AuthenticationResponse object (with details such as the login status, token, and user details) or null if the login fails.
- **AuthenticationResponse:**
 - This is a **DTO** that encapsulates the result of the login attempt, including the user's ID, email, token, and a boolean indicating whether the login was successful.

3. The Register Method

Task<AuthenticationResponse?> Register(RegisterRequest registerRequest);

- **Purpose:**
 - The Register method handles the **user registration use case**. It allows users to create a new account by providing the necessary details such as email, password, name, and gender, and it returns an AuthenticationResponse that represents the status of the registration.
- **Parameters:**
 - RegisterRequest registerRequest:
 - The registerRequest is a **DTO** that encapsulates the user's **email**, **password**, **name**, and **gender**. This DTO ensures that only the relevant data required for user registration is passed to the service layer.
- **Return Type:**
 - Task<AuthenticationResponse?>:
 - Like the Login method, the Register method is asynchronous. Registering a user typically involves I/O operations such as storing the user in the database and sending confirmation emails.
 - The return type is an AuthenticationResponse?, which indicates whether the registration was successful. If successful, it will return a response with the user's details and a token; otherwise, it will return null.
- **AuthenticationResponse:**
 - The AuthenticationResponse DTO also applies here, representing the status of the registration attempt and containing data such as the newly created user's ID and authentication token.

DTOs in Service Contracts

In both methods, the parameters and return values are **DTOs** (Data Transfer Objects). DTOs are essential in **Clean Architecture** because they isolate the business layer from the external world. The business logic operates on these DTOs rather than interacting directly with HTTP requests or database models, keeping it clean and decoupled.

- **LoginRequest:**
 - A DTO that encapsulates the **email** and **password** fields required for the login process.
- **RegisterRequest:**
 - A DTO that contains the information required for user registration: **email**, **password**, **name**, and **gender**.

- **AuthenticationResponse:**

- A DTO that represents the outcome of both the login and registration processes. It contains the user's ID, email, name, token, and a boolean indicating whether the operation was successful.

Summary of Service ContractCode and Concepts

1. Service Contract (IUserService):

- The IUserService defines two essential business use cases: user login and user registration.
- These methods operate on **DTOs** (like LoginRequest and RegisterRequest) and return an AuthenticationResponse, which represents the outcome of the operation.

2. Asynchronous Programming:

- Both methods (Login and Register) are **asynchronous** because they likely involve I/O-bound tasks such as interacting with databases or external services (like token generation). Using asynchronous methods ensures that the application remains responsive.

3. Business Logic Encapsulation:

- The service contract abstracts away the **business logic**. While the actual implementation resides in other layers (Application or Infrastructure), the core layer defines the contract that must be adhered to by the implementation.

4. DTOs:

- The use of **DTOs** (like LoginRequest, RegisterRequest, and AuthenticationResponse) ensures that the business logic is separated from external concerns like HTTP requests. DTOs provide a clear and consistent way to pass data between layers.

Clean Architecture - Validators in Core Layer

In **Clean Architecture**, the **Core** layer defines business logic, and validation is a crucial part of ensuring the integrity of data before it enters the system. Validation is typically handled by **validators** in the Core layer to ensure that the data passed to the system meets the necessary business rules.

Here, we are using **FluentValidation**, a popular .NET library, to implement validation logic in a clean, readable, and flexible way. FluentValidation allows you to define rules for different properties of your DTOs, ensuring that the input data adheres to certain conditions before the application processes it.

LoginRequestValidator

The first validator is for the **LoginRequest** DTO, which ensures that the login data (email and password) is valid.

```
using eCommerce.Core.DTO;
```

```
using FluentValidation;
```

```
namespace eCommerce.Core.Validators;
```

```
public class LoginRequestValidator : AbstractValidator<LoginRequest>
{
    public LoginRequestValidator()
    {
        //Email
        RuleFor(temp => temp.Email)
            .NotEmpty().WithMessage("Email is required")
            .EmailAddress().WithMessage("Invalid email address format");

        //Password
        RuleFor(temp => temp.Password)
            .NotEmpty().WithMessage("Password is required");
    }
}
```

Conceptual Breakdown of LoginRequestValidator

1. AbstractValidator<LoginRequest>

- **Purpose:**
 - This class inherits from `AbstractValidator<T>`, where `T` is the type of the object to be validated. In this case, it's validating a `LoginRequest` object.
 - It provides a base class for creating validation rules for DTOs.

2. Validating the Email Property

```
RuleFor(temp => temp.Email)
```

```
.NotEmpty().WithMessage("Email is required")
```

```
.EmailAddress().WithMessage("Invalid email address format");
```

- **RuleFor(temp => temp.Email):**
 - This defines a validation rule for the Email property of LoginRequest.
- **NotEmpty():**
 - This rule ensures that the Email field is not empty. If it is empty, the specified message **"Email is required"** is returned.
- **EmailAddress():**
 - This rule ensures that the Email field contains a valid email address. If the email format is invalid, the message **"Invalid email address format"** is returned.

3. Validating the Password Property

```
RuleFor(temp => temp.Password)
```

```
.NotEmpty().WithMessage("Password is required");
```

- **RuleFor(temp => temp.Password):**
 - This defines a validation rule for the Password property of LoginRequest.
- **NotEmpty():**
 - This rule ensures that the Password field is not empty. If it is empty, the message **"Password is required"** is returned.

RegisterRequestValidator

The second validator is for the **RegisterRequest** DTO, which validates registration data such as email, password, name, and gender.

```
using eCommerce.Core.DTO;
```

```
using FluentValidation;
```

```
namespace eCommerce.Core.Validators;
```

```
public class RegisterRequestValidator : AbstractValidator<RegisterRequest>
{
    public RegisterRequestValidator()
    {
        //Email
        RuleFor(temp => temp.Email)
            .NotEmpty().WithMessage("Email is required")
            .EmailAddress().WithMessage("Invalid email address format");

        //Password
        RuleFor(temp => temp.Password)
            .NotEmpty().WithMessage("Password is required");

        // Validate the PersonName property.
        RuleFor(request => request.PersonName)
            .NotEmpty().WithMessage("PersonName can't be blank")
            .Length(1, 50).WithMessage("Person Name should be 1 to 50 characters long");

        // Validate the Gender property.
        RuleFor(request => request.Gender)
            .IsInEnum().WithMessage("Invalid gender option");
    }
}
```

Conceptual Breakdown of RegisterRequestValidator

1. AbstractValidator<RegisterRequest>

- This class validates the RegisterRequest DTO, and it defines rules for properties such as email, password, name, and gender.

2. Validating the Email Property

RuleFor(temp => temp.Email)

.NotEmpty().WithMessage("Email is required")

.EmailAddress().WithMessage("Invalid email address format");

- The rules here are the same as in LoginRequestValidator. It ensures that the email is not empty and that it follows a valid email address format.

3. Validating the Password Property

RuleFor(temp => temp.Password)

.NotEmpty().WithMessage("Password is required");

- Like the email, this ensures that the password is not empty. This is a basic validation that could be expanded upon to include additional rules like length and complexity (e.g., requiring numbers and special characters).

4. Validating the PersonName Property

RuleFor(request => request.PersonName)

.NotEmpty().WithMessage("PersonName can't be blank")

.Length(1, 50).WithMessage("Person Name should be 1 to 50 characters long");

- **Purpose:**
 - This rule validates the PersonName field of the RegisterRequest.
- **NotEmpty():**
 - Ensures that the PersonName is not blank. If it is, the message "**PersonName can't be blank**" is returned.
- **Length(1, 50):**
 - This ensures that the name must be between **1** and **50** characters in length. If it's shorter or longer than this range, the message "**Person Name should be 1 to 50 characters long**" is returned.

5. Validating the Gender Property

```
RuleFor(request => request.Gender)
```

```
.IsInEnum().WithMessage("Invalid gender option");
```

- **Purpose:**
 - The Gender property must have a valid enum value from GenderOptions (i.e., Male, Female, or Others). If the gender is not part of the enum, an **"Invalid gender option"** message is returned.
- **IsInEnum():**
 - This ensures that the provided gender value matches one of the allowed enum values. This validation is especially useful to prevent invalid or unexpected values from being passed.

FluentValidation Concepts

1. Rule Definitions

- **RuleFor():** This is the key function in FluentValidation that defines a rule for a specific property.
 - Each rule targets a particular property of the DTO (e.g., Email, Password).

2. Validation Methods

- **NotEmpty():** Ensures that a field has a value and is not null or whitespace.
- **WithMessage():** Attaches a custom error message when the validation fails. This improves the clarity of error handling for end users.
- **EmailAddress():** Ensures that a string follows a valid email format.
- **Length():** Enforces that a string property falls within a specific length range.
- **IsInEnum():** Ensures that a property value matches a valid enum option.

3. Asynchronous Validation

- While these validators are synchronous, FluentValidation supports **asynchronous validation** as well, which is useful when rules depend on external systems (like checking if an email already exists in the database).

Summary of Validations Code and Concepts

1. Validators in Clean Architecture:

- Validators, defined in the **Core** layer, ensure that data entering the system is valid and follows business rules. By defining validation rules here, you decouple validation logic from the rest of the application, promoting reusability and testability.

2. FluentValidation:

- FluentValidation provides a **fluent interface** for defining validation rules, making the code more readable and expressive. It allows for complex validation scenarios with ease.

3. LoginRequestValidator:

- Ensures that the email and password in a login attempt are present and valid.

4. RegisterRequestValidator:

- Validates user registration data, including checks for a valid email format, non-empty password, valid name length, and gender matching one of the allowed enum values.

5. Best Practices:

- It's essential to place validation in the **Core** layer because this keeps business logic clean and ensures that the domain rules are applied consistently across the application.

Mappers in Core Layer (AutoMapper)

In the **Core** layer of Clean Architecture, mapping is a crucial aspect. You often need to convert between various layers' models, such as mapping data transfer objects (DTOs) to entities and vice versa. To automate this conversion and avoid repetitive code, we use **AutoMapper**, a library that helps map properties between objects, particularly when working with DTOs, view models, and entities.

What is AutoMapper?

AutoMapper is an object-object mapping library for .NET, used to automatically map properties between objects of different types. It eliminates the need for manually assigning properties from one object to another, especially when both objects have similar properties but are used for different purposes (e.g., DTOs vs Entities).

AutoMapper relies on configuration classes called **Profiles** to define how the mappings between source and destination types should occur.

Key Concepts of AutoMapper

- **Profiles:** AutoMapper uses Profile classes to configure mappings. Each profile contains rules for converting from one type to another.
- **CreateMap:** This method inside the profile defines a map between a source type and a destination type.
- **ForMember:** This method is used to customize how specific properties are mapped. You can specify which property of the destination object should be mapped from which property of the source object.
- **Ignore:** Sometimes you don't want to map certain properties, especially when they are computed or generated after the mapping. In such cases, Ignore() is used.

Mapper: ApplicationUser to AuthenticationResponse

The first mapper is for converting the **ApplicationUser** entity to an **AuthenticationResponse** DTO. This mapping is useful when, after user authentication, you want to send a response to the client with limited but relevant user data.

Code Example:

```
using AutoMapper;

using eCommerce.Core.DTO;
using eCommerce.Core.Entities;

namespace eCommerce.Core.Mappers;

public class ApplicationUserMappingProfile : Profile
{
    public ApplicationUserMappingProfile()
    {
        CreateMap<ApplicationUser, AuthenticationResponse>()
            .ForMember(dest => dest.UserID, opt => opt.MapFrom(src => src.UserID))
            .ForMember(dest => dest.Email, opt => opt.MapFrom(src => src.Email))
            .ForMember(dest => dest.PersonName, opt => opt.MapFrom(src => src.PersonName))
            .ForMember(dest => dest.Gender, opt => opt.MapFrom(src => src.Gender))
            .ForMember(dest => dest.Success, opt => opt.Ignore()) // Computed or added later
            .ForMember(dest => dest.Token, opt => opt.Ignore()) // Token is not part of ApplicationUser
    }
}
```

```
;
}
}
```

Conceptual Breakdown:

1. **CreateMap<ApplicationUser, AuthenticationResponse>:**

- This line defines a mapping from the ApplicationUser entity to the AuthenticationResponse DTO. AutoMapper will automatically try to match properties with the same names between the source (ApplicationUser) and destination (AuthenticationResponse).

2. **ForMember Usage:**

- **ForMember(dest => dest.UserID, opt => opt.MapFrom(src => src.UserID)):**
 - This explicitly defines that the UserID property of AuthenticationResponse will be mapped from the UserID property of ApplicationUser. Even though AutoMapper can infer this automatically for same-named properties, specifying it manually provides clarity.
- **ForMember(dest => dest.Email, opt => opt.MapFrom(src => src.Email)):**
 - Similarly, the Email property from ApplicationUser is mapped to AuthenticationResponse.
- **ForMember(dest => dest.Success, opt => opt.Ignore()):**
 - The Success field in AuthenticationResponse is not mapped from any property in ApplicationUser. This field will likely be set manually during the authentication process.
- **ForMember(dest => dest.Token, opt => opt.Ignore()):**
 - The Token property is ignored, as it will be generated after authentication and is not part of the ApplicationUser entity.

Use Case:

- This mapping is commonly used after a user successfully logs in. The application retrieves the user details from the database (as an ApplicationUser object), maps it to AuthenticationResponse, and adds any other necessary information (like Success and Token) before sending it as a response.

Mapper: RegisterRequest to ApplicationUser

The second mapper is for converting a **RegisterRequest** DTO to an **ApplicationUser** entity. This is necessary when registering a new user, as the request data must be converted into an entity that can be stored in the database.

Code Example:

```
using AutoMapper;

using eCommerce.Core.DTO;

using eCommerce.Core.Entities;

namespace eCommerce.Core.Mappers;

public class RegisterRequestMappingProfile : Profile
{
    public RegisterRequestMappingProfile()
    {
        CreateMap<RegisterRequest, ApplicationUser>()
            .ForMember(dest => dest.Email, opt => opt.MapFrom(src => src.Email))
            .ForMember(dest => dest.PersonName, opt => opt.MapFrom(src => src.PersonName))
            .ForMember(dest => dest.Gender, opt => opt.MapFrom(src => src.Gender.ToString())) // Convert
            enum to string
            .ForMember(dest => dest.Password, opt => opt.MapFrom(src => src.Password));
    }
}
```

Conceptual Breakdown:

1. **CreateMap<RegisterRequest, ApplicationUser>:**

- This line defines the mapping from the RegisterRequest DTO to the ApplicationUser entity. This is useful for converting registration data into an entity that can be saved to the database.

2. **ForMember Usage:**

- **ForMember(dest => dest.Email, opt => opt.MapFrom(src => src.Email)):**
 - Maps the Email field of the RegisterRequest to the Email field of the ApplicationUser. This is straightforward since both properties have the same name and type.

- **ForMember(dest => dest.PersonName, opt => opt.MapFrom(src => src.PersonName)):**
 - Maps the PersonName field of the RegisterRequest to the PersonName field of the ApplicationUser.
- **ForMember(dest => dest.Gender, opt => opt.MapFrom(src => src.Gender.ToString())):**
 - The Gender property in RegisterRequest is of type GenderOptions (an enum), but in ApplicationUser, it's stored as a string. This rule uses ToString() to convert the enum to its string representation.
- **ForMember(dest => dest.Password, opt => opt.MapFrom(src => src.Password)):**
 - The password is directly mapped from the DTO to the entity. In a real-world scenario, passwords should be hashed before storing them, but that logic would typically happen elsewhere, not in the mapping.

Use Case:

- When a user registers, the data submitted in a RegisterRequest is converted to an ApplicationUser entity, which is then saved to the database. This mapping ensures the DTO is properly translated into a format suitable for persistence.

Summary of AutoMapper Code and Concepts

1. **AutoMapper in Clean Architecture:**
 - AutoMapper simplifies the mapping process between different layers (DTOs, entities). It's especially useful when DTOs have properties with the same names as the entities they map to, allowing automatic mapping.
2. **Profiles in AutoMapper:**
 - **Profiles** are a way to organize mappings. Each Profile defines rules for converting between different types (e.g., ApplicationUser to AuthenticationResponse).
3. **Custom Mapping with ForMember:**
 - The ForMember() method is used to explicitly define how a property should be mapped, offering flexibility. Properties that don't need to be mapped (like computed properties) can be ignored using Ignore().
4. **ApplicationUser to AuthenticationResponse:**
 - The mapper converts ApplicationUser to AuthenticationResponse, ensuring that only necessary fields are returned to the client after a successful login.
5. **RegisterRequest to ApplicationUser:**
 - Converts RegisterRequest to ApplicationUser, preparing the data for storage during user registration.

Services in the Core Layer

In the **Core** layer of a Clean Architecture solution, the service layer defines business logic for the application. It handles interactions between repositories (which deal with data persistence) and external controllers or consumers (such as APIs). This is where most of the application's business rules are implemented, and it's a key layer for encapsulating the domain logic.

In this case, we have a `UserService` that handles user-related operations like **login** and **registration**. These methods interact with a repository that manages the persistence of user data and return an appropriate response to the client.

The service also leverages **AutoMapper** to map between DTOs and domain entities, ensuring that only necessary data is exposed externally.

Key Concepts

1. Dependency Injection (DI)

ASP.NET Core provides dependency injection (DI) by default, which allows you to easily inject required services into classes. In this example, the `UserService` uses DI to inject the `IUsersRepository` and `IMapper` objects.

2. AutoMapper in Services

The service uses **AutoMapper** to map between **DTOs** and **Entities**:

- Mapping from `ApplicationUser` (entity) to `AuthenticationResponse` (DTO).
- Mapping from `RegisterRequest` (DTO) to `ApplicationUser` (entity).

This mapping reduces boilerplate code that would otherwise involve manually assigning each field from the entity to the DTO and vice versa.

3. Repository Pattern

The service interacts with a **repository** that abstracts data access logic. This repository is responsible for fetching and saving user data, and by abstracting it, the service does not need to know the details of how data is stored (database, file, etc.). This separation of concerns leads to cleaner, more maintainable code.

4. DTOs for API Responses

DTOs (Data Transfer Objects) are used to send and receive data. For login and registration, the service returns an `AuthenticationResponse`, which contains information like `UserID`, `Email`, and `Token`. Using DTOs ensures that only necessary data is shared, keeping sensitive information (e.g., password) hidden.

5. with Expression for Immutable Objects

C# record types, like `AuthenticationResponse`, are immutable by default, meaning you can't modify their properties once set. However, with the **with expression**, you can create a new object by copying an existing one, while modifying some of its properties. This is useful for returning a new `AuthenticationResponse` object with updated `Success` and `Token` fields.

Code Walkthrough

Here's the complete code for `UsersService`, followed by a breakdown of each part.

```
using AutoMapper;

using eCommerce.Core.DTO;

using eCommerce.Core.Entities;

using eCommerce.Core.RepositoryContracts;

using eCommerce.Core.ServiceContracts;

namespace eCommerce.Core.Services;

internal class UsersService : IUsersService
{
    private readonly IUsersRepository _usersRepository;

    private readonly IMapper _mapper;

    public UsersService(IUsersRepository usersRepository, IMapper mapper)
    {
        _usersRepository = usersRepository;

        _mapper = mapper;
    }
}
```

```

public async Task<AuthenticationResponse?> Login(LoginRequest loginRequest)
{
    ApplicationUser? user = await _usersRepository.GetUserByEmailAndPassword(loginRequest.Email,
loginRequest.Password);

    if (user == null)
    {
        return null;
    }

    // Return AuthenticationResponse with success and token
    return _mapper.Map<AuthenticationResponse>(user) with { Success = true, Token = "token" };
}

public async Task<AuthenticationResponse?> Register(RegisterRequest registerRequest)
{
    // Create ApplicationUser object from RegisterRequest using AutoMapper
    ApplicationUser user = _mapper.Map<ApplicationUser>(registerRequest);
    ApplicationUser? registeredUser = await _usersRepository.AddUser(user);

    if (registeredUser == null)
    {
        return null;
    }

    // Return AuthenticationResponse with success and token
    return _mapper.Map<AuthenticationResponse>(registeredUser) with { Success = true, Token =
"token" };
}
}

```

Conceptual Breakdown

1. Constructor Injection

```
public UsersService(IUsersRepository usersRepository, IMapper mapper)
{
    _usersRepository = usersRepository;
    _mapper = mapper;
}
```

- **Dependency Injection:** The constructor injects two dependencies: `IUsersRepository` and `IMapper`. The repository handles data access, while `AutoMapper` handles object mapping.
- These dependencies are essential to keeping the service isolated from low-level logic (like database queries) and focusing purely on business rules.

2. Login Method

```
public async Task<AuthenticationResponse?> Login(LoginRequest loginRequest)
{
    ApplicationUser? user = await _usersRepository.GetUserByEmailAndPassword(loginRequest.Email,
    loginRequest.Password);

    if (user == null)
    {
        return null;
    }

    return _mapper.Map<AuthenticationResponse>(user) with { Success = true, Token = "token" };
}
```

- **LoginRequest:** The method accepts a `LoginRequest` DTO that contains the user's email and password. This DTO simplifies the request payload from the client.
- **Repository Call:**
 - It uses the `IUsersRepository.GetUserByEmailAndPassword()` method to fetch the user from the data store based on the credentials.
 - If no user is found (the credentials are invalid), it returns null.

- **Mapping:**
 - If the user is found, it uses AutoMapper to convert the ApplicationUser entity into an AuthenticationResponse DTO.
 - The with expression modifies the returned object by setting Success = true and generating a placeholder Token.
- **Token Generation:**
 - In a real-world scenario, this is where you'd generate a JWT or some other form of authentication token. For now, the placeholder "token" is used.

3. Register Method

```
public async Task<AuthenticationResponse?> Register(RegisterRequest registerRequest)
{
    // Create ApplicationUser object from RegisterRequest using AutoMapper
    ApplicationUser user = _mapper.Map<ApplicationUser>(registerRequest);
    ApplicationUser? registeredUser = await _usersRepository.AddUser(user);

    if (registeredUser == null)
    {
        return null;
    }

    return _mapper.Map<AuthenticationResponse>(registeredUser) with { Success = true, Token =
    "token" };
}
```

- **RegisterRequest:** Accepts a RegisterRequest DTO, which contains details like email, password, name, and gender.
- **AutoMapper Mapping:**
 - AutoMapper converts the RegisterRequest into an ApplicationUser entity. This conversion includes mapping properties like Email, Password, and PersonName.
- **Repository Call:**
 - The repository's AddUser() method is called to add the new user to the data store. If the user is successfully added, the method returns the registered user.
 - If registration fails for any reason (e.g., a database error), the method returns null.

- **Response Mapping:**
 - Similar to the Login method, the ApplicationUser is mapped to an AuthenticationResponse DTO, and the with expression sets Success = true and generates a placeholder Token.

Summary of Key Concepts of "Services" in "Core" layer

1. **Service Layer:**
 - Handles business logic and is responsible for orchestrating interactions between the repository and external consumers (controllers, clients).
2. **AutoMapper:**
 - Used to map between domain models (entities) and DTOs (for API communication), reducing boilerplate code.
3. **Repository Pattern:**
 - Repositories abstract the data access logic. The service uses these repositories to interact with the data store without worrying about the specifics of how data is stored or retrieved.
4. **DTOs:**
 - Used to encapsulate the data sent or received from the client. This layer of abstraction ensures that only necessary information is exposed to the client, enhancing security.
5. **Immutable Records and with Expressions:**
 - Records in C# are immutable, meaning once created, their properties can't be changed. However, you can create a new object with modified properties using the with expression.

By implementing services this way, we ensure that our application is maintainable, testable, and adheres to clean architecture principles.

Dependency Injection File in Core Layer

In the **Core** layer of a Clean Architecture solution, dependency injection (DI) is crucial for managing service lifetimes and dependencies. The DI configuration helps in setting up the application's services and ensuring they are available for use across different parts of the application.

The `DependencyInjection` class in the **Core** layer provides an extension method to configure and register services with the .NET Core dependency injection container. This allows you to inject services into other parts of the application, such as controllers or other services.

Key Concepts

1. Dependency Injection (DI)

- **Dependency Injection** is a design pattern used to implement IoC (Inversion of Control), where the control of dependencies is inverted from the class itself to an external container. This promotes loose coupling and improves testability.

2. Service Lifetime Management

- **Transient:** Services are created each time they are requested. Ideal for lightweight, stateless services.
- **Scoped:** Services are created once per request. Useful for services that should be shared within a single request.
- **Singleton:** Services are created once and shared across all requests. Best for stateless services that do not need to be recreated.

3. Adding Services to the DI Container

- **Adding Services:** You register services with the dependency injection container using methods like `AddTransient`, `AddScoped`, or `AddSingleton`.

Code Walkthrough

Here's the complete code for the `DependencyInjection` class in the Core layer, followed by a detailed explanation.

```
using eCommerce.Core.ServiceContracts;
using eCommerce.Core.Services;
using eCommerce.Core.Validators;
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;

namespace eCommerce.Core;
```

```
public static class DependencyInjection
{
    /// <summary>
    /// Extension method to add core services to the dependency injection container
    /// </summary>
    /// <param name="services"></param>
    /// <returns></returns>
    public static IServiceCollection AddCore(this IServiceCollection services)
    {
        // TO DO: Add services to the IoC container
        // Core services often include validation, caching and other business components.

        // Registering the UsersService as a transient service
        services.AddTransient<IUsersService, UsersService>();

        // Registering FluentValidation validators from the assembly containing the LoginRequestValidator
        services.AddValidatorsFromAssemblyContaining<LoginRequestValidator>();

        return services;
    }
}
```


Conceptual Breakdown

1. Extension Method for IServiceCollection

```
public static class DependencyInjection
{
    public static IServiceCollection AddCore(this IServiceCollection services)
    {
        // Code to add services here
        return services;
    }
}
```

- **Static Class and Method:**
 - The DependencyInjection class is a static class, and AddCore is a static extension method for IServiceCollection. This is a common pattern for configuring DI in .NET Core applications.
- **Extension Method:**
 - The AddCore method is an extension method, which means it extends the IServiceCollection interface. This allows you to call it like an instance method on IServiceCollection.

2. Registering Services

```
services.AddTransient<IUserService, UserService>();
```

- **Service Registration:**
 - The AddTransient method registers UserService with the DI container. This means that a new instance of UserService will be created each time it is requested.
- **Interface to Implementation Mapping:**
 - IUserService is the interface that UserService implements. By registering this mapping, you ensure that whenever IUserService is requested, an instance of UserService will be provided.

3. Registering Validators

`services.AddValidatorsFromAssemblyContaining<LoginRequestValidator>();`

- **FluentValidation Integration:**
 - The `AddValidatorsFromAssemblyContaining` method registers all validators in the assembly containing the specified validator (in this case, `LoginRequestValidator`). This is a convenient way to add all validators without needing to register each one individually.
- **Validators:**
 - `FluentValidation` is used for validating data transfer objects (DTOs) and ensures that incoming data adheres to specified rules. By registering validators, you enable automatic validation within the application.

Summary of Key Concepts in "Dependency Injection" file in "Core" layer:

1. **Dependency Injection (DI):**
 - A design pattern used to manage dependencies between classes, promoting loose coupling and testability.
2. **Service Lifetimes:**
 - Services can be registered with different lifetimes (Transient, Scoped, Singleton), affecting how and when instances are created.
3. **Extension Methods for IServiceCollection:**
 - Used to configure and register services with the DI container in a clean and modular way.
4. **AutoMapper and Validators:**
 - **AutoMapper** maps between DTOs and entities, simplifying data transformation.
 - **FluentValidation** provides a clean and expressive way to validate DTOs and other objects.

By following these practices, you ensure that your application's dependencies are well-managed, and its services are easy to maintain and test.

DbContext in Infrastructure Layer

In the **Infrastructure** layer of a Clean Architecture solution, the DbContext class is responsible for managing database connections and providing an abstraction for data access. In this specific case, the DapperDbContext class uses **Dapper** (a micro ORM) for database operations and **Npgsql** (a .NET data provider for PostgreSQL).

Key Concepts

1. Dapper and Npgsql

- **Dapper:**
 - A lightweight Object-Relational Mapper (ORM) for .NET. It simplifies data access and is known for its high performance compared to other ORMs like Entity Framework.
- **Npgsql:**
 - A .NET data provider for PostgreSQL. It allows .NET applications to interact with PostgreSQL databases.

2. DbContext Pattern

- **DbContext:**
 - Typically refers to Entity Framework Core's context class, but here it's used more generically. It provides methods to manage database connections and perform operations.

3. IConfiguration Interface

- **IConfiguration:**
 - Used to read configuration settings from various sources (e.g., appsettings.json, environment variables). In this case, it retrieves the database connection string.

Code Walkthrough

Here's the complete code for the `DapperDbContext` class, followed by a detailed explanation.

```
using Microsoft.Extensions.Configuration;
```

```
using Npgsql;
```

```
using System.Data;
```

```
namespace eCommerce.Infrastructure.DbContext;
```

```
public class DapperDbContext
```

```
{
```

```
    private readonly IConfiguration _configuration;
```

```
    private readonly IDbConnection _connection;
```

```
    public DapperDbContext(IConfiguration configuration)
```

```
    {
```

```
        _configuration = configuration;
```

```
        string? connectionString = _configuration.GetConnectionString("PostgresConnection");
```

```
        // Create a new NpgsqlConnection with the retrieved connection string
```

```
        _connection = new NpgsqlConnection(connectionString);
```

```
    }
```

```
    public IDbConnection DbConnection => _connection;
```

```
}
```

Conceptual Breakdown

1. Constructor and Configuration

```
public DapperDbContext(IConfiguration configuration)
{
    _configuration = configuration;

    string? connectionString = _configuration.GetConnectionString("PostgresConnection");

    // Create a new NpgsqlConnection with the retrieved connection string
    _connection = new NpgsqlConnection(connectionString);
}
```

- **Constructor Injection:**
 - The constructor accepts an `IConfiguration` instance, which is injected via dependency injection. This instance provides access to configuration settings, including connection strings.
- **Retrieving Connection String:**
 - `GetConnectionString("PostgresConnection")` retrieves the connection string named "PostgresConnection" from the configuration. This string is used to establish a connection to the PostgreSQL database.
- **NpgsqlConnection:**
 - `NpgsqlConnection` is created using the connection string. This object manages the connection to the PostgreSQL database and is used for executing queries and commands.

2. DbConnection Property

```
public IDbConnection DbConnection => _connection;
```

- **IDbConnection Interface:**
 - The `IDbConnection` interface is a common abstraction for database connections in ADO.NET. By exposing `_connection` as `IDbConnection`, the class adheres to a common interface, promoting flexibility and testability.
- **Property Getter:**
 - The `DbConnection` property provides access to the underlying database connection. This allows other components (e.g., repositories or services) to use this connection for database operations without directly managing connection details.

Summary of Key Concepts in DbContext

1. **Dapper and Npgsql:**
 - **Dapper** is used for lightweight data access, and **Npgsql** is the data provider for PostgreSQL.
2. **DbContext Pattern:**
 - Manages database connections and provides an abstraction for data access, regardless of whether you use Entity Framework or another data access technology.
3. **IConfiguration:**
 - Used to retrieve configuration settings, such as database connection strings, from various sources.
4. **IDbConnection Interface:**
 - Provides a common abstraction for database connections, allowing for flexibility and testability.

By using the `DapperDbContext` class, you efficiently manage your database connections and maintain separation of concerns between data access and business logic. This approach aligns with Clean Architecture principles, promoting a modular and maintainable codebase.

Repository in Infrastructure Layer

In the **Infrastructure** layer, the `UsersRepository` class handles data access logic, interfacing with the database using **Dapper**. This class implements the `IUsersRepository` contract, defining methods for interacting with the `ApplicationUser` entities.

Key Concepts

1. Dapper

- **Dapper:**
 - A micro ORM (Object-Relational Mapper) that provides a way to interact with the database using SQL queries while mapping results to objects.

2. Repository Pattern

- **Repository Pattern:**
 - A design pattern that abstracts data access logic, providing a clean API for accessing data and separating the data layer from the business logic.

3. Dependency Injection

- **Dependency Injection (DI):**
 - Used to provide the `DapperDbContext` instance to the repository, ensuring loose coupling and promoting testability.

Code Walkthrough

Here's the complete code for the `UsersRepository` class, followed by a detailed explanation.

```
using Dapper;

using eCommerce.Core.DTO;

using eCommerce.Core.Entities;

using eCommerce.Core.RepositoryContracts;

using eCommerce.Infrastructure.DbContext;

namespace eCommerce.Infrastructure.Repositories;

internal class UsersRepository : IUsersRepository
{
    private readonly DapperDbContext _dbContext;

    public UsersRepository(DapperDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public async Task<ApplicationUser?> AddUser(ApplicationUser user)
    {
        // Generate a new unique user ID for the user
```

```
user.UserID = Guid.NewGuid();
```

```
// SQL Query to insert user data into the "Users" table
```

```
string query = "INSERT INTO public.\"Users\"(\"UserID\", \"Email\", \"PersonName\", \"Gender\", \"Password\") VALUES(@UserID, @Email, @PersonName, @Gender, @Password)";
```

```
int rowCountAffected = await _dbContext.DbConnection.ExecuteAsync(query, user);
```

```
if (rowCountAffected > 0 )
```

```
{
```

```
    return user;
```

```
}
```

```
else
```

```
{
```

```
    return null;
```

```
}
```

```
}
```

```
public async Task<ApplicationUser?> GetUserByEmailAndPassword(string? email, string? password)
```

```
{
```

```
    // SQL query to select a user by Email and Password
```

```
    string query = "SELECT * FROM public.\"Users\" WHERE \"Email\"=@Email AND \"Password\"=@Password";
```

```
    var parameters = new { Email = email, Password = password };
```

```
    ApplicationUser? user = await
```

```
_dbContext.DbConnection.QueryFirstOrDefaultAsync<ApplicationUser>(query, parameters);
```

```
    return user;
```

```
}
```

```
}
```


Conceptual Breakdown

1. Constructor and Dependency Injection

```
public UsersRepository(DapperDbContext dbContext)
{
    _dbContext = dbContext;
}
```

- **Constructor Injection:**

- The constructor accepts an instance of DapperDbContext, which is injected via DI. This allows the repository to use the database connection provided by DapperDbContext.

2. Adding a User

```
public async Task<ApplicationUser?> AddUser(ApplicationUser user)
{
    user.UserID = Guid.NewGuid(); // Generate a new unique user ID

    string query = "INSERT INTO public.\"Users\"(\"UserID\", \"Email\", \"PersonName\", \"Gender\", \"Password\") VALUES(@UserID, @Email, @PersonName, @Gender, @Password)";

    int rowCountAffected = await _dbContext.DbConnection.ExecuteAsync(query, user);

    if (rowCountAffected > 0 )
    {
        return user;
    }
    else
    {
        return null;
    }
}
```

- **Generate Unique ID:**

- user.UserID = Guid.NewGuid(); generates a new GUID for the UserID, ensuring each user has a unique identifier.

- **SQL Query:**
 - The SQL INSERT query adds a new user to the "Users" table. The @UserID, @Email, @PersonName, @Gender, and @Password parameters are replaced with the values from the user object.
- **ExecuteAsync:**
 - ExecuteAsync executes the SQL command asynchronously. The method returns the number of rows affected. If greater than 0, it indicates success, and the user object is returned; otherwise, null is returned.

3. Retrieving a User

```
public async Task<ApplicationUser?> GetUserByEmailAndPassword(string? email, string? password)
{
    string query = "SELECT * FROM public.\"Users\" WHERE \"Email\"=@Email AND \"Password\"=@Password";
    var parameters = new { Email = email, Password = password };

    ApplicationUser? user = await
        _dbContext.DbConnection.QueryFirstOrDefaultAsync<ApplicationUser>(query, parameters);

    return user;
}
```

- **SQL Query:**
 - The SQL SELECT query retrieves a user based on their Email and Password. The parameters @Email and @Password are replaced with the values from the parameters object.
- **QueryFirstOrDefaultAsync:**
 - QueryFirstOrDefaultAsync executes the query asynchronously and returns the first matching record or null if no match is found.

Summary of Key Concepts of Repository

1. Dapper:

- A micro ORM that simplifies data access with SQL queries, offering high performance and ease of use.

2. Repository Pattern:

- Provides a clean API for data access and abstracts data operations, promoting separation of concerns.

3. Dependency Injection (DI):

- Ensures loose coupling and testability by providing necessary dependencies to classes.

4. SQL Operations:

- **Insert:** Adds new records to the database.
- **Select:** Retrieves records based on specific criteria.

By using the UsersRepository class, you efficiently manage data access operations for user entities, keeping data-related logic separate from business logic and aligning with Clean Architecture principles.

Dependency Injection in Infrastructure Layer

In the **Infrastructure** layer, the DependencyInjection class is responsible for registering services with the dependency injection (DI) container. This setup allows various components of the application to obtain the required services through DI, promoting modularity and testability.

Key Concepts

1. Dependency Injection (DI)

• Dependency Injection:

- A design pattern that allows objects to receive their dependencies from an external source rather than creating them internally. This promotes loose coupling, modularity, and ease of testing.

2. Service Registration

- **Service Registration:**
 - The process of adding services to the DI container. These services can then be injected into other components, such as controllers or services, at runtime.

3. Service Lifetimes

- **Service Lifetimes:**
 - **Transient:** Services are created each time they are requested. Used for lightweight, stateless services.
 - **Scoped:** Services are created once per request. Ideal for services that need to maintain state during a single request.
 - **Singleton:** A single instance is created and shared throughout the application's lifetime. Suitable for services that are expensive to create or need to maintain a single state.

Code Walkthrough

Here's the complete code for the DependencyInjection class in the Infrastructure layer, followed by a detailed explanation.

```
using eCommerce.Core.RepositoryContracts;

using eCommerce.Infrastructure.DbContext;

using eCommerce.Infrastructure.Repositories;

using Microsoft.Extensions.DependencyInjection;

namespace eCommerce.Infrastructure;

public static class DependencyInjection
{
    /// <summary>
    /// Extension method to add infrastructure services to the dependency injection container
    /// </summary>
    /// <param name="services"></param>
    /// <returns></returns>
```

```
public static IServiceCollection AddInfrastructure(this IServiceCollection services)
{
    // TO DO: Add services to the IoC container
    // Infrastructure services often include data access, caching and other low-level components.

    services.AddTransient<IUsersRepository, UsersRepository>();

    services.AddTransient<DapperDbContext>();

    return services;
}
```

Conceptual Breakdown

1. Extension Method

```
public static IServiceCollection AddInfrastructure(this IServiceCollection services)
{
    // TO DO: Add services to the IoC container
    // Infrastructure services often include data access, caching and other low-level components.
}
```

- **Extension Method:**
 - This method is defined as a static method in a static class. It extends `IServiceCollection`, allowing the addition of infrastructure-specific services to the DI container.

2. Registering Services

```
services.AddTransient<IUsersRepository, UsersRepository>();
```

- **Service Registration:**
 - `AddTransient<IUsersRepository, UsersRepository>()` registers the `UsersRepository` class as an implementation of the `IUsersRepository` interface. This means whenever `IUsersRepository` is requested, an instance of `UsersRepository` will be provided.
- **Transient Lifetime:**
 - This specifies that a new instance of `UsersRepository` will be created each time it is requested.

```
services.AddTransient<DapperDbContext>();
```

- **DapperDbContext Registration:**
 - `AddTransient<DapperDbContext>()` registers `DapperDbContext` as a transient service. This means a new instance of `DapperDbContext` will be created each time it is needed.

3. Returning IServiceCollection

```
return services;
```

- **Fluent Configuration:**
 - Returning the `IServiceCollection` instance allows for chaining multiple service registrations and configurations within the DI setup.

Summary of Key Concepts of Dependency Injection in Infrastructure Layer

1. **Dependency Injection (DI):**
 - A design pattern that promotes loose coupling and modularity by injecting dependencies rather than creating them internally.
2. **Service Registration:**
 - The process of adding services to the DI container to be injected where needed.
3. **Service Lifetimes:**
 - **Transient:** Services created each time requested.
 - **Scoped:** Services created once per request.
 - **Singleton:** A single shared instance throughout the application's lifetime.
4. **Extension Method:**
 - Allows the addition of services to `IServiceCollection`, enabling a modular and organized setup of service registrations.

By using the `DependencyInjection` class, you ensure that your infrastructure services, such as repositories and data contexts, are correctly registered and managed by the DI container, supporting the Clean Architecture principles and enhancing the maintainability of your application.

Program.cs File in API Layer

The `Program.cs` file in an ASP.NET Core application configures and sets up the application's services, middleware, and request pipeline. It is essential for initializing the application and specifying how various components interact with each other.

Key Concepts

1. WebApplication

- **WebApplication:**
 - A class that provides a simplified way to configure and build an ASP.NET Core application. It consolidates the functionalities of `IHostBuilder` and `IWebHostBuilder` into one class.

2. Service Registration

- **Service Registration:**
 - Adding services to the DI container, which are then available for dependency injection throughout the application.

3. Middleware

- **Middleware:**
 - Components that handle HTTP requests and responses in the request pipeline. Middleware can handle tasks like error handling, authentication, and logging.

4. Swagger/OpenAPI

- **Swagger/OpenAPI:**
 - Tools for generating interactive API documentation and specifications.

5. CORS

- **CORS (Cross-Origin Resource Sharing):**
 - A security feature that controls how resources are shared between different origins (domains).

Code Walkthrough

Here's a detailed explanation of each section of the Program.cs file.

```
using eCommerce.Infrastructure;  
using eCommerce.Core;  
using eCommerce.API.Middlewares;  
using System.Text.Json.Serialization;  
using eCommerce.Core.Mappers;  
using FluentValidation.AspNetCore;
```

```
var builder = WebApplication.CreateBuilder(args);
```

- **WebApplication.CreateBuilder:**
 - Initializes a new instance of the WebApplicationBuilder class, which is used to configure and build the web application. args represents command-line arguments passed to the application.

```
// Add Infrastructure services  
builder.Services.AddInfrastructure();  
builder.Services.AddCore();
```

- **Service Registration:**
 - AddInfrastructure and AddCore methods register services from the Infrastructure and Core layers, respectively, with the DI container.

```
// Add controllers to the service collection  
builder.Services.AddControllers().AddJsonOptions(options => {  
    options.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter());  
});
```


- **AddControllers:**
 - Registers controllers with the DI container and sets up the MVC framework.
- **AddJsonOptions:**
 - Configures JSON serialization options for the application. JsonStringEnumConverter ensures that enums are serialized as their string representations instead of numeric values.

```
builder.Services.AddAutoMapper(typeof(ApplicationUserMappingProfile).Assembly);
```

- **AddAutoMapper:**
 - Registers AutoMapper with the DI container. This configuration scans the assembly for mapping profiles, such as ApplicationUserMappingProfile, to set up object-to-object mappings.

```
// FluentValidations
```

```
builder.Services.AddFluentValidationAutoValidation();
```

- **AddFluentValidationAutoValidation:**
 - Registers FluentValidation and configures it to automatically validate models based on the validators defined in the application.

```
// Add API explorer services
```

```
builder.Services.AddEndpointsApiExplorer();
```

- **AddEndpointsApiExplorer:**
 - Adds services required for endpoint exploration, enabling API discovery and documentation.

```
// Add swagger generation services to create swagger specification
```

```
builder.Services.AddSwaggerGen();
```

- **AddSwaggerGen:**
 - Registers Swagger services to generate API documentation and interactive UI. Swagger helps in creating a visual representation of the API endpoints and their details.

```
// Add cors services
builder.Services.AddCors(options =>
{
    options.AddDefaultPolicy(builder => {
        builder.WithOrigins("http://localhost:4200")
        .AllowAnyMethod()
        .AllowAnyHeader();
    });
});
```

- **AddCors:**

- Configures Cross-Origin Resource Sharing (CORS) policies. The default policy allows requests from http://localhost:4200 (commonly used for Angular applications) and permits any HTTP method and header.

```
// Build the web application
```

```
var app = builder.Build();
```

- **Build:**

- Builds the WebApplication instance from the configuration defined in the WebApplicationBuilder.

```
app.UseExceptionHandlerMiddleware();
```

- **UseExceptionHandlerMiddleware:**

- Adds custom exception handling middleware to the request pipeline. This middleware handles exceptions globally and provides appropriate responses or logs.

```
// Routing
```

```
app.UseRouting();
```

```
app.UseSwagger(); // Adds endpoint that can serve the swagger.json
```

```
app.UseSwaggerUI(); // Adds Swagger UI (interactive page to explore and test API endpoints)
```

```
app.UseCors();
```

- **UseRouting:**
 - Adds routing middleware to the request pipeline, enabling routing based on the request URL.
- **UseSwagger:**
 - Adds middleware to serve the Swagger specification (swagger.json) endpoint.
- **UseSwaggerUI:**
 - Adds middleware to serve the Swagger UI, providing an interactive interface for testing and exploring the API.
- **UseCors:**
 - Adds CORS middleware to apply the configured CORS policy to incoming requests.

```
// Auth
```

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

- **UseAuthentication:**
 - Adds authentication middleware to the pipeline, enabling user authentication.
- **UseAuthorization:**
 - Adds authorization middleware to the pipeline, enforcing authorization policies.

```
// Controller routes
```

```
app.MapControllers();
```

- **MapControllers:**
 - Maps controller endpoints to routes, enabling the application to respond to HTTP requests routed to controllers.

app.Run();

- **Run:**
 - Starts the web application and begins listening for incoming HTTP requests.

Summary of Key Concepts of Program.cs file in "API" layer

1. **WebApplication:**
 - Simplified class for building and configuring an ASP.NET Core application.
2. **Service Registration:**
 - Configuring services and middleware in the DI container.
3. **Middleware:**
 - Components that handle requests and responses in the pipeline.
4. **Swagger/OpenAPI:**
 - Tools for API documentation and interactive testing.
5. **CORS:**
 - Security feature controlling resource sharing between different origins.
6. **FluentValidation:**
 - Library for automatically validating models based on defined rules.

By configuring services, middleware, and routes in the Program.cs file, you set up a comprehensive request pipeline and ensure that the application components are correctly wired together.

appsettings.json File in API Layer

The appsettings.json file is a configuration file used in ASP.NET Core applications to manage settings and configurations for different aspects of the application. It is a JSON file where you can specify various settings like logging, connection strings, application settings, etc.

Key Concepts

1. Configuration System

- **Configuration System:**
 - ASP.NET Core uses a flexible configuration system that allows you to load settings from various sources, including JSON files, environment variables, and command-line arguments.

2. Logging Configuration

- **Logging:**
 - ASP.NET Core provides built-in logging services. You can configure log levels for different categories to control the verbosity of logging output.

3. Connection Strings

- **Connection Strings:**
 - A connection string specifies how the application connects to a database. It includes details like the server address, database name, and authentication credentials.

Code Walkthrough

Here's a detailed explanation of each section of the appsettings.json file.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings":
```

```
{  
  "PostgresConnection": "Host=localhost; Port=5432; Database=eCommerceUsers;  
Username=postgres; Password=admin"  
}  
}
```

1. Logging Configuration

```
"Logging": {  
  "LogLevel": {  
    "Default": "Information",  
    "Microsoft.AspNetCore": "Warning"  
  }  
}
```

- **Logging:**
 - Configures the logging system for the application.
- **LogLevel:**
 - Specifies the minimum log level for different categories of logs.
 - **Default:**
 - Sets the default log level to Information, meaning all logs of level Information and above (e.g., Warning, Error) will be recorded.
 - **Microsoft.AspNetCore:**
 - Sets the log level for the Microsoft.AspNetCore namespace to Warning. This means only warnings and errors from ASP.NET Core will be logged, reducing the verbosity compared to the default setting.

2. Allowed Hosts

```
"AllowedHosts": "*"
```

- **AllowedHosts:**
 - Specifies which hosts are allowed to access the application. The wildcard * allows any host to make requests to the application, which is suitable for development environments but should be restricted in production for security reasons.

3. Connection Strings

"ConnectionStrings":

```
{  
  "PostgresConnection": "Host=localhost; Port=5432; Database=eCommerceUsers;  
Username=postgres; Password=admin"  
}
```

- **ConnectionStrings:**
 - Contains the connection string settings for various databases used by the application.
- **PostgresConnection:**
 - Defines the connection string for connecting to a PostgreSQL database.
 - **Host:**
 - Specifies the database server's address (localhost means the server is running on the same machine as the application).
 - **Port:**
 - The port number used by the PostgreSQL server (5432 is the default port for PostgreSQL).
 - **Database:**
 - The name of the database to connect to (eCommerceUsers in this case).
 - **Username:**
 - The username used to authenticate with the database (postgres).
 - **Password:**
 - The password associated with the username (admin).

Summary of Key Concepts of appsettings.json

1. **Configuration System:**
 - ASP.NET Core's flexible system for managing application settings from multiple sources.
2. **Logging:**
 - Controls the verbosity of logs based on categories and severity levels.
3. **Connection Strings:**
 - Specifies how the application connects to databases, including server details and authentication credentials.

The appsettings.json file centralizes configuration settings, making it easier to manage and adjust various aspects of the application without changing code.

ExceptionHandlerMiddleware in API Layer

The ExceptionHandlerMiddleware class is a custom middleware designed to handle exceptions that occur during the processing of HTTP requests in an ASP.NET Core application. This middleware captures exceptions, logs them, and provides a consistent error response to clients.

Key Concepts

1. Middleware

- **Middleware:**
 - Components in the request pipeline that handle requests and responses. They are used to perform operations like logging, authentication, exception handling, etc.
- **RequestDelegate:**
 - Represents a method that can process HTTP requests. It is used by middleware to call the next component in the pipeline.

2. Exception Handling

- **Exception Handling:**
 - The process of catching and managing exceptions to prevent application crashes and provide meaningful error responses.
- **Logging:**
 - Capturing and recording details of exceptions for debugging and monitoring purposes.

Code Walkthrough

ExceptionHandlerMiddleware Class

```
using Microsoft.AspNetCore.Builder;
```

```
using Microsoft.AspNetCore.Http;
```



```
using System.Threading.Tasks;
```

```
namespace eCommerce.API.Middlewares;
```

```
public class ExceptionHandlingMiddleware
```

```
{
```

```
    private readonly RequestDelegate _next;
```

```
    private readonly ILogger<ExceptionHandlingMiddleware> _logger;
```

```
    public ExceptionHandlingMiddleware(RequestDelegate next,  
    ILogger<ExceptionHandlingMiddleware> logger)
```

```
    {
```

```
        _next = next;
```

```
        _logger = logger;
```

```
    }
```

```
    public async Task Invoke(HttpContext httpContext)
```

```
    {
```

```
        try
```

```
        {
```

```
            await _next(httpContext);
```

```
        }
```

```
        catch (Exception ex)
```

```
        {
```

```
            //Log the exception type and message
```

```
            _logger.LogError($"{ex.GetType().ToString()}: {ex.Message}");
```

```
            if (ex.InnerException is not null)
```

```
            {
```

```
                //Log the inner exception type and message
```

```
                _logger.LogError($"{ex.InnerException.GetType().ToString()}: {ex.InnerException.Message}");
```

```

    }

    httpContext.Response.StatusCode = 500; //Internal Server Error

    await httpContext.Response.WriteAsJsonAsync(new { Message = ex.Message, Type =
ex.GetType().ToString() });
    }
}
}

```

- **Constructor:**

```

public ExceptionHandlingMiddleware(RequestDelegate next,
ILogger<ExceptionHandlingMiddleware> logger)
{
    _next = next;
    _logger = logger;
}

```

- **RequestDelegate next:**

- Represents the next delegate (middleware) in the pipeline.

- **ILogger<ExceptionHandlingMiddleware> logger:**

- Provides logging capabilities specific to this middleware.

- **Invoke Method:**

```

public async Task Invoke(HttpContext httpContext)
{
    try
    {
        await _next(httpContext);
    }
    catch (Exception ex)
    {
        //Log the exception type and message
        _logger.LogError($"{ex.GetType().ToString()}: {ex.Message}");
    }
}

```

```

if (ex.InnerException is not null)
{
    //Log the inner exception type and message
    _logger.LogError($"{ex.InnerException.GetType().ToString()}:
{ex.InnerException.Message}");
}

httpContext.Response.StatusCode = 500; //Internal Server Error

await httpContext.Response.WriteAsJsonAsync(new { Message = ex.Message, Type =
ex.GetType().ToString() });
}
}

```

- **Try-Catch Block:**

- **await _next(httpContext);:**
 - Calls the next middleware in the pipeline.
- **Catch Block:**
 - Catches any exceptions thrown by the subsequent middlewares or request handlers.
 - **Logging:**
 - Logs the type and message of the exception and any inner exceptions.
 - **Response:**
 - Sets the HTTP response status code to 500 (Internal Server Error).
 - Writes a JSON response containing the exception message and type.

ExceptionHandlerMiddlewareExtensions Class

```
public static class ExceptionHandlingMiddlewareExtensions
{
    public static IApplicationBuilder UseExceptionHandlerMiddleware(this IApplicationBuilder builder)
    {
        return builder.UseMiddleware<ExceptionHandlerMiddleware>();
    }
}
```

- **Extension Method:**
 - Provides a convenient way to add the ExceptionHandlingMiddleware to the HTTP request pipeline.
 - **UseExceptionHandlerMiddleware Method:**
 - **this IApplicationBuilder builder:**
 - Allows extension methods to be called on IApplicationBuilder.
 - **builder.UseMiddleware<ExceptionHandlerMiddleware>():**
 - Adds the ExceptionHandlingMiddleware to the pipeline.

Summary of Key Concepts of ExceptionHandlingMiddleware

1. **Middleware:**
 - Handles HTTP requests and responses, and can be used for cross-cutting concerns like exception handling.
2. **Exception Handling:**
 - Catches and manages exceptions to prevent application crashes and provide meaningful error responses.
3. **Logging:**
 - Captures details of exceptions for debugging and monitoring.
4. **Extension Methods:**
 - Provide a convenient way to configure and add middleware to the request pipeline.

The ExceptionHandlingMiddleware ensures that exceptions are logged and handled gracefully, improving the reliability and maintainability of the application.

AuthController in API Layer

The AuthController class is an ASP.NET Core MVC controller responsible for handling authentication-related API requests, such as user registration and login. It interacts with the IUserService to process these requests and return appropriate responses.

Concepts

1. ASP.NET Core Controllers

- **Controller:**
 - A class that handles HTTP requests and returns responses. It typically interacts with services and repositories to process business logic.
- **ControllerBase:**
 - A base class for controllers that do not require views. It provides methods for creating HTTP responses.
- **ApiController Attribute:**
 - Indicates that the controller responds to web API requests. It also enables some conventions like automatic model validation.
- **Route Attribute:**
 - Specifies the URL pattern for the controller's actions. In this case, [Route("api/[controller]")] maps to api/auth.

2. Dependency Injection

- **Constructor Injection:**
 - The controller's constructor receives an instance of IUserService, which is injected by ASP.NET Core's dependency injection system.

3. HTTP Actions

- **IActionResult:**
 - Represents the result of an action method. It allows returning different types of HTTP responses.
- **HttpPost Attribute:**
 - Specifies that the action method responds to HTTP POST requests.
- **Status Codes:**
 - BadRequest(400): Indicates that the request is invalid.
 - Unauthorized(401): Indicates that authentication is required or invalid.
 - Ok(200): Indicates that the request was successful.

Code Walkthrough

AuthController Class

```
using eCommerce.Core.DTO;
using eCommerce.Core.ServiceContracts;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;

namespace eCommerce.API.Controllers
{
    [Route("api/[controller]")] //api/auth
    [ApiController]
    public class AuthController : ControllerBase
    {
        private readonly IUsersService _userService;

        public AuthController(IUsersService userService)
        {
            _userService = userService;
        }
    }
}
```

```

//Endpoint for user registration use case
[HttpPost("register")] //POST api/auth/register
public async Task<IActionResult> Register(RegisterRequest registerRequest)
{
    //Check for invalid registerRequest
    if (registerRequest == null)
    {
        return BadRequest("Invalid registration data");
    }

    //Call the UsersService to handle registration
    AuthenticationResponse? authenticationResponse = await
    _userService.Register(registerRequest);

    if (authenticationResponse == null || authenticationResponse.Success == false)
    {
        return BadRequest(authenticationResponse);
    }

    return Ok(authenticationResponse);
}

//Endpoint for user login use case
[HttpPost("login")]
public async Task<IActionResult> Login(LoginRequest loginRequest)
{
    //Check for invalid LoginRequest
    if (loginRequest == null)
    {
        return BadRequest("Invalid login data");
    }

```

```

    }

    AuthenticationResponse? authenticationResponse = await _userService.Login(loginRequest);

    if (authenticationResponse == null || authenticationResponse.Success == false)
    {
        return Unauthorized(authenticationResponse);
    }

    return Ok(authenticationResponse);
}
}
}

```

- **Constructor:**

```

public AuthController(IUserService userService)
{
    _userService = userService;
}

```

- **IUserService userService:**

- Injected service for user-related operations, such as registration and login.

- **Register Method:**

```

[HttpPost("register")] //POST api/auth/register
public async Task<IActionResult> Register(RegisterRequest registerRequest)
{
    //Check for invalid registerRequest
    if (registerRequest == null)
    {
        return BadRequest("Invalid registration data");
    }
}

```



```

//Call the UsersService to handle registration

AuthenticationResponse? authenticationResponse = await
_usersService.Register(registerRequest);

if (authenticationResponse == null || authenticationResponse.Success == false)
{
    return BadRequest(authenticationResponse);
}

return Ok(authenticationResponse);
}

```

- **[HttpPost("register")]:**
 - Maps this method to handle POST requests at api/auth/register.
- **Check for null:**
 - Ensures that the registerRequest object is not null before proceeding.
- **Service Call:**
 - Calls IUsersService.Register() to process registration and receive an AuthenticationResponse.
- **Response Handling:**
 - Returns BadRequest if registration fails or the response indicates failure.
 - Returns Ok with the AuthenticationResponse if registration succeeds.

- **Login Method:**

```

[HttpPost("login")]
public async Task<IActionResult> Login(LoginRequest loginRequest)
{
    //Check for invalid LoginRequest
    if (loginRequest == null)
    {
        return BadRequest("Invalid login data");
    }
}

```

```
AuthenticationResponse? authenticationResponse = await  
_userService.Login(loginRequest);
```

```
if (authenticationResponse == null || authenticationResponse.Success == false)  
{  
    return Unauthorized(authenticationResponse);  
}
```

```
return Ok(authenticationResponse);  
}
```

- **[HttpPost("login")]:**
 - Maps this method to handle POST requests at api/auth/login.
- **Check for null:**
 - Ensures that the loginRequest object is not null before proceeding.
- **Service Call:**
 - Calls IUserService.Login() to authenticate the user and receive an AuthenticationResponse.
- **Response Handling:**
 - Returns Unauthorized if login fails or the response indicates failure.
 - Returns Ok with the AuthenticationResponse if login succeeds.

Summary of Key Concepts of AuthController

1. **Controllers:**
 - Handle HTTP requests and return responses, typically interacting with services.
2. **Dependency Injection:**
 - Allows for the injection of services into controllers, promoting loose coupling and testability.
3. **HTTP Actions:**
 - Use attributes like [HttpPost] to map HTTP requests to action methods.
 - Return IActionResult to provide various HTTP responses.

4. **Validation:**

- Check for null or invalid data before processing requests to ensure that only valid data is handled.

The AuthController provides endpoints for user registration and login, utilizing dependency injection and service contracts to manage authentication logic and ensure consistent error handling and responses.

.NET Microservices – Azure DevOps and AKS

Section 3: Products Microservice - Notes

Entities in DataAccessLayer

The Product class represents the entity model for products in the Products microservice. It defines the structure of the Product data that will be stored in the database.

Concepts

1. Entity Framework Core (EF Core) Entity

- **Entity:**
 - Represents a table in the database. Each instance of the entity corresponds to a row in the table.
- **DataAnnotations:**
 - Used for configuring entity properties and validation in EF Core.
- **Primary Key:**
 - Defined by [Key], it uniquely identifies each record in the table.

Code Walkthrough

Product Class

```
using System.ComponentModel.DataAnnotations;
```

```
namespace eCommerce.DataAccessLayer.Entities;
```

```
public class Product
{
    [Key]
    public Guid ProductID { get; set; }

    public string ProductName { get; set; }
    public string Category { get; set; }
```

```
public double? UnitPrice { get; set; }  
public int? QuantityInStock { get; set; }  
}
```

- **ProductID:**
 - **Type:** Guid
 - **Attributes:** [Key]
 - **Description:** Unique identifier for each product. Marked with [Key] to indicate that it is the primary key of the Product table.
- **ProductName:**
 - **Type:** string
 - **Description:** Name of the product.
- **Category:**
 - **Type:** string
 - **Description:** Category under which the product is listed.
- **UnitPrice:**
 - **Type:** double?
 - **Description:** Price of a single unit of the product. It is nullable (double?) because it might not always be specified.
- **QuantityInStock:**
 - **Type:** int?
 - **Description:** Number of units of the product available in stock. It is nullable (int?) because it might not always be specified.

Summary of Key Concepts of "Entities"

1. **Entity Definition:**
 - Product class defines the structure of the Product table in the database, including properties and their types.
2. **Primary Key:**
 - ProductID is marked with [Key] to indicate it is the primary key for the Product table.
3. **Nullable Properties:**
 - UnitPrice and QuantityInStock are nullable to handle cases where these values might not be provided.

The Product class is a straightforward representation of a product in the system, with attributes that define its unique identifier, name, category, price, and stock quantity.

DbContext in DataAccessLayer

The ApplicationDbContext class is a key component of the data access layer in the Products microservice, responsible for interacting with the database using Entity Framework Core (EF Core).

Concepts

1. DbContext

- **DbContext:**
 - Represents a session with the database and provides methods for querying and saving data. It acts as a bridge between the application and the database.
 - It includes DbSet<TEntity> properties for each entity type that the context manages.

2. DbSet<TEntity>

- **DbSet<TEntity>:**
 - Represents a collection of entities of a specific type that can be queried from the database.
 - Provides methods to perform CRUD (Create, Read, Update, Delete) operations.

3. OnModelCreating Method

- **OnModelCreating:**
 - This method is used to configure the model and relationships using Fluent API. It is where you define custom configurations and constraints.

Code Walkthrough

ApplicationDbContext Class

```
using eCommerce.DataAccessLayer.Entities;
```

```
using Microsoft.EntityFrameworkCore;
```

```
namespace eCommerce.DataAccessLayer.Context;
```

```
public class ApplicationDbContext : DbContext
```

```
{
```

```
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options): base(options)
```

```
    {
```

```
    }
```

```
    public DbSet<Product> Products { get; set; }
```

```
    protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```
    {
```

```
        base.OnModelCreating(modelBuilder);
```

```
    }
```

```
}
```

- **Constructor:**

```
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options):
```

```
        base(options)
```

```
    {
```

```
    }
```

- **Purpose:** Initializes a new instance of ApplicationDbContext with the specified options. DbContextOptions<ApplicationDbContext> provides configuration settings for the context, such as the database provider and connection string.

- **DbSet<Product>:**

```
public DbSet<Product> Products { get; set; }
```

- **Purpose:** Represents the collection of Product entities in the context. This property allows querying and saving Product entities to the Products table in the database.

- **OnModelCreating Method:**

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
}
```

- **Purpose:** Configures the model and relationships using Fluent API. By calling `base.OnModelCreating(modelBuilder)`, it ensures that any configurations from the base class are also applied. This method is currently empty but can be used to add custom configurations, such as entity relationships or constraints, in the future.

Summary of Key Concepts of DbContext

1. **DbContext:**

- `ApplicationDbContext` is responsible for managing the connection to the database and providing access to Product entities.

2. **DbSet<TEntity>:**

- `DbSet<Product>` allows querying and manipulating Product entities in the Products table.

3. **OnModelCreating:**

- Used to configure the model and relationships, allowing for custom configurations if needed.

The `ApplicationDbContext` class serves as the main interface for interacting with the database in the Products microservice, providing the necessary methods and properties for managing Product entities.

Repository Contract in DataAccessLayer

The IProductsRepository interface defines the contract for accessing and manipulating Product data in the Products table. It specifies various methods for CRUD operations and querying the database.

Concepts

1. Repository Pattern

- **Repository Pattern:**
 - A design pattern that abstracts the data access logic and provides a simple interface for CRUD operations. It helps in managing data retrieval and storage, promoting separation of concerns and testability.

2. Asynchronous Programming

- **Asynchronous Programming:**
 - Techniques that allow operations to run in the background without blocking the main thread. Methods returning Task or Task<T> are typically used to perform non-blocking I/O operations.

3. Expressions

- **Expression<Func<T, bool>>:**
 - Represents a lambda expression that can be compiled into a query. It allows dynamic filtering of data based on conditions.

Code Walkthrough

IProductsRepository Interface

```
using eCommerce.DataAccessLayer.Entities;
```

```
using System.Linq.Expressions;
```

```
namespace eCommerce.DataAccessLayer.RepositoryContracts;
```

```
/// <summary>
```

```
/// Represents a repository for managing 'products' table
```

```
/// </summary>
```

```
public interface IProductsRepository
```

```

{
    /// <summary>
    /// Retrieves all products asynchronously
    /// </summary>
    /// <returns>Returns all products from the table</returns>
    Task<IEnumerable<Product>> GetProducts();

    /// <summary>
    /// Retrieves all products based on the specified condition asynchronously.
    /// </summary>
    /// <param name="conditionExpression">The condition to filter products</param>
    /// <returns>Returning a collection of matching products</returns>
    Task<IEnumerable<Product?>> GetProductsByCondition(Expression<Func<Product, bool>>
conditionExpression);

    /// <summary>
    /// Retrieves a single product based on the specified condition asynchronously
    /// </summary>
    /// <param name="conditionExpression">The condition to filter the product</param>
    /// <returns>Returns a single product or null if not found</returns>
    Task<Product?> GetProductByCondition(Expression<Func<Product, bool>> conditionExpression);

    /// <summary>
    /// Adds a new product into the products table asynchronously
    /// </summary>
    /// <param name="product">The product to be added</param>
    /// <returns>Returns the added product object or null if unsuccessful</returns>
    Task<Product?> AddProduct(Product product);

    /// <summary>
    /// Updates an existing product asynchronously.

```

```
/// </summary>
/// <param name="product">The product to be updated</param>
/// <returns>Returns the updated product; or null if not found</returns>
Task<Product?> UpdateProduct(Product product);
```

```
/// <summary>
/// Deletes the product asynchronously
/// </summary>
/// <param name="productID">The product ID to be deleted</param>
/// <returns>Returns true if the deletion is successful, false otherwise.</returns>
Task<bool> DeleteProduct(Guid productID);
}
```

- **GetProducts Method:**

```
Task<IEnumerable<Product>> GetProducts();
```

- **Purpose:** Asynchronously retrieves all Product entities from the database. Returns a collection of products.

- **GetProductsByCondition Method:**

```
Task<IEnumerable<Product?>> GetProductsByCondition(Expression<Func<Product, bool>>
conditionExpression);
```

- **Purpose:** Asynchronously retrieves products based on a condition specified by the conditionExpression. Returns a collection of products that match the condition.

- **GetProductByCondition Method:**

```
Task<Product?> GetProductByCondition(Expression<Func<Product, bool>>
conditionExpression);
```

- **Purpose:** Asynchronously retrieves a single product based on the conditionExpression. Returns a single product or null if no product matches the condition.

- **AddProduct Method:**

```
Task<Product?> AddProduct(Product product);
```

- **Purpose:** Asynchronously adds a new Product entity to the database. Returns the added product object or null if the operation was unsuccessful.

- **UpdateProduct Method:**

Task<Product?> UpdateProduct(Product product);

- **Purpose:** Asynchronously updates an existing Product entity in the database. Returns the updated product or null if the product was not found.

- **DeleteProduct Method:**

Task<bool> DeleteProduct(Guid productID);

- **Purpose:** Asynchronously deletes a Product entity based on the provided productID. Returns true if the deletion was successful and false otherwise.

Summary of Key Concepts of Repository Contract

1. **Repository Pattern:**

- Abstracts data access logic, providing a clear interface for CRUD operations.

2. **Asynchronous Programming:**

- Uses Task to perform non-blocking I/O operations, improving performance and responsiveness.

3. **Expressions:**

- Allows dynamic querying of data based on runtime conditions, facilitating flexible and powerful data access.

The IProductsRepository interface outlines the necessary methods for managing Product entities, promoting a clean separation between data access and business logic.

Repository Implementation in DataAccessLayer

The ProductsRepository class implements the IProductsRepository interface, providing concrete methods for accessing and manipulating Product data using Entity Framework Core. It interacts with the database through the ApplicationDbContext context.

Code Walkthrough

ProductsRepository Class

```
using eCommerce.DataAccessLayer.Entities;
using eCommerce.DataAccessLayer.Context;
using eCommerce.DataAccessLayer.RepositoryContracts;
using Microsoft.EntityFrameworkCore;
using System.Linq.Expressions;

namespace eCommerce.DataAccessLayer.Repositories;

public class ProductsRepository : IProductsRepository
{
    private readonly ApplicationDbContext _dbContext;

    public ProductsRepository(ApplicationDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public async Task<Product?> AddProduct(Product product)
    {
        _dbContext.Products.Add(product);
        await _dbContext.SaveChangesAsync();
        return product;
    }

    public async Task<bool> DeleteProduct(Guid productID)
```

```

{
    Product? existingProduct = await _dbContext.Products.FirstOrDefaultAsync(temp =>
temp.ProductID == productID);
    if (existingProduct == null)
    {
        return false;
    }

    _dbContext.Products.Remove(existingProduct);
    int affectedRowCount = await _dbContext.SaveChangesAsync();
    return affectedRowCount > 0;
}

public async Task<Product?> GetProductByCondition(Expression<Func<Product, bool>>
conditionExpression)
{
    return await _dbContext.Products.FirstOrDefaultAsync(conditionExpression);
}

public async Task<IEnumerable<Product>> GetProducts()
{
    return await _dbContext.Products.ToListAsync();
}

public async Task<IEnumerable<Product?>> GetProductsByCondition(Expression<Func<Product,
bool>> conditionExpression)
{
    return await _dbContext.Products.Where(conditionExpression).ToListAsync();
}

public async Task<Product?> UpdateProduct(Product product)
{

```

```

    Product? existingProduct = await _dbContext.Products.FirstOrDefaultAsync(temp =>
temp.ProductID == product.ProductID);

    if (existingProduct == null)
    {
        return null;
    }

    existingProduct.ProductName = product.ProductName;
    existingProduct.UnitPrice = product.UnitPrice;
    existingProduct.QuantityInStock = product.QuantityInStock;
    existingProduct.Category = product.Category;

    await _dbContext.SaveChangesAsync();

    return existingProduct;
}
}

```

Key Methods

1. AddProduct

```

public async Task<Product?> AddProduct(Product product)
{
    _dbContext.Products.Add(product);
    await _dbContext.SaveChangesAsync();
    return product;
}

```

- **Purpose:** Adds a new product to the Products table and saves the changes to the database. Returns the added product.

2. DeleteProduct

```

public async Task<bool> DeleteProduct(Guid productID)
{

```

```

    Product? existingProduct = await _dbContext.Products.FirstOrDefaultAsync(temp =>
temp.ProductID == productID);

    if (existingProduct == null)
    {
        return false;
    }

```

```

_dbContext.Products.Remove(existingProduct);

int affectedRowCount = await _dbContext.SaveChangesAsync();

return affectedRowCount > 0;
}

```

- **Purpose:** Deletes a product with the specified productID. Returns true if the deletion was successful, otherwise false.

3. **GetProductByCondition**

```

public async Task<Product?> GetProductByCondition(Expression<Func<Product, bool>>
conditionExpression)
{
    return await _dbContext.Products.FirstOrDefaultAsync(conditionExpression);
}

```

- **Purpose:** Retrieves a single product that matches the specified condition. Returns the product or null if not found.

4. **GetProducts**

```

public async Task<IEnumerable<Product>> GetProducts()
{
    return await _dbContext.Products.ToListAsync();
}

```

- **Purpose:** Retrieves all products from the Products table. Returns a collection of all products.

5. **GetProductsByCondition**

```
public async Task<IEnumerable<Product?>>
GetProductsByCondition(Expression<Func<Product, bool>> conditionExpression)
{
    return await _dbContext.Products.Where(conditionExpression).ToListAsync();
}
```

- **Purpose:** Retrieves products that match the specified condition. Returns a collection of matching products.

6. **UpdateProduct**

```
public async Task<Product?> UpdateProduct(Product product)
{
    Product? existingProduct = await _dbContext.Products.FirstOrDefaultAsync(temp =>
temp.ProductID == product.ProductID);

    if (existingProduct == null)
    {
        return null;
    }

    existingProduct.ProductName = product.ProductName;
    existingProduct.UnitPrice = product.UnitPrice;
    existingProduct.QuantityInStock = product.QuantityInStock;
    existingProduct.Category = product.Category;

    await _dbContext.SaveChangesAsync();

    return existingProduct;
}
```

- **Purpose:** Updates an existing product. Returns the updated product or null if the product was not found.

Summary of Repository Implementation

- **CRUD Operations:** ProductsRepository implements methods for creating, reading, updating, and deleting Product entities using Entity Framework Core.
- **Asynchronous Programming:** Methods use Task to perform non-blocking database operations, ensuring responsiveness and scalability.
- **Entity Framework Core:** Utilizes DbContext and LINQ to query and manipulate data, leveraging the capabilities of EF Core for ORM (Object-Relational Mapping).

Dependency Injection in DataAccessLayer

The DependencyInjection class in the DataAccessLayer is responsible for registering data access services with the IoC (Inversion of Control) container. This includes configuring the database context and repository services.

Here's a detailed look at the code and its purpose:

Code Walkthrough

```
using eCommerce.DataAccessLayer.Context;
using eCommerce.DataAccessLayer.Repositories;
using eCommerce.DataAccessLayer.RepositoryContracts;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

namespace eCommerce.ProductsService.DataAccessLayer;

public static class DependencyInjection
{
    public static IServiceCollection AddDataAccessLayer(this IServiceCollection services, IConfiguration configuration)
    {
        // TO DO: Add Data Access Layer services into the IoC container
```

```
// Configure DbContext to use MySQL with the connection string from configuration
services.AddDbContext<ApplicationDbContext>(options => {
    options.UseMySQL(configuration.GetConnectionString("DefaultConnection"));
});

// Register repository with a scoped lifetime
services.AddScoped<IProductsRepository, ProductsRepository>();

return services;
}
}
```

Key Components

1. DbContext Configuration

```
services.AddDbContext<ApplicationDbContext>(options => {
    options.UseMySQL(configuration.GetConnectionString("DefaultConnection"));
});
```

- **Purpose:** Configures ApplicationDbContext to use MySQL as the database provider. It retrieves the connection string named "DefaultConnection" from the application's configuration settings.
- **Method:** AddDbContext<TContext> registers the ApplicationDbContext with the dependency injection container and configures it to use MySQL. Note that UseMySQL is used here, but if you are using PostgreSQL, you would typically use UseNpgsql.

2. Repository Registration

```
services.AddScoped<IProductsRepository, ProductsRepository>();
```

- **Purpose:** Registers the ProductsRepository class as the implementation of the IProductsRepository interface.
- **Lifetime:** Scoped lifetime ensures that a single instance of the repository is used within a single request scope, which is suitable for managing database transactions.

Usage

To use the `AddDataAccessLayer` method, you would typically call it from the `Startup.cs` or `Program.cs` file of your application:

```
public class Startup
{
    public void ConfigureServices(IServiceCollection services)
    {
        // Other service registrations

        services.AddDataAccessLayer(Configuration);
    }
}
```

Summary of Dependency Injection `DataAccessLayer`

- **DbContext Configuration:** Registers `ApplicationDbContext` with MySQL configuration, allowing it to interact with the database.
- **Repository Registration:** Adds `ProductsRepository` as the implementation of `IProductsRepository`, ensuring it is available for dependency injection.
- **Scoped Lifetime:** Ensures that the repository and `DbContext` are created and disposed of within the same request scope, promoting efficient resource management.

DTOs in the Business Logic Layer

The Data Transfer Objects (DTOs) in the Business Logic Layer are used to encapsulate data for various operations related to Product. These DTOs facilitate the transfer of data between different layers of the application, such as between the API layer and the business logic layer.

Here's a breakdown of each DTO:

1. CategoryOptions Enum

```
namespace eCommerce.BusinessLogicLayer.DTO;
```

```
public enum CategoryOptions
```

```
{
```

```
    Electronics, HomeAppliances, Furniture, Accessories
```

```
}
```

- **Purpose:** Defines the categories that products can belong to. This enum provides a set of predefined values that help in categorizing products.

2. ProductAddRequest Record

```
namespace eCommerce.BusinessLogicLayer.DTO;
```

```
public record ProductAddRequest(string ProductName, CategoryOptions Category, double? UnitPrice, int? QuantityInStock)
```

```
{
```

```
    public ProductAddRequest() : this(default, default, default, default)
```

```
{
```

```
}
```

```
}
```

- **Purpose:** Represents the data required to add a new product.
- **Properties:**
 - ProductName: The name of the product.
 - Category: The category of the product, using the CategoryOptions enum.
 - UnitPrice: The price of the product.
 - QuantityInStock: The quantity available in stock.

- **Constructor:** Includes a parameterless constructor that sets all properties to their default values, useful for scenarios where default initialization is required.

3. ProductResponse Record

```
namespace eCommerce.BusinessLogicLayer.DTO;
```

```
public record ProductResponse(Guid ProductID, string ProductName, CategoryOptions  
Category, double? UnitPrice, int? QuantityInStock)
```

```
{  
  
    public ProductResponse() : this(default, default, default, default, default)  
  
    {  
  
    }  
  
}
```

- **Purpose:** Represents the data returned when querying for a product. This DTO is used to return the details of a product.
- **Properties:**
 - ProductID: The unique identifier of the product.
 - ProductName: The name of the product.
 - Category: The category of the product.
 - UnitPrice: The price of the product.
 - QuantityInStock: The quantity available in stock.
- **Constructor:** Includes a parameterless constructor for default initialization.

4. ProductUpdateRequest Record

```
namespace eCommerce.BusinessLogicLayer.DTO;
```

```
public record ProductUpdateRequest(Guid ProductID, string ProductName, CategoryOptions  
Category, double? UnitPrice, int? QuantityInStock)
```

```
{  
  
    public ProductUpdateRequest() : this(default, default, default, default, default)  
  
    {  
  
    }  
  
}
```

}

- **Purpose:** Represents the data required to update an existing product.
- **Properties:**
 - ProductID: The unique identifier of the product being updated.
 - ProductName: The name of the product.
 - Category: The category of the product.
 - UnitPrice: The price of the product.
 - QuantityInStock: The quantity available in stock.
- **Constructor:** Includes a parameterless constructor for default initialization.

Summary of DTO

- **Enums and Records:** Enums define fixed sets of values (e.g., CategoryOptions), while records are used for encapsulating data with immutable properties (e.g., ProductAddRequest, ProductResponse, ProductUpdateRequest).
- **Default Constructors:** Each DTO includes a parameterless constructor for cases where default values are needed.

These DTOs facilitate the interaction between different layers by providing structured and typed data for product-related operations.

Service Contracts in the Business Logic Layer

The service contracts define the operations that the business logic layer will expose for managing products. These operations interact with the data access layer and convert data into the format required by the API layer or other consumers.

Here's a breakdown of each service contract:

1. IProductsService Interface

```
using eCommerce.DataAccessLayer.Entities;
```

```
using eCommerce.BusinessLogicLayer.DTO;
```

```
using System.Linq.Expressions;
```

```
namespace eCommerce.BusinessLogicLayer.ServiceContracts;
```

```
public interface IProductsService
```

```
{
```

```
    /// <summary>
```

```
    /// Retrieves the list of products from the products repository.
```

```
    /// </summary>
```

```
    /// <returns>Returns list of ProductResponse objects.</returns>
```

```
    Task<List<ProductResponse?>> GetProducts();
```

```
    /// <summary>
```

```
    /// Retrieves list of products matching the given condition.
```

```
    /// </summary>
```

```
    /// <param name="conditionExpression">Expression that represents the condition to  
    check.</param>
```

```
    /// <returns>Returns matching products.</returns>
```

```
    Task<List<ProductResponse?>> GetProductsByCondition(Expression<Func<Product, bool>>  
    conditionExpression);
```

```
    /// <summary>
```



```

    /// Returns a single product that matches the given condition.
    /// </summary>

    /// <param name="conditionExpression">Expression that represents the condition to
    check.</param>

    /// <returns>Returns matching product or null.</returns>

    Task<ProductResponse?> GetProductByCondition(Expression<Func<Product, bool>>
    conditionExpression);

    /// <summary>
    /// Adds (inserts) a product into the table using the products repository.
    /// </summary>

    /// <param name="productAddRequest">Product to insert.</param>

    /// <returns>Product after inserting or null if unsuccessful.</returns>

    Task<ProductResponse?> AddProduct(ProductAddRequest productAddRequest);

    /// <summary>
    /// Updates the existing product based on the ProductID.
    /// </summary>

    /// <param name="productUpdateRequest">Product data to update.</param>

    /// <returns>Returns product object after successful updation; otherwise null.</returns>

    Task<ProductResponse?> UpdateProduct(ProductUpdateRequest productUpdateRequest);

    /// <summary>
    /// Deletes an existing product based on the given product ID.
    /// </summary>

    /// <param name="productID">ProductID to search and delete.</param>

    /// <returns>Returns true if the deletion is successful; otherwise false.</returns>

    Task<bool> DeleteProduct(Guid productID);
}

```

Summary of Service Contract Methods:

1. **GetProducts:**
 - **Purpose:** Retrieves all products from the repository.
 - **Returns:** A list of ProductResponse objects.
2. **GetProductsByCondition:**
 - **Purpose:** Retrieves products that match a specified condition.
 - **Parameters:** conditionExpression - An expression representing the filter condition.
 - **Returns:** A list of ProductResponse objects matching the condition.
3. **GetProductByCondition:**
 - **Purpose:** Retrieves a single product that matches a specified condition.
 - **Parameters:** conditionExpression - An expression representing the filter condition.
 - **Returns:** A single ProductResponse object or null if no product matches.
4. **AddProduct:**
 - **Purpose:** Adds a new product to the repository.
 - **Parameters:** productAddRequest - Data required to create a new product.
 - **Returns:** The added ProductResponse object or null if the addition was unsuccessful.
5. **UpdateProduct:**
 - **Purpose:** Updates an existing product in the repository.
 - **Parameters:** productUpdateRequest - Data required to update an existing product.
 - **Returns:** The updated ProductResponse object or null if the update was unsuccessful.
6. **DeleteProduct:**
 - **Purpose:** Deletes a product from the repository based on its ID.
 - **Parameters:** productID - The ID of the product to be deleted.
 - **Returns:** True if the deletion was successful, otherwise false.

Usage

- **Data Access:** These service methods interact with the data access layer (IProductsRepository) to perform CRUD operations.
- **Business Logic:** The business logic layer applies additional logic or transformations on the data as needed.

- **API Layer:** The API layer consumes these services to expose functionality through HTTP endpoints.

By defining these contracts, the business logic layer provides a clear API for managing products, ensuring that the data access and presentation layers remain decoupled.

Mappers in the Business Logic Layer

The mappers in the Business Logic Layer use AutoMapper to map between Data Transfer Objects (DTOs) and entities. This approach ensures that the application's different layers remain decoupled and that data is transformed correctly as it flows through the system.

1. ProductAddRequest to Product Mapping Profile

This profile handles the mapping from ProductAddRequest (DTO) to Product (entity) for when new products are added.

```
using AutoMapper;
```

```
using eCommerce.DataAccessLayer.Entities;
```

```
using eCommerce.BusinessLogicLayer.DTO;
```

```
namespace eCommerce.BusinessLogicLayer.Mappers;
```

```
public class ProductAddRequestToProductMappingProfile : Profile
```

```
{
```

```
    public ProductAddRequestToProductMappingProfile()
```

```
    {
```

```
        CreateMap<ProductAddRequest, Product>()
```

```
            .ForMember(dest => dest.ProductName, opt => opt.MapFrom(src => src.ProductName))
```

```
            .ForMember(dest => dest.Category, opt => opt.MapFrom(src => src.Category))
```

```
            .ForMember(dest => dest.UnitPrice, opt => opt.MapFrom(src => src.UnitPrice))
```

```
            .ForMember(dest => dest.QuantityInStock, opt => opt.MapFrom(src => src.QuantityInStock))
```

```
            .ForMember(dest => dest.ProductID, opt => opt.Ignore()); // ProductID is ignored during addition
```

```
    }
```

```
}
```

- **Purpose:** Converts the ProductAddRequest DTO to a Product entity when a new product is being created.
- **Key Points:**
 - Ignores ProductID as it will be generated during the insert operation.

2. Product to ProductResponse Mapping Profile

This profile maps the Product entity to the ProductResponse DTO for returning product details to the client.

```
using AutoMapper;
```

```
using eCommerce.DataAccessLayer.Entities;
```

```
using eCommerce.BusinessLogicLayer.DTO;
```

```
namespace eCommerce.BusinessLogicLayer.Mappers;
```

```
public class ProductToProductResponseMappingProfile : Profile
{
    public ProductToProductResponseMappingProfile()
    {
        CreateMap<Product, ProductResponse>()
            .ForMember(dest => dest.ProductName, opt => opt.MapFrom(src => src.ProductName))
            .ForMember(dest => dest.Category, opt => opt.MapFrom(src => src.Category))
            .ForMember(dest => dest.UnitPrice, opt => opt.MapFrom(src => src.UnitPrice))
            .ForMember(dest => dest.QuantityInStock, opt => opt.MapFrom(src => src.QuantityInStock))
            .ForMember(dest => dest.ProductID, opt => opt.MapFrom(src => src.ProductID)); // Maps the
            ProductID
    }
}
```

- **Purpose:** Converts the Product entity into a ProductResponse DTO for output to the client.
- **Key Points:**
 - All fields, including ProductID, are directly mapped.

3. ProductUpdateRequest to Product Mapping Profile

This profile maps from the ProductUpdateRequest DTO to the Product entity for updating existing products.

```
using AutoMapper;
```

```
using eCommerce.DataAccessLayer.Entities;
```

```
using eCommerce.BusinessLogicLayer.DTO;
```

```
namespace eCommerce.BusinessLogicLayer.Mappers;
```

```
public class ProductUpdateRequestToProductMappingProfile : Profile
```

```
{
```

```
    public ProductUpdateRequestToProductMappingProfile()
```

```
    {
```

```
        CreateMap<ProductUpdateRequest, Product>()
```

```
            .ForMember(dest => dest.ProductName, opt => opt.MapFrom(src => src.ProductName))
```

```
            .ForMember(dest => dest.Category, opt => opt.MapFrom(src => src.Category))
```

```
            .ForMember(dest => dest.UnitPrice, opt => opt.MapFrom(src => src.UnitPrice))
```

```
            .ForMember(dest => dest.QuantityInStock, opt => opt.MapFrom(src => src.QuantityInStock))
```

```
            .ForMember(dest => dest.ProductID, opt => opt.MapFrom(src => src.ProductID)); // Maps  
ProductID for the update
```

```
    }
```

```
}
```

- **Purpose:** Converts the ProductUpdateRequest DTO to a Product entity for updates.
- **Key Points:**
 - All fields, including ProductID, are directly mapped, which is important for identifying the product to update.

Summary of Usage of Mappers in BusinessLogicLayer

1. ProductAddRequestToProductMappingProfile:

- Maps from ProductAddRequest DTO to the Product entity during product addition.

2. ProductToProductResponseMappingProfile:

- Maps from Product entity to ProductResponse DTO when returning product details to clients.

3. ProductUpdateRequestToProductMappingProfile:

- Maps from ProductUpdateRequest DTO to the Product entity when updating product details.

These mappings allow the service layer to handle complex transformations in a clean and efficient way, abstracting the data transformations between different layers of the application.

Validators in the Business Logic Layer

The validators ensure that the data passed to the system through ProductAddRequest and ProductUpdateRequest is valid and adheres to the necessary business rules before further processing occurs. FluentValidation is used for this purpose, providing a fluent interface to define validation rules.

1. ProductAddRequestValidator

This validator is used for validating ProductAddRequest DTO, ensuring the product creation request meets specific criteria.

using eCommerce.BusinessLogicLayer.DTO;

using FluentValidation;

namespace eCommerce.BusinessLogicLayer.Validators;

```
public class ProductAddRequestValidator : AbstractValidator<ProductAddRequest>
{
    public ProductAddRequestValidator()
    {
```

```

//ProductName
RuleFor(temp => temp.ProductName)
    .NotEmpty().WithMessage("Product Name can't be blank");

//Category
RuleFor(temp => temp.Category)
    .IsInEnum().WithMessage("Category can't be blank");

//UnitPrice
RuleFor(temp => temp.UnitPrice)
    .InclusiveBetween(0, double.MaxValue).WithMessage($"Unit Price should be between 0 and {double.MaxValue}");

//QuantityInStock
RuleFor(temp => temp.QuantityInStock)
    .InclusiveBetween(0, int.MaxValue).WithMessage($"Quantity in Stock should be between 0 and {int.MaxValue}");
}
}

```

- **Key Validations:**

- **ProductName:** Cannot be empty.
- **Category:** Must be one of the predefined enum values.
- **UnitPrice:** Must be a positive value between 0 and double.MaxValue.
- **QuantityInStock:** Must be a positive integer between 0 and int.MaxValue.

2. ProductUpdateRequestValidator

This validator is used for validating the ProductUpdateRequest DTO, ensuring the product update request is correct before updating an existing product.

```
using eCommerce.BusinessLogicLayer.DTO;
```

```
using FluentValidation;
```

```
namespace eCommerce.BusinessLogicLayer.Validators;
```

```
public class ProductUpdateRequestValidator : AbstractValidator<ProductUpdateRequest>
{
    public ProductUpdateRequestValidator()
    {
        //ProductID
        RuleFor(temp => temp.ProductID)
            .NotEmpty().WithMessage("Product ID can't be blank");

        //ProductName
        RuleFor(temp => temp.ProductName)
            .NotEmpty().WithMessage("Product Name can't be blank");

        //Category
        RuleFor(temp => temp.Category)
            .IsInEnum().WithMessage("Category can't be blank");

        //UnitPrice
        RuleFor(temp => temp.UnitPrice)
            .InclusiveBetween(0, double.MaxValue).WithMessage($"Unit Price should be between 0 and {double.MaxValue}");

        //QuantityInStock
        RuleFor(temp => temp.QuantityInStock)
```



```
.InclusiveBetween(0, int.MaxValue).WithMessage($"Quantity in Stock should be between 0 and {int.MaxValue}");  
}  
}
```

- **Key Validations:**

- **ProductID:** Must not be empty to identify the product to update.
- **ProductName:** Cannot be empty.
- **Category:** Must be a valid enum value.
- **UnitPrice:** Must be between 0 and double.MaxValue.
- **QuantityInStock:** Must be a positive integer between 0 and int.MaxValue.

Summary of Validators in the Business Logic Layer

1. **ProductAddRequestValidator:** Validates that the required fields for adding a product, such as product name, category, unit price, and quantity in stock, are provided and valid.
2. **ProductUpdateRequestValidator:** Ensures that when updating a product, the product ID, along with other necessary fields like name, category, unit price, and quantity, are correct and valid.

These validators enforce the necessary business rules before data is passed further into the service layer or repository layer.

Services in BusinessLogicLayer

The ProductsService class provides the business logic for interacting with product-related operations. It sits between the data access layer (repositories) and external requests, applying validation, mapping, and additional business rules.

Key Components:

- **Validators:** Uses FluentValidation for validating incoming requests (e.g., ProductAddRequest and ProductUpdateRequest).
- **Mapper:** AutoMapper is used to map between the request DTOs (ProductAddRequest, ProductUpdateRequest) and entity models (Product) and also between entities and response DTOs (ProductResponse).
- **Repository:** Interacts with the repository layer to handle data access operations.

Constructor

The constructor receives dependencies via Dependency Injection:

```
public ProductsService(
    IValidator<ProductAddRequest> productAddRequestValidator,
    IValidator<ProductUpdateRequest> productUpdateRequestValidator,
    IMapper mapper,
    IProductsRepository productsRepository)
{
    _productAddRequestValidator = productAddRequestValidator;
    _productUpdateRequestValidator = productUpdateRequestValidator;
    _mapper = mapper;
    _productsRepository = productsRepository;
}
```

Methods Overview

1. AddProduct

Adds a new product to the repository after validation.

```
public async Task<ProductResponse?> AddProduct(ProductAddRequest productAddRequest)
{
    if (productAddRequest == null) throw new
    ArgumentNullException(nameof(productAddRequest));

    ValidationResult validationResult = await
    _productAddRequestValidator.ValidateAsync(productAddRequest);

    if (!validationResult.IsValid)
    {
        string errors = string.Join(", ", validationResult.Errors.Select(temp =>
        temp.ErrorMessage));

        throw new ArgumentException(errors);
    }

    Product productInput = _mapper.Map<Product>(productAddRequest);
    Product? addedProduct = await _productsRepository.AddProduct(productInput);

    if (addedProduct == null) return null;

    ProductResponse addedProductResponse =
    _mapper.Map<ProductResponse>(addedProduct);

    return addedProductResponse;
}
```

- **Validation:** Ensures the incoming ProductAddRequest is valid.
- **Mapping:** Maps ProductAddRequest to Product entity using AutoMapper.
- **Repository Interaction:** Calls the AddProduct method in the repository.
- **Response:** Maps the added Product entity to a ProductResponse.

2. DeleteProduct

Deletes a product by its ID.

```
public async Task<bool> DeleteProduct(Guid productID)
{
    Product? existingProduct = await _productsRepository.GetProductByCondition(temp =>
temp.ProductID == productID);

    if (existingProduct == null) return false;

    bool isDeleted = await _productsRepository.DeleteProduct(productID);

    return isDeleted;
}
```

- **Product Existence:** Checks if the product exists before attempting deletion.
- **Repository Interaction:** Deletes the product by ID if found.

3. GetProductByCondition

Retrieves a product based on a condition (e.g., by ID or other criteria).

```
public async Task<ProductResponse?> GetProductByCondition(Expression<Func<Product,
bool>> conditionExpression)
{
    Product? product = await
_productsRepository.GetProductByCondition(conditionExpression);

    if (product == null) return null;

    ProductResponse productResponse = _mapper.Map<ProductResponse>(product);

    return productResponse;
}
```

- **Repository Interaction:** Fetches a product based on a condition.
- **Mapping:** Maps the fetched Product to a ProductResponse.

4. GetProducts

Retrieves a list of all products.

```
public async Task<List<ProductResponse?>> GetProducts()
```

```

{
    IEnumerable<Product?> products = await _productsRepository.GetProducts();

    IEnumerable<ProductResponse?> productResponses =
    _mapper.Map<IEnumerable<ProductResponse>>(products);

    return productResponses.ToList();
}

```

- **Repository Interaction:** Retrieves all products from the repository.
- **Mapping:** Maps the collection of Product entities to a list of ProductResponse objects.

5. GetProductsByCondition

Retrieves a list of products based on a condition (e.g., category or stock availability).

```

public async Task<List<ProductResponse?>>
GetProductsByCondition(Expression<Func<Product, bool>> conditionExpression)
{
    IEnumerable<Product?> products = await
    _productsRepository.GetProductsByCondition(conditionExpression);

    IEnumerable<ProductResponse?> productResponses =
    _mapper.Map<IEnumerable<ProductResponse>>(products);

    return productResponses.ToList();
}

```

- **Repository Interaction:** Fetches products matching the specified condition.
- **Mapping:** Maps the collection of Product entities to ProductResponse.

6. UpdateProduct

Updates an existing product based on the provided request.

```

public async Task<ProductResponse?> UpdateProduct(ProductUpdateRequest
productUpdateRequest)
{
    Product? existingProduct = await _productsRepository.GetProductByCondition(temp =>
temp.ProductID == productUpdateRequest.ProductID);

    if (existingProduct == null) throw new ArgumentException("Invalid Product ID");
}

```

```

        ValidationResult validationResult = await
        _productUpdateRequestValidator.ValidateAsync(productUpdateRequest);

        if (!validationResult.IsValid)
        {
            string errors = string.Join(" ", validationResult.Errors.Select(temp =>
            temp.ErrorMessage));

            throw new ArgumentException(errors);
        }

        Product product = _mapper.Map<Product>(productUpdateRequest);

        Product? updatedProduct = await _productsRepository.UpdateProduct(product);

        ProductResponse? updatedProductResponse =
        _mapper.Map<ProductResponse>(updatedProduct);

        return updatedProductResponse;
    }

```

- **Validation:** Validates the update request.
- **Repository Interaction:** Updates the product if valid.
- **Mapping:** Maps both request and response between DTO and entity.

Summary of Services in BusinessLogicLayer

The ProductsService ensures that data is properly validated and mapped before interacting with the repository. It provides all essential product-related business operations such as adding, updating, retrieving, and deleting products, leveraging validation through FluentValidation and mapping through AutoMapper.

DependencyInjection in BusinessLogicLayer

This DependencyInjection class configures the services required for the Business Logic Layer (BLL) and registers them in the application's IoC container.

Key Components

- **AutoMapper**: Registers the mapping profiles used for converting between DTOs and entities.
- **FluentValidation**: Adds validators for handling validation of incoming requests.
- **ProductsService**: Registers the ProductsService to be used whenever the IProductsService contract is requested.

Implementation

```
using eCommerce.BusinessLogicLayer.Mappers;
using eCommerce.BusinessLogicLayer.ServiceContracts;
using Microsoft.Extensions.DependencyInjection;
using eCommerce.BusinessLogicLayer.Validators;
using FluentValidation;

namespace eCommerce.ProductsService.BusinessLogicLayer
{
    public static class DependencyInjection
    {
        {
            public static IServiceCollection AddBusinessLogicLayer(this IServiceCollection services)
            {
                // Register AutoMapper with all mapping profiles from the assembly

                services.AddAutoMapper(typeof(ProductAddRequestToProductMappingProfile).Assembly);

                // Register FluentValidation validators from the same assembly as
                ProductAddRequestValidator

                services.AddValidatorsFromAssemblyContaining<ProductAddRequestValidator>();

                // Register ProductsService with the IProductsService interface
```

```

        services.AddScoped<IProductsService,
eCommerce.BusinessLogicLayer.Services.ProductsService>();

        return services;
    }
}
}

```

Service Registrations

1. AutoMapper:

- The AddAutoMapper() method registers the mapping profiles in the specified assembly, enabling the use of AutoMapper for object transformation.
- All profiles in the assembly containing ProductAddRequestToProductMappingProfile are scanned and added to the IoC container.

2. FluentValidation:

- The AddValidatorsFromAssemblyContaining<ProductAddRequestValidator>() method registers all validators in the assembly where ProductAddRequestValidator exists.
- This allows any request DTO to be validated using the registered validators.

3. ProductsService:

- Registers the ProductsService class as the implementation of the IProductsService interface.
- This ensures that when IProductsService is requested, an instance of ProductsService is injected.

Summary of DependencyInjection in BusinessLogicLayer

This DependencyInjection class configures the necessary components of the Business Logic Layer, ensuring that:

- Mapping profiles and validators are registered and available.
- The business service (ProductsService) is accessible via dependency injection through its interface (IProductsService).

This setup allows for a clean separation of concerns and scalable service registration in the ASP.NET Core application.

appsettings.json in API Layer

The appsettings.json file contains configuration settings for the application, including logging, allowed hosts, and connection strings.

Configuration Structure

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost; Port=3306; Database=ecommerceproductsdatabase; User ID=root; Password=admin"
  }
}
```

Explanation of Key Sections

1. Logging:

- Defines the log levels for the application.
- Default is set to Information to log general information messages.
- Microsoft.AspNetCore is set to Warning, which logs warnings and above for the ASP.NET Core infrastructure.

2. AllowedHosts:

- Set to "*", allowing the application to accept requests from any host.
- In production, you can specify allowed host names for security.

3. ConnectionStrings:

- Defines the DefaultConnection string for connecting to the MySQL database.
- The connection string includes:
 - Server=localhost: The database is hosted locally.
 - Port=3306: The port MySQL uses.
 - Database=ecommerceproductsdatabase: The name of the database.
 - User ID=root: The root username for authentication.
 - Password=admin: The password for the database user.

Usage in Code

This file is referenced in the Program.cs or Startup.cs to configure logging and database connection for the API.

Example of retrieving the connection string in code:

```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");  
builder.Services.AddDbContext<ApplicationDbContext>(options =>  
{  
    options.UseMySQL(connectionString);  
});
```

Summary of appsettings.json in API Layer

This appsettings.json file handles key configuration settings for the API, including logging, host settings, and database connection strings, making it easier to manage and modify configuration without hardcoding values.

ExceptionHandlerMiddleware in the API Layer

This ExceptionHandlerMiddleware is a custom middleware for handling exceptions in the API layer of your application. It catches any unhandled exceptions during the request pipeline, logs the error details, and returns a JSON response with the error message.

Code Explanation

Middleware Class (ExceptionHandlerMiddleware)

```
namespace eCommerce.ProductsMicroService.API.Middleware;

public class ExceptionHandlerMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionHandlerMiddleware> _logger;

    public ExceptionHandlerMiddleware(RequestDelegate next,
    ILogger<ExceptionHandlerMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task Invoke(HttpContext httpContext)
    {
        try
        {
            await _next(httpContext);
        }
        catch (Exception ex)
        {
            if (ex.InnerException != null)
            {
```

```

        _logger.LogError("{ExceptionType} {ExceptionMessage}",
ex.InnerException.GetType().ToString(), ex.InnerException.Message);
    }
    else
    {
        _logger.LogError("{ExceptionType} {ExceptionMessage}", ex.GetType().ToString(),
ex.Message);
    }

    // Return status code 500 for server error
    httpContext.Response.StatusCode = 500;

    // Write a JSON response with exception details
    await httpContext.Response.WriteAsJsonAsync(new {
        Message = ex.Message,
        Type = ex.GetType().ToString()
    });
}
}
}

```

Middleware Extension (ExceptionHandlerMiddlewareExtensions)

```

public static class ExceptionHandlingMiddlewareExtensions
{
    public static IApplicationBuilder UseExceptionHandlerMiddleware(this IApplicationBuilder
builder)
    {
        return builder.UseMiddleware<ExceptionHandlerMiddleware>();
    }
}

```

Key Features

1. Logging:

- Catches all unhandled exceptions and logs their details using ILogger<ExceptionHandlerMiddleware>.
- If the exception has an InnerException, it logs that as well.

2. Error Response:

- The middleware sets the HTTP status code to 500 (Internal Server Error).
- A JSON object is returned in the response, containing:
 - The exception message.
 - The exception type.

3. Exception Handling:

- Catches and processes exceptions that occur in the request pipeline, ensuring no unhandled exceptions propagate further.

Usage in Program.cs

To activate the middleware, it must be added to the request pipeline in Program.cs or Startup.cs:

```
var builder = WebApplication.CreateBuilder(args);
```

```
var app = builder.Build();
```

```
// Add the exception handling middleware
```

```
app.UseExceptionHandlerMiddleware();
```

```
app.Run();
```

Advantages of ExceptionHandlingMiddleware

- **Centralized Error Handling:** All exceptions are handled in one place, ensuring consistent behavior for unhandled exceptions.
- **Logging:** Detailed logging of exceptions helps in debugging and monitoring.
- **JSON Error Response:** Provides structured JSON responses that are easier for API clients to consume and process.

This middleware ensures graceful error handling, improved logging, and better error reporting in the API layer.

API Endpoints in the API Layer

The ProductAPIEndpoints class defines REST API endpoints for managing products in the eCommerce platform. These endpoints interact with the business logic layer to handle common product-related operations such as retrieving, adding, updating, and deleting products.

Endpoints Explanation

1. GET /api/products

Fetches all products.

```
app.MapGet("/api/products", async (IProductsService productsService) =>
{
    List<ProductResponse?> products = await productsService.GetProducts();
    return Results.Ok(products);
});
```

- **Description:** Retrieves all available products.
- **Returns:** A list of ProductResponse objects in the response.

2. GET /api/products/search/product-id/{ProductID}

Fetches a product by its unique ProductID.

```
app.MapGet("/api/products/search/product-id/{ProductID:guid}", async (IProductsService
productsService, Guid ProductID) =>
{
    ProductResponse? product = await productsService.GetProductByCondition(temp =>
temp.ProductID == ProductID);
    return Results.Ok(product);
});
```

- **Description:** Retrieves a product that matches the given ProductID.
- **Returns:** A single ProductResponse object or null if no matching product is found.

3. GET /api/products/search/{SearchString}

Searches for products by name or category.

```
app.MapGet("/api/products/search/{SearchString}", async (IProductsService productsService, string SearchString) =>
```

```
{  
    List<ProductResponse?> productsByProductName = await  
    productsService.GetProductsByCondition(temp => temp.ProductName != null &&  
    temp.ProductName.Contains(SearchString, StringComparison.OrdinalIgnoreCase));
```

```
  
    List<ProductResponse?> productsByCategory = await  
    productsService.GetProductsByCondition(temp => temp.Category != null &&  
    temp.Category.Contains(SearchString, StringComparison.OrdinalIgnoreCase));
```

```
  
    var products = productsByProductName.Union(productsByCategory);
```

```
  
    return Results.Ok(products);
```

```
});
```

- **Description:** Searches for products by matching the SearchString with product names or categories.
- **Returns:** A list of matching products.

4. POST /api/products

Adds a new product.

```
app.MapPost("/api/products", async (IProductsService productsService,  
IValidator<ProductAddRequest> productAddRequestValidator, ProductAddRequest  
productAddRequest) =>
```

```
{
```

```
    // Validation using FluentValidation
```

```
    ValidationResult validationResult = await  
    productAddRequestValidator.ValidateAsync(productAddRequest);
```

```
  
    if (!validationResult.IsValid)
```

```
{
```

```
    Dictionary<string, string[]> errors = validationResult.Errors
```

```

        .GroupBy(temp => temp.PropertyName)
        .ToDictionary(grp => grp.Key, grp => grp.Select(err => err.ErrorMessage).ToArray());
return Results.ValidationProblem(errors);
}

var addedProductResponse = await productService.AddProduct(productAddRequest);
if (addedProductResponse != null)
    return Results.Created($" /api/products/search/product-
id/{addedProductResponse.ProductID}", addedProductResponse);
else
    return Results.Problem("Error in adding product");
});

```

- **Description:** Adds a new product to the system.
- **Validates:** The product information using FluentValidation.
- **Returns:** The newly added product, or an error message if the operation fails.

5. PUT /api/products

Updates an existing product.

```

app.MapPut("/api/products", async (IProductsService productService,
IValidator<ProductUpdateRequest> productUpdateRequestValidator, ProductUpdateRequest
productUpdateRequest) =>
{
    ValidationResult validationResult = await
productUpdateRequestValidator.ValidateAsync(productUpdateRequest);

    if (!validationResult.IsValid)
    {
        Dictionary<string, string[]> errors = validationResult.Errors
        .GroupBy(temp => temp.PropertyName)
        .ToDictionary(grp => grp.Key, grp => grp.Select(err => err.ErrorMessage).ToArray());
return Results.ValidationProblem(errors);
}
}

```



```

var updatedProductResponse = await productService.UpdateProduct(productUpdateRequest);
if (updatedProductResponse != null)
    return Results.Ok(updatedProductResponse);
else
    return Results.Problem("Error in updating product");
});

```

- **Description:** Updates an existing product's details.
- **Validates:** The updated information using FluentValidation.
- **Returns:** The updated product details, or an error message if the operation fails.

6. DELETE /api/products/{ProductID}

Deletes a product by its ProductID.

```

app.MapDelete("/api/products/{ProductID:guid}", async (IProductsService productService, Guid
ProductID) =>

```

```

{
    bool isDeleted = await productService.DeleteProduct(ProductID);
    if (isDeleted)
        return Results.Ok(true);
    else
        return Results.Problem("Error in deleting product");
});

```

- **Description:** Deletes the product that matches the provided ProductID.
- **Returns:** A success message if the deletion is successful, or an error if it fails.

Summary of API Endpoints in the API Layer

These API endpoints handle various product management operations such as retrieving, adding, updating, and deleting products using services and validators from the business logic layer. The usage of FluentValidation ensures that the incoming data is validated before processing, while the Results object simplifies generating responses.

Program.cs File in API Layer

This Program.cs file sets up and configures the services, middleware, and API endpoints for the eCommerce Products Microservice. Below is a breakdown of the configuration.

1. Builder Configuration

```
var builder = WebApplication.CreateBuilder(args);
```

- The `WebApplication.CreateBuilder(args)` method initializes a new `WebApplicationBuilder`, setting up the application host.

2. Adding Services

```
// Add DAL and BLL services
```

```
builder.Services.AddDataAccessLayer(builder.Configuration);
```

```
builder.Services.AddBusinessLogicLayer();
```

- **Data Access Layer (DAL)** and **Business Logic Layer (BLL)** services are registered using extension methods `AddDataAccessLayer` and `AddBusinessLogicLayer`.

```
builder.Services.AddControllers();
```

- Adds the **MVC controllers** for handling incoming HTTP requests.

3. Fluent Validation Setup

```
// Fluent Validations
```

```
builder.Services.AddFluentValidationAutoValidation();
```

- Registers Fluent Validation for automatic validation of models in the API.

4. JSON Enum Converter

```
// Add model binder to read values from JSON to enum
```

```
builder.Services.ConfigureHttpJsonOptions(options => {
```

```
    options.SerializerOptions.Converters.Add(new JsonStringEnumConverter());
```

```
});
```

- Configures JSON serialization to support conversion between enum values and strings.

5. Swagger Configuration

```
// Add Swagger services
```

```
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen();
```

- **Swagger** is added for API documentation and testing. The AddSwaggerGen method configures Swagger UI generation for the project.

6. CORS Configuration

```
// CORS
```

```
builder.Services.AddCors(options => {
```

```
    options.AddDefaultPolicy(builder => {
```

```
        builder.WithOrigins("http://localhost:4200")
```

```
            .AllowAnyMethod()
```

```
            .AllowAnyHeader();
```

```
    });
```

```
});
```

- **CORS (Cross-Origin Resource Sharing)** is configured to allow specific domains (in this case, http://localhost:4200, which might be for an Angular frontend) to make requests to the API.

7. Application Middleware Pipeline

```
var app = builder.Build();
```

- After configuring services, the application is built using builder.Build().

8. Middleware Setup

```
// Exception Handling Middleware
```

```
app.UseExceptionHandlerMiddleware();
```

- Adds custom **exception handling middleware** to handle any exceptions that occur during request processing.

```
app.UseRouting();
```

- Enables **routing** for the API.

9. Additional Middleware

```
// CORS
```

```
app.UseCors();
```

- Applies the configured **CORS policy**.

```
// Swagger
```

```
app.UseSwagger();
```

```
app.UseSwaggerUI();
```

- Enables **Swagger** for API documentation at runtime.

```
// HTTPS, Authentication, and Authorization
```

```
app.UseHttpsRedirection();
```

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

- Redirects HTTP requests to HTTPS, and configures **authentication** and **authorization** middleware.

10. API Endpoint Mapping

```
app.MapControllers();
```

```
app.MapProductAPIEndpoints();
```

- Maps **controller-based endpoints** and the custom-defined **product API endpoints** to the application.

11. Running the Application

```
app.Run();
```

- Starts the application and begins listening for incoming HTTP requests.

Summary of Program.cs File in API Layer

This Program.cs file sets up the core structure of the API, including service registration for the business logic and data access layers, CORS policy, Fluent Validation, Swagger, and middleware such as exception handling. It also maps the API endpoints for product management.

.NET Microservices – Azure DevOps and AKS

Section 4: Docker - Notes

Deployment Workflow

Understanding Deployment Workflow

Deployment workflow is a critical concept to grasp when discussing containerization, as it helps to understand why containers are needed and how they improve the software development process.

1. Traditional Deployment Workflow

In traditional software deployment workflows, artifacts (like application binaries or packages) move through various stages of development, testing, and production environments. Here's a typical flow:

1. **Development:** Developers write code and build applications on their local machines. They might use specific versions of libraries and tools that are installed locally.
2. **QA (Quality Assurance):** The application is then deployed to a QA environment, where testers verify its functionality and stability. This environment might have different configurations, libraries, or versions of dependencies compared to the development environment.
3. **Staging:** Before production, the application is deployed to a staging environment. This environment mirrors the production setup as closely as possible. It ensures that any issues that didn't surface in QA are identified before going live.
4. **Production:** Finally, the application is deployed to the production environment where it is available to end-users.

2. Issues with Traditional Deployment

- **Dependency Management:** Each environment may have different versions of libraries or tools. Ensuring consistency across environments can be challenging and may lead to “works on my machine” problems.
- **Configuration Differences:** Different environments often require different configurations. Managing these configurations and ensuring they are correctly applied can be error-prone.
- **Environment Setup:** Setting up environments consistently is often time-consuming. Any differences between environments can result in bugs or issues that are hard to trace.

3. Role of Containerization

Containerization addresses these issues by encapsulating applications and their dependencies into a single, portable unit called a container. Here's how it helps:

- **Consistency Across Environments:** Containers include everything needed to run an application: code, runtime, system tools, libraries, and settings. This ensures that the application runs consistently regardless of where the container is deployed.
- **Simplified Deployment:** Containers provide a consistent environment, reducing the effort required to set up and configure deployment environments. This makes it easier to move applications from development to production with fewer surprises.
- **Dependency Management:** Containers handle dependencies internally. Once a container image is created, it contains all dependencies required to run the application, eliminating the problem of differing dependencies across environments.

4. Deployment Workflow with Docker

Using Docker, the deployment workflow typically involves the following steps:

1. **Build Docker Image:** Developers create a Dockerfile defining the application's environment. Docker builds an image from this file, which includes all dependencies and configurations.
2. **Test Docker Image:** The Docker image is tested in different environments (e.g., QA, staging) to ensure it works as expected.
3. **Deploy Docker Container:** The Docker image is deployed as a container in the production environment. The container runs the application in a consistent environment, ensuring that it works the same way as it did in development and testing.
4. **Update and Rollback:** Updating the application involves building a new Docker image and redeploying it. Rollbacks are also straightforward, as you can revert to a previous image if needed.

Key Points to Remember (For Interview Preparation)

- **Deployment Workflow Stages:** Development, QA, Staging, Production.
- **Challenges in Traditional Deployment:** Dependency management, configuration differences, environment setup.
- **Containerization Benefits:** Consistency across environments, simplified deployment, dependency management.
- **Docker Workflow:** Build Docker image, test Docker image, deploy Docker container, update and rollback.

Hypervisor and Its Limitations

1. What is a Hypervisor?

A hypervisor, also known as a virtual machine monitor (VMM), is a software layer that allows multiple operating systems (OS) to run on a single physical machine. It manages and allocates resources between the host machine and multiple guest virtual machines (VMs).

Types of Hypervisors:

- **Type 1 Hypervisor (Bare-metal):** Runs directly on the physical hardware without an underlying host OS. Examples include VMware ESXi, Microsoft Hyper-V, and Xen. It offers better performance and resource efficiency because it operates directly on hardware.
- **Type 2 Hypervisor (Hosted):** Runs on top of a conventional operating system. Examples include VMware Workstation, Oracle VirtualBox, and Parallels Desktop. It relies on the host OS for device drivers and system resources, which can introduce additional overhead.

2. Limitations of Hypervisors

- **Resource Overhead:** Hypervisors require significant system resources to run. Each virtual machine includes a full OS, which consumes additional memory and CPU compared to running processes natively on the host OS.
- **Performance:** While virtualization provides isolation, the overhead of managing multiple VMs can lead to performance degradation compared to running applications directly on the host OS.
- **Complexity:** Managing and maintaining multiple VMs, each with its own OS, can be complex and resource-intensive. Configuration, updates, and security patches need to be handled individually for each VM.
- **Disk Space:** Each VM requires its own disk image, which can quickly consume storage space. This can be especially problematic if multiple VMs are used for testing and development.
- **Boot Time:** VMs typically have longer boot times because each virtual machine must start its own operating system. This can delay application deployment and testing.

3. How Docker Addresses Hypervisor Limitations

Docker containers provide a more lightweight and efficient alternative to hypervisors:

- **Efficiency:** Containers share the host OS kernel but run in isolated user spaces. This reduces overhead as containers do not require a full OS for each instance.
- **Speed:** Containers start up almost instantly compared to VMs because they do not need to boot an entire OS. This speed improves development and deployment cycles.
- **Resource Utilization:** Containers are more resource-efficient, requiring less memory and storage compared to VMs. This efficiency allows for higher density of application instances on the same hardware.

- **Simplicity:** Managing containers is simpler compared to managing multiple VMs. Containers can be easily started, stopped, and scaled, and they use a single OS image, reducing complexity.

Key Points to Remember (For Interview Preparation)

- **Hypervisor Types:** Type 1 (bare-metal) and Type 2 (hosted).
- **Limitations of Hypervisors:** Resource overhead, performance issues, complexity, disk space usage, longer boot times.
- **Docker Advantages Over Hypervisors:** Efficiency, faster startup, better resource utilization, simpler management.

Introduction to Docker

1. What is Docker?

Docker is an open-source platform that automates the deployment, scaling, and management of applications using containerization technology. It allows developers to package applications and their dependencies into a standardized unit called a container, which can be run consistently across various environments.

2. Key Concepts

- **Containers:** A lightweight, standalone, and executable package that includes everything needed to run an application—code, runtime, libraries, and system tools. Containers are isolated from each other and from the host system, ensuring that applications run the same way regardless of where they are deployed.
- **Images:** A read-only template used to create containers. Images contain the application code, runtime, libraries, and dependencies. They are built from a Dockerfile and serve as the blueprint for containers.
- **Docker Engine:** The core component of Docker, responsible for running and managing containers. It consists of the Docker daemon, which handles container operations, and the Docker CLI (Command-Line Interface), which provides commands to interact with Docker.
- **Dockerfile:** A text file with a set of instructions to build a Docker image. It defines the base image, installs dependencies, and specifies how the application should be run within the container.
- **Docker Hub:** A cloud-based registry service where Docker images can be stored, shared, and managed. It provides a centralized place for distributing container images.

3. How Docker Works

- **Containerization:** Docker packages applications and their dependencies into containers, ensuring consistency across different environments. Containers run on any system that has Docker installed, without the need for additional configuration or modifications.
- **Isolation:** Docker uses features of the host OS, such as namespaces and cgroups, to provide isolation between containers. Each container operates in its own environment with its own file system, processes, and network settings.
- **Portability:** Docker containers can be run on any platform that supports Docker, including different operating systems and cloud environments. This portability simplifies development and deployment processes.
- **Efficiency:** Docker containers are lightweight and share the host OS kernel, reducing resource consumption compared to traditional virtual machines. This allows for better utilization of system resources and faster application startup times.

4. Use Cases for Docker

- **Development:** Docker simplifies development workflows by providing a consistent environment across different stages of the application lifecycle. Developers can build and test applications in containers, ensuring that they work the same way in production.
- **Testing:** Docker allows for the creation of isolated testing environments that mimic production systems. This ensures that tests are run in a controlled and predictable manner, reducing the risk of environmental issues affecting test results.
- **Deployment:** Docker enables the deployment of applications as containers, making it easier to scale and manage applications in production environments. Containers can be quickly started, stopped, and updated without affecting other parts of the system.
- **Microservices:** Docker is ideal for deploying microservices architectures, where applications are composed of small, loosely coupled services. Each service can be developed, tested, and deployed independently as a separate container.

Key Points to Remember (For Interview Preparation)

- **Docker Overview:** Automates deployment and management of applications using containerization.
- **Core Concepts:** Containers, images, Docker Engine, Dockerfile, Docker Hub.
- **Key Features:** Containerization, isolation, portability, efficiency.
- **Use Cases:** Development, testing, deployment, microservices.

Benefits of Containerization

1. Consistency Across Environments

- **Problem:** Applications often behave differently in development, testing, and production environments due to variations in configurations, dependencies, and system libraries.
- **Solution:** Containers package applications along with their dependencies, ensuring that they run consistently across different environments. This eliminates "works on my machine" issues and simplifies troubleshooting and debugging.

2. Isolation and Security

- **Isolation:** Containers run in isolated environments, which means that processes running inside one container do not interfere with those running in another. This isolation helps in avoiding conflicts between applications and improves system stability.
- **Security:** Containers provide an additional layer of security by isolating applications from the host system and other containers. They use namespaces and cgroups to enforce boundaries and limit resource usage, reducing the potential impact of security vulnerabilities.

3. Portability

- **Cross-Platform Compatibility:** Docker containers can be run on any system that has Docker installed, regardless of the underlying operating system or hardware. This portability ensures that applications can be easily moved between different environments, such as from a developer's laptop to a cloud server.
- **Cloud Flexibility:** Containers are well-suited for cloud environments, where they can be deployed and scaled across various cloud providers. This flexibility allows organizations to choose the best cloud platform for their needs.

4. Resource Efficiency

- **Lightweight:** Containers share the host OS kernel, which makes them more lightweight compared to traditional virtual machines (VMs) that require separate operating systems. This reduces overhead and resource consumption, allowing more containers to run on the same hardware.
- **Fast Startup:** Containers can be started and stopped quickly, which improves the efficiency of deploying and scaling applications. This fast startup time is beneficial for both development and production environments.

5. Simplified Deployment and Management

- **Automation:** Docker simplifies the deployment process by automating the creation and management of containers. Deployment scripts and orchestration tools can be used to manage containerized applications, reducing manual effort and errors.
- **Version Control:** Docker images can be versioned and tagged, making it easy to track changes and roll back to previous versions if needed. This version control enhances the ability to manage application updates and deployments.

6. Scalability and Flexibility

- **Scalability:** Containers can be easily scaled horizontally by adding more instances to handle increased load. Orchestration tools like Kubernetes can automate the scaling process and manage container clusters.
- **Microservices:** Docker is well-suited for microservices architectures, where applications are composed of small, independent services. Each service can be deployed and scaled separately, providing flexibility and agility in application development and deployment.

Key Points to Remember (For Interview Preparation)

- **Consistency:** Containers ensure consistent behavior across development, testing, and production environments.
- **Isolation & Security:** Containers provide process isolation and additional security layers.
- **Portability:** Containers are portable across different systems and cloud environments.
- **Efficiency:** Containers are lightweight, have fast startup times, and share the host OS kernel.
- **Deployment & Management:** Simplifies deployment through automation and version control.
- **Scalability:** Containers can be scaled horizontally and are ideal for microservices architectures.

Docker Container vs Virtual Machines (VMs)

1. Basic Concepts

- **Docker Containers:**
 - **Definition:** Containers are lightweight, portable units that package applications and their dependencies together. They share the host operating system (OS) kernel but run in isolated user spaces.
 - **Structure:** Containers use OS-level virtualization to achieve isolation. They include only the application and its dependencies, which makes them smaller in size compared to VMs.

- **Virtual Machines (VMs):**

- **Definition:** VMs are emulated hardware environments that run a full operating system (OS) on top of a hypervisor. Each VM includes its own OS, applications, and virtualized hardware.
- **Structure:** VMs use hardware-level virtualization. Each VM is a complete replica of a physical machine, including its own OS and virtual hardware components.

2. Resource Efficiency

- **Docker Containers:**

- **Lightweight:** Containers are smaller because they share the host OS kernel. They only include the application and necessary dependencies, which results in lower resource consumption.
- **Fast Startup:** Containers can be started and stopped quickly due to their lightweight nature. This is ideal for scenarios that require rapid scaling and deployment.

- **Virtual Machines:**

- **Heavyweight:** VMs are larger because they include a full OS and virtual hardware. This results in higher resource usage, including more memory and storage.
- **Slower Startup:** Starting a VM involves booting up a complete OS, which takes more time compared to starting a container.

3. Isolation

- **Docker Containers:**

- **Process-Level Isolation:** Containers use namespaces and cgroups to provide process isolation. While containers are isolated from each other and the host, they still share the same OS kernel.
- **Security:** Containers offer process isolation but may have limited security compared to VMs. Security measures need to be implemented to ensure that vulnerabilities in one container do not affect others.

- **Virtual Machines:**

- **Hardware-Level Isolation:** VMs provide stronger isolation as they emulate separate physical machines. Each VM runs its own OS, which provides a higher level of separation between VMs.
- **Security:** VMs offer a more secure isolation boundary because they do not share the host OS kernel. This can reduce the risk of vulnerabilities affecting multiple VMs.

4. Management and Deployment

- **Docker Containers:**
 - **Simpler Deployment:** Containers are easier to deploy and manage due to their lightweight nature. Docker images can be versioned and deployed quickly.
 - **Container Orchestration:** Tools like Docker Compose and Kubernetes are used to manage and orchestrate containers, enabling automated deployment, scaling, and management.
- **Virtual Machines:**
 - **Complex Deployment:** VMs require more complex deployment processes, including provisioning and configuring virtual hardware. They are often managed using hypervisors and tools like VMware vSphere or Microsoft Hyper-V.
 - **VM Orchestration:** Tools like VMware vCenter and Microsoft SCVMM are used for managing VMs and their lifecycle. They provide features for scaling and high availability but may require more overhead.

5. Use Cases

- **Docker Containers:**
 - **Microservices:** Ideal for deploying microservices architectures where applications are composed of small, independent services.
 - **Development and Testing:** Suitable for development and testing environments where consistency and rapid provisioning are important.
- **Virtual Machines:**
 - **Legacy Applications:** Suitable for running legacy applications that require a full OS and virtualized hardware.
 - **Complete Isolation:** Useful in scenarios where strong isolation and security are required, such as multi-tenant environments.

Key Points to Remember (For Interview Preparation)

- **Containers vs VMs:** Containers are lightweight and share the host OS kernel, while VMs are heavier and include a full OS and virtual hardware.
- **Resource Efficiency:** Containers offer lower resource consumption and faster startup times compared to VMs.
- **Isolation:** Containers provide process-level isolation, whereas VMs offer hardware-level isolation and stronger security.
- **Management:** Containers are easier to manage and deploy with orchestration tools, while VMs require more complex management and provisioning.

How Docker Images and Containers (Layers) Work Internally

1. Docker Images

- **Definition:** A Docker image is a snapshot of a filesystem that contains the application code, libraries, and dependencies required to run a container. It serves as the blueprint from which containers are created.
- **Layers:**
 - **Concept:** Docker images are built in layers. Each layer represents a set of changes or additions made to the filesystem. These layers are stacked on top of each other to form the final image.
 - **Base Layers:** The bottom layer is usually a base image (like an OS image), and subsequent layers are added on top, each representing modifications such as installed packages or application files.
 - **Layer Caching:** Docker uses caching to avoid rebuilding layers unnecessarily. If a layer hasn't changed, Docker reuses the cached version, which speeds up the build process.
- **Example:**
 - **Base Image:** FROM mcr.microsoft.com/dotnet/aspnet:8.0 provides the .NET runtime environment.
 - **Add Files:** COPY . . adds application files to the image.
 - **Build Commands:** RUN dotnet build compiles the application.

2. Docker Containers

- **Definition:** A Docker container is an instance of a Docker image. It is a running process with its own isolated filesystem, network, and process space. Containers are ephemeral and can be started, stopped, and deleted as needed.
- **Layers in Containers:**
 - **Overlay Filesystem:** Containers use a union filesystem (like OverlayFS) to combine the layers of the image. The container adds a writable layer on top of the read-only layers of the image. This writable layer is where changes to the filesystem occur during runtime.
 - **File Changes:** Changes made inside a container are stored in the writable layer and do not affect the underlying image. When the container is removed, these changes are discarded unless they are committed to a new image.
- **Example:**

- **Running a Container:** `docker run my-image` starts a container from the `my-image` image.
- **Writable Layer:** Any files created or modified in the container are stored in the container's writable layer.

3. Image Creation and Layers

- **Build Process:**
 - **Dockerfile:** Defines the steps to build an image. Each command in the Dockerfile creates a new layer in the image.
 - **Caching:** Docker caches layers to optimize the build process. If a layer hasn't changed, Docker reuses the cached version.
- **Optimization Tips:**
 - **Minimize Layers:** Combine commands in the Dockerfile to reduce the number of layers. For example, use `RUN apt-get update && apt-get install -y package` instead of separate `RUN` commands.
 - **Order of Instructions:** Place frequently changing commands towards the bottom of the Dockerfile to leverage caching effectively.

4. Docker Image Storage and Distribution

- **Image Storage:**
 - **Local Storage:** Images are stored locally on the Docker host. You can list and inspect local images using commands like `docker images` and `docker inspect`.
 - **Remote Repositories:** Images can be pushed to and pulled from remote repositories (like Docker Hub) for distribution.
- **Example:**
 - **List Images:** `docker images` lists all images stored locally.
 - **Push to Repository:** `docker push my-repo/my-image` uploads the image to a remote repository.
 - **Pull from Repository:** `docker pull my-repo/my-image` downloads the image from a remote repository.

Key Points to Remember (For Interview Preparation)

- **Docker Images:** Composed of layers, with each layer representing a set of changes. Layers are cached to optimize the build process.
- **Docker Containers:** Instances of images that run with an isolated filesystem. Containers use a writable layer on top of the image layers.

- **Layer Caching:** Docker reuses cached layers to speed up the build process. Minimizing layers and ordering instructions can improve build efficiency.
- **Image Storage:** Images are stored locally or in remote repositories. Commands like `docker push` and `docker pull` handle image distribution.

Introduction to Docker Hub

1. What is Docker Hub?

- **Definition:** Docker Hub is a cloud-based repository service where Docker users can store, share, and manage Docker images. It acts as a central hub for finding and distributing container images.
- **Features:**
 - **Public Repositories:** Users can publish images to public repositories that are accessible to everyone.
 - **Private Repositories:** Users can also create private repositories for storing images that should be restricted to specific users or organizations.
 - **Automated Builds:** Docker Hub can automatically build images from source code stored in version control systems like GitHub or Bitbucket.
 - **Web Interface:** Docker Hub provides a web-based interface to manage repositories, view image details, and configure repository settings.

2. Basic Operations with Docker Hub

- **Creating an Account:**
 - **Sign Up:** Go to Docker Hub and sign up for a free account. You can also use an existing Docker ID if you have one.
 - **Profile Setup:** After creating an account, you can set up your profile and manage your repositories from the Docker Hub dashboard.
- **Working with Repositories:**
 - **Create a Repository:** In Docker Hub, you can create a new repository for your images. You can choose to make it public or private.
 - **Push an Image:** After building a Docker image locally, you can push it to Docker Hub using `docker push <repository>/<image>:<tag>`.
 - **Pull an Image:** To use an image from Docker Hub, you can pull it using `docker pull <repository>/<image>:<tag>`.

- **Example Commands:**
 - **Login to Docker Hub:** docker login (Enter your Docker ID and password when prompted)
 - **Push an Image:** docker push myusername/my-repo:latest
 - **Pull an Image:** docker pull myusername/my-repo:latest

3. Managing Docker Hub Repositories

- **Repository Settings:**
 - **Visibility:** Configure the visibility of your repositories (public or private).
 - **Collaborators:** Add collaborators to your private repositories to allow them access.
 - **Automated Builds:** Link your repository to a version control system to enable automated builds.
- **Web Interface:**
 - **Dashboard:** The Docker Hub dashboard allows you to view your repositories, check image details, and manage settings.
 - **Search:** You can search for public images on Docker Hub using keywords or filter by tags.

4. Security Considerations

- **Private Repositories:** Use private repositories for sensitive or proprietary images to control access.
- **Access Control:** Manage access permissions to your repositories and images carefully.

Key Points to Remember (For Interview Preparation)

- **Docker Hub:** A central repository service for storing and sharing Docker images, with both public and private options.
- **Operations:** Basic commands include docker push for uploading images and docker pull for downloading images.
- **Repository Management:** Includes creating repositories, setting visibility, adding collaborators, and configuring automated builds.
- **Security:** Use private repositories for sensitive images and manage access permissions.

Docker Architecture

1. Docker Components

- **Docker Daemon (dockerd):**
 - **Role:** The Docker Daemon is a background process that manages Docker containers. It handles container creation, management, and networking.
 - **Function:** Listens for Docker API requests and handles container operations. It can communicate with other Docker Daemons to manage containers across different hosts.
- **Docker Client (docker):**
 - **Role:** The Docker Client is the command-line interface that allows users to interact with the Docker Daemon.
 - **Function:** Sends commands to the Docker Daemon using the Docker API. Commands include `docker run`, `docker ps`, `docker build`, etc.
- **Docker Images:**
 - **Role:** Docker Images are read-only templates used to create Docker containers. They contain the application code, libraries, and dependencies.
 - **Function:** Images serve as the blueprint for containers. They are built using a Dockerfile and can be pulled from or pushed to Docker Hub or other registries.
- **Docker Containers:**
 - **Role:** Docker Containers are instances of Docker Images. They are isolated, lightweight, and run the application with its dependencies.
 - **Function:** Containers are created from images and run the application in a separate environment. Containers are ephemeral and can be stopped or removed without affecting the underlying image.
- **Docker Registries:**
 - **Role:** Registries store Docker Images. Docker Hub is a public registry, while Docker also supports private registries.
 - **Function:** Images can be pushed to and pulled from registries. Registries provide a centralized location for image distribution and storage.
- **Docker Compose:**
 - **Role:** Docker Compose is a tool for defining and running multi-container Docker applications.
 - **Function:** Uses a `docker-compose.yml` file to configure and run multiple containers with a single command. It simplifies managing complex applications with multiple services.

2. Docker Architecture Diagram

The Docker architecture can be visualized as follows:

1. User Interaction:

- Users interact with Docker through the Docker Client, issuing commands and API requests.

2. Docker Daemon:

- The Docker Daemon processes commands from the Docker Client. It communicates with Docker Registries to pull images and manage containers.

3. Docker Images:

- Docker Images are pulled from or pushed to Docker Registries. They serve as the base for creating containers.

4. Docker Containers:

- Containers are created from images and run isolated applications. They can be managed by the Docker Daemon.

5. Docker Registries:

- Registries store Docker Images and provide them to Docker Daemons as needed.

3. Docker Networking

• Default Network Types:

- **Bridge Network:** The default network for Docker containers. Containers on the same bridge network can communicate with each other.
- **Host Network:** Uses the host's network stack directly, bypassing Docker's network isolation.
- **Overlay Network:** Allows containers running on different Docker hosts to communicate securely.
- **Macvlan Network:** Assigns a MAC address to a container, allowing it to appear as a physical device on the network.

• Custom Networks:

- **Creating Custom Networks:** You can create custom networks for containers using the `docker network create` command.
- **Connecting Containers:** Containers can be connected to custom networks to communicate with each other.

4. Docker Volumes

- **Role:** Docker Volumes are used to persist data generated by and used by Docker containers.
- **Function:** Volumes allow data to be stored outside the container's filesystem, ensuring that data is preserved even if the container is removed.
- **Commands:**
 - **Create a Volume:** `docker volume create <volume-name>`
 - **Mount a Volume:** `docker run -v <volume-name>:<container-path> <image>`

Key Points to Remember (For Interview Preparation)

- **Docker Components:** Understand the roles of Docker Daemon, Docker Client, Docker Images, Docker Containers, Docker Registries, and Docker Compose.
- **Architecture:** Be able to explain how the Docker Client interacts with the Docker Daemon, and how Docker Images and Containers fit into the architecture.
- **Networking:** Familiarize yourself with default network types and their use cases, as well as custom network creation and container connectivity.
- **Volumes:** Know the purpose of Docker Volumes for data persistence and basic volume commands.

Dockerfile and All Important Commands

1. Dockerfile Basics

- **Purpose:** A Dockerfile is a script with a series of instructions to build a Docker Image. Each instruction creates a layer in the image.
- **Syntax:** The Dockerfile uses a specific syntax for instructions, which Docker understands to build and configure the image.

2. Key Dockerfile Instructions

- **FROM:**
 - **Purpose:** Specifies the base image for the Docker image. All subsequent instructions are applied on top of this base.
 - **Example:** FROM mcr.microsoft.com/dotnet/aspnet:8.0
- **WORKDIR:**
 - **Purpose:** Sets the working directory inside the container for subsequent instructions.
 - **Example:** WORKDIR /app
- **COPY:**
 - **Purpose:** Copies files or directories from the host into the container.
 - **Example:** COPY ["ProductsMicroService.API/ProductsMicroService.API.csproj", "ProductsMicroService.API/"]
- **RUN:**
 - **Purpose:** Executes commands inside the container during the image build process.
 - **Example:** RUN dotnet restore
"./ProductsMicroService.API/./ProductsMicroService.API.csproj"
- **EXPOSE:**
 - **Purpose:** Documents the port number on which the container will listen. This does not publish the port but serves as documentation.
 - **Example:** EXPOSE 8080
- **ENV:**
 - **Purpose:** Sets environment variables inside the container.
 - **Example:** ENV MYSQL_HOST=localhost
- **ENTRYPOINT:**
 - **Purpose:** Specifies the command to run when the container starts. It defines the default executable for the container.
 - **Example:** ENTRYPOINT ["dotnet", "ProductsMicroService.API.dll"]
- **CMD:**
 - **Purpose:** Provides default arguments for the ENTRYPOINT command. It can be overridden when running the container.
 - **Example:** CMD ["--help"]
- **VOLUME:**

- **Purpose:** Creates a mount point with the specified path and marks it as holding externally mounted volumes from native host or other containers.
- **Example:** VOLUME ["/data"]
- **USER:**
 - **Purpose:** Sets the user name or UID to use when running the image.
 - **Example:** USER app

3. Example Dockerfile Explanation

Base image for the runtime environment

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base

USER app

WORKDIR /app

EXPOSE 8080

EXPOSE 8081

Image for building the application

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build

ARG BUILD_CONFIGURATION=Release

WORKDIR /src

COPY ["ProductsMicroService.API/ProductsMicroService.API.csproj", "ProductsMicroService.API/"]

RUN dotnet restore "./ProductsMicroService.API/./ProductsMicroService.API.csproj"

COPY . .

WORKDIR "/src/ProductsMicroService.API"

RUN dotnet build "./ProductsMicroService.API.csproj" -c \$BUILD_CONFIGURATION -o /app/build

Publish the application

FROM build AS publish

ARG BUILD_CONFIGURATION=Release

RUN dotnet publish "./ProductsMicroService.API.csproj" -c \$BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false

Final image to run the application

FROM base AS final

WORKDIR /app

COPY --from=publish /app/publish .

ENV MYSQL_HOST=localhost

ENV MYSQL_PASSWORD=admin

ENTRYPOINT ["dotnet", "ProductsMicroService.API.dll"]

- **Explanation:**
 - **Base Stage:** Sets up the runtime environment with the ASP.NET image and exposes ports.
 - **Build Stage:** Uses the .NET SDK image to restore, build, and publish the application.
 - **Publish Stage:** Publishes the application output.
 - **Final Stage:** Combines the runtime environment with the published output and sets up environment variables and the entry point.

4. Docker Commands for Dockerfile

- **Building an Image:** `docker build -t <image-name>:<tag> .`
 - **Purpose:** Builds the Docker image from the Dockerfile in the current directory.
- **Listing Images:** `docker images` or `docker image ls`
 - **Purpose:** Lists all available Docker images on the local machine.
- **Running a Container:** `docker run <image-name>:<tag>`
 - **Purpose:** Runs a container from the specified image.
- **Inspecting an Image:** `docker inspect <image-id>`
 - **Purpose:** Provides detailed information about the Docker image.

Key Points to Remember (For Interview Preparation)

- **Dockerfile Instructions:** Be familiar with common Dockerfile instructions (FROM, WORKDIR, COPY, RUN, EXPOSE, ENV, ENTRYPOINT, CMD, VOLUME, USER) and their purposes.
- **Multi-Stage Builds:** Understand how multi-stage builds work to optimize Docker images by separating build and runtime environments.
- **Docker Commands:** Know the commands for building, running, and managing Docker images and containers.

Creating Docker Images and Containers in Terminal (Windows)

1. Prerequisites

- **Docker Installation:** Ensure Docker Desktop is installed on your Windows machine. It can be downloaded from the Docker website.
- **Docker Daemon:** Make sure the Docker Daemon is running. Docker Desktop should start it automatically.

2. Building a Docker Image

- **Navigate to the Directory:** Open Command Prompt or PowerShell and navigate to the directory containing your Dockerfile.

```
cd path\to\your\project
```

- **Build the Image:** Use the docker build command to create an image from the Dockerfile.

```
docker build -t <image-name>:<tag> .
```

- **Parameters:**

- -t <image-name>:<tag>: Specifies the name and tag for the image.
- .: Refers to the current directory where the Dockerfile is located.

Example:

```
docker build -t myapp:latest .
```

- **Verify the Image:** List Docker images to confirm that your image was created.

```
docker images
```

- **Output:**

- REPOSITORY: The name of the image.
- TAG: The tag of the image.
- IMAGE ID: The unique ID of the image.
- CREATED: When the image was created.
- SIZE: The size of the image.

3. Running a Docker Container

- **Run the Container:** Use the docker run command to start a container from your image.

`docker run -d -p <host-port>:<container-port> --name <container-name> <image-name>:<tag>`

- **Parameters:**

- `-d`: Runs the container in detached mode (in the background).
- `-p <host-port>:<container-port>`: Maps a port on your host to a port in the container.
- `--name <container-name>`: Assigns a name to the container.
- `<image-name>:<tag>`: Specifies the image to use.

Example:

`docker run -d -p 8080:80 --name myapp-container myapp:latest`

- **Verify the Running Container:** List running containers to check if your container is running.

`docker ps`

- **Output:**

- **CONTAINER ID:** The unique ID of the container.
- **IMAGE:** The image used by the container.
- **COMMAND:** The command running inside the container.
- **CREATED:** When the container was created.
- **STATUS:** The status of the container.
- **PORTS:** Port mappings.
- **NAMES:** The name of the container.

- **Access the Container Logs:** Check the logs of your running container to see its output.

`docker logs <container-name>`

Example:

`docker logs myapp-container`

- **Stop and Remove the Container:** Stop and remove a running container.

- `docker stop <container-name>`

`docker rm <container-name>`

Example:

```
docker stop myapp-container
```

```
docker rm myapp-container
```

- **Remove an Image:** Delete an image from your local Docker registry.

```
docker rmi <image-name>:<tag>
```

Example:

```
docker rmi myapp:latest
```

4. Common Issues

- **Port Conflicts:** Ensure that the port you map (host-port) is not already in use on your machine.
- **Docker Daemon Not Running:** Verify that Docker Desktop is running and the Docker Daemon is active.

Key Points to Remember (For Interview Preparation)

- **Docker Commands:** Know the basic Docker commands for building (docker build), running (docker run), and managing images and containers (docker ps, docker images, docker logs, docker stop, docker rm, docker rmi).
- **Port Mapping:** Understand how to map host ports to container ports and why this is important for accessing containerized applications.
- **Detached Mode:** Be familiar with the use of detached mode (-d) for running containers in the background.

Docker Hub Accounts

1. Introduction to Docker Hub

- **What is Docker Hub?** Docker Hub is a cloud-based repository service where Docker images can be stored and shared. It's Docker's default registry and provides public and private repositories.
- **Features:**
 - **Public Repositories:** Free access to Docker images shared by the community.
 - **Private Repositories:** Secure storage for private Docker images (requires a subscription).
 - **Automated Builds:** Automatically build images from your GitHub or Bitbucket repository.
 - **Webhooks:** Trigger actions (like deployments) when new images are pushed.

2. Creating a Docker Hub Account

- **Sign Up:**
 - Visit the Docker Hub website.
 - Click on "Sign Up" and fill out the registration form with your email, username, and password.
 - Verify your email address to activate your account.

3. Logging In

- **Via Docker CLI:**
 - Open Command Prompt or PowerShell.
 - Use the docker login command to authenticate to Docker Hub.

docker login

- **Parameters:**
 - **Username:** Your Docker Hub username.
 - **Password:** Your Docker Hub password.
- **Example:**

docker login -u your-username -p your-password

- **Via Docker Desktop:**
 - Open Docker Desktop.
 - Go to the "Sign In" option and provide your Docker Hub credentials.

4. Pushing Docker Images to Docker Hub

- **Tagging the Image:**

- Before pushing an image, it needs to be tagged with the repository name on Docker Hub.

```
docker tag <local-image>:<tag> <dockerhub-username>/<repository>:<tag>
```

Example:

```
docker tag myapp:latest your-username/myapp:latest
```

- **Pushing the Image:**

- Use the docker push command to upload your image to Docker Hub.

```
docker push <dockerhub-username>/<repository>:<tag>
```

Example:

```
docker push your-username/myapp:latest
```

- **Verifying the Push:**

- Go to your Docker Hub repository page and verify that the image appears in your repository list.

5. Pulling Docker Images from Docker Hub

- **Pulling an Image:**

- Use the docker pull command to download an image from Docker Hub to your local machine.

```
docker pull <dockerhub-username>/<repository>:<tag>
```

Example:

```
docker pull your-username/myapp:latest
```

6. Managing Docker Hub Repositories

- **Creating a New Repository:**

- On Docker Hub, go to your profile and click on “Create Repository”.
- Provide a repository name and description.
- Set repository visibility (public or private).
- Click “Create”.

- **Managing Repositories:**

- Navigate to your repositories on Docker Hub to manage settings, view tags, and delete images.
- **Repository Permissions:**
 - Set permissions for who can access your private repositories. This can be configured under repository settings.

7. Docker Hub Best Practices

- **Tagging:** Use meaningful tags (e.g., v1.0, latest, dev, prod) to distinguish between different versions of your images.
- **Security:** Use private repositories for sensitive or proprietary images and ensure you set appropriate permissions.
- **Documentation:** Document your images with a README file in the repository to provide users with information about the image.

Key Points to Remember (For Interview Preparation)

- **Docker Hub Overview:** Understand the purpose and features of Docker Hub, including public and private repositories.
- **Account Management:** Know how to create, log in, and manage a Docker Hub account.
- **Image Operations:** Be familiar with commands for pushing (docker push) and pulling (docker pull) Docker images.
- **Repository Management:** Understand how to create and manage Docker repositories on Docker Hub.

Pushing Docker Images

1. Preparation

- **Ensure Image is Built:** Before pushing an image, ensure it is built locally. Verify by listing Docker images.

docker images

- **Tag Image Appropriately:** Tag the image to match the repository naming convention on Docker Hub. This helps organize and locate your images.

docker tag <local-image>:<tag> <dockerhub-username>/<repository>:<tag>

Example:

docker tag myapp:latest your-username/myapp:latest

2. Pushing the Image

- **Push Command:** Use `docker push` to upload the image to Docker Hub. The image must be tagged with the Docker Hub repository path.

`docker push <dockerhub-username>/<repository>:<tag>`

Example:

`docker push your-username/myapp:latest`

- **Progress Monitoring:** Docker CLI shows progress during the push, including layers being uploaded.

3. Verifying the Push

- **Docker Hub Website:** After pushing, verify by visiting your Docker Hub repository. You should see the newly pushed image and its tags.
- **Docker CLI:** You can also pull the image from Docker Hub to verify it has been successfully pushed.

`docker pull <dockerhub-username>/<repository>:<tag>`

4. Common Issues and Solutions

- **Authentication Errors:** Ensure you are logged in (`docker login`). Verify credentials if there are issues.
- **Repository Not Found:** Make sure the repository name and tag are correct. The repository should exist on Docker Hub.
- **Image Not Found Locally:** Check if the image is built and tagged correctly.

5. Automating Image Builds and Pushes

- **CI/CD Integration:** Automate image builds and pushes using CI/CD pipelines with tools like GitHub Actions, GitLab CI, or Jenkins.
- **Docker Hub Webhooks:** Configure webhooks to trigger builds or deployments when new images are pushed.

Key Points to Remember (For Interview Preparation)

- **Tagging:** Understand the importance of tagging images correctly for pushing to Docker Hub.
- **Push Command:** Be familiar with the `docker push` command and its usage.

- **Verification:** Know how to verify that the image has been pushed successfully using both Docker CLI and Docker Hub website.
- **Troubleshooting:** Be aware of common issues and their solutions related to pushing images.

Installing Docker on Linux

1. Overview

- **Supported Distro:** Docker supports major Linux distributions like Ubuntu, CentOS, and Debian. Ensure you're following the right steps for your specific distro.

2. Installing Docker on Ubuntu

- **Update Package Index:**

```
sudo apt-get update
```

- **Install Required Packages:** These packages allow apt to use packages over HTTPS.

```
sudo apt-get install apt-transport-https ca-certificates curl software-properties-common
```

- **Add Docker's Official GPG Key:**

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

- **Add Docker Repository:**

```
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu  
$(lsb_release -cs) stable"
```

- **Update Package Index Again:**

```
sudo apt-get update
```

- **Install Docker Engine:**

```
sudo apt-get install docker-ce
```

- **Verify Installation:**

```
sudo systemctl status docker
```

Ensure the Docker service is active and running. You can also check the version:

```
docker --version
```

3. Installing Docker on CentOS

- **Update Package Index:**

```
sudo yum update
```

- **Install Required Packages:**

```
sudo yum install -y yum-utils
```

- **Add Docker Repository:**

```
sudo yum-config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
```

- **Install Docker Engine:**

```
sudo yum install docker-ce
```

- **Start Docker Service:**

```
sudo systemctl start docker
```

- **Enable Docker to Start on Boot:**

```
sudo systemctl enable docker
```

- **Verify Installation:**

```
udo systemctl status docker
```

Check Docker version:

```
docker --version
```

4. Post-Installation Steps

- **Manage Docker as a Non-Root User:** To avoid using sudo with Docker commands, add your user to the Docker group.

`sudo usermod -aG docker $USER`

Log out and back in to apply the group changes.

- **Testing Docker Installation:**

`docker run hello-world`

This command runs a test container to verify that Docker is correctly installed and operational.

5. Common Issues and Solutions

- **Permission Denied:** If you encounter permission issues, ensure your user is added to the Docker group and that you've logged out and back in.
- **Service Not Starting:** Check Docker logs for errors and ensure the Docker service is correctly installed.

Key Points to Remember (For Interview Preparation)

- **Installation Steps:** Understand the step-by-step process for installing Docker on different Linux distributions.
- **Post-Installation Configuration:** Know how to manage Docker as a non-root user and verify the installation.
- **Common Issues:** Be prepared to troubleshoot common installation issues.

Pulling Docker Images on Linux

1. Overview

- **Docker Images:** Images are read-only templates used to create Docker containers. Pulling images means downloading these templates from a Docker registry to your local machine.

2. Prerequisites

- **Docker Installed:** Ensure Docker is installed and running on your Linux machine.

- **Docker Hub Account (Optional):** While not strictly necessary for pulling images, having an account allows access to private images and more features.

3. Pulling Images from Docker Hub

- **Basic Command Syntax:**

```
docker pull [OPTIONS] NAME[:TAG | @DIGEST]
```

- **NAME:** The name of the image.
- **TAG:** Optional tag to specify the version (default is latest).
- **DIGEST:** Optional digest for a specific image version.

- **Example: Pulling the Latest Ubuntu Image**

```
docker pull ubuntu
```

This command pulls the latest Ubuntu image from Docker Hub.

- **Example: Pulling a Specific Version**

```
docker pull ubuntu:20.04
```

This command pulls Ubuntu version 20.04.

- **Example: Pulling an Image from a Private Repository**

```
docker pull myrepository/myimage:tag
```

Replace myrepository, myimage, and tag with your repository name, image name, and tag respectively.

4. Verifying Pulled Images

- **List Pulled Images:**

```
docker images
```

This command lists all images on your local machine.

- **Inspect Image Details:**

```
docker inspect IMAGE_ID
```

Replace IMAGE_ID with the ID of the image you want to inspect. This provides detailed information about the image.

5. Common Issues and Solutions

- **Authentication Issues:** If pulling from a private repository, ensure you're logged in using docker login with the correct credentials.
- **Network Issues:** If you encounter network problems, check your internet connection and Docker daemon configuration.
- **Image Not Found:** Verify the image name and tag are correct. Check the repository URL if using a private registry.

6. Cleaning Up

- **Remove Unused Images:** To free up space, remove unused images with:

`docker image prune`

To remove all images not used by any containers:

`docker image prune -a`

Key Points to Remember (For Interview Preparation)

- **Basic Commands:** Know how to use docker pull to get images and how to specify tags.
- **Verification:** Be familiar with commands to list and inspect images.
- **Common Issues:** Understand potential problems with authentication, network issues, and image names.

Docker Networks

1. Overview

- **Docker Networks:** Docker networking allows containers to communicate with each other and with external systems. Docker provides several built-in network drivers to facilitate different networking needs.

2. Docker Network Drivers

- **Bridge Network:**

- **Default Network Driver:** Used when creating containers without specifying a network.
- **Isolated Network:** Containers on the same bridge network can communicate with each other but not with the host or containers on other networks.
- **Command to Create:**

```
docker network create --driver bridge my_bridge_network
```

-

- **Host Network:**

- **Direct Access:** The container shares the host's network stack, which means it has direct access to the host's network interfaces.
- **Command to Create:**

```
docker network create --driver host my_host_network
```

-

- **Overlay Network:**

- **Multi-Host Communication:** Allows containers across different Docker hosts to communicate.
- **Requires Swarm Mode:** Typically used in Docker Swarm for clustering.
- **Command to Create:**

```
docker network create --driver overlay my_overlay_network
```

-

- **Macvlan Network:**

- **Direct Network Interface:** Assigns a unique MAC address to each container, allowing them to appear as physical devices on the network.
- **Use Cases:** Useful for legacy applications requiring direct network access.
- **Command to Create:**

```
docker network create --driver macvlan --subnet=192.168.1.0/24 --gateway=192.168.1.1  
my_macvlan_network
```

-

- **None Network:**

- **No Networking:** Containers using this driver have no network access.
- **Command to Create:**

```
docker network create --driver none my_none_network
```

3. Managing Docker Networks

- **List Networks:**

`docker network ls`

This command lists all networks available on your Docker host.

- **Inspect Network:**

`docker network inspect NETWORK_NAME`

Replace NETWORK_NAME with the name of the network you want to inspect. This provides detailed information about the network configuration.

- **Remove Network:**

`docker network rm NETWORK_NAME`

Removes the specified network. Ensure no containers are connected to the network before removing it.

4. Connecting Containers to Networks

- **Run Container on Specific Network:**

`docker run --network NETWORK_NAME my_image`

Connects the container to the specified network.

- **Connect Existing Container to Network:**

`docker network connect NETWORK_NAME CONTAINER_NAME`

Connects an existing container to a specified network.

- **Disconnect Container from Network:**

`docker network disconnect NETWORK_NAME CONTAINER_NAME`

Disconnects a container from a specified network.

5. Advanced Networking

- **Port Binding:** Expose container ports to the host using `-p` or `--publish` option:

`docker run -p HOST_PORT:CONTAINER_PORT my_image`

- **DNS Configuration:** Containers can use Docker's internal DNS to resolve service names to container IP addresses.
- **Network Scopes:** In Docker Swarm, you can create global or local scopes for services to control network visibility.

Key Points to Remember (For Interview Preparation)

- **Network Types:** Understand the different Docker network drivers and their use cases.
- **Network Management:** Be familiar with commands to list, inspect, create, and remove networks.
- **Container Connectivity:** Know how to connect and disconnect containers from networks and manage port bindings.

Sample Code

#See <https://aka.ms/customizecontainer> to learn how to customize your debug container and how Visual Studio uses this Dockerfile to build your images for faster debugging.

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
```

```
USER app
```

```
WORKDIR /app
```

```
EXPOSE 8080
```

```
EXPOSE 8081
```

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
```

```
ARG BUILD_CONFIGURATION=Release
```

```
WORKDIR /src
```

```
COPY ["ProductsMicroService.API/ProductsMicroService.API.csproj", "ProductsMicroService.API/"]
```

```
RUN dotnet restore "./ProductsMicroService.API/./ProductsMicroService.API.csproj"
```

```
COPY . .
```

```
WORKDIR "/src/ProductsMicroService.API"
```

```
RUN dotnet build "./ProductsMicroService.API.csproj" -c $BUILD_CONFIGURATION -o /app/build
```

```
FROM build AS publish
```

```
ARG BUILD_CONFIGURATION=Release
```

```
RUN dotnet publish "./ProductsMicroService.API.csproj" -c $BUILD_CONFIGURATION -o  
/app/publish /p:UseAppHost=false
```

```
FROM base AS final
```

```
WORKDIR /app
```

```
COPY --from=publish /app/publish .
```

```
ENV MYSQL_HOST=localhost
```

```
ENV MYSQL_PASSWORD=admin
```

```
ENTRYPOINT ["dotnet", "ProductsMicroService.API.dll"]
```

Sample Code Explanation

Comment:

See <https://aka.ms/customizecontainer> to learn how to customize your debug container and how Visual Studio uses this Dockerfile to build your images for faster debugging.

- **Purpose:** This is a comment that provides a link to further documentation on customizing containers and explains that Visual Studio uses this Dockerfile during the debugging process.
- **Note:** Comments in Dockerfiles are prefixed with # and are ignored during execution.

Base Image:

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
```

- **Purpose:** This line specifies the **base image** for the container. It pulls the official **ASP.NET Core 8.0 runtime** image from Microsoft Container Registry (MCR).
- **FROM:** It indicates the starting image upon which the following instructions will be built.
- **mcr.microsoft.com/dotnet/aspnet:8.0:** Specifies the source of the image (MCR) and the version (8.0) of the .NET ASP.NET runtime.
- **AS base:** Assigns an alias base to this image so it can be referred to later in the Dockerfile.

USER Instruction:

```
USER app
```

- **Purpose:** Switches the user context within the container to a user named app. This is for **security** purposes to avoid running applications as the root user.

- **USER app:** Tells the container to run subsequent commands as the app user instead of the default root user.

Set Working Directory:

WORKDIR /app

- **Purpose:** Sets the **working directory** within the container to /app. Any commands or file operations will take place in this directory unless otherwise specified.
- **/app:** The directory where the application code or runtime files will reside.

Expose Ports:

EXPOSE 8080

EXPOSE 8081

- **Purpose:** These lines tell Docker to **expose** ports 8080 and 8081 to the host machine. This allows external clients to connect to the container through these ports.
- **EXPOSE:** Does not actually open the ports, but serves as a declaration of intention. The ports must be mapped when running the container using the -p option.

SDK Base Image (for Build Stage):

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build

- **Purpose:** This starts a new stage of the Docker build process, called build, using the .NET **SDK image** (not just the runtime) from version 8.0. The SDK image includes tools needed for building, restoring dependencies, and publishing the .NET application.
- **mcr.microsoft.com/dotnet/sdk:8.0:** Pulls the .NET SDK version 8.0 from the MCR.

Build Configuration Argument:

ARG BUILD_CONFIGURATION=Release

- **Purpose:** Defines a build-time argument BUILD_CONFIGURATION, which is used to specify the build mode (Release by default). It can be changed when building the Docker image.
- **ARG:** Defines a variable that can be passed to the build context.

Set Working Directory for Source Code:

WORKDIR /src

- **Purpose:** Sets the working directory to /src, which will be used to store the application source code.

Copy Project File:

`COPY ["ProductsMicroService.API/ProductsMicroService.API.csproj", "ProductsMicroService.API/"]`

- **Purpose:** Copies the .csproj file (project file) from the local directory into the container, placing it inside ProductsMicroService.API/.
- **COPY:** The COPY command takes two arguments: the first is the source file on the host system, and the second is the destination directory in the container.

Restore Dependencies:

`RUN dotnet restore "./ProductsMicroService.API/./ProductsMicroService.API.csproj"`

- **Purpose:** Runs the **dotnet restore** command to restore any NuGet packages or dependencies required by the project. This will only run within the container for the copied .csproj file.
- **dotnet restore:** Fetches all necessary dependencies listed in the .csproj file, enabling the project to be built successfully.

Copy Remaining Source Code:

`COPY . .`

- **Purpose:** Copies all the remaining source code from the local directory to the container. The source files are copied to the /src directory (which is the current working directory).
- **COPY . .:** The first . refers to the current directory on the host, and the second . refers to the current working directory in the container.

Set Working Directory for Project:

`WORKDIR "/src/ProductsMicroService.API"`

- **Purpose:** Switches the working directory to the folder containing the actual application, ProductsMicroService.API.

Build the Application:

`RUN dotnet build "./ProductsMicroService.API.csproj" -c $BUILD_CONFIGURATION -o /app/build`

- **Purpose:** Runs the dotnet build command to compile the project in the specified build configuration (Release by default). The build output is placed in the /app/build directory inside the container.
- **\$BUILD_CONFIGURATION:** Refers to the argument specified earlier (default is Release).

Publish the Application:

FROM build AS publish

ARG BUILD_CONFIGURATION=Release

RUN dotnet publish "./ProductsMicroService.API.csproj" -c \$BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false

- **FROM build AS publish:** Starts a new stage, using the result of the previous build stage as the base. This is where the final publish step takes place.
- **dotnet publish:** Publishes the application in the specified configuration (default is Release), generating optimized files ready for deployment. The files are placed in the /app/publish directory.
- **/p=false:** Disables the use of the application host, which can help in reducing the image size.

Final Stage:

FROM base AS final

WORKDIR /app

COPY --from=publish /app/publish .

- **FROM base AS final:** Uses the base stage (with the runtime) to run the final container. This is the **multi-stage build** where only the runtime is used for efficiency.
- **WORKDIR /app:** Sets the working directory to /app, where the application will run.
- **COPY --from=publish /app/publish .:** Copies the output of the publish stage into the /app directory in the final container.

Environment Variables:

ENV MYSQL_HOST=localhost

ENV MYSQL_PASSWORD=admin

- **Purpose:** Sets environment variables MYSQL_HOST and MYSQL_PASSWORD, which can be used by the application at runtime. These environment variables are used to configure database connectivity.
- **ENV:** Defines environment variables inside the container.

Entry Point:

ENTRYPOINT ["dotnet", "ProductsMicroService.API.dll"]

- **Purpose:** Specifies the **entry point** for the container when it starts. This tells Docker to execute the command `dotnet ProductsMicroService.API.dll`, which starts the .NET application.
- **ENTRYPOINT:** Defines the main command that runs when the container is launched.

Summary:

- This **multi-stage Dockerfile** builds, publishes, and runs a .NET Core microservice.
- The build and publish stages use the .NET SDK image to compile the application.
- The final stage uses a lightweight runtime image (`aspnet:8.0`) to run the compiled app.
- It sets up necessary environment variables and exposes ports for external access.

.NET Microservices – Azure DevOps and AKS

Section 5: Docker Compose - Notes

Introduction to Docker Compose

Docker Compose is a tool that simplifies the management of multi-container Docker applications. It allows you to define and run applications with multiple services (such as web servers, databases, caching layers, etc.) using a single configuration file. Instead of manually running and linking several containers, Docker Compose orchestrates everything with one command.

Docker Compose uses a YAML file (`docker-compose.yml`) to configure the services, networks, and volumes that the application needs. With Compose, all the complexity of setting up and managing multiple Docker containers is abstracted into a single file, making it much easier to manage and scale applications.

Key Use Cases of Docker Compose

1. **Multi-Container Applications:** Many modern applications require multiple services to function, such as databases, APIs, message brokers, etc. Docker Compose allows you to define and link these services easily.
2. **Development Environments:** Compose is widely used in development to simulate a full stack of services locally. Developers can spin up containers for web apps, databases, and message queues with a single command.
3. **Testing and CI/CD:** In continuous integration pipelines, Docker Compose is often used to quickly spin up test environments, where the application and all its dependencies can be run in containers. This ensures consistent environments for testing.

Benefits of Docker Compose

1. **Simplified Multi-Container Management**
 - Without Docker Compose, managing multiple services requires manually running several Docker commands for each service, linking containers, and managing their lifecycles. With Compose, all services are defined in a YAML file, and one command (`docker-compose up`) can start and configure the entire application stack.
2. **Declarative Service Definitions**
 - The `docker-compose.yml` file provides a declarative way to define services, volumes, and networks. This makes it easy to version control your container setups and share configurations across teams.

3. Consistent Environments

- Compose ensures that the services defined in your YAML file behave the same way across different environments, whether it's a local machine or a production environment. This reduces the chance of environment-specific bugs.

4. Easy Networking

- Docker Compose automatically sets up networks for services so they can communicate with each other. Services are identified by their names, making it easy to create service dependencies. For example, your web app can simply reference the database service by name to connect.

5. Scalability

- Docker Compose allows you to scale services effortlessly. You can specify how many instances of a service you want to run, and Compose will handle the networking and load balancing between them. This is useful for simulating production-like environments.

6. Volume Management

- Compose provides built-in support for persistent storage with volumes, ensuring that data remains available even when containers are stopped and restarted. This is crucial for services like databases where data persistence is required.

7. Isolation

- Compose keeps services isolated from each other unless explicitly defined to communicate. This ensures that different services don't interfere with each other unless necessary, improving the stability of your environment.

How Docker Compose Works

1. **Defining the Application:** All services, networks, and volumes are defined in a single `docker-compose.yml` file.
2. **Starting Services:** With the command `docker-compose up`, Docker Compose reads the YAML file and brings up all defined services. It ensures that any necessary networks and volumes are created, and it starts the containers.
3. **Stopping Services:** The command `docker-compose down` stops and removes all running containers and networks defined by the Compose file. Optionally, you can also remove volumes and images created by Compose.

Real-World Example

Let's say you're developing an e-commerce application that consists of the following services:

- **Web Server:** Runs your application code (e.g., Node.js or ASP.NET Core).
- **Database:** Stores the application's data (e.g., MySQL or PostgreSQL).

- **Cache:** Speeds up the application by caching data (e.g., Redis).

Instead of manually configuring each of these services and their networks, you can define them in a `docker-compose.yml` file. This file will specify how these containers should run, which ports to expose, how they communicate, and how their data is persisted. One command, `docker-compose up`, will start the entire application with all these services working together.

Advantages Over Docker Run

While Docker commands can individually create, start, and stop containers, Docker Compose provides a higher level of abstraction for orchestrating multiple containers together. Key differences:

1. **Single Command:** Docker Compose eliminates the need to issue multiple `docker run` commands. Everything is defined in a configuration file, and you only need one command to start the entire application.
2. **Easy Maintenance:** Making changes to your environment (e.g., adding a new service, modifying a configuration) is easier with Docker Compose. You modify the YAML file and simply restart the services. Docker Compose handles container creation, destruction, and updates without requiring complex commands.

Docker Compose YAML File and Important Properties

The core of Docker Compose is the **`docker-compose.yml`** file. This file contains the configuration for all the services, networks, and volumes that are needed for the application. It is written in **YAML (YAML Ain't Markup Language)**, which is a human-readable data format, making it easy to define the setup for multi-container applications.

Let's dive into the structure of a Docker Compose file and the most important properties.

Basic Structure of `docker-compose.yml`

A typical `docker-compose.yml` file contains three main sections:

1. **Version:** Specifies the version of Docker Compose file format.
2. **Services:** Defines the individual containers (e.g., web, database, cache) that make up the application.
3. **Networks and Volumes:** Optionally defines custom networks and volumes for inter-service communication and data persistence.

Here's an example of a basic Docker Compose file:

```
version: '3.8' # Specifies the version of the Docker Compose file format
```

```
services:    # Defines all the services (containers)

web:         # Defines a service named 'web'

  image: nginx:latest # Pulls the NGINX image from Docker Hub

  ports:

    - "8080:80"      # Maps port 80 in the container to port 8080 on the host machine


db:          # Defines a service named 'db' (database)

  image: mysql:5.7    # Pulls the MySQL image from Docker Hub

  environment:        # Passes environment variables to the container

    MYSQL_ROOT_PASSWORD: example


networks:    # Optionally define custom networks

default:     # 'default' network is created automatically

  driver: bridge # Default network driver is bridge (used for inter-container communication)
```

Key Properties in docker-compose.yml

1. version

version: '3.8'

- Specifies the version of Docker Compose syntax being used.
- As of now, 3.8 is commonly used, and it provides a set of features for modern Compose files.
- Ensure compatibility between the Docker engine and Docker Compose version by checking which versions support the specific features you need.

2. services

3. services:

4. web:

5. image: nginx:latest

6. ports:

- "8080:80"

- **Services** are the individual containers that form your multi-container application. Each service has its own configuration like which Docker image to use, how to expose ports, and environment variables.
- **image**: Specifies the Docker image to use for this service. If the image doesn't exist locally, it is pulled from a Docker registry like Docker Hub.
- **ports**: Maps ports between the host and the container. In the example, port 8080 on the host is mapped to port 80 in the NGINX container (commonly used for web traffic).

7. **build**

8. services:

9. web:

10. build:

11. context: ./app

dockerfile: Dockerfile

- Instead of pulling an image from a registry, you can **build** your own image directly within the Compose file.
- **context**: Specifies the build context (i.e., the directory containing your Dockerfile and source code).
- **dockerfile**: Points to a specific Dockerfile, in case it's not located in the root directory of the build context.

12. **environment**

13. services:

14. db:

15. image: mysql:5.7

16. environment:

MYSQL_ROOT_PASSWORD: example

- The **environment** property allows you to pass environment variables into your container. These variables are often required for configuring services like databases (e.g., setting up passwords or connection details).
- Environment variables defined here are equivalent to using the `-e` flag with `docker run`.

17. **volumes**

18. services:

19. web:

20. volumes:

- ./code:/var/www/html

- **Volumes** allow you to mount a directory from the host machine into the container. This is useful for persisting data or for sharing source code between the host and the container.
- In this example, the local directory ./code is mounted to /var/www/html inside the container.
- Volumes are often used with databases to persist data, so it remains intact even if the container is restarted.

21. **depends_on**

22. services:

23. web:

24. depends_on:

- db

- The **depends_on** property defines dependencies between services. In this example, the web service depends on the db (database) service. This ensures that Docker Compose starts the db service before the web service.
- However, **depends_on** does not wait for the database service to be ready—it only ensures that it is started first. For readiness checks, you may need to implement health checks.

25. **networks**

26. networks:

27. app-network:

driver: bridge

- Docker Compose can define custom **networks** to control how containers communicate with each other.
- By default, services within the same Compose file are placed in a shared network, but you can also define your own.
- **driver**: Specifies the type of network driver (e.g., bridge, overlay).

28. **volumes (external)**

29. volumes:

30. data-volume:

external: true

- You can define **external volumes**, which are shared across multiple Docker Compose projects or containers. These volumes are not deleted when the services are taken down.

Example of a Comprehensive docker-compose.yml

version: '3.8'

services:

web:

build:

context: .

dockerfile: Dockerfile

ports:

- "8080:80"

depends_on:

- db

environment:

- APP_ENV=development

db:

image: mysql:5.7

environment:

MYSQL_ROOT_PASSWORD: example

volumes:

- db-data:/var/lib/mysql

networks:

app-network:

driver: bridge

volumes:

db-data:

external: false

- **web** service is built from a Dockerfile, exposing port 8080 and depending on the db service.
- **db** service uses the MySQL image, with a password passed through environment variables, and data is persisted using a volume (db-data).
- **networks**: Custom network app-network is defined to allow services to communicate.
- **volumes**: A local volume db-data is created to store MySQL data.

Benefits of Using docker-compose.yml

1. **Simplified Configuration**: All services, volumes, and networks are defined in one file, making the infrastructure easier to understand and modify.
2. **Portability**: The Compose file can be shared across teams or environments, ensuring consistency across different setups.
3. **Service Isolation**: Compose isolates services by default, allowing only defined services to communicate with each other.
4. **Easy Scaling**: With docker-compose, scaling services is as easy as adding scale parameters or adjusting the YAML file.
5. **Automation**: Docker Compose abstracts complex setups, reducing the number of manual Docker commands needed to start multi-container environments.

Docker Compose Commands in Terminal (Windows)

Docker Compose provides a powerful command-line interface (CLI) to manage multi-container applications. These commands help you define, run, stop, and manage your services as described in the docker-compose.yml file.

Let's go through the most commonly used Docker Compose commands, particularly in the Windows terminal.

1. docker-compose up

The docker-compose up command is the most fundamental one. It creates and starts all services defined in your docker-compose.yml file.

docker-compose up

- **How it works:**
 - It reads the docker-compose.yml file and starts all the containers in the order specified by depends_on if defined.
 - If any services are not built (e.g., if they use a build directive), Compose will build the necessary images.
 - If the services are already running, the command will leave them as-is, unless changes have been made.
- **Flags:**
 - -d: Runs the containers in detached mode (i.e., in the background).

docker-compose up -d

Running in detached mode lets you continue using the terminal while the services run in the background.

- --build: Forces a rebuild of the service images before starting the containers.

docker-compose up --build

This is helpful when you've made changes to the Dockerfile or source code and need to rebuild the service images.

- --force-recreate: Recreates all containers even if their configuration or images haven't changed.

docker-compose up --force-recreate

Use this when you need to refresh your container environment without changing the underlying configuration.

2. docker-compose down

The docker-compose down command stops and removes all containers, networks, and volumes that were created by docker-compose up.

docker-compose down

- **How it works:**

- This command stops all the services and then removes the containers.
- By default, it doesn't remove any volumes unless explicitly specified.

- **Flags:**

- `--volumes`: Removes all volumes that were created by Docker Compose.

`docker-compose down --volumes`

This is useful when you want to clear out all persistent data (e.g., databases) and start fresh.

- `--rmi all`: Removes all the images created by the docker-compose setup.

`docker-compose down --rmi all`

This is useful if you want to free up disk space by removing unused images.

3. docker-compose build

The `docker-compose build` command builds or rebuilds the images for services defined in the Compose file. This is particularly useful if you're developing locally and need to continuously build and test changes.

`docker-compose build`

- **How it works:**

- It reads the build context in the `docker-compose.yml` file and builds Docker images for the specified services.

- **Flags:**

- `--no-cache`: Forces Docker Compose to build the images without using the cache.

`docker-compose build --no-cache`

This can be used when changes to the Dockerfile aren't being picked up, or when you want to ensure everything is built from scratch.

- `--pull`: Always attempts to pull the latest version of the base images before building.

`docker-compose build --pull`

4. docker-compose stop

The `docker-compose stop` command stops all running containers without removing them.

`docker-compose stop`

- **How it works:**

- Unlike docker-compose down, which stops and removes containers, docker-compose stop only stops the containers. They remain in a stopped state and can be restarted later using docker-compose start.

5. docker-compose start

The docker-compose start command starts existing containers that were stopped with docker-compose stop.

docker-compose start

- **How it works:**

- This is used to bring back stopped containers without recreating or rebuilding them.

6. docker-compose logs

The docker-compose logs command displays the logs from all containers defined in the docker-compose.yml file. This is useful for debugging and monitoring services in real time.

docker-compose logs

- **Flags:**

- -f: Follows the log output in real-time (similar to tail -f).

docker-compose logs -f

This flag is very useful when monitoring the live behavior of your services.

7. docker-compose ps

The docker-compose ps command shows the status of the containers created by Docker Compose.

docker-compose ps

- **How it works:**

- It lists all the running containers, their states (e.g., running, exited), ports exposed, and the command used to start them.
- This command helps in identifying if a service is running as expected or has crashed.

8. docker-compose exec

The docker-compose exec command allows you to run arbitrary commands inside a running service container. This is useful for debugging and inspecting containers.

docker-compose exec <service-name> <command>

- **Example:**

docker-compose exec web bash

- This command opens an interactive Bash shell inside the web container. You can then inspect the file system, logs, or manually run commands.

9. docker-compose scale

The docker-compose scale command allows you to scale a specific service up or down. This means you can increase or decrease the number of container instances for a given service.

docker-compose up --scale <service-name>=<number-of-instances>

- **Example:**

docker-compose up --scale web=3

- This command starts three instances of the web service and balances them across the defined network.
- **Note:** Scaling is useful in development to simulate production-like environments with multiple instances of the same service (e.g., for load balancing).

10. docker-compose config

The docker-compose config command validates and displays the configuration file. This is helpful to ensure that the docker-compose.yml file is correct before running any commands.

docker-compose config

- **How it works:**

- It parses the docker-compose.yml file and checks for syntax errors, missing properties, or deprecated options.

11. docker-compose pull

The docker-compose pull command pulls the latest version of images defined in the Compose file. It is useful when you want to make sure that you're using the most up-to-date images for services that don't use local builds.

```
docker-compose pull
```

12. docker-compose restart

The docker-compose restart command restarts all or specific services. It's a useful command to refresh services without taking down the entire environment.

```
docker-compose restart <service-name>
```

Common Workflow Using Docker Compose Commands

1. Start Your Services:

```
docker-compose up -d
```

2. Check Running Containers:

```
docker-compose ps
```

3. Monitor Logs:

```
docker-compose logs -f
```

4. Execute a Command in a Service:

```
docker-compose exec <service> <command>
```

5. Scale Services:

```
docker-compose up --scale web=3
```

6. Stop and Remove All Services:

```
docker-compose down --volumes
```


Key Points to Remember (for Interview Preparation)

- **docker-compose up** starts and creates services, networks, and volumes defined in docker-compose.yml.
- **docker-compose down** stops and removes services, networks, and volumes, unless volumes are preserved explicitly.
- **docker-compose build** builds services from a Dockerfile if defined.
- **docker-compose exec** allows you to run commands inside a running container.

Scaling is possible using the `--scale` flag to simulate load balancing and multi-instance setups.

Log monitoring and **service inspection** are done using `docker-compose logs` and `docker-compose ps`.

Sample Code

version: "3.8"

services:

mysql-container:

image: mysql:8.3.0

environment:

- MYSQL_ROOT_PASSWORD=admin

ports:

- "3306:3306"

volumes:

- ./mysql-init:/docker-entrypoint-initdb.d

networks:

- ecommerce-network

hostname: mysql-host-productsmicroservice

products-microservice:

image: harshamicroservices/ecommerce-products-microservice:v1.0

environment:

MYSQL_HOST: mysql-host-productsmicroservice

MYSQL_PASSWORD: admin

ports:

- "8080:8080"

networks:

- ecommerce-network

postgres-container:

image: postgres:16.1

environment:

- POSTGRES_USER=postgres
- POSTGRES_PASSWORD=admin
- POSTGRES_DB=eCommerceUsers

ports:

- "5432:5432"

volumes:

- ./postgres-init:/docker-entrypoint-initdb.d

networks:

- ecommerce-network

users-microservice:

image: harshamicroservices/ecommerce-users-microservice:v1.0

environment:

- POSTGRES_HOST=postgres-container
- POSTGRES_PASSWORD=admin

ports:

- "9090:9090"

networks:

- ecommerce-network

networks:

ecommerce-network:

driver: bridge

Sample Code Explanation

1. version: "3.8"

- **Explanation:**
 - Specifies the version of Docker Compose syntax used in this file. Version 3.8 is compatible with newer Docker Compose features.
 - Each version brings new functionalities and deprecates older ones, so using the correct version ensures compatibility with the Docker Engine and features like networks and volumes.

2. services:

- **Explanation:**
 - This is the main section where you define your application services. Each service runs in its own container, and you can configure how they interact with each other.

MySQL Service Definition (mysql-container):

3. mysql-container:

- **Explanation:**
 - This defines a container named mysql-container that will run the MySQL database.
 - Services are given unique names, and this will be used as a reference elsewhere (like in networking or linking).

4. image: mysql:8.3.0

- **Explanation:**
 - This line pulls the MySQL Docker image version 8.3.0 from Docker Hub. This container will run a MySQL database instance.

5. environment:

- **Explanation:**
 - Environment variables are used to pass configuration parameters to the container.

6. - MYSQL_ROOT_PASSWORD=admin

- **Explanation:**
 - This sets the environment variable MYSQL_ROOT_PASSWORD to admin, which configures the root password for the MySQL server.

7. ports:

- **Explanation:**
 - This maps ports between the container and the host machine so that the service is accessible outside of the Docker network.

8. - "3306:3306"

- **Explanation:**
 - The first 3306 is the port on the host, and the second 3306 is the port inside the container. This mapping exposes MySQL on port 3306 (the default MySQL port) to the host machine.
 - You can connect to the MySQL instance on localhost:3306 from the host.

9. volumes:

- **Explanation:**
 - This section mounts volumes to persist data or execute scripts. Volumes allow files from the host system to be made available inside the container.

10. - ./mysql-init:/docker-entrypoint-initdb.d

- **Explanation:**
 - This maps the ./mysql-init directory on the host to /docker-entrypoint-initdb.d inside the container.
 - The MySQL container will run any SQL scripts placed in this directory during initialization, which is a common way to pre-load the database with schema or seed data.

11. networks:

- **Explanation:**
 - Defines the network that the service will be a part of. Networking allows containers to communicate with each other.

12. - ecommerce-network

- **Explanation:**
 - Specifies that this container is part of the ecommerce-network, enabling communication with other containers on this network.

13. hostname: mysql-host-productsmicroservice

- **Explanation:**
 - Sets the hostname of the MySQL container to mysql-host-productsmicroservice, which other services (like the Products Microservice) can use to connect to this container.

Products Microservice (products-microservice):

14. products-microservice:

- **Explanation:**
 - Defines a service named products-microservice, which is a microservice for handling product-related functionality in the eCommerce system.

15. image: harshamicroservices/ecommerce-products-microservice:v1.0

- **Explanation:**
 - This service will run a pre-built Docker image harshamicroservices/ecommerce-products-microservice:v1.0 (presumably hosted on Docker Hub or a private repository). This image is for the Products Microservice.

16. environment:

- **Explanation:**
 - Passes environment variables required by the service to function properly.

17. MYSQL_HOST: mysql-host-productsmicroservice

- **Explanation:**
 - This sets the MYSQL_HOST environment variable to mysql-host-productsmicroservice, which was defined as the hostname for the MySQL container. This tells the microservice where to connect to the MySQL database.

18. MYSQL_PASSWORD: admin

- **Explanation:**
 - The MYSQL_PASSWORD environment variable is set to admin, which is the password to connect to the MySQL database. This should match the password used in the MySQL container.

19. ports:

- **Explanation:**
 - Maps the internal port of the service to the host machine.

20. - "8080:8080"

- **Explanation:**
 - Maps port 8080 inside the container to port 8080 on the host machine. This exposes the Products Microservice to localhost:8080.

21. networks:

- **Explanation:**
 - The products-microservice is also connected to the ecommerce-network, enabling communication with the MySQL container.

PostgreSQL Service Definition (postgres-container):

22. postgres-container:

- **Explanation:**
 - Defines a service named postgres-container that will run a PostgreSQL database instance for managing user data.

23. image: postgres:16.1

- **Explanation:**
 - This line pulls the PostgreSQL Docker image version 16.1 from Docker Hub.

24. environment:

- **Explanation:**
 - Sets the necessary environment variables for PostgreSQL.

25. - POSTGRES_USER=postgres

- **Explanation:**
 - This sets the POSTGRES_USER environment variable, specifying the username to be used when connecting to PostgreSQL. The username is postgres.

26. - POSTGRES_PASSWORD=admin

- **Explanation:**
 - This sets the POSTGRES_PASSWORD environment variable, which is the password to authenticate the POSTGRES_USER.

27. - POSTGRES_DB=eCommerceUsers

- **Explanation:**
 - Specifies the name of the database that will be created by default, which in this case is eCommerceUsers.

28. ports:

- **Explanation:**
 - Maps the internal PostgreSQL port to the host machine.

29. - "5432:5432"

- **Explanation:**
 - Maps port 5432 inside the container to port 5432 on the host machine. PostgreSQL is now accessible via localhost:5432.

30. volumes:

- **Explanation:**
 - Mounts the `./postgres-init` directory on the host to the `/docker-entrypoint-initdb.d` directory in the container. This allows initialization scripts for PostgreSQL.

31. - `./postgres-init:/docker-entrypoint-initdb.d`

- **Explanation:**
 - Any SQL scripts inside the `./postgres-init` folder on the host will be executed when the container initializes.

32. networks:

- **Explanation:**
 - Connects the PostgreSQL container to the `ecommerce-network` so that it can communicate with other services.

Users Microservice (users-microservice):

33. users-microservice:

- **Explanation:**
 - Defines a service named `users-microservice` for handling user-related functionality in the eCommerce system.

34. image: `harshamicroservices/ecommerce-users-microservice:v1.0`

- **Explanation:**
 - This service will run a pre-built Docker image for the Users Microservice, pulled from the Docker registry.

35. environment:

- **Explanation:**
 - Sets environment variables for connecting to the PostgreSQL database.

36. - `POSTGRES_HOST=postgres-container`

- **Explanation:**

- This sets the POSTGRES_HOST environment variable to postgres-container, allowing the microservice to connect to the PostgreSQL container.

37. - POSTGRES_PASSWORD=admin

- **Explanation:**
 - The POSTGRES_PASSWORD environment variable is set to admin, which should match the password defined in the postgres-container.

38. ports:

- **Explanation:**
 - Exposes the Users Microservice on port 9090.

39. - "9090:9090"

- **Explanation:**
 - Maps port 9090 in the container to port 9090 on the host machine, allowing access to the Users Microservice at localhost:9090.

40. networks:

- **Explanation:**
 - Connects the users-microservice to the ecommerce-network, allowing it to communicate with the PostgreSQL container and other services.

.NET Microservices – Azure DevOps and AKS

Section 6: Orders Microservice - Notes

Orders Microservice:

Code Explanation - Entities

Order Class:

Purpose: The Order class represents an order in an eCommerce system, including details like order ID, user ID, order date, total bill, and a list of items included in the order.

using MongoDB.Bson.Serialization.Attributes;

namespace eCommerce.OrdersMicroservice.DataAccessLayer.Entities

{

 // Represents an order in the system

 public class Order

 {

 // The unique identifier for the document in MongoDB

 [BsonId]

 [BsonRepresentation(MongoDB.Bson.BsonType.String)]

 public Guid _id { get; set; }

 // Unique identifier for the order

 [BsonRepresentation(MongoDB.Bson.BsonType.String)]

 public Guid OrderID { get; set; }

 // Unique identifier for the user who placed the order

 [BsonRepresentation(MongoDB.Bson.BsonType.String)]

 public Guid UserID { get; set; }

 // The date when the order was placed

 [BsonRepresentation(MongoDB.Bson.BsonType.String)]

```

public DateTime OrderDate { get; set; }

// The total amount billed for the order
[BsonRepresentation(MongoDB.Bson.BsonType.Double)]
public decimal TotalBill { get; set; }

// List of items included in the order
public List<OrderItem> OrderItems { get; set; } = new List<OrderItem>();
}
}

```

Explanation:

1. **using MongoDB.Bson.Serialization.Attributes;**
 - This directive allows the use of MongoDB-specific attributes that help in mapping C# classes to MongoDB documents.
2. **namespace eCommerce.OrdersMicroservice.DataAccessLayer.Entities**
 - Defines the namespace for organizing code and preventing name conflicts.
3. **public class Order**
 - Declares a public class named Order. The public access modifier means that this class can be accessed from other assemblies.
4. **[BsonId]**
 - Specifies that the following property is the primary key of the MongoDB document. MongoDB uses this to uniquely identify each document.
5. **[BsonRepresentation(MongoDB.Bson.BsonType.String)]**
 - Indicates that the Guid property should be stored as a string in MongoDB. This helps with the representation of Guid in MongoDB, which doesn't have a direct Guid type.
6. **public Guid _id { get; set; }**
 - Declares a property named _id of type Guid. This serves as the primary key for the MongoDB document.
7. **[BsonRepresentation(MongoDB.Bson.BsonType.String)]**
 - Again, specifies that the following property should be stored as a string in MongoDB.
8. **public Guid OrderID { get; set; }**
 - Declares a property for storing the unique order identifier. This is the ID that identifies an order within the system.

9. **[BsonRepresentation(MongoDB.Bson.BsonType.String)]**

- Specifies that the Guid should be stored as a string in MongoDB.

10. **public Guid UserID { get; set; }**

- Declares a property for storing the unique user identifier who placed the order.

11. **[BsonRepresentation(MongoDB.Bson.BsonType.String)]**

- Specifies that the DateTime should be stored as a string in MongoDB.

12. **public DateTime OrderDate { get; set; }**

- Declares a property for storing the date when the order was placed.

13. **[BsonRepresentation(MongoDB.Bson.BsonType.Double)]**

- Specifies that the decimal should be stored as a double in MongoDB. MongoDB does not have a direct decimal type, so double is used as an approximation.

14. **public decimal TotalBill { get; set; }**

- Declares a property for storing the total bill amount for the order.

15. **public List<OrderItem> OrderItems { get; set; } = new List<OrderItem>();**

- Declares a property for storing a list of OrderItem objects associated with the order. Initializes it with an empty list to ensure it's never null.

OrderItem Class:

Purpose: The OrderItem class represents an individual item within an order, including details like the product ID, unit price, quantity, and total price.

using MongoDB.Bson.Serialization.Attributes;

namespace eCommerce.OrdersMicroservice.DataAccessLayer.Entities

{

 // Represents an item in an order

 public class OrderItem

 {

 // The unique identifier for the document in MongoDB

 [BsonId]

 [BsonRepresentation(MongoDB.Bson.BsonType.String)]

 public Guid _id { get; set; }

```

// Unique identifier for the product
[BsonRepresentation(MongoDB.Bson.BsonType.String)]
public Guid ProductID { get; set; }

// The price of one unit of the product
[BsonRepresentation(MongoDB.Bson.BsonType.Double)]
public decimal UnitPrice { get; set; }

// The quantity of the product ordered
[BsonRepresentation(MongoDB.Bson.BsonType.Int32)]
public int Quantity { get; set; }

// The total price for the ordered quantity of the product
[BsonRepresentation(MongoDB.Bson.BsonType.Double)]
public decimal TotalPrice { get; set; }
}
}

```

Explanation:

1. **using MongoDB.Bson.Serialization.Attributes;**
 - Imports the necessary attributes for mapping C# properties to MongoDB document fields.
2. **namespace eCommerce.OrdersMicroservice.DataAccessLayer.Entities**
 - Defines the namespace for the OrderItem class.
3. **public class OrderItem**
 - Declares a public class named OrderItem. This class represents an item within an order.
4. **[BsonId]**
 - Marks the following property as the primary key for the MongoDB document.
5. **[BsonRepresentation(MongoDB.Bson.BsonType.String)]**
 - Specifies that the Guid property should be represented as a string in MongoDB.

6. **public Guid _id { get; set; }**
 - Declares a property named _id to serve as the primary key for the OrderItem document.
7. **[BsonRepresentation(MongoDB.Bson.BsonType.String)]**
 - Specifies that the Guid should be represented as a string in MongoDB.
8. **public Guid ProductID { get; set; }**
 - Declares a property for storing the unique identifier of the product.
9. **[BsonRepresentation(MongoDB.Bson.BsonType.Double)]**
 - Specifies that the decimal value should be stored as a double in MongoDB.
10. **public decimal UnitPrice { get; set; }**
 - Declares a property for storing the price per unit of the product.
11. **[BsonRepresentation(MongoDB.Bson.BsonType.Int32)]**
 - Specifies that the int value should be stored as an integer in MongoDB.
12. **public int Quantity { get; set; }**
 - Declares a property for storing the quantity of the product ordered.
13. **[BsonRepresentation(MongoDB.Bson.BsonType.Double)]**
 - Specifies that the decimal value should be stored as a double in MongoDB.
14. **public decimal TotalPrice { get; set; }**
 - Declares a property for storing the total price for the ordered quantity of the product.

Code Explanation - Repository Contracts

IOrdersRepository Interface

Purpose: The IOrdersRepository interface defines the contract for interacting with the Order collection in the data access layer. It provides methods for CRUD operations and querying the Order collection.

```
using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;
```

```
using MongoDB.Driver;
```

```
namespace eCommerce.OrdersMicroservice.DataAccessLayer.RepositoryContracts
```

```
{
```

```
// Defines the contract for operations related to Orders in the data layer
public interface IOrdersRepository
{
    /// <summary>
    /// Retrieves all Orders asynchronously
    /// </summary>
    /// <returns>Returns all orders from the orders collection</returns>
    Task<IEnumerable<Order>> GetOrders();

    /// <summary>
    /// Retrieves all Orders based on the specified condition asynchronously
    /// </summary>
    /// <param name="filter">The condition to filter orders</param>
    /// <returns>Returning a collection of matching orders</returns>
    Task<IEnumerable<Order?>> GetOrdersByCondition(FilterDefinition<Order> filter);

    /// <summary>
    /// Retrieves a single order based on the specified condition asynchronously
    /// </summary>
    /// <param name="filter">The condition to filter Orders</param>
    /// <returns>Returning matching order</returns>
    Task<Order?> GetOrderByCondition(FilterDefinition<Order> filter);

    /// <summary>
    /// Adds a new Order into the Orders collection asynchronously
    /// </summary>
    /// <param name="order">The order to be added</param>
    /// <returns>Returns the added Order object or null if unsuccessful</returns>
    Task<Order?> AddOrder(Order order);

    /// <summary>
```

```

    /// Updates an existing order asynchronously
    /// </summary>
    /// <param name="order">The order to be added</param>
    /// <returns>Returns the updated order object; or null if not found</returns>
    Task<Order?> UpdateOrder(Order order);

    /// <summary>
    /// Deletes the order asynchronously
    /// </summary>
    /// <param name="orderId">The Order ID based on which we need to delete the
    order</param>
    /// <returns>Returns true if the deletion is successful, false otherwise</returns>
    Task<bool> DeleteOrder(Guid orderId);
}
}

```

Explanation:

1. **using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;**
 - Imports the Order class from the Entities namespace to be used in this interface.
2. **using MongoDB.Driver;**
 - Imports MongoDB's driver library for using MongoDB-specific types and operations.
3. **namespace eCommerce.OrdersMicroservice.DataAccessLayer.RepositoryContracts**
 - Defines the namespace for organizing repository contract interfaces.
4. **public interface IOrdersRepository**
 - Declares a public interface named IOrdersRepository. Interfaces define contracts for classes that implement them.
5. **/// <summary> ... /// <returns>**
 - XML comments used to document the purpose and behavior of each method in the interface. These comments are used by documentation tools and IDEs to provide descriptions and generate API documentation.
6. **Task<IEnumerable<Order>> GetOrders();**
 - Defines a method to asynchronously retrieve all orders. Returns a Task that, when completed, will provide an IEnumerable<Order> representing all orders in the collection.

7. **Task<IEnumerable<Order?>> GetOrdersByCondition(FilterDefinition<Order> filter);**
 - Defines a method to asynchronously retrieve orders that match a specific filter condition. Uses FilterDefinition<Order> from MongoDB to specify the filter.
8. **Task<Order?> GetOrderByCondition(FilterDefinition<Order> filter);**
 - Defines a method to asynchronously retrieve a single order that matches a filter condition. Returns a single Order object or null if no matching order is found.
9. **Task<Order?> AddOrder(Order order);**
 - Defines a method to asynchronously add a new order to the collection. Returns the added Order object or null if the addition was unsuccessful.
10. **Task<Order?> UpdateOrder(Order order);**
 - Defines a method to asynchronously update an existing order. Returns the updated Order object or null if the order was not found.
11. **Task<bool> DeleteOrder(Guid orderID);**
 - Defines a method to asynchronously delete an order based on its OrderID. Returns true if the deletion was successful, otherwise false.

Code Explanation - Repository

OrdersRepository Class

```
using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;  
using eCommerce.OrdersMicroservice.DataAccessLayer.RepositoryContracts;  
using MongoDB.Driver;
```

```
namespace eCommerce.OrdersMicroservice.DataAccessLayer.Repositories
```

```
{
```

```
    public class OrdersRepository : IOrdersRepository
```

```
    {
```

```
        private readonly IMongoCollection<Order> _orders;
```

```
        private readonly string collectionName = "orders";
```

```
        public OrdersRepository(IMongoDatabase mongoDatabase)
```

```
        {
```

```

        _orders = mongoDatabase.GetCollection<Order>(collectionName);
    }

    public async Task<Order?> AddOrder(Order order)
    {
        order.OrderID = Guid.NewGuid(); // Assign a new unique OrderID
        order._id = order.OrderID; // Set _id to the same value as OrderID

        foreach (OrderItem orderItem in order.OrderItems)
        {
            orderItem._id = Guid.NewGuid(); // Assign a new unique ID to each order item
        }

        await _orders.InsertOneAsync(order); // Insert the order into the collection
        return order; // Return the inserted order
    }

    public async Task<bool> DeleteOrder(Guid orderID)
    {
        FilterDefinition<Order> filter = Builders<Order>.Filter.Eq(temp => temp.OrderID, orderID); //
        Create filter to find order

        Order? existingOrder = (await _orders.FindAsync(filter)).FirstOrDefault(); // Find the order
        that matches the filter

        if (existingOrder == null)
        {
            return false; // Return false if the order was not found
        }

        DeleteResult deleteResult = await _orders.DeleteOneAsync(filter); // Delete the order that
        matches the filter
    }

```

```
        return deleteResult.DeletedCount > 0; // Return true if the deletion was successful, otherwise
false
    }
}
```

```
public async Task<Order?> GetOrderByCondition(FilterDefinition<Order> filter)
{
    return (await _orders.FindAsync(filter)).FirstOrDefault(); // Find and return a single order that
matches the filter
}
```

```
public async Task<IEnumerable<Order>> GetOrders()
{
    return (await _orders.FindAsync(Builders<Order>.Filter.Empty)).ToList(); // Find and return all
orders
}
```

```
public async Task<IEnumerable<Order?>> GetOrdersByCondition(FilterDefinition<Order> filter)
{
    return (await _orders.FindAsync(filter)).ToList(); // Find and return orders that match the filter
}
```

```
public async Task<Order?> UpdateOrder(Order order)
{
    FilterDefinition<Order> filter = Builders<Order>.Filter.Eq(temp => temp.OrderID,
order.OrderID); // Create filter to find order

    Order? existingOrder = (await _orders.FindAsync(filter)).FirstOrDefault(); // Find the order
that matches the filter

    if (existingOrder == null)
    {
        return null; // Return null if the order was not found
    }
}
```

```
}
```

```
order._id = existingOrder._id; // Set the _id of the updated order to the existing order's _id
```

```
ReplaceOneResult replaceOneResult = await _orders.ReplaceOneAsync(filter, order); //  
Replace the existing order with the updated order
```

```
return order; // Return the updated order
```

```
}
```

```
}
```

```
}
```

Explanation:

AddOrder Method

1. **public async Task<Order?> AddOrder(Order order)**
 - Defines an asynchronous method that returns a Task with an Order object, which may be null.
2. **order.OrderID = Guid.NewGuid();**
 - Generates a new unique identifier for the OrderID property of the order using Guid.NewGuid().
3. **order._id = order.OrderID;**
 - Sets the _id field to the same value as OrderID. In MongoDB, _id is the primary key and must be unique.
4. **foreach (OrderItem orderItem in order.OrderItems)**
 - Iterates over each OrderItem in the OrderItems list to assign unique IDs.
5. **orderItem._id = Guid.NewGuid();**
 - Generates a new unique identifier for each OrderItem.
6. **await _orders.InsertOneAsync(order);**
 - Asynchronously inserts the order into the MongoDB collection _orders.
7. **return order;**
 - Returns the inserted order object.

DeleteOrder Method

1. **public async Task<bool> DeleteOrder(Guid orderID)**
 - Defines an asynchronous method that returns a Task with a boolean value indicating success or failure.
2. **FilterDefinition<Order> filter = Builders<Order>.Filter.Eq(temp => temp.OrderID, orderID);**
 - Creates a filter to find an order based on its OrderID. The Builders<Order>.Filter.Eq method specifies an equality condition.
3. **Order? existingOrder = (await _orders.FindAsync(filter)).FirstOrDefault();**
 - Asynchronously finds the order that matches the filter. FirstOrDefault() returns the first matching order or null if no match is found.
4. **if (existingOrder == null)**
 - Checks if the existingOrder is null, meaning no order was found.
5. **return false;**
 - Returns false if the order was not found.
6. **DeleteResult deleteResult = await _orders.DeleteOneAsync(filter);**
 - Asynchronously deletes the order that matches the filter.
7. **return deleteResult.DeletedCount > 0;**
 - Returns true if the DeletedCount is greater than 0, indicating that at least one document was deleted.

GetOrderByCondition Method

1. **public async Task<Order?> GetOrderByCondition(FilterDefinition<Order> filter)**
 - Defines an asynchronous method that returns a Task with an Order object, which may be null.
2. **return (await _orders.FindAsync(filter)).FirstOrDefault();**
 - Asynchronously finds and returns the first order that matches the filter. FirstOrDefault() returns null if no match is found.

GetOrders Method

1. **public async Task<IEnumerable<Order>> GetOrders()**
 - Defines an asynchronous method that returns a Task with an IEnumerable<Order>.
2. **return (await _orders.FindAsync(Builders<Order>.Filter.Empty)).ToList();**
 - Asynchronously finds and returns all orders. Builders<Order>.Filter.Empty specifies no filter, so all documents are retrieved. ToList() converts the result to a list.

GetOrdersByCondition Method

1. **public async Task<IEnumerable<Order?>> GetOrdersByCondition(FilterDefinition<Order> filter)**
 - Defines an asynchronous method that returns a Task with an IEnumerable<Order?>.
2. **return (await _orders.FindAsync(filter)).ToList();**
 - Asynchronously finds and returns all orders that match the filter. ToList() converts the result to a list.

UpdateOrder Method

1. **public async Task<Order?> UpdateOrder(Order order)**
 - Defines an asynchronous method that returns a Task with an Order object, which may be null.
2. **FilterDefinition<Order> filter = Builders<Order>.Filter.Eq(temp => temp.OrderID, order.OrderID);**
 - Creates a filter to find an order based on its OrderID.
3. **Order? existingOrder = (await _orders.FindAsync(filter)).FirstOrDefault();**
 - Asynchronously finds the order that matches the filter. FirstOrDefault() returns the first matching order or null if no match is found.
4. **if (existingOrder == null)**
 - Checks if the existingOrder is null, meaning no order was found.
5. **return null;**
 - Returns null if the order was not found.
6. **order._id = existingOrder._id;**
 - Sets the _id field of the order to the _id of the existing order. This ensures that the same document is updated.
7. **ReplaceOneResult replaceOneResult = await _orders.ReplaceOneAsync(filter, order);**

- Asynchronously replaces the existing order with the updated order.

8. **return order;**

- Returns the updated order object.

Code Explanation - DependencyInjection in DataAccessLayer

DependencyInjection Class

The DependencyInjection class configures and registers services related to data access in the dependency injection container. This setup allows the services to be injected into other components of the application.

```
using eCommerce.OrdersMicroservice.DataAccessLayer.RepositoryContracts;
```

```
using eCommerce.OrdersMicroservice.DataAccessLayer.Repositories;
```

```
using Microsoft.Extensions.Configuration;
```

```
using Microsoft.Extensions.DependencyInjection;
```

```
using MongoDB.Driver;
```

```
namespace eCommerce.OrdersMicroservice.DataAccessLayer
```

```
{
```

```
    public static class DependencyInjection
```

```
    {
```

```
        public static IServiceCollection AddDataAccessLayer(this IServiceCollection services,
        IConfiguration configuration)
```

```
        {
```

```
            //TO DO: Add data access layer services into the IoC container
```

```
            string connectionStringTemplate = configuration.GetConnectionString("MongoDB");
```

```
            string connectionString = connectionStringTemplate
```

```
                .Replace("$MONGO_HOST", Environment.GetEnvironmentVariable("MONGODB_HOST"))
```

```
                .Replace("$MONGO_PORT", Environment.GetEnvironmentVariable("MONGODB_PORT"));
```

```
            services.AddSingleton<IMongoClient>(new MongoClient(connectionString));
```

```

services.AddScoped<IMongoDatabase>(provider =>
{
    IMongoClient client = provider.GetRequiredService<IMongoClient>();

    return
client.GetDatabase(Environment.GetEnvironmentVariable("MONGODB_DATABASE"));

});

services.AddScoped<IOrdersRepository, OrdersRepository>();

return services;
}
}
}

```

Explanation:

AddDataAccessLayer Method

1. **public static IServiceCollection AddDataAccessLayer(this IServiceCollection services, IConfiguration configuration)**
 - Defines a static extension method for IServiceCollection to register data access layer services. The method accepts IConfiguration for accessing configuration settings.
2. **string connectionStringTemplate = configuration.GetConnectionString("MongoDB")!;**
 - Retrieves the connection string template for MongoDB from the configuration settings. The ! operator is used to assert that the value is not null.
3. **string connectionString = connectionStringTemplate**
 - Initializes a new string connectionString by replacing placeholders with actual environment variable values.
4. **.Replace("\$MONGO_HOST", Environment.GetEnvironmentVariable("MONGODB_HOST"))**
 - Replaces the \$MONGO_HOST placeholder in the connection string with the value from the environment variable MONGODB_HOST.
5. **.Replace("\$MONGO_PORT", Environment.GetEnvironmentVariable("MONGODB_PORT"));**
 - Replaces the \$MONGO_PORT placeholder in the connection string with the value from the environment variable MONGODB_PORT.
6. **services.AddSingleton<IMongoClient>(new MongoClient(connectionString));**
 - Registers IMongoClient as a singleton service in the dependency injection container. MongoClient is created using the connection string. Singleton means that only one

instance of IMongoClient will be created and used throughout the application's lifetime.

7. **services.AddScoped<IMongoDatabase>(provider =>**
 - Registers IMongoDatabase as a scoped service. Scoped means that a new instance is created for each request within a scope, such as an HTTP request.
8. **IMongoClient client = provider.GetRequiredService<IMongoClient>();**
 - Retrieves the IMongoClient instance from the service provider. GetRequiredService ensures that the service is available and throws an exception if it is not.
9. **return**
client.GetDatabase(Environment.GetEnvironmentVariable("MONGODB_DATABASE"));
 - Uses the IMongoClient instance to get the MongoDB database specified by the environment variable MONGODB_DATABASE.
10. **services.AddScoped<IOrdersRepository, OrdersRepository>();**
 - Registers IOrdersRepository with its implementation OrdersRepository as a scoped service. This means that each request will receive a new instance of OrdersRepository, but the same instance will be used throughout the scope of the request.
11. **return services;**
 - Returns the IServiceCollection for chaining additional service registrations.

Code Explanation - DTO in BusinessLogicLayer

These Data Transfer Object (DTO) classes are used to transfer data between different layers of the application, such as between the API and the business logic layer. Each record class represents a different type of data structure for various operations involving orders and related entities.

OrderAddRequest Class

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;

```
public record OrderAddRequest(Guid UserID, DateTime OrderDate, List<OrderItemAddRequest>
OrderItems)
{
    public OrderAddRequest() : this(default, default, default)
    {
```

```
}  
}
```

Explanation:

- **public record OrderAddRequest(Guid UserID, DateTime OrderDate, List<OrderItemAddRequest> OrderItems)**
 - Defines a record type OrderAddRequest that contains data for creating a new order.
 - UserID: The ID of the user placing the order.
 - OrderDate: The date the order was placed.
 - OrderItems: A list of items in the order, each represented by OrderItemAddRequest.
- **public OrderAddRequest() : this(default, default, default)**
 - Defines a parameterless constructor that initializes the record with default values.

OrderItemAddRequest Class

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;

```
public record OrderItemAddRequest(Guid ProductID, decimal UnitPrice, int Quantity)  
{  
    public OrderItemAddRequest() : this(default, default, default)  
    {  
    }  
}
```

Explanation:

- **public record OrderItemAddRequest(Guid ProductID, decimal UnitPrice, int Quantity)**
 - Defines a record type OrderItemAddRequest for specifying details about an item in an order.
 - ProductID: The ID of the product.
 - UnitPrice: The price per unit of the product.
 - Quantity: The number of units of the product.
- **public OrderItemAddRequest() : this(default, default, default)**
 - Defines a parameterless constructor that initializes the record with default values.

OrderItemResponse Class

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;

```
public record OrderItemResponse(Guid ProductID, decimal UnitPrice, int Quantity, decimal
TotalPrice, string? ProductName, string? Category)
{
    public OrderItemResponse() : this(default, default, default, default, default, default)
    {
    }
}
```

Explanation:

- **public record OrderItemResponse(Guid ProductID, decimal UnitPrice, int Quantity, decimal TotalPrice, string? ProductName, string? Category)**
 - Defines a record type OrderItemResponse for returning item details in responses.
 - ProductID: The ID of the product.
 - UnitPrice: The price per unit of the product.
 - Quantity: The number of units ordered.
 - TotalPrice: The total price for the quantity ordered.
 - ProductName: The name of the product (optional).
 - Category: The category of the product (optional).
- **public OrderItemResponse() : this(default, default, default, default, default, default)**
 - Defines a parameterless constructor that initializes the record with default values.

OrderItemUpdateRequest Class

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;

```
public record OrderItemUpdateRequest(Guid ProductID, decimal UnitPrice, int Quantity)
{
    public OrderItemUpdateRequest() : this(default, default, default)
    {
    }
}
```

Explanation:

- **public record OrderItemUpdateRequest(Guid ProductID, decimal UnitPrice, int Quantity)**
 - Defines a record type OrderItemUpdateRequest for updating details of an existing order item.
 - ProductID: The ID of the product.
 - UnitPrice: The updated price per unit of the product.
 - Quantity: The updated number of units ordered.
- **public OrderItemUpdateRequest() : this(default, default, default)**
 - Defines a parameterless constructor that initializes the record with default values.

OrderResponse Class

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;

```
public record OrderResponse(Guid OrderID, Guid UserID, decimal TotalBill, DateTime OrderDate,
List<OrderItemResponse> OrderItems, string? UserPersonName, string? Email)
{
    public OrderResponse() : this(default, default, default, default, default, default, default)
    {
    }
}
```

Explanation:

- **public record OrderResponse(Guid OrderID, Guid UserID, decimal TotalBill, DateTime OrderDate, List<OrderItemResponse> OrderItems, string? UserPersonName, string? Email)**
 - Defines a record type OrderResponse for returning order details in responses.
 - OrderID: The unique ID of the order.
 - UserID: The ID of the user who placed the order.
 - TotalBill: The total amount for the order.
 - OrderDate: The date the order was placed.
 - OrderItems: A list of items in the order, each represented by OrderItemResponse.
 - UserPersonName: The name of the user who placed the order (optional).
 - Email: The email of the user who placed the order (optional).

- **public OrderResponse() : this(default, default, default, default, default, default, default, default)**
 - Defines a parameterless constructor that initializes the record with default values.

OrderUpdateRequest Class

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;

public record OrderUpdateRequest(Guid OrderID, Guid UserID, DateTime OrderDate,
List<OrderItemUpdateRequest> OrderItems)

```
{
    public OrderUpdateRequest() : this(default, default, default, default)
    {
    }
}
```

Explanation:

- **public record OrderUpdateRequest(Guid OrderID, Guid UserID, DateTime OrderDate, List<OrderItemUpdateRequest> OrderItems)**
 - Defines a record type OrderUpdateRequest for updating an existing order.
 - OrderID: The ID of the order to be updated.
 - UserID: The ID of the user placing the update.
 - OrderDate: The updated date of the order.
 - OrderItems: A list of updated items in the order, each represented by OrderItemUpdateRequest.
- **public OrderUpdateRequest() : this(default, default, default, default)**
 - Defines a parameterless constructor that initializes the record with default values.

ProductDTO Class

```
namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
```

```
public record ProductDTO(Guid ProductID, string? ProductName, string? Category, double UnitPrice, int QuantityInStock);
```

Explanation:

- **public record ProductDTO(Guid ProductID, string? ProductName, string? Category, double UnitPrice, int QuantityInStock)**
 - Defines a record type ProductDTO for transferring product details.
 - ProductID: The ID of the product.
 - ProductName: The name of the product (optional).
 - Category: The category of the product (optional).
 - UnitPrice: The price per unit of the product.
 - QuantityInStock: The quantity of the product in stock.

UserDTO Class

```
namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
```

```
public record UserDTO(Guid UserID, string? Email, string? PersonName, string Gender);
```

Explanation:

- **public record UserDTO(Guid UserID, string? Email, string? PersonName, string Gender)**
 - Defines a record type UserDTO for transferring user details.
 - UserID: The ID of the user.
 - Email: The email address of the user (optional).
 - PersonName: The name of the user (optional).
 - Gender: The gender of the user.

Code Explanation - HttpClient classes in BusinessLogicLayer

These classes, UsersMicroserviceClient and ProductsMicroserviceClient, are responsible for making HTTP requests to other microservices to fetch user and product details, respectively. Each class utilizes HttpClient to send requests and handle responses.

UsersMicroserviceClient Class

```
using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;

using System.Net.Http.Json;

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.HttpClients;

public class UsersMicroserviceClient
{
    private readonly HttpClient _httpClient;

    public UsersMicroserviceClient(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<UserDTO?> GetUserByUserID(Guid userID)
    {
        HttpResponseMessage response = await _httpClient.GetAsync($"api/users/{userID}");

        if (!response.IsSuccessStatusCode)
        {
            if (response.StatusCode == System.Net.HttpStatusCode.NotFound)
            {
                return null;
            }

            else if (response.StatusCode == System.Net.HttpStatusCode.BadRequest)
```

```
{  
    throw new HttpRequestException("Bad request", null,  
System.Net.HttpStatusCode.BadRequest);  
}  
else  
{  
    throw new HttpRequestException($"Http request failed with status code  
{response.StatusCode}");  
}  
}
```

```
UserDTO? user = await response.Content.ReadFromJsonAsync<UserDTO>();
```

```
if (user == null)  
{  
    throw new ArgumentException("Invalid User ID");  
}  
  
return user;  
}  
}
```


Explanation:

- **using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;**
 - Imports the DTO namespace to use the UserDTO class.
- **using System.Net.Http.Json;**
 - Imports the System.Net.Http.Json namespace to use extension methods like ReadFromJsonAsync.
- **public class UsersMicroserviceClient**
 - Defines a class UsersMicroserviceClient to handle HTTP requests related to user data.
- **private readonly HttpClient _httpClient;**
 - Declares a private read-only field to store an instance of HttpClient for making HTTP requests.
- **public UsersMicroserviceClient(HttpClient httpClient)**
 - Defines a constructor that initializes _httpClient with an instance provided through dependency injection.
- **public async Task<UserDTO?> GetUserByUserID(Guid userID)**
 - Defines an asynchronous method to get user details by their ID.
- **HttpResponseMessage response = await _httpClient.GetAsync(\$" /api/users/{userID}");**
 - Sends a GET request to the /api/users/{userID} endpoint to retrieve user data.
- **if (!response.IsSuccessStatusCode)**
 - Checks if the response indicates a failure (status code not in the range 200-299).
- **if (response.StatusCode == System.Net.HttpStatusCode.NotFound)**
 - Returns null if the response status code is 404 Not Found.
- **else if (response.StatusCode == System.Net.HttpStatusCode.BadRequest)**
 - Throws an HttpRequestException with a "Bad request" message if the status code is 400 Bad Request.
- **else**
 - Throws a generic HttpRequestException for other status codes indicating request failure.
- **UserDTO? user = await response.Content.ReadFromJsonAsync<UserDTO>();**
 - Deserializes the response content to an instance of UserDTO.
- **if (user == null)**

- Throws an ArgumentException if deserialization results in null, indicating an invalid user ID.
- **return user;**
 - Returns the deserialized UserDTO object.

ProductsMicroserviceClient Class

```
using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
```

```
using System.Net.Http.Json;
```

```
namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.HttpClients;
```

```
public class ProductsMicroserviceClient
```

```
{
```

```
    private readonly HttpClient _httpClient;
```

```
    public ProductsMicroserviceClient(HttpClient httpClient)
```

```
    {
```

```
        _httpClient = httpClient;
```

```
    }
```

```
    public async Task<ProductDTO?> GetProductByProductID(Guid productID)
```

```
    {
```

```
        HttpResponseMessage response = await _httpClient.GetAsync($"/api/products/search/product-id/{productID}");
```

```
        if (!response.IsSuccessStatusCode)
```

```
        {
```

```
            if (response.StatusCode == System.Net.HttpStatusCode.NotFound)
```

```
            {
```

```
                return null;
```

```
            }
```

```

else if (response.StatusCode == System.Net.HttpStatusCode.BadRequest)
{
    throw new HttpRequestException("Bad request", null,
System.Net.HttpStatusCode.BadRequest);
}
else
{
    throw new HttpRequestException($"Http request failed with status code
{response.StatusCode}");
}
}

```

```

ProductDTO? product = await response.Content.ReadFromJsonAsync<ProductDTO>();

```

```

if (product == null)
{
    throw new ArgumentException("Invalid Product ID");
}

```

```

return product;

```

```

}

```

```

}

```

Explanation:

- **using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;**
 - Imports the DTO namespace to use the ProductDTO class.
- **using System.Net.Http.Json;**
 - Imports the System.Net.Http.Json namespace to use extension methods like ReadFromJsonAsync.
- **public class ProductsMicroserviceClient**
 - Defines a class ProductsMicroserviceClient to handle HTTP requests related to product data.

- **private readonly HttpClient _httpClient;**
 - Declares a private read-only field to store an instance of HttpClient for making HTTP requests.
- **public ProductsMicroserviceClient(HttpClient httpClient)**
 - Defines a constructor that initializes _httpClient with an instance provided through dependency injection.
- **public async Task<ProductDTO?> GetProductByProductID(Guid productID)**
 - Defines an asynchronous method to get product details by their ID.
- **HttpResponseMessage response = await _httpClient.GetAsync(\$"/api/products/search/product-id/{productID}");**
 - Sends a GET request to the /api/products/search/product-id/{productID} endpoint to retrieve product data.
- **if (!response.IsSuccessStatusCode)**
 - Checks if the response indicates a failure (status code not in the range 200-299).
- **if (response.StatusCode == System.Net.HttpStatusCode.NotFound)**
 - Returns null if the response status code is 404 Not Found.
- **else if (response.StatusCode == System.Net.HttpStatusCode.BadRequest)**
 - Throws an HttpRequestException with a "Bad request" message if the status code is 400 Bad Request.
- **else**
 - Throws a generic HttpRequestException for other status codes indicating request failure.
- **ProductDTO? product = await response.Content.ReadFromJsonAsync<ProductDTO>();**
 - Deserializes the response content to an instance of ProductDTO.
- **if (product == null)**
 - Throws an ArgumentException if deserialization results in null, indicating an invalid product ID.
- **return product;**
 - Returns the deserialized ProductDTO object.

Code Explanation - AutoMapper classes in BusinessLogicLayer

These classes define mapping profiles for converting between various data transfer objects (DTOs) and data access layer entities. The AutoMapper library is used to streamline these conversions, which helps in keeping the business logic clean and focused on its core functionality.

OrderAddRequestToOrderMappingProfile Class

```
using AutoMapper;

using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;

namespace eCommerce.ordersMicroservice.BusinessLogicLayer.Mappers;

public class OrderAddRequestToOrderMappingProfile : Profile
{
    public OrderAddRequestToOrderMappingProfile()
    {
        CreateMap<OrderAddRequest, Order>()
            .ForMember(dest => dest.UserID, opt => opt.MapFrom(src => src.UserID))
            .ForMember(dest => dest.OrderDate, opt => opt.MapFrom(src => src.OrderDate))
            .ForMember(dest => dest.OrderItems, opt => opt.MapFrom(src => src.OrderItems))
            .ForMember(dest => dest.OrderID, opt => opt.Ignore())
            .ForMember(dest => dest._id, opt => opt.Ignore())
            .ForMember(dest => dest.TotalBill, opt => opt.Ignore());
    }
}
```

Explanation:

- **CreateMap<OrderAddRequest, Order>()**
 - Defines a mapping from OrderAddRequest to Order.
- **.ForMember(dest => dest.OrderID, opt => opt.Ignore())**
 - Specifies that the OrderID property in the Order entity should be ignored (not mapped from OrderAddRequest).
- **.ForMember(dest => dest._id, opt => opt.Ignore())**

- Specifies that the `_id` property in the Order entity should be ignored.
- **`.ForMember(dest => dest.TotalBill, opt => opt.Ignore())`**
 - Specifies that the TotalBill property in the Order entity should be ignored.

OrderItemAddRequestToOrderItemMappingProfile Class

```
using AutoMapper;
```

```
using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
```

```
using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;
```

```
namespace eCommerce.ordersMicroservice.BusinessLogicLayer.Mappers;
```

```
public class OrderItemAddRequestToOrderItemMappingProfile : Profile
{
    public OrderItemAddRequestToOrderItemMappingProfile()
    {
        CreateMap<OrderItemAddRequest, OrderItem>()
            .ForMember(dest => dest.ProductID, opt => opt.MapFrom(src => src.ProductID))
            .ForMember(dest => dest.UnitPrice, opt => opt.MapFrom(src => src.UnitPrice))
            .ForMember(dest => dest.Quantity, opt => opt.MapFrom(src => src.Quantity))
            .ForMember(dest => dest.TotalPrice, opt => opt.Ignore())
            .ForMember(dest => dest._id, opt => opt.Ignore());
    }
}
```

Explanation:

- **`CreateMap<OrderItemAddRequest, OrderItem>()`**
 - Defines a mapping from OrderItemAddRequest to OrderItem.
- **`.ForMember(dest => dest.TotalPrice, opt => opt.Ignore())`**
 - Specifies that the TotalPrice property in the OrderItem entity should be ignored.
- **`.ForMember(dest => dest._id, opt => opt.Ignore())`**
 - Specifies that the `_id` property in the OrderItem entity should be ignored.

OrderItemToOrderItemResponseMappingProfile Class

```
using AutoMapper;

using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;

namespace eCommerce.ordersMicroservice.BusinessLogicLayer.Mappers;

public class OrderItemToOrderItemResponseMappingProfile : Profile
{
    public OrderItemToOrderItemResponseMappingProfile()
    {
        CreateMap<OrderItem, OrderItemResponse>()
            .ForMember(dest => dest.ProductID, opt => opt.MapFrom(src => src.ProductID))
            .ForMember(dest => dest.UnitPrice, opt => opt.MapFrom(src => src.UnitPrice))
            .ForMember(dest => dest.Quantity, opt => opt.MapFrom(src => src.Quantity))
            .ForMember(dest => dest.TotalPrice, opt => opt.MapFrom(src => src.TotalPrice));
    }
}
```

Explanation:

- **CreateMap<OrderItem, OrderItemResponse>()**
 - Defines a mapping from OrderItem to OrderItemResponse.

OrderItemUpdateRequestToOrderItemMappingProfile Class

```
using AutoMapper;

using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;

namespace eCommerce.ordersMicroservice.BusinessLogicLayer.Mappers;

public class OrderItemUpdateRequestToOrderItemMappingProfile : Profile
{

```

```

public OrderItemUpdateRequestToOrderItemMappingProfile()
{
    CreateMap<OrderItemUpdateRequest, OrderItem>()
        .ForMember(dest => dest.ProductID, opt => opt.MapFrom(src => src.ProductID))
        .ForMember(dest => dest.UnitPrice, opt => opt.MapFrom(src => src.UnitPrice))
        .ForMember(dest => dest.Quantity, opt => opt.MapFrom(src => src.Quantity))
        .ForMember(dest => dest.TotalPrice, opt => opt.Ignore())
        .ForMember(dest => dest._id, opt => opt.Ignore());
}
}

```

Explanation:

- **CreateMap<OrderItemUpdateRequest, OrderItem>()**
 - Defines a mapping from OrderItemUpdateRequest to OrderItem.

OrderToOrderResponseMappingProfile Class

```

using AutoMapper;

using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;

namespace eCommerce.ordersMicroservice.BusinessLogicLayer.Mappers;

public class OrderToOrderResponseMappingProfile : Profile
{
    public OrderToOrderResponseMappingProfile()
    {
        CreateMap<Order, OrderResponse>()
            .ForMember(dest => dest.OrderID, opt => opt.MapFrom(src => src.OrderID))
            .ForMember(dest => dest.UserID, opt => opt.MapFrom(src => src.UserID))
            .ForMember(dest => dest.OrderDate, opt => opt.MapFrom(src => src.OrderDate))
            .ForMember(dest => dest.OrderItems, opt => opt.MapFrom(src => src.OrderItems))
            .ForMember(dest => dest.TotalBill, opt => opt.MapFrom(src => src.TotalBill));
    }
}

```



```
}  
}
```

Explanation:

- **CreateMap<Order, OrderResponse>()**
 - Defines a mapping from Order to OrderResponse.

OrderUpdateRequestToOrderMappingProfile Class

```
using AutoMapper;
```

```
using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
```

```
using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;
```

```
namespace eCommerce.ordersMicroservice.BusinessLogicLayer.Mappers;
```

```
public class OrderUpdateRequestToOrderMappingProfile : Profile
```

```
{  
    public OrderUpdateRequestToOrderMappingProfile()  
    {  
        CreateMap<OrderUpdateRequest, Order>()  
            .ForMember(dest => dest.OrderID, opt => opt.MapFrom(src => src.OrderID))  
            .ForMember(dest => dest.UserID, opt => opt.MapFrom(src => src.UserID))  
            .ForMember(dest => dest.OrderDate, opt => opt.MapFrom(src => src.OrderDate))  
            .ForMember(dest => dest.OrderItems, opt => opt.MapFrom(src => src.OrderItems))  
            .ForMember(dest => dest._id, opt => opt.Ignore())  
            .ForMember(dest => dest.TotalBill, opt => opt.Ignore());  
    }  
}
```

Explanation:

- **CreateMap<OrderUpdateRequest, Order>()**
 - Defines a mapping from OrderUpdateRequest to Order.

ProductDTOToOrderItemResponseMappingProfile Class

```
using AutoMapper;

using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;

namespace eCommerce.ordersMicroservice.BusinessLogicLayer.Mappers;

public class ProductDTOToOrderItemResponseMappingProfile : Profile
{
    public ProductDTOToOrderItemResponseMappingProfile()
    {
        CreateMap<ProductDTO, OrderItemResponse>()
            .ForMember(dest => dest.ProductName, opt => opt.MapFrom(src => src.ProductName))
            .ForMember(dest => dest.Category, opt => opt.MapFrom(src => src.Category));
    }
}
```

Explanation:

- **CreateMap<ProductDTO, OrderItemResponse>()**
 - Defines a mapping from ProductDTO to OrderItemResponse.

UserDTOToOrderResponseMappingProfile Class

```
using AutoMapper;

using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.Mappers;

public class UserDTOToOrderResponseMappingProfile : Profile
{
    public UserDTOToOrderResponseMappingProfile()
    {
        CreateMap<UserDTO, OrderResponse>()
            .ForMember(dest => dest.UserPersonName, opt => opt.MapFrom(src => src.PersonName))
    }
}
```

```

        .ForMember(dest => dest.Email, opt => opt.MapFrom(src => src.Email));
    }
}

```

Explanation:

- **CreateMap<UserDTO, OrderResponse>()**
 - Defines a mapping from UserDTO to OrderResponse.

Code Explanation - Validators in BusinessLogicLayer

These validator classes use FluentValidation to enforce rules on various DTOs. They ensure that the input data meets the specified requirements before any further processing is done. This helps in maintaining data integrity and preventing invalid data from being handled by the business logic.

OrderAddRequestValidator Class

```

using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
using FluentValidation;

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.Validators;

public class OrderAddRequestValidator : AbstractValidator<OrderAddRequest>
{
    public OrderAddRequestValidator()
    {
        // UserID
        RuleFor(temp => temp.UserID)
            .NotEmpty().WithErrorCode("User ID can't be blank");

        // OrderDate
        RuleFor(temp => temp.OrderDate)
            .NotEmpty().WithErrorCode("Order Date can't be blank");
    }
}

```

```
// OrderItems

RuleFor(temp => temp.OrderItems)

    .NotEmpty().WithErrorCode("Order Items can't be blank");

}

}
```

Explanation:

- **RuleFor(temp => temp.UserID)**
 - Ensures that the UserID field is not empty.
- **RuleFor(temp => temp.OrderDate)**
 - Ensures that the OrderDate field is not empty.
- **RuleFor(temp => temp.OrderItems)**
 - Ensures that the OrderItems collection is not empty.

OrderItemAddRequestValidator Class

```
using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;

using FluentValidation;
```

```
namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.Validators;
```

```
public class OrderItemAddRequestValidator : AbstractValidator<OrderItemAddRequest>
{
    public OrderItemAddRequestValidator()
    {
        // ProductID

        RuleFor(temp => temp.ProductID)

            .NotEmpty().WithErrorCode("Product ID can't be blank");


        // UnitPrice

        RuleFor(temp => temp.UnitPrice)

            .NotEmpty().WithErrorCode("Unit Price can't be blank")

            .GreaterThan(0).WithErrorCode("Unit Price can't be less than or equal to zero");
    }
}
```

```

// Quantity
RuleFor(temp => temp.Quantity)
    .NotEmpty().WithErrorCode("Quantity can't be blank")
    .GreaterThan(0).WithErrorCode("Quantity can't be less than or equal to zero");
}
}

```

Explanation:

- **RuleFor(temp => temp.ProductID)**
 - Ensures that the ProductID field is not empty.
- **RuleFor(temp => temp.UnitPrice)**
 - Ensures that the UnitPrice is not empty and is greater than zero.
- **RuleFor(temp => temp.Quantity)**
 - Ensures that the Quantity is not empty and is greater than zero.

OrderItemUpdateRequestValidator Class

```

using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
using FluentValidation;

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.Validators;

public class OrderItemUpdateRequestValidator : AbstractValidator<OrderItemUpdateRequest>
{
    public OrderItemUpdateRequestValidator()
    {
        // ProductID
        RuleFor(temp => temp.ProductID)
            .NotEmpty().WithErrorCode("Product ID can't be blank");

        // UnitPrice
        RuleFor(temp => temp.UnitPrice)

```

```

        .NotEmpty().WithErrorCode("Unit Price can't be blank")
        .GreaterThan(0).WithErrorCode("Unit Price can't be less than or equal to zero");

// Quantity
RuleFor(temp => temp.Quantity)
    .NotEmpty().WithErrorCode("Quantity can't be blank")
    .GreaterThan(0).WithErrorCode("Quantity can't be less than or equal to zero");
}
}

```

Explanation:

- **RuleFor(temp => temp.ProductID)**
 - Ensures that the ProductID field is not empty.
- **RuleFor(temp => temp.UnitPrice)**
 - Ensures that the UnitPrice is not empty and is greater than zero.
- **RuleFor(temp => temp.Quantity)**
 - Ensures that the Quantity is not empty and is greater than zero.

OrderUpdateRequestValidator Class

```

using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
using FluentValidation;

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.Validators;

public class OrderUpdateRequestValidator : AbstractValidator<OrderUpdateRequest>
{
    public OrderUpdateRequestValidator()
    {
        // OrderID
        RuleFor(temp => temp.OrderID)
            .NotEmpty().WithErrorCode("Order ID can't be blank");
    }
}

```

```

// UserID
RuleFor(temp => temp.UserID)
    .NotEmpty().WithErrorCode("User ID can't be blank");

// OrderDate
RuleFor(temp => temp.OrderDate)
    .NotEmpty().WithErrorCode("Order Date can't be blank");

// OrderItems
RuleFor(temp => temp.OrderItems)
    .NotEmpty().WithErrorCode("Order Items can't be blank");
}
}

```

Explanation:

- **RuleFor(temp => temp.OrderID)**
 - Ensures that the OrderID field is not empty.
- **RuleFor(temp => temp.UserID)**
 - Ensures that the UserID field is not empty.
- **RuleFor(temp => temp.OrderDate)**
 - Ensures that the OrderDate field is not empty.
- **RuleFor(temp => temp.OrderItems)**
 - Ensures that the OrderItems collection is not empty.

Summary

- **OrderAddRequestValidator:** Validates that UserID, OrderDate, and OrderItems are not empty.
- **OrderItemAddRequestValidator:** Validates that ProductID is not empty, UnitPrice is not empty and greater than zero, and Quantity is not empty and greater than zero.
- **OrderItemUpdateRequestValidator:** Same as OrderItemAddRequestValidator but for update scenarios.
- **OrderUpdateRequestValidator:** Validates that OrderID, UserID, OrderDate, and OrderItems are not empty.

Code Explanation - Service Contracts in BusinessLogicLayer

The IOrdersService interface defines the contract for managing orders in the application. It includes methods for retrieving, adding, updating, and deleting orders. Each method's description helps in understanding its functionality and the expected outcomes. Here's a detailed look at each method in the IOrdersService interface:

IOrdersService Interface

```
using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;
using MongoDB.Driver;

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer.ServiceContracts;

public interface IOrdersService
{
    /// <summary>
    /// Retrieves the list of orders from the orders repository
    /// </summary>
    /// <returns>Returns list of OrderResponse objects</returns>
    Task<List<OrderResponse?>> GetOrders();

    /// <summary>
    /// Returns list of orders matching with given condition
    /// </summary>
    /// <param name="filter">Expression that represents condition to check</param>
    /// <returns>Returns matching orders as OrderResponse objects</returns>
    Task<List<OrderResponse?>> GetOrdersByCondition(FilterDefinition<Order> filter);

    /// <summary>
```



```

/// Returns a single order that matches with given condition
/// </summary>
/// <param name="filter">Expression that represents the condition to check</param>
/// <returns>Returns matching order object as OrderResponse; or null if not found</returns>
Task<OrderResponse?> GetOrderByCondition(FilterDefinition<Order> filter);

/// <summary>
/// Add (inserts) order into the collection using orders repository
/// </summary>
/// <param name="orderAddRequest">Order to insert</param>
/// <returns>Returns OrderResponse object that contains order details after inserting; or returns
null if insertion is unsuccessful.</returns>
Task<OrderResponse?> AddOrder(OrderAddRequest orderAddRequest);

/// <summary>
/// Updates the existing order based on the OrderID
/// </summary>
/// <param name="orderUpdateRequest">Order data to update</param>
/// <returns>Returns order object after successful updation; otherwise null</returns>
Task<OrderResponse?> UpdateOrder(OrderUpdateRequest orderUpdateRequest);

/// <summary>
/// Deletes an existing order based on given order id
/// </summary>
/// <param name="orderId">OrderID to search and delete</param>
/// <returns>Returns true if the deletion is successful; otherwise false</returns>
Task<bool> DeleteOrder(Guid orderId);
}

```

Method Details

1. GetOrders

- **Purpose:** Retrieves all orders from the repository.
- **Returns:** A list of OrderResponse objects representing all orders.
- **Use Case:** To fetch and display all orders for administrative or analytical purposes.

2. GetOrdersByCondition

- **Purpose:** Retrieves orders that match a specific condition.
- **Parameters:**
 - filter: A FilterDefinition<Order> representing the condition to filter the orders.
- **Returns:** A list of OrderResponse objects that match the condition.
- **Use Case:** To fetch orders based on specific criteria, such as date range, user ID, or status.

3. GetOrderByCondition

- **Purpose:** Retrieves a single order that matches a specific condition.
- **Parameters:**
 - filter: A FilterDefinition<Order> representing the condition to filter the order.
- **Returns:** An OrderResponse object if a match is found; otherwise, returns null.
- **Use Case:** To fetch a specific order based on a unique condition.

4. AddOrder

- **Purpose:** Adds a new order to the repository.
- **Parameters:**
 - orderAddRequest: An OrderAddRequest object containing the details of the order to be added.
- **Returns:** An OrderResponse object representing the newly added order, or null if the addition fails.
- **Use Case:** To insert a new order into the system.

5. UpdateOrder

- **Purpose:** Updates an existing order based on its ID.
- **Parameters:**
 - orderUpdateRequest: An OrderUpdateRequest object containing the updated order details.

- **Returns:** An OrderResponse object representing the updated order, or null if the update fails.
- **Use Case:** To modify the details of an existing order.

6. DeleteOrder

- **Purpose:** Deletes an order from the repository.
- **Parameters:**
 - **orderId:** The ID of the order to be deleted.
- **Returns:** True if the deletion is successful; otherwise, false.
- **Use Case:** To remove an order from the system, typically done for order cancellations or deletions.

Code Explanation - OrdersService in BusinessLogicLayer

The OrdersService class implements the IOrdersService interface and provides the business logic for managing orders in the system. This service handles various operations such as adding, updating, deleting, and retrieving orders.

OrdersService:

```
using AutoMapper;

using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;

using eCommerce.OrdersMicroservice.BusinessLogicLayer.HttpClients;

using eCommerce.OrdersMicroservice.BusinessLogicLayer.ServiceContracts;

using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;

using eCommerce.OrdersMicroservice.DataAccessLayer.RepositoryContracts;

using FluentValidation;

using FluentValidation.Results;

using MongoDB.Driver;

namespace eCommerce.ordersMicroservice.BusinessLogicLayer.Services;

public class OrdersService : IOrdersService
{
```

```
private readonly IValidator<OrderAddRequest> _orderAddRequestValidator;

private readonly IValidator<OrderItemAddRequest> _orderItemAddRequestValidator;

private readonly IValidator<OrderUpdateRequest> _orderUpdateRequestValidator;

private readonly IValidator<OrderItemUpdateRequest> _orderItemUpdateRequestValidator;

private readonly IMapper _mapper;

private IOOrdersRepository _ordersRepository;

private UsersMicroserviceClient _usersMicroserviceClient;

private ProductsMicroserviceClient _productsMicroserviceClient;
```

```
public OrdersService(IOOrdersRepository ordersRepository, IMapper mapper,
IValidator<OrderAddRequest> orderAddRequestValidator, IValidator<OrderItemAddRequest>
orderItemAddRequestValidator, IValidator<OrderUpdateRequest> orderUpdateRequestValidator,
IValidator<OrderItemUpdateRequest> orderItemUpdateRequestValidator, UsersMicroserviceClient
usersMicroserviceClient, ProductsMicroserviceClient productsMicroserviceClient)
```

```
{

    _orderAddRequestValidator = orderAddRequestValidator;

    _orderItemAddRequestValidator = orderItemAddRequestValidator;

    _orderUpdateRequestValidator = orderUpdateRequestValidator;

    _orderItemUpdateRequestValidator = orderItemUpdateRequestValidator;

    _mapper = mapper;

    _ordersRepository = ordersRepository;

    _usersMicroserviceClient = usersMicroserviceClient;

    _productsMicroserviceClient = productsMicroserviceClient;

}
```

```
public async Task<OrderResponse?> AddOrder(OrderAddRequest orderAddRequest)

{

    if (orderAddRequest == null)

    {

        throw new ArgumentNullException(nameof(orderAddRequest));

    }

}
```

```

        ValidationResult orderAddRequestValidationResult = await
_orderAddRequestValidator.ValidateAsync(orderAddRequest);

        if (!orderAddRequestValidationResult.IsValid)
        {
            string errors = string.Join(" ", orderAddRequestValidationResult.Errors.Select(temp =>
temp.ErrorMessage));

            throw new ArgumentException(errors);
        }

        List<ProductDTO?> products = new List<ProductDTO?>();

        foreach (OrderItemAddRequest orderItemAddRequest in orderAddRequest.OrderItems)
        {
            ValidationResult orderItemAddRequestValidationResult = await
_orderItemAddRequestValidator.ValidateAsync(orderItemAddRequest);

            if (!orderItemAddRequestValidationResult.IsValid)
            {
                string errors = string.Join(" ", orderItemAddRequestValidationResult.Errors.Select(temp =>
temp.ErrorMessage));

                throw new ArgumentException(errors);
            }

            ProductDTO? product = await
_productsMicroserviceClient.GetProductByProductID(orderItemAddRequest.ProductID);

            if (product == null)
            {
                throw new ArgumentException("Invalid Product ID");
            }

            products.Add(product);
        }

        UserDTO? user = await _usersMicroserviceClient.GetUserByUserID(orderAddRequest.UserID);

```

```
if (user == null)
{
    throw new ArgumentException("Invalid User ID");
}
```

```
Order orderInput = _mapper.Map<Order>(orderAddRequest);
foreach (OrderItem orderItem in orderInput.OrderItems)
{
    orderItem.TotalPrice = orderItem.Quantity * orderItem.UnitPrice;
}
orderInput.TotalBill = orderInput.OrderItems.Sum(temp => temp.TotalPrice);
```

```
Order? addedOrder = await _ordersRepository.AddOrder(orderInput);
```

```
if (addedOrder == null)
{
    return null;
}
```

```
OrderResponse addedOrderResponse = _mapper.Map<OrderResponse>(addedOrder);
```

```
if (addedOrderResponse != null)
{
    foreach (OrderItemResponse orderItemResponse in addedOrderResponse.OrderItems)
    {
        ProductDTO? productDTO = products.FirstOrDefault(temp => temp.ProductID ==
orderItemResponse.ProductID);
        if (productDTO != null)
        {
            _mapper.Map(productDTO, orderItemResponse);
        }
    }
}
```

```

    }

    if (user != null)
    {
        _mapper.Map(user, addedOrderResponse);
    }
}

return addedOrderResponse;
}

public async Task<OrderResponse?> UpdateOrder(OrderUpdateRequest orderUpdateRequest)
{
    if (orderUpdateRequest == null)
    {
        throw new ArgumentNullException(nameof(orderUpdateRequest));
    }

    ValidationResult orderUpdateRequestValidationResult = await
    _orderUpdateRequestValidator.ValidateAsync(orderUpdateRequest);
    if (!orderUpdateRequestValidationResult.IsValid)
    {
        string errors = string.Join(", ", orderUpdateRequestValidationResult.Errors.Select(temp =>
temp.ErrorMessage));
        throw new ArgumentException(errors);
    }

    List<ProductDTO> products = new List<ProductDTO>();

    foreach (OrderItemUpdateRequest orderItemUpdateRequest in orderUpdateRequest.OrderItems)
    {

```

```
ValidationResult orderItemUpdateRequestValidationResult = await
_orderItemUpdateRequestValidator.ValidateAsync(orderItemUpdateRequest);

if (!orderItemUpdateRequestValidationResult.IsValid)
{
    string errors = string.Join(" ", orderItemUpdateRequestValidationResult.Errors.Select(temp =>
temp.ErrorMessage));

    throw new ArgumentException(errors);
}
```

```
ProductDTO? product = await
_productsMicroserviceClient.GetProductByProductID(orderItemUpdateRequest.ProductID);

if (product == null)
{
    throw new ArgumentException("Invalid Product ID");
}
```

```
products.Add(product);
}
```

```
UserDTO? user = await _usersMicroserviceClient.GetUserByUserID(orderUpdateRequest.UserID);

if (user == null)
{
    throw new ArgumentException("Invalid User ID");
}
```

```
Order orderInput = _mapper.Map<Order>(orderUpdateRequest);

foreach (OrderItem orderItem in orderInput.OrderItems)
{
    orderItem.TotalPrice = orderItem.Quantity * orderItem.UnitPrice;
}

orderInput.TotalBill = orderInput.OrderItems.Sum(temp => temp.TotalPrice);
```



```
Order? updatedOrder = await _ordersRepository.UpdateOrder(orderInput);
```

```
if (updatedOrder == null)
```

```
{
```

```
    return null;
```

```
}
```

```
OrderResponse updatedOrderResponse = _mapper.Map<OrderResponse>(updatedOrder);
```

```
if (updatedOrderResponse != null)
```

```
{
```

```
    foreach (OrderItemResponse orderItemResponse in updatedOrderResponse.OrderItems)
```

```
    {
```

```
        ProductDTO? productDTO = products.FirstOrDefault(temp => temp.ProductID ==  
orderItemResponse.ProductID);
```

```
        if (productDTO != null)
```

```
        {
```

```
            _mapper.Map(productDTO, orderItemResponse);
```

```
        }
```

```
    }
```

```
if (user != null)
```

```
{
```

```
    _mapper.Map(user, updatedOrderResponse);
```

```
}
```

```
}
```

```
return updatedOrderResponse;
```

```
}
```

```
public async Task<bool> DeleteOrder(Guid orderID)
```

```

{
    FilterDefinition<Order> filter = Builders<Order>.Filter.Eq(temp => temp.OrderID, orderID);
    Order? existingOrder = await _ordersRepository.GetOrderByCondition(filter);

    if (existingOrder == null)
    {
        return false;
    }

    bool isDeleted = await _ordersRepository.DeleteOrder(orderID);
    return isDeleted;
}

public async Task<OrderResponse?> GetOrderByCondition(FilterDefinition<Order> filter)
{
    Order? order = await _ordersRepository.GetOrderByCondition(filter);
    if (order == null)
    {
        return null;
    }

    OrderResponse orderResponse = _mapper.Map<OrderResponse>(order);

    if (orderResponse != null)
    {
        foreach (OrderItemResponse orderItemResponse in orderResponse.OrderItems)
        {
            ProductDTO? productDTO = await
            _productsMicroserviceClient.GetProductByProductID(orderItemResponse.ProductID);
            if (productDTO != null)
            {

```

```

        _mapper.Map(productDTO, orderItemResponse);
    }
}

UserDTO? user = await _usersMicroserviceClient.GetUserByUserID(orderResponse.UserID);
if (user != null)
{
    _mapper.Map(user, orderResponse);
}

return orderResponse;
}

public async Task<List<OrderResponse?>> GetOrdersByCondition(FilterDefinition<Order> filter)
{
    IEnumerable<Order?> orders = await _ordersRepository.GetOrdersByCondition(filter);

    IEnumerable<OrderResponse?> orderResponses =
_mapper.Map<IEnumerable<OrderResponse>>(orders);

    foreach (OrderResponse? orderResponse in orderResponses)
    {
        if (orderResponse != null)
        {
            foreach (OrderItemResponse orderItemResponse in orderResponse.OrderItems)
            {
                ProductDTO? productDTO = await
_productsMicroserviceClient.GetProductByProductID(orderItemResponse.ProductID);
                if (productDTO != null)
                {
                    _mapper.Map(productDTO, orderItemResponse);
                }
            }
        }
    }
}

```

```

    }

    UserDTO? user = await _usersMicroserviceClient.GetUserByUserID(orderResponse.UserID);
    if (user != null)
    {
        _mapper.Map(user, orderResponse);
    }
}

return orderResponses.ToList();
}

public async Task<List<OrderResponse?>> GetOrders()
{
    IEnumerable<Order?> orders = await _ordersRepository.GetOrders();
    IEnumerable<OrderResponse?> orderResponses =
    _mapper.Map<IEnumerable<OrderResponse>>(orders);

    foreach (OrderResponse? orderResponse in orderResponses)
    {
        if (orderResponse != null)
        {
            foreach (OrderItemResponse orderItemResponse in orderResponse.OrderItems)
            {
                ProductDTO? productDTO = await
                _productsMicroserviceClient.GetProductByProductID(orderItemResponse.ProductID);
                if (productDTO != null)
                {
                    _mapper.Map(productDTO, orderItemResponse);
                }
            }
        }
    }
}

```

```

        UserDTO? user = await _usersMicroserviceClient.GetUserByUserID(orderResponse.UserID);
        if (user != null)
        {
            _mapper.Map(user, orderResponse);
        }
    }
}

return orderResponses.ToList();
}
}

```

Explanation

- **Dependencies:** The constructor injects various dependencies, including repositories, clients, and validators.
- **AddOrder:**
 - Validates the incoming request.
 - Retrieves products and user data from microservices.
 - Maps the request to an Order entity and calculates total prices.
 - Adds the order to the repository and maps the result to a response object.
- **UpdateOrder:**
 - Similar to AddOrder, but updates an existing order.
 - Retrieves and validates order items and user.
 - Updates the order in the repository and returns the updated response.
- **DeleteOrder:**
 - Deletes an order based on its ID.
 - Checks if the order exists before attempting deletion.
- **GetOrderByCondition:**
 - Retrieves a single order based on a filter.
 - Maps the order to a response object, including product and user details.

- **GetOrdersByCondition:**
 - Retrieves a list of orders based on a filter.
 - Maps each order and its details to response objects.
- **GetOrders:**
 - Retrieves all orders.
 - Maps each order to a response object with product and user details.

Code Explanation - Dependency Injection in BusinessLogicLayer

The DependencyInjection class in the BusinessLogicLayer is designed to register services and validators into the dependency injection container. This setup ensures that the business logic layer's dependencies are properly managed and injected throughout the application.

DependencyInjection Class

```
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using FluentValidation;
using eCommerce.OrdersMicroservice.BusinessLogicLayer.Validators;
using eCommerce.ordersMicroservice.BusinessLogicLayer.Mappers;
using eCommerce.OrdersMicroservice.BusinessLogicLayer.ServiceContracts;
using eCommerce.ordersMicroservice.BusinessLogicLayer.Services;

namespace eCommerce.OrdersMicroservice.BusinessLogicLayer;

public static class DependencyInjection
{
    public static IServiceCollection AddBusinessLogicLayer(this IServiceCollection services,
    IConfiguration configuration)
    {
        // Register all validators from the assembly containing the OrderAddRequestValidator
        services.AddValidatorsFromAssemblyContaining<OrderAddRequestValidator>();
    }
}
```

```

    // Register AutoMapper profiles from the assembly containing the
    OrderAddRequestToOrderMappingProfile

    services.AddAutoMapper(typeof(OrderAddRequestToOrderMappingProfile).Assembly);

    // Register the OrdersService implementation for the IOrdersService interface
    services.AddScoped<IOrdersService, OrdersService>();

    return services;
}
}

```

Explanation

1. **AddValidatorsFromAssemblyContaining<OrderAddRequestValidator>():**
 - This line registers all validators in the assembly where OrderAddRequestValidator is located.
 - FluentValidation uses this to automatically discover and register all validator types, so you don't need to add each validator individually.
2. **AddAutoMapper(typeof(OrderAddRequestToOrderMappingProfile).Assembly):**
 - This registers AutoMapper profiles from the assembly containing the OrderAddRequestToOrderMappingProfile.
 - AutoMapper is used to map between different object models (e.g., mapping DTOs to entities). By specifying the assembly, AutoMapper will look for any profiles in that assembly.
3. **AddScoped<IOrdersService, OrdersService>():**
 - Registers the OrdersService class as the implementation for the IOrdersService interface.
 - The Scoped lifetime means that a new instance of OrdersService will be created for each request within the scope (typically a web request).
4. **Return services:**
 - The services object is returned, allowing for method chaining and adding more services or configurations if needed.

Code Explanation - appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "MongoDB": "mongodb://$MONGO_HOST:$MONGO_PORT"
  }
}
```

This file is used to configure various settings for your ASP.NET Core application. Here's what each section does:

Logging Configuration

```
"Logging": {
  "LogLevel": {
    "Default": "Information",
    "Microsoft.AspNetCore": "Warning"
  }
}
```

- **Default:** Sets the default logging level to Information. This means that logs of level Information and higher (like Warning, Error, Critical) will be recorded.
- **Microsoft.AspNetCore:** Sets the logging level for ASP.NET Core to Warning. Only warnings and errors from ASP.NET Core will be logged.

AllowedHosts

```
"AllowedHosts": "*"
```

- *: Allows requests from any host. In production, you might restrict this to specific hosts for security reasons.

ConnectionStrings

```
"ConnectionStrings": {  
  "MongoDB": "mongodb://$MONGO_HOST:$MONGO_PORT"  
}
```

- **MongoDB**: Contains the connection string for MongoDB. Replace \$MONGO_HOST and \$MONGO_PORT with actual values. For example:

```
"ConnectionStrings": {  
  "MongoDB": "mongodb://localhost:27017"  
}
```

This connection string tells your application how to connect to the MongoDB database. In practice, use environment variables to inject these values securely.

Code Explanation - Exception Handling Middleware in API layer

This middleware component is designed to handle exceptions that occur during the processing of HTTP requests in your ASP.NET Core application.

ExceptionHandlerMiddleware Class

```
public class ExceptionHandlingMiddleware  
{  
    private readonly RequestDelegate _next;  
    private readonly ILogger<ExceptionHandlerMiddleware> _logger;  
  
    public ExceptionHandlingMiddleware(RequestDelegate next,  
    ILogger<ExceptionHandlerMiddleware> logger)  
    {  
        _next = next;
```

```

    _logger = logger;
}

public async Task Invoke(HttpContext httpContext)
{
    try
    {
        await _next(httpContext);
    }
    catch (Exception ex)
    {
        if (ex.InnerException != null)
        {
            _logger.LogError("{ExceptionType} {ExceptionMessage}",
                ex.InnerException.GetType().ToString(), ex.InnerException.Message);
        }
        else
        {
            _logger.LogError("{ExceptionType} {ExceptionMessage}", ex.GetType().ToString(), ex.Message);
        }
    }

    httpContext.Response.StatusCode = 500;

    await httpContext.Response.WriteAsJsonAsync(new { Message = ex.Message, Type =
        ex.GetType().ToString() });
}
}

```

- **Purpose:** Catches exceptions that occur during the request processing pipeline.
- **Constructor:** Initializes the middleware with the next request delegate and a logger.
- **Invoke Method:**
 - Wraps the next middleware in a try-catch block.
 - Logs detailed error information including the type and message of the exception.

- Sets the HTTP response status code to 500 (Internal Server Error).
- Returns a JSON response with the error message and type.

Code Explanation - Program.cs in API layer

This file configures and sets up the services and middleware for the ASP.NET Core application.

```
using eCommerce.OrdersMicroservice.DataAccessLayer;
using eCommerce.OrdersMicroservice.BusinessLogicLayer;
using FluentValidation.AspNetCore;
using eCommerce.OrdersMicroservice.API.Middleware;
using eCommerce.OrdersMicroservice.BusinessLogicLayer.HttpClients;

var builder = WebApplication.CreateBuilder(args);

// Add DAL and BLL services
builder.Services.AddDataAccessLayer(builder.Configuration);
builder.Services.AddBusinessLogicLayer(builder.Configuration);

builder.Services.AddControllers();

// FluentValidation
builder.Services.AddFluentValidationAutoValidation();

// Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// CORS
builder.Services.AddCors(options => {
    options.AddDefaultPolicy(builder =>
```

```

{
    builder.WithOrigins("http://localhost:4200")
        .AllowAnyMethod()
        .AllowAnyHeader();
});

});

// HTTP Clients for microservices

builder.Services.AddHttpClient<UsersMicroserviceClient>(client => {

    client.BaseAddress = new
Uri($"http://{builder.Configuration["UsersMicroserviceName"]}:{builder.Configuration["UsersMicroservicePort"]}");

});

builder.Services.AddHttpClient<ProductsMicroserviceClient>(client => {

    client.BaseAddress = new
Uri($"http://{builder.Configuration["ProductsMicroserviceName"]}:{builder.Configuration["ProductsMicroservicePort"]}");

});

var app = builder.Build();

// Middleware

app.UseExceptionHandlerMiddleware();

app.UseRouting();

// CORS

app.UseCors();

// Swagger

app.UseSwagger();

app.UseSwaggerUI();

```

```
// Authentication & Authorization

// app.UseHttpsRedirection(); // Uncomment if HTTPS redirection is needed

app.UseAuthentication();

app.UseAuthorization();


// Endpoints

app.MapControllers();


app.Run();
```

Key Components:

1. Service Registration:

- AddDataAccessLayer: Registers the data access layer services.
- AddBusinessLogicLayer: Registers business logic layer services.
- AddControllers: Adds services for controllers.
- AddFluentValidationAutoValidation: Configures FluentValidation.
- AddEndpointsApiExplorer and AddSwaggerGen: Configures Swagger for API documentation.

2. CORS Policy:

- Allows requests from `http://localhost:4200` and permits any method and header.

3. HTTP Clients:

- Configures `UsersMicroserviceClient` and `ProductsMicroserviceClient` with base addresses derived from configuration settings.

4. Middleware Configuration:

- UseExceptionHandlerMiddleware: Adds custom exception handling middleware.
- UseRouting: Adds routing middleware.
- UseCors: Adds CORS middleware.
- UseSwagger and UseSwaggerUI: Adds Swagger documentation middleware.
- UseAuthentication and UseAuthorization: Configures authentication and authorization.
- MapControllers: Maps controller endpoints.

5. Application Build and Run:

- Builds and runs the web application.

Code Explanation - Orders API Controller in API layer

This controller handles HTTP requests for managing orders within the Orders microservice. It utilizes the `IOrdersService` interface to perform various operations related to orders.

```
using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
using eCommerce.OrdersMicroservice.BusinessLogicLayer.ServiceContracts;
using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;
using Microsoft.AspNetCore.Mvc;
using MongoDB.Driver;
```

```
namespace OrdersMicroservice.API.ApiControllers;
```

```
[Route("api/[controller]")]
```

```
[ApiController]
```

```
public class OrdersController : ControllerBase
```

```
{
```

```
    private readonly IOrdersService _ordersService;
```

```
    public OrdersController(IOrdersService ordersService)
```

```
    {
```

```
        _ordersService = ordersService;
```

```
    }
```

```
// GET: /api/Orders
```

```
[HttpGet]
```

```
public async Task<IEnumerable<OrderResponse?>> Get()
```

```
{
```

```

    List<OrderResponse?> orders = await _ordersService.GetOrders();
    return orders;
}

```

```

// GET: /api/Orders/search/orderid/{orderId}
[HttpGet("search/orderid/{orderId}")]
public async Task<OrderResponse?> GetOrderByOrderID(Guid orderId)
{
    FilterDefinition<Order> filter = Builders<Order>.Filter.Eq(temp => temp.OrderID, orderId);
    OrderResponse? order = await _ordersService.GetOrderByCondition(filter);
    return order;
}

```

```

// GET: /api/Orders/search/productid/{productId}
[HttpGet("search/productid/{productId}")]
public async Task<IEnumerable<OrderResponse?>> GetOrdersByProductID(Guid productId)
{
    FilterDefinition<Order> filter = Builders<Order>.Filter.ElemMatch(temp => temp.OrderItems,
        Builders<OrderItem>.Filter.Eq(tempProduct => tempProduct.ProductID, productId));
    List<OrderResponse?> orders = await _ordersService.GetOrdersByCondition(filter);
    return orders;
}

```

```

// GET: /api/Orders/search/orderDate/{orderDate}
[HttpGet("/search/orderDate/{orderDate}")]
public async Task<IEnumerable<OrderResponse?>> GetOrdersByOrderDate(DateTime orderDate)
{
    FilterDefinition<Order> filter = Builders<Order>.Filter.Eq(temp =>
temp.OrderDate.ToString("yyyy-MM-dd"), orderDate.ToString("yyyy-MM-dd"));
    List<OrderResponse?> orders = await _ordersService.GetOrdersByCondition(filter);
    return orders;
}

```

```
}
```

```
// POST api/Orders
```

```
[HttpPost]
```

```
public async Task<IActionResult> Post(OrderAddRequest orderAddRequest)
```

```
{
```

```
    if (orderAddRequest == null)
```

```
    {
```

```
        return BadRequest("Invalid order data");
```

```
    }
```

```
    OrderResponse? orderResponse = await _ordersService.AddOrder(orderAddRequest);
```

```
    if (orderResponse == null)
```

```
    {
```

```
        return Problem("Error in adding order");
```

```
    }
```

```
    return Created($"api/Orders/search/orderid/{orderResponse?.OrderID}", orderResponse);
```

```
}
```

```
// PUT api/Orders/{orderId}
```

```
[HttpPut("{orderId}")]
```

```
public async Task<IActionResult> Put(Guid orderId, OrderUpdateRequest orderUpdateRequest)
```

```
{
```

```
    if (orderUpdateRequest == null)
```

```
    {
```

```
        return BadRequest("Invalid order data");
```

```
    }
```

```
    if (orderId != orderUpdateRequest.OrderID)
```



```

    {
        return BadRequest("OrderID in the URL doesn't match with the OrderID in the Request
body");
    }

    OrderResponse? orderResponse = await _ordersService.UpdateOrder(orderUpdateRequest);

    if (orderResponse == null)
    {
        return Problem("Error in updating order");
    }

    return Ok(orderResponse);
}

// DELETE api/Orders/{orderId}
[HttpDelete("{orderId}")]
public async Task<ActionResult> Delete(Guid orderId)
{
    if (orderId == Guid.Empty)
    {
        return BadRequest("Invalid order ID");
    }

    bool isDeleted = await _ordersService.DeleteOrder(orderID);

    if (!isDeleted)
    {
        return Problem("Error in deleting order");
    }
}

```

```
        return Ok(isDeleted);
    }
}
```

Key Endpoints:

1. **GET /api/Orders:** Retrieves a list of all orders.
2. **GET /api/Orders/search/orderid/{orderId}:** Retrieves an order by its OrderID.
3. **GET /api/Orders/search/productid/{productID}:** Retrieves orders containing a specific ProductID.
4. **GET /api/Orders/search/orderDate/{orderDate}:** Retrieves orders by a specific order date.
5. **POST /api/Orders:** Adds a new order. Expects an OrderAddRequest object in the request body.
6. **PUT /api/Orders/{orderId}:** Updates an existing order. Expects an OrderUpdateRequest object in the request body.
7. **DELETE /api/Orders/{orderId}:** Deletes an order by its OrderID.

Error Handling:

- **400 Bad Request:** Returns if the request body is invalid or OrderID mismatch.
- **500 Internal Server Error:** Returns if there's an issue with adding, updating, or deleting an order.

Detailed Explanation of OrdersController

The OrdersController is part of the API layer in an ASP.NET Core application and manages HTTP requests related to orders. It utilizes the IOrdersService interface to perform operations on orders. Here's a breakdown of each part of the controller:

Namespace and Using Directives

```
using eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO;
using eCommerce.OrdersMicroservice.BusinessLogicLayer.ServiceContracts;
using eCommerce.OrdersMicroservice.DataAccessLayer.Entities;
using Microsoft.AspNetCore.Mvc;
```

using MongoDB.Driver;

- **eCommerce.OrdersMicroservice.BusinessLogicLayer.DTO**: Contains data transfer objects (DTOs) used for communication between the API and business logic layers.
- **eCommerce.OrdersMicroservice.BusinessLogicLayer.ServiceContracts**: Contains service interfaces that define business logic operations.
- **eCommerce.OrdersMicroservice.DataAccessLayer.Entities**: Contains entity classes representing the data structure in MongoDB.
- **Microsoft.AspNetCore.Mvc**: Provides the base classes for MVC controllers and actions.
- **MongoDB.Driver**: Provides MongoDB-specific features, such as filters and queries.

Controller Definition

namespace OrdersMicroservice.API.ApiControllers;

[Route("api/[controller]")]

[ApiController]

public class OrdersController : ControllerBase

{

private readonly IOOrdersService _ordersService;

public OrdersController(IOOrdersService ordersService)

{

_ordersService = ordersService;

}

- **[Route("api/[controller]")]**: Defines the base route for all actions in this controller. [controller] is replaced with the controller name (in this case, Orders).
- **[ApiController]**: Makes the controller an API controller with automatic model state validation and binding source parameter inference.

GET: /api/Orders

[HttpGet]

public async Task<IEnumerable<OrderResponse?>> Get()

{

List<OrderResponse?> orders = await _ordersService.GetOrders();

```

return orders;
}

```

- **[HttpGet]**: Specifies that this action responds to HTTP GET requests.
- **Get()**: Retrieves all orders from the data source.
- **_ordersService.GetOrders()**: Calls the business logic layer to get a list of orders.
- **IEnumerable<OrderResponse?>**: Returns a collection of OrderResponse objects.

GET: /api/Orders/search/orderid/{orderId}

```
[HttpGet("search/orderid/{orderId}")]
```

```
public async Task<OrderResponse?> GetOrderByOrderID(Guid orderId)
```

```

{
    FilterDefinition<Order> filter = Builders<Order>.Filter.Eq(temp => temp.OrderID, orderId);
    OrderResponse? order = await _ordersService.GetOrderByCondition(filter);
    return order;
}

```

- **[HttpGet("search/orderid/{orderId}")]**: Routes to this action when the URL matches /api/Orders/search/orderid/{orderId}.
- **GetOrderByOrderID(Guid orderId)**: Retrieves a specific order based on the provided orderId.
- **FilterDefinition<Order> filter**: Creates a MongoDB filter to match the OrderID.
- **_ordersService.GetOrderByCondition(filter)**: Calls the service layer to get the order matching the filter.

GET: /api/Orders/search/productid/{productID}

```
[HttpGet("search/productid/{productID}")]
```

```
public async Task<IEnumerable<OrderResponse?>> GetOrdersByProductID(Guid productID)
```

```

{
    FilterDefinition<Order> filter = Builders<Order>.Filter.ElemMatch(temp => temp.OrderItems,
        Builders<OrderItem>.Filter.Eq(tempProduct => tempProduct.ProductID, productID));
    List<OrderResponse?> orders = await _ordersService.GetOrdersByCondition(filter);
    return orders;
}

```

- **[HttpGet("search/productid/{productID}")]**: Routes to this action when the URL matches /api/Orders/search/productid/{productID}.
- **GetOrdersByProductID(Guid productID)**: Retrieves orders containing a specific product.
- **FilterDefinition<Order> filter**: Creates a filter to match orders that have an item with the given ProductID.
- **_ordersService.GetOrdersByCondition(filter)**: Calls the service layer to get orders matching the filter.

GET: /api/Orders/search/orderDate/{orderDate}

[HttpGet("/search/orderDate/{orderDate}")]

```
public async Task<IEnumerable<OrderResponse?>> GetOrdersByOrderDate(DateTime orderDate)
{
    FilterDefinition<Order> filter = Builders<Order>.Filter.Eq(temp => temp.OrderDate.ToString("yyyy-MM-dd"), orderDate.ToString("yyyy-MM-dd"));

    List<OrderResponse?> orders = await _ordersService.GetOrdersByCondition(filter);

    return orders;
}
```

- **[HttpGet("/search/orderDate/{orderDate}")]**: Routes to this action when the URL matches /search/orderDate/{orderDate}.
- **GetOrdersByOrderDate(DateTime orderDate)**: Retrieves orders based on the order date.
- **FilterDefinition<Order> filter**: Creates a filter to match orders with the specified orderDate.
- **_ordersService.GetOrdersByCondition(filter)**: Calls the service layer to get orders matching the filter.

POST: api/Orders

[HttpPost]

```
public async Task<ActionResult> Post(OrderAddRequest orderAddRequest)
{
    if (orderAddRequest == null)
    {
        return BadRequest("Invalid order data");
    }
}
```

```

OrderResponse? orderResponse = await _ordersService.AddOrder(orderAddRequest);

if (orderResponse == null)
{
    return Problem("Error in adding order");
}

return Created($"api/Orders/search/orderid/{orderResponse?.OrderID}", orderResponse);
}

```

- **[HttpPost]**: Specifies that this action responds to HTTP POST requests.
- **Post(OrderAddRequest orderAddRequest)**: Adds a new order. Expects an OrderAddRequest object in the request body.
- **BadRequest("Invalid order data")**: Returns if the request body is null.
- **_ordersService.AddOrder(orderAddRequest)**: Calls the service layer to add the new order.
- **Created(\$"api/Orders/search/orderid/{orderResponse?.OrderID}", orderResponse)**: Returns a 201 Created response with the location of the newly created order and the order details.

PUT: api/Orders/{orderId}

[HttpPut("{orderId}")]

```

public async Task<IActionResult> Put(Guid orderId, OrderUpdateRequest orderUpdateRequest)
{
    if (orderUpdateRequest == null)
    {
        return BadRequest("Invalid order data");
    }

    if (orderId != orderUpdateRequest.OrderID)
    {
        return BadRequest("OrderID in the URL doesn't match with the OrderID in the Request body");
    }
}

```

```
OrderResponse? orderResponse = await _ordersService.UpdateOrder(orderUpdateRequest);
```

```
if (orderResponse == null)
```

```
{
```

```
    return Problem("Error in updating order");
```

```
}
```

```
return Ok(orderResponse);
```

```
}
```

- **[HttpPut("{orderId}")]**: Routes to this action when the URL matches /api/Orders/{orderId}.
- **Put(Guid orderId, OrderUpdateRequest orderUpdateRequest)**: Updates an existing order. Expects an OrderUpdateRequest object in the request body.
- **BadRequest("Invalid order data")**: Returns if the request body is null.
- **BadRequest("OrderID in the URL doesn't match with the OrderID in the Request body")**: Returns if the OrderID in the URL does not match the one in the request body.
- **_ordersService.UpdateOrder(orderUpdateRequest)**: Calls the service layer to update the order.
- **Ok(orderResponse)**: Returns a 200 OK response with the updated order details.

DELETE: api/Orders/{orderId}

```
[HttpDelete("{orderId}")]
```

```
public async Task<IActionResult> Delete(Guid orderId)
```

```
{
```

```
    if (orderId == Guid.Empty)
```

```
    {
```

```
        return BadRequest("Invalid order ID");
```

```
    }
```

```
bool isDeleted = await _ordersService.DeleteOrder(orderID);
```

```
if (!isDeleted)
```

```
{
```

```
        return Problem("Error in deleting order");  
    }  
}
```

```
    return Ok(isDeleted);  
}
```

- **[HttpDelete("{orderId}")]**: Routes to this action when the URL matches /api/Orders/{orderId}.
- **Delete(Guid orderId)**: Deletes an order by its OrderID.
- **BadRequest("Invalid order ID")**: Returns if the orderId is empty.
- **_ordersService.DeleteOrder(orderID)**: Calls the service layer to delete the order.
- **Ok(isDeleted)**: Returns a 200 OK response indicating whether the order was successfully deleted.

.NET Microservices – Azure DevOps and AKS

Section 7: Sync Microservice Communication - Notes

Microservice Communication Patterns

In microservice architecture, services are typically deployed independently and communicate with each other to accomplish business goals. One common way for microservices to interact is through **HTTP-based communication**. Communication between microservices can generally be categorized into two main patterns:

1. **Synchronous Communication (Sync)**
2. **Asynchronous Communication (Async)**

Both patterns have their own advantages, challenges, and specific scenarios where they are more suited.

Synchronous Communication (Sync)

Definition:

Synchronous communication means that the client service sends a request to the server service and waits for the response before proceeding further. The process is "blocking" in nature, i.e., the client service is halted until a response is received.

The most common protocol for synchronous communication in microservices is **HTTP** with **REST APIs**, although other protocols like **gRPC** are also used.

Advantages of Synchronous Communication

1. **Simple and Intuitive:**
Synchronous communication is easy to understand and implement. It's very similar to how traditional applications communicate using HTTP/REST calls.
2. **Immediate Feedback:**
Clients receive an immediate response, allowing them to make decisions based on real-time data.
3. **Consistency:**
Since a response is required immediately, there is often a higher level of consistency in the state of the system because the client waits for a completed operation before continuing.
4. **Suitable for Request-Response Workflows:**
If the interaction involves asking for data or requesting the completion of an operation with immediate feedback (e.g., retrieving user details, processing a payment), synchronous communication is ideal.

Challenges of Synchronous Communication

1. **Blocking Nature:**
The client is blocked while waiting for a response, which can lead to poor performance if the server is slow or unresponsive.
2. **Tight Coupling:**
Synchronous communication creates tighter coupling between services because one service's availability and response time directly affects the calling service.
3. **Scalability Issues:**
If the number of calls grows, synchronous communication can cause bottlenecks. If the dependent service has a slowdown, it can ripple through the entire system.
4. **Timeouts and Reliability:**
Network failures or timeouts can become problematic. A failure in one microservice can cascade into the failure of other services if they depend on a synchronous response.

Scenarios Where Synchronous Communication is Best

- **Real-Time Data Requests:**
When you need an immediate response to proceed with further actions. For example, when you need to retrieve details from an inventory service before placing an order.
- **Client-Facing Operations:**
When the client expects a response in real time, such as web or mobile applications that require data from a microservice (e.g., showing the current balance in a banking app).
- **Read Operations:**
For operations that primarily involve retrieving data without complex or long-running processes.

Asynchronous Communication (Async)

Definition:

Asynchronous communication is non-blocking. The client service sends a request and does not wait for a response. The server service processes the request independently, and the client can continue its own workflow. Responses are usually sent via callbacks, message queues, or event streams.

Common implementations of asynchronous communication include:

- **Message Brokers** like RabbitMQ, Kafka, or Azure Service Bus
- **Event-Driven Architectures** using Event Sourcing

Advantages of Asynchronous Communication

1. **Decoupling:**
Asynchronous communication allows services to be more loosely coupled. Since services do not have to wait for one another, they can be more independent.
2. **Scalability:**
Asynchronous systems can handle a large number of requests without creating bottlenecks. The decoupled nature allows each service to scale independently.
3. **Non-Blocking:**
The client is not blocked while waiting for a response, making the system more efficient, especially when dealing with long-running tasks (e.g., order fulfillment, background processing).
4. **Resilience and Fault Tolerance:**
Even if one microservice is temporarily unavailable, requests can be queued and processed later. This ensures the system remains operational even in the face of partial failures.

Challenges of Asynchronous Communication

1. **Complexity:**
Handling asynchronous workflows can be more complex than synchronous communication. You have to manage message queues, callbacks, and retries, making error handling more difficult.
2. **Eventual Consistency:**
Asynchronous systems are typically **eventually consistent**. This means that while data will be consistent over time, there may be temporary discrepancies across services, which could lead to issues in highly transactional systems.
3. **Latency:**
Since the response is not immediate, there can be delays in processing. This may not be acceptable in scenarios where real-time feedback is required.
4. **Monitoring and Debugging:**
It can be harder to monitor and trace asynchronous calls, especially in distributed systems, where messages may be processed at different times and locations.

Scenarios Where Asynchronous Communication is Best

- **Long-Running Processes:**
For workflows that require a lot of processing time (e.g., batch jobs, order fulfillment, or report generation), asynchronous communication allows services to handle tasks in the background.
- **Event-Driven Systems:**
Systems built on an event-driven architecture where actions are triggered by certain events (e.g., sending an email after an order is placed) are best suited for asynchronous communication.
- **High Availability and Scalability:**
When services need to handle a high volume of requests, asynchronous patterns can scale more efficiently. For example, in a system where hundreds of thousands of requests are placed, asynchronous message queues can handle the load without causing delays.

Comparison: Synchronous vs Asynchronous Microservice Communication

1. Definition

- **Synchronous Communication:** This method requires the services to wait for a response from the other service before continuing with their own processing. Typically, HTTP/REST and gRPC are used for synchronous communication.
- **Asynchronous Communication:** In this method, services do not wait for a response from the other service. They continue processing and handle responses or notifications when they are available. This often uses message queues, such as RabbitMQ or Kafka.

2. Communication Flow

- **Synchronous Communication:** The client sends a request to the server and waits for the server to process the request and send back a response. The client is blocked until the response is received.
- **Asynchronous Communication:** The client sends a request to the server and proceeds without waiting for a response. The server processes the request and sends a notification or response back to the client when it is ready, which can be handled through callbacks or message queues.

3. Performance

- **Synchronous Communication:** Performance can be affected by the response time of the server. If the server is slow, the client's performance and throughput are impacted as it has to wait.
- **Asynchronous Communication:** Generally improves performance and throughput, as the client does not have to wait for the server's response and can handle multiple requests or perform other tasks in parallel.

4. Complexity

- **Synchronous Communication:** Easier to implement and understand because it follows a straightforward request-response model.
- **Asynchronous Communication:** More complex to implement due to the need for additional components such as message brokers, and managing the state of the communication and handling retries or failures.

5. Scalability

- **Synchronous Communication:** Scalability can be challenging as the system needs to handle a large number of simultaneous requests and responses. Scaling often involves increasing server capacity or implementing load balancing.
- **Asynchronous Communication:** More scalable as it decouples request processing from response handling. It allows the system to handle high loads more effectively and can process messages at different rates without affecting the client's performance.

6. Error Handling

- **Synchronous Communication:** Errors are typically handled immediately by sending an error response to the client. This can be straightforward but may impact the user experience if not managed properly.
- **Asynchronous Communication:** Error handling can be more complex, involving mechanisms for retries, error queues, or compensating actions. However, it can provide a more resilient system that can recover from transient issues.

7. Use Cases

- **Synchronous Communication:** Suitable for scenarios where real-time responses are critical, such as user interactions, financial transactions, or data retrieval operations where immediate feedback is required.
- **Asynchronous Communication:** Ideal for scenarios where tasks can be processed independently and do not require immediate responses, such as background jobs, notifications, or event-driven architectures.

Synchronous communication is straightforward and suitable for real-time interactions but can suffer from performance bottlenecks. Asynchronous communication, while more complex, offers better performance and scalability, making it suitable for high-load and decoupled systems.

Key Points for Interview Preparation

1. **Synchronous Communication:**
 - Blocks the client until a response is received.
 - Ideal for real-time, request-response workflows.

- Simpler but leads to tight coupling between services.
- Introduces challenges with scalability and fault tolerance.

2. **Asynchronous Communication:**

- Non-blocking, allowing services to work independently.
- Suitable for long-running tasks and event-driven architectures.
- More complex to implement but highly scalable and fault-tolerant.
- Introduces the concept of eventual consistency, which can be a challenge in transactional systems.

3. **Decoupling and Scalability:**

- Async communication allows for more decoupling between services, which is essential for scaling large systems.

4. **Eventual Consistency:**

- Be prepared to discuss how to handle eventual consistency and its implications on the business logic.

5. **Trade-offs:**

- Understanding when to use sync vs async is crucial. In an interview, emphasize the trade-offs between simplicity and flexibility, performance, and consistency.

Docker Compose Launch Profiles in Visual Studio

When developing microservices, especially with Docker, Visual Studio provides built-in support for managing and running multiple services using Docker Compose. A **Docker Compose launch profile** helps you configure how your services are built, run, and tested during development.

This section covers the following key aspects:

- Overview of Docker Compose Launch Profiles in Visual Studio
- Step-by-step guide to creating and using launch profiles
- Configuration options in launchSettings.json
- Best practices for using Docker Compose in Visual Studio
- Key points for interview preparation

1. Overview of Docker Compose Launch Profiles

A **launch profile** in Visual Studio allows developers to specify how a microservice should be launched for debugging and testing purposes. When working with multiple microservices, using Docker Compose helps spin up and coordinate multiple containers seamlessly. Visual Studio integrates with Docker Compose to provide a streamlined way to:

- Build, run, and debug multiple microservices together.
- Set up environment variables, ports, and volumes for individual services.
- Simulate a production-like environment locally.

The **launchSettings.json** file defines how Visual Studio should handle the project during the debugging phase. This file includes important configurations like the launch profile (which defines the environment, startup URL, and more), as well as settings for each microservice that Docker Compose will manage.

2. Creating and Using Docker Compose Launch Profiles

Step 1: Add Docker Support to Your Project

1. **Right-click on your solution** in Visual Studio and select **"Add" > "Container Orchestrator Support"**.
2. Choose **Docker Compose** as your orchestration tool. Visual Studio will automatically generate a `docker-compose.yml` file and integrate Docker support into your project.

Step 2: Create a Launch Profile

1. After adding Docker Compose support, Visual Studio will generate a **docker-compose launch profile** inside the `launchSettings.json` file.
2. You can view and modify this file by navigating to the **Properties** folder of the main project.

Step 3: Configure the Launch Profile

In the `launchSettings.json`, you'll see a "docker-compose" profile that looks something like this:

```
{  
  "profiles": {  
    "docker-compose": {  
      "commandName": "DockerCompose",  
      "serviceName": "webapi",  
      "dockerComposeFilePath": "docker-compose.yml",  
    }  
  }  
}
```

```

    "launchBrowser": true,
    "launchUrl": "{Scheme}://{ServiceHost}:{ServicePort}",
    "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
    }
}
}
}
}

```

- **commandName:** Defines how Visual Studio will launch the project. For Docker Compose, it should always be "DockerCompose".
- **serviceName:** This is the name of the service specified in the docker-compose.yml file. Visual Studio will run this service when you start the project.
- **dockerComposeFilePath:** Specifies the path to the docker-compose.yml file that defines how services are built and run.
- **launchBrowser:** If set to true, Visual Studio will automatically launch a web browser when the application starts.
- **launchUrl:** Specifies the URL that the browser should navigate to when the application launches.
- **environmentVariables:** Allows you to set environment variables for the application (such as ASPNETCORE_ENVIRONMENT).

Step 4: Customize the Docker Compose File

You can configure your services in the docker-compose.yml file, which defines how Docker should build and run your microservices. Here's a basic example:

```

version: '3.8'

services:
  products-microservice:
    image: harshhamicroservices/ecommerce-products-microservice:v1.0
    environment:
      - MYSQL_HOST=mysql-host
      - MYSQL_PASSWORD=admin
    ports:
      - "8080:8080"

```


users-microservice:

image: harshamicroservices/ecommerce-users-microservice:v1.0

environment:

- POSTGRES_HOST=postgres-host
- POSTGRES_PASSWORD=admin

ports:

- "9090:9090"

Visual Studio will use this Docker Compose file to coordinate the two microservices (products-microservice and users-microservice). The docker-compose launch profile ensures that both services are up and running when you start the solution in Visual Studio.

3. Configuration Options in launchSettings.json

When working with Docker Compose in Visual Studio, the launchSettings.json file plays a critical role in defining the development environment. Let's dive into some important properties:

- **commandName:**
The commandName for Docker Compose should always be "DockerCompose", which tells Visual Studio to use Docker Compose as the execution engine.
- **serviceName:**
Defines which service from the docker-compose.yml file should be the primary service for debugging. For example, "serviceName": "products-microservice" means that the products-microservice service will be launched and debugged.
- **launchBrowser:**
This boolean flag determines whether Visual Studio should automatically launch the default browser after starting the service. Set this to false if you don't want the browser to open automatically.
- **launchUrl:**
Defines the URL that Visual Studio should open when the application is launched. Placeholders like {Scheme}, {ServiceHost}, and {ServicePort} allow for dynamic URL generation.
- **environmentVariables:**
Set environment variables for your service here. This is especially useful for configuring the environment (e.g., Development, Production) or any other custom environment variables your service requires (like database credentials).

4. Best Practices for Using Docker Compose in Visual Studio

1. **Use Separate Compose Files for Development and Production:**
Keep separate docker-compose.yml files for development and production environments. For instance, use docker-compose.override.yml for development settings such as mounting local volumes for code hot-reloading, and reserve docker-compose.yml for production-like configurations.
2. **Leverage Debugging with Docker Compose:**
Visual Studio integrates with Docker to provide debugging capabilities for multiple services. This means you can set breakpoints, inspect variables, and step through code across microservices.
3. **Maintain Environment-Specific Settings:**
Use environment variables to manage environment-specific settings, like database connection strings and API keys. These can be specified in the launchSettings.json or passed directly through the docker-compose.yml file.
4. **Optimize for Rebuilds:**
Make sure to structure your Dockerfile in a way that minimizes rebuild times. For example, place frequently changing files like source code lower in the file and install dependencies first to leverage Docker's caching mechanism.

Key Points for Interview Preparation

1. **Launch Profiles:**
 - A launch profile defines how Visual Studio runs your application (including environment variables, ports, and URLs).
 - Docker Compose launch profiles are used when multiple microservices need to be run together.
2. **launchSettings.json:**
 - Contains key configurations for running the service in Visual Studio. It defines the commandName, serviceName, launchBrowser, and environment variables.
 - The "DockerCompose" command triggers the execution of services defined in docker-compose.yml.
3. **Visual Studio Integration:**
 - Visual Studio provides tight integration with Docker Compose, allowing you to manage and debug multiple services seamlessly.
4. **Environment Variables:**
 - Environment variables defined in launchSettings.json or the docker-compose.yml file allow you to configure microservices for different environments (e.g., Dev, QA, Production).

5. Service Coordination:

- Docker Compose enables you to run, test, and debug multiple microservices together, simulating real-world production environments locally.

Docker Compose in Visual Studio - Code Explanation

version: '3.4'

services:

ordersmicroservice.api:

image: ordersmicroserviceapi

build:

context: .

dockerfile: OrdersMicroservice.API/Dockerfile

environment:

- MONGODB_HOST=mongodb-container
- MONGODB_PORT=27017
- MONGODB_DATABASE=OrdersDatabase
- UsersMicroserviceName=users-microservice
- UsersMicroservicePort=9090
- ProductsMicroserviceName=products-microservice
- ProductsMicroservicePort=8080

ports:

- "7000:8080"

networks:

- orders-mongodb-network
- ecommerce-network

depends_on:

- mongodb-container

mongodb-container:

image: mongo:latest

ports:

- "27017:27017"

volumes:

- ../mongodb-init:/docker-entrypoint-initdb.d

networks:

- orders-mongodb-network

products-microservice:

image: products-microservice:latest

environment:

- ASPNETCORE_HTTP_PORTS=8080
- ASPNETCORE_ENVIRONMENT=Development
- MYSQL_HOST=mysql-container
- MYSQL_PORT=3306
- MYSQL_DATABASE=ecommerceproductsdatabase
- MYSQL_USER=root
- MYSQL_PASSWORD=admin

ports:

- "6000:8080"

networks:

- products-mysql-network
- ecommerce-network

depends_on:

- mysql-container

mysql-container:

image: mysql:8.0

environment:

- MYSQL_ROOT_PASSWORD=admin

ports:

- "3307:3306"

volumes:

- ../mysql-init:/docker-entrypoint-initdb.d

networks:

- products-mysql-network

users-microservice:

image: users-microservice:latest

environment:

- ASPNETCORE_HTTP_PORTS=8080
- ASPNETCORE_HTTP_ENVIRONMENT=Development
- POSTGRES_HOST=postgres-container
- POSTGRES_PORT=5432
- POSTGRES_DATABASE=eCommerceUsers
- POSTGRES_USER=postgres
- POSTGRES_PASSWORD=admin

ports:

- "5000:9090"

networks:

- users-postgres-network
- ecommerce-network

depends_on:

- postgres-container

postgres-container:

image: postgres:13

environment:

- POSTGRES_USER=postgres
- POSTGRES_PASSWORD=admin
- POSTGRES_DB=eCommerceUsers

ports:

- "5433:5432"

volumes:

- ../postgres-init:/docker-entrypoint-initdb.d

networks:

- users-postgres-network

networks:

orders-mongodb-network:

driver: bridge

products-mysql-network:

driver: bridge

users-postgres-network:

driver: bridge

ecommerce-network:

driver: bridge

1. Version of Docker Compose

version: '3.4'

- **Explanation:** This specifies the version of the Docker Compose file format. Version 3.4 is used to define services, networks, and volumes. It is compatible with the latest features, making it a good choice for modern applications like microservices.

2. Services Definition

This section defines the microservices and their configurations.

Orders Microservice

services:

ordersmicroservice.api:

image: ordersmicroserviceapi

build:

context: .

dockerfile: OrdersMicroservice.API/Dockerfile

- **Explanation:**

- services: This block defines each service (or container) that will be launched.
- ordersmicroservice.api: The name of the service for the Orders Microservice.
- image: The Docker image that will be used for the service. This can either point to an existing image (e.g., from Docker Hub) or be built from the source code.
- build: Specifies the build context and Dockerfile.
 - context: The current directory (.) is used as the build context. This means the source files for the Docker image are located in the current directory.
 - dockerfile: This points to the path of the Dockerfile that will be used to build the image (OrdersMicroservice.API/Dockerfile).

Environment Variables

environment:

- MONGODB_HOST=mongodb-container
- MONGODB_PORT=27017
- MONGODB_DATABASE=OrdersDatabase
- UsersMicroserviceName=users-microservice
- UsersMicroservicePort=9090
- ProductsMicroserviceName=products-microservice
- ProductsMicroservicePort=8080

- **Explanation:** The environment block sets environment variables for the service, which are passed to the application running inside the container.
 - MONGODB_HOST: The hostname for the MongoDB service. This is the name of the service (mongodb-container), which allows the Orders Microservice to connect to MongoDB.

- MONGODB_PORT: The port MongoDB will use.
- MONGODB_DATABASE: The name of the database to connect to.
- UsersMicroserviceName, UsersMicroservicePort: The name and port for the Users Microservice.
- ProductsMicroserviceName, ProductsMicroservicePort: The name and port for the Products Microservice.
- These variables allow the Orders Microservice to communicate with the MongoDB database and other microservices.

Ports Configuration

ports:

- "7000:8080"

- **Explanation:**

- This exposes port 8080 (the internal port of the container where the application runs) to port 7000 on the host machine. It allows external traffic to access the Orders Microservice via `http://localhost:7000`.

Networks and Dependencies

networks:

- orders-mongodb-network

- ecommerce-network

depends_on:

- mongodb-container

- **Explanation:**

- networks: Specifies the networks the service is connected to. Here, the Orders Microservice is connected to two networks:
 - orders-mongodb-network: To communicate with the MongoDB container.
 - ecommerce-network: To communicate with other services in the eCommerce system.
- depends_on: This indicates that the Orders Microservice depends on the mongodb-container. Docker Compose ensures that the MongoDB container is up and running before the Orders Microservice starts.

3. MongoDB Container

mongodb-container:

image: mongo:latest

ports:

- "27017:27017"

volumes:

- ../mongodb-init:/docker-entrypoint-initdb.d

networks:

- orders-mongodb-network

- **Explanation:**

- image: This pulls the latest MongoDB image from Docker Hub (mongo:latest).
- ports: Maps MongoDB's internal port (27017) to the host machine's port (27017).
- volumes: This mounts a local directory (../mongodb-init) to the MongoDB container at /docker-entrypoint-initdb.d. This is often used to initialize MongoDB with predefined scripts or data.
- networks: Connects the MongoDB container to the orders-mongodb-network to allow communication between it and the Orders Microservice.

4. Products Microservice

products-microservice:

image: products-microservice:latest

environment:

- ASPNETCORE_HTTP_PORTS=8080

- ASPNETCORE_ENVIRONMENT=Development

- MYSQL_HOST=mysql-container

- MYSQL_PORT=3306

- MYSQL_DATABASE=ecommerceproductsdatabase

- MYSQL_USER=root

- MYSQL_PASSWORD=admin

ports:

- "6000:8080"

networks:

- products-mysql-network

- ecommerce-network

depends_on:

- mysql-container

- **Explanation:**

- image: Specifies the Docker image for the Products Microservice.
- environment: Defines environment variables such as the HTTP ports, environment (Development), and MySQL connection settings (MYSQL_HOST, MYSQL_PORT, etc.).
- ports: Exposes port 8080 (internal) to port 6000 on the host.
- networks: Connects this service to products-mysql-network and ecommerce-network.
- depends_on: Ensures that the MySQL container is up and running before starting the Products Microservice.

5. MySQL Container

mysql-container:

image: mysql:8.0

environment:

- MYSQL_ROOT_PASSWORD=admin

ports:

- "3307:3306"

volumes:

- ../mysql-init:/docker-entrypoint-initdb.d

networks:

- products-mysql-network

- **Explanation:**

- image: Pulls the MySQL 8.0 image from Docker Hub.
- environment: Sets the root password for MySQL (MYSQL_ROOT_PASSWORD=admin).
- ports: Maps port 3306 (MySQL's default port) to 3307 on the host.

- volumes: Mounts a local directory (../mysql-init) to initialize the database.
- networks: Connects the MySQL container to products-mysql-network.

6. Users Microservice

users-microservice:

image: users-microservice:latest

environment:

- ASPNETCORE_HTTP_PORTS=8080
- ASPNETCORE_HTTP_ENVIRONMENT=Development
- POSTGRES_HOST=postgres-container
- POSTGRES_PORT=5432
- POSTGRES_DATABASE=eCommerceUsers
- POSTGRES_USER=postgres
- POSTGRES_PASSWORD=admin

ports:

- "5000:9090"

networks:

- users-postgres-network
- ecommerce-network

depends_on:

- postgres-container

- **Explanation:**

- image: Specifies the Docker image for the Users Microservice.
- environment: Defines environment variables for ASP.NET Core and PostgreSQL connection details.
- ports: Exposes port 9090 (internal) to port 5000 on the host.
- networks: Connects to users-postgres-network and ecommerce-network.
- depends_on: Ensures that the PostgreSQL container starts before the Users Microservice.

7. PostgreSQL Container

postgres-container:

image: postgres:13

environment:

- POSTGRES_USER=postgres
- POSTGRES_PASSWORD=admin
- POSTGRES_DB=eCommerceUsers

ports:

- "5433:5432"

volumes:

- ../postgres-init:/docker-entrypoint-initdb.d

networks:

- users-postgres-network

- **Explanation:**

- image: Uses the PostgreSQL 13 image from Docker Hub.
- environment: Defines PostgreSQL user, password, and database.
- ports: Maps PostgreSQL's port 5432 to port 5433 on the host.
- volumes: Mounts initialization scripts for the database from a local directory.
- networks: Connects to users-postgres-network.

8. Networks Definition

networks:

orders-mongodb-network:

driver: bridge

products-mysql-network:

driver: bridge

users-postgres-network:

driver: bridge

ecommerce-network:

driver: bridge

- **Explanation:**

- This section defines the networks used by the services.
- driver: bridge: Uses the bridge network driver, which allows containers to communicate with each other as if they are on the same network. This ensures services in different microservices can communicate efficiently.

Summary

This Docker Compose file defines a multi-container environment for an **eCommerce system** consisting of:

- Orders Microservice (with MongoDB).
- Products Microservice (with MySQL).
- Users Microservice (with PostgreSQL).

.NET Microservices – Azure DevOps and AKS

Section 8: Fault Tolerance - Notes

Introduction and Fault Tolerance in Microservices

Fault tolerance refers to the ability of a system to continue functioning even when some of its components fail. In microservices architecture, individual services may fail due to various reasons, such as network issues, service unavailability, or hardware problems. Fault-tolerant systems ensure that these failures do not bring down the entire application and offer mechanisms to gracefully handle or recover from these errors.

Why is Fault Tolerance Crucial for Microservices?

Microservices are designed to be independent, small, and decentralized. While this makes the system more flexible and scalable, it also introduces additional complexities. Here's why fault tolerance is essential in microservices:

1. **Distributed Nature:** Microservices run across multiple servers, environments, and networks. This distributed architecture increases the chances of partial failure (where one or more microservices fail while others continue to function). Ensuring fault tolerance helps mitigate the impact of these failures.
2. **Service Dependencies:** Microservices often depend on each other to complete business logic. For example, an eCommerce system might have separate services for user management, order processing, payment, and inventory. If the payment service fails, the entire transaction could fail unless proper fault tolerance mechanisms are in place.
3. **Continuous Availability:** In high-availability systems, especially in cloud-based applications, downtime can have serious consequences. Customers expect services to be available 24/7, and even small outages can lead to revenue loss, poor user experience, or reputational damage.
4. **Network Latency and Timeouts:** Since microservices communicate over the network using HTTP or other protocols, network latency or timeouts can occur. Fault tolerance mechanisms help ensure that temporary issues like network delays don't crash the system.

Key Fault Tolerance Techniques in Microservices:

Here are the primary strategies to build fault tolerance into microservices:

- **Retry Mechanisms:** If a request to a microservice fails due to a temporary issue, retrying the request after a short delay can often succeed. This ensures transient errors don't cause the service to fail.
- **Exponential Backoff:** A retry mechanism where the time between retries increases exponentially to reduce load on the system and prevent further failures.

- **Circuit Breaker Pattern:** When a service is consistently failing, the circuit breaker pattern can prevent further requests to the failing service, giving it time to recover. Once the service appears to be healthy again, the circuit breaker will "close," allowing requests through.
- **Bulkhead Isolation:** This strategy isolates failures within one part of the system, preventing them from cascading and affecting other parts. This is particularly useful in microservices, where a failure in one service shouldn't take down the entire application.
- **Fallbacks:** When a service fails, having a fallback (such as a cached response or a default value) ensures that the system can still provide some functionality instead of completely failing.

Importance of Polly in Fault Tolerance for ASP.NET Core

In ASP.NET Core, the **Polly** library is a robust and popular library for implementing fault tolerance strategies, including retries, circuit breakers, timeouts, and fallback policies. Polly allows developers to easily add fault tolerance strategies to their microservices with minimal effort.

- Polly provides a fluent API, making it easy to configure different fault-tolerance mechanisms.
- Polly integrates seamlessly with ASP.NET Core's dependency injection (DI) and HTTP clients, allowing for efficient handling of service-to-service communication failures.

Example: Microservice Scenario

Imagine an online shopping system where:

- The **PaymentService** may go down temporarily due to heavy traffic.
- The **InventoryService** might face intermittent failures due to data inconsistency.

In such cases:

- The retry mechanism (via Polly) can handle temporary PaymentService failures.
- Circuit breakers can ensure that, when InventoryService is repeatedly failing, the service is not flooded with requests, giving it time to recover.

Key Points to Remember (For Interview Preparation)

- **Fault Tolerance:** Crucial in microservices to ensure the system remains available and resilient even when individual services fail.
- **Microservices and Failure:** Service dependencies and network issues can lead to partial failures; fault tolerance helps mitigate the effects.
- **Retry Mechanisms:** Simple retry mechanisms help with transient errors.
- **Exponential Backoff and Circuit Breaker:** These advanced techniques help prevent overwhelming a failing service.

- **Polly in ASP.NET Core:** Polly is a powerful library for implementing fault tolerance patterns in .NET-based microservices.

Polly Library in ASP.NET Core

The **Polly** library is an open-source .NET library that provides a comprehensive set of fault-handling techniques like retries, circuit breakers, timeouts, fallback strategies, and bulkhead isolation. It enables you to build resilient, fault-tolerant systems by adding these policies to your ASP.NET Core applications with minimal effort.

Polly integrates smoothly with ASP.NET Core, especially for **HTTP clients** where service-to-service communication is common. It's most often used with `HttpClientFactory`, but can also be used with other dependencies and services.

Why Polly?

1. **Resiliency:** Adds resilience to your microservices by handling transient faults such as network issues, service timeouts, or intermittent service failures.
2. **Ease of Use:** Offers a fluent API, making it straightforward to define complex policies.
3. **Composability:** Polly allows you to combine multiple fault-handling policies (e.g., retries with circuit breakers) in a seamless manner.
4. **Flexibility:** You can use Polly for both synchronous and asynchronous operations.

Using Polly in ASP.NET Core with `HttpClientFactory`

Polly is commonly used with ASP.NET Core's `HttpClientFactory` to handle HTTP requests in a resilient way. The `HttpClientFactory` is a powerful feature introduced in ASP.NET Core 2.1, allowing you to create and manage instances of `HttpClient` in a flexible and robust manner.

Example Setup in ASP.NET Core:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddHttpClient<IProductService, ProductService>()
        .AddTransientHttpErrorPolicy(policyBuilder =>
            policyBuilder.WaitAndRetryAsync(3, retryAttempt =>
                TimeSpan.FromSeconds(Math.Pow(2, retryAttempt))
            )
        );
}
```



```
    )  
    );  
}
```

In the above example, Polly is used with `HttpClientFactory` to define a retry policy that retries a failed request up to 3 times, with an exponential backoff ($2^{\text{retryAttempt}}$ seconds).

Key Concepts:

- **AddHttpClient:** Registers an `HttpClient` for dependency injection with a specific retry policy.
- **WaitAndRetryAsync:** Defines a retry policy that waits for an increasing amount of time between retries, improving the system's resilience to transient errors.

WaitAndRetry Policy

The **WaitAndRetry** policy in Polly is used to retry a failed operation after waiting for a specified duration. It's a basic form of fault tolerance designed for handling transient faults (temporary issues that can succeed after a short delay).

How it Works:

- Polly attempts an action (such as an HTTP request).
- If the action fails, it waits for a predefined period (e.g., 2 seconds) and then retries the action.
- This process repeats until the maximum number of retries is reached or the action succeeds.

Example with WaitAndRetry:

```
public void ConfigureServices(IServiceCollection services)  
{  
    services.AddHttpClient<IProductService, ProductService>()  
        .AddTransientHttpErrorPolicy(policyBuilder =>  
            policyBuilder.WaitAndRetryAsync(3, retryAttempt =>  
                TimeSpan.FromSeconds(2))  
        );  
}
```

In this case, Polly will wait 2 seconds between each retry, up to a maximum of 3 retries.

Exponential Backoff

The **Exponential Backoff** is a more advanced version of the WaitAndRetry policy. Instead of waiting for a fixed duration between retries, the wait time increases exponentially with each retry attempt (e.g., 2^1 seconds, 2^2 seconds, 2^3 seconds, and so on). This helps prevent flooding a failing service with too many requests in a short period, giving it more time to recover.

Why Use Exponential Backoff?

1. **Reduces Load:** By gradually increasing the time between retries, you reduce the load on both the service and the client.
2. **Improves Recovery Chances:** If the service is under heavy load or temporarily unavailable, exponential backoff allows more time for it to recover.

Code Example for Exponential Backoff:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddHttpClient<IProductService, ProductService>()
        .AddTransientHttpErrorPolicy(policyBuilder =>
            policyBuilder.WaitAndRetryAsync(3, retryAttempt =>
                TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)) // 2^retryAttempt seconds
            )
        );
}
```

In this example:

- **1st retry:** Waits for 2 seconds (2^1)
- **2nd retry:** Waits for 4 seconds (2^2)
- **3rd retry:** Waits for 8 seconds (2^3)

Key Points to Remember (For Interview Preparation)

- **Polly Library:** Essential for implementing fault tolerance in ASP.NET Core applications. It offers various fault-handling policies such as retry, circuit breaker, timeout, and fallback.
- **WaitAndRetry Policy:** A simple retry mechanism where the system retries failed requests after waiting for a defined period.
- **Exponential Backoff:** An advanced retry strategy where the wait time between retries increases exponentially, helping prevent overwhelming a failing service.

- **Integration with HttpClientFactory:** Polly works seamlessly with HttpClientFactory in ASP.NET Core to make service-to-service communication more resilient.

Circuit Breakers

A **circuit breaker** is a fault-tolerance mechanism designed to prevent a system from repeatedly trying to execute an operation that is likely to fail. It helps to avoid overwhelming the failing service by "breaking" the circuit after a predefined number of failures, effectively blocking further attempts until the service is deemed healthy again.

Think of it as a real-life electrical circuit breaker: when too much current flows, it "trips" and prevents further current flow, protecting the system. Similarly, when too many consecutive failures occur in a service, the circuit breaker trips, preventing more requests and allowing the system to recover.

How Circuit Breaker Works:

- **Closed State:** Initially, the circuit is "closed," meaning that requests are allowed to flow through to the service.
- **Open State:** After a predefined number of consecutive failures, the circuit "opens," preventing further requests from being sent to the failing service. During this state, the system immediately fails fast for any new requests.
- **Half-Open State:** After a cool-down period, the circuit moves to a "half-open" state. During this period, a limited number of requests are allowed to pass through to test if the service has recovered.
- If the test requests succeed, the circuit is closed again. If they fail, the circuit reopens.

Code Example for Circuit Breaker with Polly:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddHttpClient<IProductService, ProductService>()
        .AddTransientHttpErrorPolicy(policyBuilder =>
            policyBuilder.CircuitBreakerAsync(
                handledEventsAllowedBeforeBreaking: 5, // Number of exceptions before the circuit
                opens
                durationOfBreak: TimeSpan.FromSeconds(30) // Time the circuit remains open
            )
        );
}
```

```
);  
}
```

In this example:

- After **5 consecutive failures**, the circuit opens.
- The circuit remains open for **30 seconds** before moving to the half-open state, allowing the system to check if the service has recovered.

When to Use Circuit Breakers:

- **Service Overload:** To protect an already struggling service from being overwhelmed with requests.
- **Intermittent Failures:** When dealing with services that are likely to experience short periods of downtime or failure.

Handling BrokenCircuitException

When a Polly circuit breaker is in an **open** or **half-open** state, it throws a `BrokenCircuitException` for any new requests, indicating that the circuit is currently preventing further execution.

Code Example of Handling BrokenCircuitException:

```
public async Task<ProductResponse> GetProductDetailsAsync(int productId)  
{  
    try  
    {  
        // Execute request through circuit breaker  
        return await _httpClient.GetFromJsonAsync<ProductResponse>($"api/products/{productId}");  
    }  
    catch (BrokenCircuitException ex)  
    {  
        // Handle the exception by returning a fallback response or logging the issue  
        _logger.LogError("Circuit is currently open. Service unavailable: {Exception}", ex);  
        throw new ServiceUnavailableException("Service is temporarily unavailable. Please try again later.");  
    }  
}
```

```
}
```

Here, when the circuit is open and a request is made, the `BrokenCircuitException` is caught and handled gracefully, preventing the application from crashing.

Key Notes:

- You can provide **fallback** behavior (e.g., returning a default response or cached data) when a `BrokenCircuitException` occurs.
- It's essential to log these exceptions to monitor the health of your services and circuits.

Fallback Policy

The **Fallback policy** is a crucial resiliency mechanism in Polly. It defines a default or alternative response when a particular operation fails. This is useful when you want your system to degrade gracefully instead of failing entirely.

How it Works:

- If an operation fails, Polly will invoke the fallback policy, allowing you to return a predefined result or execute alternative logic.
- This is often combined with other Polly policies like retry or circuit breaker.

Example of Fallback Policy:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddHttpClient<IProductService, ProductService>()
        .AddTransientHttpErrorPolicy(policyBuilder =>
            policyBuilder.FallbackAsync(
                fallbackAction: async (ct) =>
                {
                    return new HttpResponseMessage(HttpStatusCode.OK)
                    {
                        Content = new StringContent("Fallback response: service is currently unavailable")
                    };
                },
            ),
    },
```

```

onFallbackAsync: async (exception, context) =>
{
    _logger.LogError("Fallback executed. Exception: {Exception}", exception);
}
);
}

```

In this example:

- If the request fails, Polly executes the **fallbackAction** and returns a default HTTP response with a custom message.
- You can also log the fallback execution using `onFallbackAsync`.

Use Case for Fallback:

- **Service Unavailability:** When the primary service fails, you can return a default value or cached data to prevent full system failure.

Key Points to Remember (For Interview Preparation)

- **Circuit Breaker:** Helps prevent overwhelming a failing service by blocking further requests after consecutive failures.
 - States: **Closed**, **Open**, and **Half-Open**.
 - Use to protect services under stress and handle transient failures.
- **BrokenCircuitException:** Indicates that the circuit is open, and further execution is blocked. Handle it by providing fallback responses or alternative logic.
- **Fallback Policy:** Defines a default response or alternative execution when an operation fails, ensuring the system degrades gracefully.

Timeout Policy

The **Timeout Policy** in Polly ensures that if an operation takes longer than a specified amount of time, it is automatically abandoned and throws a `TimeoutRejectedException`. This is essential in microservices, where slow responses can lead to system bottlenecks or cascading failures.

By setting a timeout, you can make sure that slow or unresponsive services don't block resources indefinitely, allowing the system to handle failures more gracefully.

How Timeout Policy Works:

- You define a time limit for the execution of an operation.
- If the operation exceeds this time, Polly cancels the execution and throws an exception.
- Timeout policies are often used with retry or circuit breaker policies to handle failures robustly.

Code Example of Timeout Policy with Polly:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddHttpClient<IProductService, ProductService>()
        .AddTransientHttpErrorPolicy(policyBuilder =>
            policyBuilder
                .Or<TimeoutRejectedException>() // Handle timeout exceptions
                .TimeoutAsync(TimeSpan.FromSeconds(5)) // Set timeout for 5 seconds
        );
}
```

In this example:

- If the HTTP call to the product service takes longer than **5 seconds**, Polly cancels the request and throws a `TimeoutRejectedException`.

When to Use a Timeout Policy:

- When interacting with external services that may have long response times.
- To prevent resource blocking in your microservices when certain services take too long to respond.

Best Practices for Timeout Policy:

- Combine with **retry policies** to handle transient failures effectively.
- Set reasonable timeout values based on the expected response times of your services.

Handling TimeoutRejectedException

When a timeout occurs and Polly cancels the operation, a `TimeoutRejectedException` is thrown. Handling this exception allows you to log the issue or provide a fallback response, preventing the entire system from failing.

Code Example of Handling TimeoutRejectedException:

```
public async Task<ProductResponse?> GetProductDetailsAsync(int productId)
{
    try
    {
        // Make an HTTP call with a timeout policy in place
        return await _httpClient.GetFromJsonAsync<ProductResponse>($"api/products/{productId}");
    }
    catch (TimeoutRejectedException ex)
    {
        // Handle the timeout exception by logging the issue
        _logger.LogError("Request timed out. Service is unresponsive: {Exception}", ex);
        throw new ServiceUnavailableException("The service timed out. Please try again later.");
    }
}
```

In this example:

- If the request takes too long and a `TimeoutRejectedException` is thrown, we catch it and log the issue.
- A custom exception (`ServiceUnavailableException`) is thrown to notify the client that the service is currently unresponsive.

Bulkhead Isolation Policy

The **Bulkhead Isolation Policy** is a crucial mechanism in fault tolerance. It prevents a system from being overwhelmed by isolating resources for different workloads. The term "bulkhead" is borrowed from shipbuilding, where a ship's compartments (bulkheads) are isolated from one another to prevent flooding from spreading.

In microservices, bulkhead isolation ensures that a failure in one part of the system doesn't consume all resources, thereby preventing other parts of the system from being affected. This is especially useful when dealing with high-load services or when certain services might fail under pressure.

How Bulkhead Isolation Works:

- You can limit the number of concurrent executions or threads allocated to a particular resource or service.
- If the limit is reached, new requests are either queued or rejected.

Code Example of Bulkhead Isolation Policy with Polly:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddHttpClient<IProductService, ProductService>()
        .AddTransientHttpErrorPolicy(policyBuilder =>
            policyBuilder.BulkheadAsync(
                maxParallelization: 10, // Limit to 10 concurrent requests
                maxQueueingActions: 20 // Queue up to 20 additional requests
            )
        );
}
```

In this example:

- A maximum of **10 concurrent requests** is allowed for the product service.
- If more than 10 requests are made simultaneously, up to **20 additional requests** can be queued.

If both the concurrency and queuing limits are exceeded, the system rejects new requests, ensuring other services can continue to operate without being affected.

When to Use Bulkhead Isolation:

- In services that deal with unpredictable loads or peak traffic.
- When you want to ensure that a single overloaded service does not consume all system resources, protecting other services from failure.

Key Points to Remember (For Interview Preparation)

- **Timeout Policy:** Automatically cancels an operation if it exceeds a defined time limit, preventing slow or unresponsive services from blocking resources. Use it to handle long-running operations.
 - Exception: `TimeoutRejectedException` is thrown when the time limit is exceeded.
- **Bulkhead Isolation Policy:** Limits the number of concurrent executions or threads for a specific service to prevent system overload. It ensures that a failure in one part of the system doesn't impact the entire system.
- **Best Practices for Combining Policies:** Use a combination of **timeout**, **retry**, **circuit breaker**, and **bulkhead** policies to build a highly resilient microservices system. Each policy addresses a different failure scenario, providing comprehensive fault tolerance

Code Explanation

This code is for implementing **fault tolerance** in an ASP.NET Core application using the **Polly** library. It handles HTTP calls to different microservices and applies various Polly policies, such as retry, circuit breaker, timeout, fallback, and bulkhead isolation, to ensure the system is resilient to transient and persistent faults.

1. Program.cs - Configuring HTTP Clients with Fault-Tolerant Policies

```
builder.Services.AddHttpClient<UsersMicroserviceClient>(client =>
{
    client.BaseAddress = new
Uri($"http://{builder.Configuration["UsersMicroserviceName"]}:{builder.Configuration["UsersMicroservicePort"]}");
})
.AddPolicyHandler(

builder.Services.BuildServiceProvider().GetRequiredService<IUsersMicroservicePolicies>().GetCombinedPolicy());
```

- `AddHttpClient<UsersMicroserviceClient>`: This creates a typed `HttpClient` for communicating with the **UsersMicroservice**.
- `client.BaseAddress = new Uri(...)`: The `BaseAddress` specifies the root URL for the **UsersMicroservice** by reading the name and port from the configuration (appsettings.json or environment variables).

- `AddPolicyHandler()`: Adds fault tolerance using Polly policies for HTTP requests.
 - `builder.Services.BuildServiceProvider().GetRequiredService<IUsersMicroservicePolicies>()`: This retrieves an implementation of `IUsersMicroservicePolicies`, which contains the policies for fault tolerance.
 - `.GetCombinedPolicy()`: This method returns a **combined policy** (retry, circuit breaker, and timeout).

```
builder.Services.AddHttpClient<ProductsMicroserviceClient>(client =>
{
    client.BaseAddress = new
Uri($"http://{builder.Configuration["ProductsMicroserviceName"]}:{builder.Configuration["Products
MicroservicePort"]}");
})
```

```
.AddPolicyHandler(
```

```
builder.Services.BuildServiceProvider().GetRequiredService<IProductsMicroservicePolicies>().GetFall
backPolicy())
```

```
.AddPolicyHandler(
```

```
builder.Services.BuildServiceProvider().GetRequiredService<IProductsMicroservicePolicies>().GetBul
kheadIsolationPolicy());
```

- `AddHttpClient<ProductsMicroserviceClient>`: Configures the `HttpClient` for **ProductsMicroservice**.
- `client.BaseAddress = new Uri(...)`: The root URL is configured similarly to the `UsersMicroservice`.
- **Policies for ProductsMicroservice:**
 - `GetFallbackPolicy()`: Adds a fallback policy, where, if the service fails, dummy data will be returned instead.
 - `GetBulkheadIsolationPolicy()`: Adds a **Bulkhead Isolation policy** to limit concurrent requests and avoid overwhelming the service.

2. Polly Policies - Interfaces and Classes

IPollyPolicies Interface

This interface defines three core Polly policies:

```
public interface IPollyPolicies
{
    IAsyncPolicy<HttpResponseMessage> GetRetryPolicy(int retryCount);

    IAsyncPolicy<HttpResponseMessage> GetCircuitBreakerPolicy(int
handledEventsAllowedBeforeBreaking, TimeSpan durationOfBreak);

    IAsyncPolicy<HttpResponseMessage> GetTimeoutPolicy(TimeSpan timeout);
}
```

- **GetRetryPolicy():** Defines a retry mechanism where requests are retried a specified number of times before failing.
- **GetCircuitBreakerPolicy():** Implements a circuit breaker pattern to temporarily block requests after repeated failures.
- **GetTimeoutPolicy():** Cancels requests that exceed a specified time limit.

IProductsMicroservicePolicies Interface

This interface defines additional policies specific to the **ProductsMicroservice**:

```
public interface IProductsMicroservicePolicies
{
    IAsyncPolicy<HttpResponseMessage> GetFallbackPolicy();

    IAsyncPolicy<HttpResponseMessage> GetBulkheadIsolationPolicy();
}
```

- **GetFallbackPolicy():** Returns a fallback response when the request fails.
- **GetBulkheadIsolationPolicy():** Limits the number of concurrent requests to avoid overloading the service.

IUsersMicroservicePolicies Interface

This interface defines the combined fault-tolerant policy for the **UsersMicroservice**:

```
public interface IUsersMicroservicePolicies
{
    IAsyncPolicy<HttpResponseMessage> GetCombinedPolicy();
}
```

3. Polly Policies Implementation

PollyPolicies Class - Core Retry, Circuit Breaker, Timeout Policies

```
public class PollyPolicies : IPollyPolicies
{
    private readonly ILogger<UsersMicroservicePolicies> _logger;

    public PollyPolicies(ILogger<UsersMicroservicePolicies> logger)
    {
        _logger = logger;
    }

    • ILogger<UsersMicroservicePolicies>: Used for logging retry attempts, circuit breaker state changes, etc.
```

GetRetryPolicy() - Retry with Exponential Backoff

```
public IAsyncPolicy<HttpResponseMessage> GetRetryPolicy(int retryCount)
{
    AsyncRetryPolicy<HttpResponseMessage> policy = Policy.HandleResult<HttpResponseMessage>(r
=> !r.IsSuccessStatusCode)
        .WaitAndRetryAsync(
            retryCount: retryCount,
            sleepDurationProvider: retryAttempt => TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)),
            onRetry: (outcome, timespan, retryAttempt, context) =>
        {
```

```

        _logger.LogInformation($"Retry {retryAttempt} after {timespan.TotalSeconds} seconds");
    });

    return policy;
}

```

- **WaitAndRetryAsync()**: Defines the retry logic.
 - retryCount: How many times the request will be retried.
 - sleepDurationProvider: The delay between retries increases exponentially ($2^{\text{retryAttempt}}$).
 - onRetry: Logs retry attempts.

GetCircuitBreakerPolicy() - Circuit Breaker Pattern

```

public IAsyncPolicy<HttpResponseMessage> GetCircuitBreakerPolicy(int
handledEventsAllowedBeforeBreaking, TimeSpan durationOfBreak)
{
    AsyncCircuitBreakerPolicy<HttpResponseMessage> policy =
    Policy.HandleResult<HttpResponseMessage>(r => !r.IsSuccessStatusCode)
    .CircuitBreakerAsync(
        handledEventsAllowedBeforeBreaking: handledEventsAllowedBeforeBreaking,
        durationOfBreak: durationOfBreak,
        onBreak: (outcome, timespan) =>
        {
            _logger.LogInformation($"Circuit breaker opened for {timespan.TotalMinutes} minutes due to
consecutive 3 failures.");
        },
        onReset: () => {
            _logger.LogInformation($"Circuit breaker closed.");
        }
    );

    return policy;
}

```

- **CircuitBreakerAsync()**: Blocks requests for a specified period if too many failures occur.
 - `handledEventsAllowedBeforeBreaking`: Number of failures allowed before the circuit "opens".
 - `durationOfBreak`: Time period the circuit remains open before retrying requests.

GetTimeoutPolicy() - Timeout Handling

```
public IAsyncPolicy<HttpResponseMessage> GetTimeoutPolicy(TimeSpan timeout)
{
    AsyncTimeoutPolicy<HttpResponseMessage> policy =
    Policy.TimeoutAsync<HttpResponseMessage>(timeout);

    return policy;
}
```

- **TimeoutAsync()**: Cancels requests that exceed the specified timeout duration.

ProductsMicroservicePolicies Class - Fallback and Bulkhead Isolation

GetBulkheadIsolationPolicy() - Bulkhead Isolation

```
public IAsyncPolicy<HttpResponseMessage> GetBulkheadIsolationPolicy()
{
    AsyncBulkheadPolicy<HttpResponseMessage> policy =
    Policy.BulkheadAsync<HttpResponseMessage>(
        maxParallelization: 2,
        maxQueuingActions: 40,
        onBulkheadRejectedAsync: (context) =>
        {
            _logger.LogWarning("BulkheadIsolation triggered. Can't send any more requests.");
            throw new BulkheadRejectedException("Bulkhead queue is full");
        });

    return policy;
}
```

- **BulkheadAsync()**: Limits the number of concurrent requests.
 - maxParallelization: Maximum concurrent requests allowed.
 - maxQueueingActions: Number of additional requests allowed in the queue.

GetFallbackPolicy() - Fallback with Dummy Data

```
public IAsyncPolicy<HttpResponseMessage> GetFallbackPolicy()
{
    AsyncFallbackPolicy<HttpResponseMessage> policy =
    Policy.HandleResult<HttpResponseMessage>(r => !r.IsSuccessStatusCode)
    .FallbackAsync(async (context) =>
    {
        _logger.LogWarning("Fallback triggered: Returning dummy data.");

        ProductDTO product = new ProductDTO(Guid.Empty, "Temporarily Unavailable", "Temporarily
        Unavailable", 0, 0);

        var response = new HttpResponseMessage(HttpStatusCode.OK)
        {
            Content = new StringContent(JsonSerializer.Serialize(product), Encoding.UTF8,
            "application/json")
        };

        return response;
    });

    return policy;
}
```

- **FallbackAsync()**: Returns dummy data if the request fails.

UsersMicroservicePolicies Class - Combined Policies

```
public IAsyncPolicy<HttpResponseMessage> GetCombinedPolicy()
{
    var retryPolicy = _pollyPolicies.GetRetryPolicy(5);
    var circuitBreakerPolicy = _pollyPolicies.GetCircuitBreakerPolicy(3, TimeSpan.FromMinutes(2));
    var timeoutPolicy = _pollyPolicies.GetTimeoutPolicy(TimeSpan.FromSeconds(5));

    AsyncPolicyWrap<HttpResponseMessage> wrappedPolicy = Policy.WrapAsync(retryPolicy,
    circuitBreakerPolicy, timeoutPolicy);

    return wrappedPolicy;
}
```

- **Policy.WrapAsync():** Combines multiple policies (retry, circuit breaker, and timeout) into a single, fault-tolerant policy for UsersMicroservice.

Summary (For Interview Preparation)

- **Retry Policy:** Retries failed requests with exponential backoff to handle transient faults.
- **Circuit Breaker:** Blocks requests temporarily after multiple consecutive failures to prevent overwhelming a failing service.
- **Timeout Policy:** Cancels requests if they take too long to prevent resource exhaustion.
- **Fallback Policy:** Returns predefined dummy data when the service fails, ensuring a graceful failure.
- **Bulkhead Isolation:** Limits the number of concurrent and queued requests to prevent overloading the service.

.NET Microservices – Azure DevOps and AKS

Section 9: Caching - Notes

Introduction to Caching

Caching is a process of temporarily storing frequently accessed data in memory, allowing faster retrieval in future requests. This reduces the need to fetch data from the primary data source (like databases) for every request. Caching plays a crucial role in enhancing the performance and scalability of applications, particularly in high-traffic environments, by reducing latency and lowering the load on databases and APIs.

Types of Caching

1. In-Memory Caching:

- Stores data directly in the memory (RAM) of the local application server.
- Examples: ASP.NET Core's **MemoryCache**.
- Suitable for small-scale applications with a single server, but doesn't work well in **distributed systems**.

2. Distributed Caching:

- Stores cache data in a centralized, shared location accessible to multiple servers.
- It's crucial in **microservices** and **cloud-based architectures**, where applications are spread across many servers.
- **Redis** is a popular choice for distributed caching.

In distributed systems, local in-memory caching may lead to inconsistencies because each server holds its own cache. A distributed cache ensures consistency across all instances of the application.

What is Redis?

Redis (Remote Dictionary Server) is an open-source, in-memory data structure store that can be used as a **cache**, database, or message broker. It is designed for high-performance operations like caching, as it stores everything in memory, making data retrieval incredibly fast. Redis supports complex data types such as strings, lists, sets, and hashes, making it versatile for many use cases.

Why Redis for Caching?

- **In-Memory:** Redis is an in-memory cache, meaning that all data is stored in RAM, ensuring very low latency in data retrieval.
- **High Performance:** Redis is capable of handling millions of requests per second due to its lightweight nature.
- **Persistence Options:** While Redis is primarily in-memory, it can persist data to disk if needed, combining speed with reliability.
- **Scalability:** Redis can operate in distributed environments, making it ideal for large-scale, cloud-based applications.
- **Data Structures:** Redis supports multiple data types (strings, lists, hashes, sets), which can cater to various caching requirements.

Example: An e-commerce site can cache user profiles, product information, and shopping cart details in Redis, which can be quickly accessed during peak traffic without needing a round trip to the database.

Key Redis Features for Caching

- **TTL (Time-To-Live):** You can set expiration times for cached data, ensuring that stale data is automatically purged.
- **Eviction Policies:** Redis offers configurable eviction policies (e.g., Least Recently Used - LRU) to remove old or unused cache entries when memory limits are reached.
- **Cluster Support:** Redis can be scaled horizontally using clusters to distribute the load across multiple nodes.
- **Pub/Sub Messaging:** Redis can also be used for messaging between microservices through its publish-subscribe mechanism.

Common Use Cases of Redis Caching in ASP.NET Core

- **Session State Management:** Redis is used to store user session data in distributed systems.
- **Output Caching:** Cache the output of expensive computations or frequently accessed web pages, improving response times.
- **Database Query Results:** Cache results of complex database queries to reduce database load.
- **API Caching:** Cache API responses, reducing the need for repetitive external calls.

Example: If you have an expensive query fetching top 10 products from a database, caching the result in Redis can reduce database load. Instead of querying the database every time, the system checks if the result exists in Redis first.

Why Use Redis over Other Caching Solutions?

- **Scalability:** Redis handles distributed caching well, making it suitable for cloud and microservice-based architectures.
- **Data Durability (Optional):** Redis can persist data to disk, ensuring durability beyond simple in-memory caching.
- **Wide Language Support:** Redis supports a wide variety of programming languages, including C#, Java, Python, and more.
- **Replication & High Availability:** Redis offers built-in replication, meaning data can be replicated across multiple servers to ensure high availability.

Redis is widely used in high-traffic environments like e-commerce platforms, content management systems, and IoT applications to ensure smooth and fast data retrieval.

Summary (for Interview Preparation)

- **What is Redis?:** A high-performance, in-memory data store used for caching, databases, and message brokering.
- **Benefits of Caching:** Faster data retrieval, reduced load on the primary data source, improved scalability, and performance.
- **Distributed Caching:** Redis enables distributed caching, making it suitable for cloud-based and microservice architectures.
- **Redis Key Features:** In-memory caching, data persistence options, TTL (Time-To-Live), eviction policies, support for complex data types, and clustering.
- **Use Cases:** Session management, output caching, database query results caching, API response caching, etc.

Redis NuGet Package

In ASP.NET Core, **Redis** is integrated into your application through the **StackExchange.Redis** NuGet package, a popular and high-performance client library used to communicate with Redis from .NET applications.

Step 1: Installing the Redis NuGet Package

To use Redis in your ASP.NET Core application, you need to install the **StackExchange.Redis** package. Here's how to do it:

1. **Via NuGet Package Manager:**
 - In Visual Studio, right-click on your project, select "Manage NuGet Packages."
 - Search for "StackExchange.Redis."
 - Select the package and install the latest stable version.
2. **Via .NET CLI:** You can also install the package using the .NET CLI by running the following command in your project directory:

```
dotnet add package StackExchange.Redis
```

This installs the **StackExchange.Redis** client, which provides easy-to-use methods for connecting to Redis and performing operations like setting and retrieving cache values.

Step 2: Configuring Redis in ASP.NET Core

After installing the NuGet package, the next step is to configure Redis as the distributed cache provider in your ASP.NET Core application.

You need to modify the Program.cs or Startup.cs file to register Redis as a service.

Here's how to set up Redis in **Program.cs**:

```
var builder = WebApplication.CreateBuilder(args);

// Configure Redis as the distributed cache
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration = builder.Configuration.GetConnectionString("RedisConnection");
    options.InstanceName = "RedisCacheInstance";
});

var app = builder.Build();
app.Run();
```

Explanation:

- **AddStackExchangeRedisCache:** This extension method configures Redis as the distributed cache provider.
- **options.Configuration:** This specifies the Redis server connection string (e.g., localhost:6379 for local testing, or the Redis server URL for production).
- **options.InstanceName:** A logical name used to prefix Redis keys, allowing multiple applications to share the same Redis instance without key collisions.

Connection String for Redis:

The Redis connection string is typically stored in the appsettings.json file:

```
{  
  "ConnectionStrings": {  
    "RedisConnection": "localhost:6379"  
  }  
}
```

- **localhost:6379** is the default address for a locally running Redis instance. Replace this with the Redis server's IP address or hostname when deploying to a production environment.

Step 3: Accessing the Distributed Cache

In ASP.NET Core, the **IDistributedCache** interface provides methods to interact with the distributed cache (which, in this case, will be Redis). The following methods are available in the **IDistributedCache** interface:

- **GetAsync(string key):** Retrieves the cached value for the given key.
- **SetAsync(string key, byte[] value):** Caches the value for the specified key.
- **RemoveAsync(string key):** Removes the cached value associated with the given key.
- **RefreshAsync(string key):** Extends the expiration of a cached item, keeping it alive.

These methods will be used to read from and write to the Redis cache.

Summary (for Interview Preparation)

- **StackExchange.Redis:** A high-performance client library used in .NET for communicating with Redis.
- **Installation:** Install the package via **NuGet** or **.NET CLI**.

- **Configuration:** Redis is registered as the distributed cache provider using `AddStackExchangeRedisCache` in the service configuration.
- **Connection String:** Typically configured in `appsettings.json`. For local Redis, use `localhost:6379`; for production, use the Redis server's IP or URL.
- **IDistributedCache** Interface: Provides methods like `GetAsync`, `SetAsync`, `RemoveAsync`, and `RefreshAsync` to work with Redis as a distributed cache.

Redis Docker Image

When developing or testing with Redis, you can easily run Redis as a Docker container instead of installing it directly on your system. Using Docker allows for quick setup and easy removal of the Redis instance when you're done.

Step 1: Pulling the Redis Docker Image

The official Redis Docker image is available on **Docker Hub**. You can pull it using the following command:

```
docker pull redis
```

This command downloads the latest official Redis image from Docker Hub, which is maintained by the Redis team.

Step 2: Running the Redis Container

After pulling the image, you can run a Redis container using the `docker run` command:

```
docker run --name redis-server -p 6379:6379 -d redis
```

- **--name redis-server:** This names the container `redis-server` for easy reference.
- **-p 6379:6379:** Maps port 6379 on your local machine to port 6379 inside the container (Redis runs on port 6379 by default).
- **-d redis:** Runs the Redis container in detached mode (in the background).

This starts a Redis container that will listen for requests on `localhost:6379`.

Step 3: Verifying Redis Is Running

To verify that the Redis container is running, use the following Docker command:

```
docker ps
```

This command lists all running containers. You should see an entry for the redis-server container, with a status indicating it is up and running.

Example output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
abcd1234	redis	"docker-entry..."	1 minute ago	Up 1 minute	0.0.0.0:6379->6379/tcp
redis-server					

The output shows that Redis is mapped to port **6379** on your local machine.

Step 4: Interacting with Redis in the Container

You can execute Redis commands directly inside the running container by using the `docker exec` command.

To open the Redis CLI inside the container, run:

```
docker exec -it redis-server redis-cli
```

This opens an interactive terminal inside the Redis container. You can now run Redis commands, such as `PING`, to check connectivity:

```
PING
```

The expected response is:

```
PONG
```

This confirms that Redis is running and responding correctly inside the Docker container.

Step 5: Stopping and Removing the Redis Container

When you are done working with the Redis container, you can stop and remove it with the following commands:

1. **Stopping the container:**

```
docker stop redis-server
```

2. **Removing the container:**


```
docker rm redis-server
```

These commands will stop the Redis container and remove it from Docker, cleaning up your environment.

Summary (for Interview Preparation)

- **Redis Docker Image:** Available on Docker Hub and can be pulled using `docker pull redis`.
- **Running the Redis Container:** Use `docker run -p 6379:6379 -d redis` to run Redis as a container, mapping it to port 6379.
- **Verifying Redis:** Use `docker ps` to check that the Redis container is running.
- **Interacting with Redis:** Use `docker exec -it redis-server redis-cli` to open the Redis CLI inside the container.
- **Stopping and Removing:** Use `docker stop` and `docker rm` to stop and remove the Redis container when it's no longer needed.

Reading and Writing into Redis Cache (using IDistributedCache)

1. Introduction to IDistributedCache

In .NET Core, the `IDistributedCache` interface provides a mechanism for working with distributed caches like Redis. This interface allows you to perform caching operations in a unified way, regardless of the caching provider (Redis, SQL Server, etc.). When using Redis, `IDistributedCache` offers an abstraction to easily interact with Redis as a key-value store.

Key methods of `IDistributedCache`:

- **GetAsync:** Retrieves a cached item.
- **SetAsync:** Stores an item in the cache.
- **RemoveAsync:** Removes an item from the cache.
- **RefreshAsync:** Refreshes the expiration of a cache entry.

When working with Redis, these methods map to common Redis commands (e.g., GET, SET, DEL).

2. Configuring Redis in the .NET Core Application

To use Redis with `IDistributedCache`, we first need to configure Redis in the application's `Program.cs` or `Startup.cs`.

```
public void ConfigureServices(IServiceCollection services)
```

```

{
    // Add Redis to IDistributedCache
    services.AddStackExchangeRedisCache(options =>
    {
        options.Configuration = Configuration.GetConnectionString("RedisConnection");
        options.InstanceName = "MyRedisInstance"; // Optional, to namespace cache keys
    });
}

```

- **AddStackExchangeRedisCache:** This method registers Redis as the distributed cache provider in .NET Core using the StackExchange.Redis library.
- **options.Configuration:** This is the Redis connection string, typically including the Redis server's IP and port.
- **options.InstanceName:** Provides an optional namespace prefix for your cache keys, which can help in multi-application environments.

3. Service Class for Caching (Using IDistributedCache)

Let's now create a simple service class that uses IDistributedCache to perform caching operations. In this example, we'll create a ProductCacheService that reads from and writes to Redis.

Example:

```

using Microsoft.Extensions.Caching.Distributed;
using System.Text.Json;
using System.Threading.Tasks;

public class ProductCacheService
{
    private readonly IDistributedCache _cache;

    public ProductCacheService(IDistributedCache cache)
    {
        _cache = cache;
    }
}

```

```
// Fetch product from cache by ID
public async Task<ProductDTO> GetProductAsync(string productId)
{
    var cachedProduct = await _cache.GetStringAsync(productId);

    if (cachedProduct != null)
    {
        return JsonSerializer.Deserialize<ProductDTO>(cachedProduct);
    }

    return null; // Product not found in cache
}

// Store product in cache with expiration
public async Task SetProductAsync(string productId, ProductDTO product)
{
    var cacheOptions = new DistributedCacheEntryOptions()
        .SetAbsoluteExpiration(TimeSpan.FromMinutes(30)); // Expiration time of 30 minutes

    var serializedProduct = JsonSerializer.Serialize(product);

    await _cache.SetStringAsync(productId, serializedProduct, cacheOptions);
}

// Remove product from cache
public async Task RemoveProductAsync(string productId)
{
    await _cache.RemoveAsync(productId);
}
}
```

Explanation of Code:

1. **Constructor Injection:** We inject the IDistributedCache service into our ProductCacheService to interact with Redis.
2. **GetProductAsync:**
 - Uses GetStringAsync to retrieve a cached product by its ID.
 - The product data, if found, is deserialized from JSON back into a ProductDTO.
3. **SetProductAsync:**
 - Takes a ProductDTO and stores it in Redis using SetStringAsync.
 - Sets an absolute expiration of 30 minutes, meaning the cache entry will be removed after that time.
4. **RemoveProductAsync:**
 - Deletes the cached product from Redis using RemoveAsync.

4. Interacting with the Service

Here's an example of how you might interact with this service in a controller:

```
[ApiController]
```

```
[Route("api/[controller]")]
```

```
public class ProductsController : ControllerBase
```

```
{
```

```
    private readonly ProductCacheService _cacheService;
```

```
    public ProductsController(ProductCacheService cacheService)
```

```
    {
```

```
        _cacheService = cacheService;
```

```
    }
```

```
    [HttpGet("{id}")]
```

```
    public async Task<ActionResult> GetProduct(string id)
```

```
    {
```

```
        var product = await _cacheService.GetProductAsync(id);
```

```

    if (product != null)
    {
        return Ok(product); // Return cached product
    }

    // Simulate fetching product from database (if not cached)
    product = await GetProductFromDatabaseAsync(id);

    if (product != null)
    {
        await _cacheService.SetProductAsync(id, product); // Cache the product
        return Ok(product);
    }

    return NotFound(); // Product not found
}

private async Task<ProductDTO> GetProductFromDatabaseAsync(string productId)
{
    // Simulated database fetch logic
    return new ProductDTO { ProductID = Guid.NewGuid(), ProductName = "Sample Product" };
}
}

```

Key Points:

- **Caching in the Controller:** Before fetching from the database, we first try to retrieve the product from Redis. If the product is not found in the cache, it's fetched from the database, cached, and then returned to the client.
- **Cache Expiration:** We set a 30-minute expiration to avoid serving outdated data indefinitely.

Summary (for Interview Preparation)

- **IDistributedCache Interface:** A standard interface in .NET Core to interact with different caching providers like Redis, SQL Server, etc.
- **StackExchange.Redis:** The package used for Redis caching in .NET Core.
- **Basic Methods:** GetAsync, SetAsync, RemoveAsync are the main methods to interact with the cache.
- **Expiration:** Cache entries can have absolute or sliding expiration.
- **Service Layer Integration:** The service layer (like ProductCacheService) abstracts caching logic, making the controller code cleaner and more maintainable.

.NET Microservices – Azure DevOps and AKS

Section 10: API Gateway using Ocelot - Notes

Introduction to API Gateway

An API Gateway is a server that acts as an API front-end, receiving API requests, routing them to the appropriate microservice, and then returning the responses back to the client. It sits between the client and your backend services, providing a single entry point for client requests.

Functions of an API Gateway:

1. **Request Routing:** Directs incoming requests to the appropriate microservice.
2. **Load Balancing:** Distributes incoming traffic evenly across multiple instances of a service.
3. **Authentication and Authorization:** Manages security by authenticating and authorizing requests.
4. **Aggregation:** Combines responses from multiple microservices into a single response for the client.
5. **Rate Limiting:** Controls the rate of requests to prevent overloading services.
6. **Caching:** Improves performance by caching frequently requested data.
7. **Logging and Monitoring:** Tracks and logs requests for monitoring and debugging purposes.
8. **Transformation:** Modifies requests and responses, such as adding headers or transforming data formats.

Why Use an API Gateway?

- **Simplifies Client Communication:** Clients make requests to a single gateway instead of multiple services.
- **Centralized Management:** Provides a central point for managing cross-cutting concerns like security, logging, and monitoring.
- **Improves Security:** Handles authentication, authorization, and SSL termination.
- **Optimizes Performance:** Supports caching and load balancing to improve response times and availability.
- **Facilitates Service Composition:** Aggregates results from multiple services into a single response.

Common API Gateway Patterns:

1. **Proxy Pattern:** The gateway simply forwards requests to the backend services.
2. **Aggregator Pattern:** Combines responses from multiple services into one response.
3. **Routing Pattern:** Routes requests based on URL paths, HTTP methods, or headers.
4. **Security Pattern:** Handles authentication, authorization, and SSL termination.

Introduction to Ocelot

Ocelot is a lightweight API Gateway built for .NET applications. It simplifies the management of microservices by providing features such as routing, load balancing, caching, and policy handling. Ocelot is particularly well-suited for environments where multiple services need to be exposed through a single entry point.

Key Features of Ocelot

1. **Routing:** Ocelot maps incoming HTTP requests to appropriate downstream services, making it easier to manage and direct traffic.
2. **Load Balancing:** Distributes requests among multiple instances of a service to ensure even distribution and high availability.
3. **Caching:** Supports response caching to reduce latency and improve performance by avoiding repeated calls to downstream services.
4. **Security:** Provides mechanisms for securing routes, including support for authentication and authorization.
5. **QoS (Quality of Service):** Uses Polly integration to handle retries, circuit breakers, and other fault tolerance mechanisms.
6. **Logging and Monitoring:** Facilitates logging and monitoring of API requests for better observability and debugging.

Setting Up Ocelot

1. Add Ocelot to Your Project

Install the Ocelot NuGet package in your ASP.NET Core project:

```
dotnet add package Ocelot
```


2. Configure Ocelot in Program.cs

Set up Ocelot in the Program.cs file of your ASP.NET Core application:

```
using Microsoft.Extensions.DependencyInjection;
using Ocelot.DependencyInjection;
using Ocelot.Middleware;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddOcelot();

var app = builder.Build();

await app.UseOcelot();

app.Run();
```

- **AddOcelot()**: Registers Ocelot services for dependency injection.
- **UseOcelot()**: Adds Ocelot middleware to the HTTP request pipeline.

Configuration File: ocelot.json

The ocelot.json file is used to configure Ocelot. It is typically placed in the root directory of your project and defines routing rules, policies, and other settings.

Basic Structure of ocelot.json:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/products",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
```

```

    "Port": 5001
  }
],
"UpstreamPathTemplate": "/products",
"UpstreamHttpMethod": [ "Get" ]
}
],
"GlobalConfiguration": {
  "BaseUrl": "http://localhost:5000"
}
}

```

- **Routes:** Defines the routing rules for how incoming requests are handled.
 - **DownstreamPathTemplate:** The path to the downstream service.
 - **DownstreamScheme:** The protocol scheme of the downstream service (http or https).
 - **DownstreamHostAndPorts:** The host and port of the downstream service.
 - **UpstreamPathTemplate:** The path used by clients to access the API Gateway.
 - **UpstreamHttpMethod:** HTTP methods that are allowed for the route (GET, POST, etc.).

- **GlobalConfiguration:** Contains global settings for Ocelot.
 - **BaseUrl:** The base URL for the API Gateway.

Key Concepts

1. **Downstream Service:** The service that Ocelot forwards the request to.
2. **Upstream Request:** The client request that reaches the API Gateway.
3. **ReRoute:** The configuration that maps upstream requests to downstream services.

ocelot.json and Its Full Properties Including Downstream and Upstream

The ocelot.json file is the heart of Ocelot's configuration. It defines how Ocelot should handle routing requests, apply policies, and manage various settings. Understanding its structure and properties is crucial for effectively setting up your API Gateway.

Basic Structure of ocelot.json

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/products",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5001
        }
      ],
      "UpstreamPathTemplate": "/products",
      "UpstreamHttpMethod": [ "Get" ]
    }
  ],
  "GlobalConfiguration": {
    "BaseUrl": "http://localhost:5000"
  }
}
```

Routes

The Routes section is where you define the routing rules. Each Route maps an incoming request to a downstream service. Here's a detailed explanation of the properties:

- **DownstreamPathTemplate:** Specifies the path to which Ocelot will forward the request. It represents the path on the downstream service.
 - Example: `/api/products` – Ocelot will route requests to this path on the downstream service.

- **DownstreamScheme:** Indicates the protocol scheme used by the downstream service. It can be http or https.
 - Example: http – The downstream service is accessed over HTTP.
- **DownstreamHostAndPorts:** An array of host and port combinations for the downstream service.
 - **Host:** The hostname or IP address of the downstream service.
 - Example: localhost – The downstream service is running on the local machine.
 - **Port:** The port number on which the downstream service is listening.
 - Example: 5001 – The downstream service listens on port 5001.
- **UpstreamPathTemplate:** Defines the path that clients use to access the API Gateway. This path is used by Ocelot to match incoming requests.
 - Example: /products – Clients send requests to /products, which are then routed to the downstream service.
- **UpstreamHttpMethod:** Specifies the HTTP methods allowed for this route. It is an array of methods like GET, POST, PUT, etc.
 - Example: ["Get"] – Only GET requests are allowed for this route.

GlobalConfiguration

The GlobalConfiguration section contains settings that apply globally across all routes.

- **BaseUrl:** Defines the base URL for the API Gateway. This is useful for setting the base address of the gateway and can be used to build full URLs for downstream services.
 - Example: http://localhost:5000 – The API Gateway listens on port 5000.

Additional Properties

Ocelot supports several additional properties in the ocelot.json configuration file for advanced scenarios:

- **AuthenticationOptions:** Configures authentication mechanisms for routes. You can specify authentication schemes, credentials, and more.
 - Example: { "AuthenticationProviderKey": "Bearer", "AllowedScopes": ["api1"] }
- **AuthorizationOptions:** Specifies authorization policies and roles required to access routes.
 - Example: { "AllowedScopes": ["api1"] }

- **QoSOptions:** Configures Quality of Service policies, such as retries and circuit breakers, using the Polly library.
 - Example: { "RetryCount": 3, "CircuitBreakerOptions": { "FailureThreshold": 0.5, "DurationOfBreak": "00:01:00" } }
- **RateLimitOptions:** Defines rate limiting rules to control the number of requests allowed from a client within a time period.
 - Example: { "ClientIdHeader": "X-Client-ID", "QuotaExceededMessage": "Rate limit exceeded" }
- **ResponseCachingOptions:** Configures caching for responses, including cache duration and the ability to vary cache based on request headers.
 - Example: { "CacheDuration": "00:05:00" }
- **ServiceDiscoveryProvider:** Defines settings for service discovery, such as using a service registry like Consul.
 - Example: { "Scheme": "http", "Host": "localhost", "Port": 8500 }
- **ReRouteOptions:** Provides additional options for configuring individual routes, such as request transformations and response transformations.
 - Example: { "RequestIdKey": "X-Request-ID", "RequestIdProvider": "Guid.NewGuid" }

Examples

Example 1: Basic Configuration

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/orders",
      "DownstreamScheme": "https",
      "DownstreamHostAndPorts": [
        {
          "Host": "order-service",
          "Port": 443
        }
      ],
      "UpstreamPathTemplate": "/orders",
```

```
    "UpstreamHttpMethod": [ "Get" ]
  }
],
"GlobalConfiguration": {
  "BaseUrl": "http://localhost:5000"
}
}
```

Example 2: Adding Rate Limiting

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/products",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5001
        }
      ],
      "UpstreamPathTemplate": "/products",
      "UpstreamHttpMethod": [ "Get" ],
      "RateLimitOptions": {
        "ClientIdHeader": "X-Client-ID",
        "QuotaExceededMessage": "Rate limit exceeded",
        "RateLimitRules": [
          {
            "Period": "1m",
            "Limit": 100
          }
        ]
      }
    }
  ]
}
```

```

    }
  }
],
"GlobalConfiguration": {
  "BaseUrl": "http://localhost:5000"
}
}

```

Key Points to Remember (for interview preparation):

1. **Routes Section:** Defines routing rules in ocelot.json, mapping upstream requests to downstream services.
2. **DownstreamPathTemplate:** Path to the downstream service that Ocelot will forward requests to.
3. **DownstreamScheme:** Protocol scheme used by the downstream service (http or https).
4. **DownstreamHostAndPorts:** Host and port of the downstream service.
5. **UpstreamPathTemplate:** Path that clients use to access the API Gateway.
6. **UpstreamHttpMethod:** Allowed HTTP methods for the route.
7. **GlobalConfiguration:** Global settings for the API Gateway, including the base URL.
8. **Advanced Properties:** Includes options for authentication, authorization, QoS, rate limiting, response caching, and service discovery.

Polly Integration with Ocelot

Polly is a .NET library that provides resilience and transient-fault-handling capabilities. It allows you to define policies such as retries, circuit breakers, timeouts, and more. Integrating Polly with Ocelot enhances the API Gateway's reliability by managing faults and improving system resilience.

Basic Integration Overview

To use Polly with Ocelot, you need to configure the Ocelot gateway to use Polly policies. This involves setting up policies for various fault-tolerance scenarios and applying them to the Routes in the ocelot.json configuration file.

Steps to Integrate Polly with Ocelot

1. Add Polly NuGet Package

First, ensure that the Polly NuGet package is installed in your project. You can add it using the Package Manager Console or the NuGet Package Manager.

```
dotnet add package Polly
```

2. Define Polly Policies

Define your Polly policies (like retries, circuit breakers, etc.) in your application code. This typically involves creating classes or methods that set up these policies.

Example: Defining Polly Policies

```
using Polly;

using Polly.CircuitBreaker;

using Polly.Retry;

using Polly.Timeout;

using Polly.Wrap;

using System;

using System.Net.Http;

public class PollyPolicies
{
    public IAsyncPolicy<HttpResponseMessage> GetRetryPolicy(int retryCount)
    {
        return Policy.HandleResult<HttpResponseMessage>(r => !r.IsSuccessStatusCode)
            .WaitAndRetryAsync(retryCount, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2,
                retryAttempt)));
    }

    public IAsyncPolicy<HttpResponseMessage> GetCircuitBreakerPolicy(int
        handledEventsAllowedBeforeBreaking, TimeSpan durationOfBreak)
    {
        return Policy.HandleResult<HttpResponseMessage>(r => !r.IsSuccessStatusCode)
            .CircuitBreakerAsync(handledEventsAllowedBeforeBreaking, durationOfBreak);
    }
}
```



```

    }

    public IAsyncPolicy<HttpResponseMessage> GetTimeoutPolicy(TimeSpan timeout)
    {
        return Policy.TimeoutAsync<HttpResponseMessage>(timeout);
    }

    public IAsyncPolicy<HttpResponseMessage> GetCombinedPolicy()
    {
        var retryPolicy = GetRetryPolicy(3);
        var circuitBreakerPolicy = GetCircuitBreakerPolicy(5, TimeSpan.FromMinutes(1));
        var timeoutPolicy = GetTimeoutPolicy(TimeSpan.FromSeconds(2));

        return Policy.WrapAsync(retryPolicy, circuitBreakerPolicy, timeoutPolicy);
    }
}

```

3. Configure Ocelot to Use Polly Policies

In your Ocelot configuration (ocelot.json), you need to integrate the Polly policies with your Routes. This typically involves using the QoSOptions to specify the policies.

Example: Integrating Polly Policies

```

{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/products",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5001
        }
      ]
    }
  ]
}

```

```

    ],
    "UpstreamPathTemplate": "/products",
    "UpstreamHttpMethod": [ "Get" ],
    "QoSOptions": {
        "ExceptionsAllowedBeforeBreaking": 3,
        "DurationOfBreak": "00:00:30",
        "TimeoutValue": 2000
    }
}
],
"GlobalConfiguration": {
    "BaseUrl": "http://localhost:5000"
}
}

```

In this configuration:

- **ExceptionsAllowedBeforeBreaking:** Specifies how many exceptions are allowed before the circuit breaker opens.
- **DurationOfBreak:** Defines how long the circuit breaker remains open before transitioning to a half-open state.
- **TimeoutValue:** Sets the timeout duration for requests.

4. Configure Ocelot Startup

In the Startup.cs or Program.cs file, ensure you register and configure the policies you have defined. This involves adding the required services and configuring Ocelot to use them.

Example: Registering and Configuring Ocelot

```

builder.Services.AddOcelot()
    .AddPolly();

await app.UseOcelot();

```

Example: Configuring QoSOptions in Code

You can also configure QoS options programmatically, which gives you more control over the policy setup.

Example: Programmatic Configuration

```
using Ocelot.DependencyInjection;  
using Ocelot.Middleware;  
using Microsoft.Extensions.DependencyInjection;  
using Polly;
```

```
builder.Services.AddOcelot()  
    .AddPolly();  
...
```

```
await app.UseOcelot();
```

Polly Integration Summary

- **Polly Policies:** Define resilience policies like retries, circuit breakers, and timeouts.
- **Ocelot Configuration:** Integrate Polly policies into the Ocelot configuration using QoSOptions.
- **Startup Configuration:** Register Ocelot and Polly in your application startup.

Key Points to Remember (for interview preparation):

1. **Polly Integration:** Use Polly to add resilience to your API Gateway by handling faults and transient issues.
2. **Policies:** Define and configure policies such as retries, circuit breakers, and timeouts.
3. **QoSOptions:** Configure Quality of Service options in ocelot.json to apply Polly policies.
4. **Startup Configuration:** Ensure Polly and Ocelot are correctly registered and configured in your application's startup code.

QoSOptions

Quality of Service (QoS) options in Ocelot allow you to manage the reliability of your API Gateway by applying various fault-tolerance policies such as circuit breakers, timeouts, and retries. These options help ensure that your gateway handles failures gracefully and maintains a good user experience even when downstream services experience issues.

QoSOptions Overview

QoSOptions provides a way to configure various policies that enhance the resilience of your API Gateway. This section covers the configuration options available for QoS and how they interact with Polly policies.

QoSOptions Properties

1. ExceptionsAllowedBeforeBreaking

- **Definition:** Specifies the number of exceptions (failed requests) that are allowed before the circuit breaker opens.
- **Purpose:** Helps in managing how many failures are tolerated before the system considers the downstream service to be unreliable.
- **Type:** Integer
- **Example:** ExceptionsAllowedBeforeBreaking: 3

2. DurationOfBreak

- **Definition:** Defines how long the circuit breaker remains open before transitioning to a half-open state. During this time, all requests to the downstream service are blocked.
- **Purpose:** Controls the duration for which the system should wait before checking if the downstream service has recovered.
- **Type:** TimeSpan (in hh:mm:ss format)
- **Example:** DurationOfBreak: "00:00:30"

3. TimeoutValue

- **Definition:** Sets the timeout duration for requests made through the API Gateway. If a request takes longer than this duration, it is considered to have failed.
- **Purpose:** Ensures that requests that take too long are not indefinitely waited on, thus preventing system bottlenecks and improving responsiveness.
- **Type:** TimeSpan (in milliseconds)
- **Example:** TimeoutValue: 2000 (2 seconds)

4. **RetryCount** (Optional in some configurations)

- **Definition:** Specifies the number of times a failed request should be retried before considering it a failure.
- **Purpose:** Provides the ability to automatically retry requests that fail due to transient issues, improving resilience and reliability.
- **Type:** Integer
- **Example:** RetryCount: 5

Example Configuration in ocelot.json

Here's how you can configure QoS options in the ocelot.json file for a ReRoute:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/products",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5001
        }
      ],
      "UpstreamPathTemplate": "/products",
      "UpstreamHttpMethod": [ "Get" ],
      "QoSOptions": {
        "ExceptionsAllowedBeforeBreaking": 3,
        "DurationOfBreak": "00:00:30",
        "TimeoutValue": 2000,
        "RetryCount": 5
      }
    }
  ],
  "GlobalConfiguration": {
```

```
"BaseUrl": "http://localhost:5000"  
}  
}
```

Applying QoSOptions Programmatically

To configure QoS options programmatically in your application code, you would typically use the Ocelot configuration API to apply these settings.

Example Code: Programmatic QoS Configuration

```
using Microsoft.Extensions.DependencyInjection;  
using Ocelot.DependencyInjection;  
using Ocelot.Middleware;  
  
builder.Services.AddOcelot()  
    .AddPolly(); // Ensure Polly is added if using Polly policies  
  
...  
  
app.UseOcelot().Wait();
```

In this example, Ocelot is configured to use the QoSOptions specified in the ocelot.json file. You can further customize the QoS settings by adding more detailed configurations in code if needed.

Key Points to Remember (for interview preparation):

1. **QoSOptions:** Configures how the API Gateway handles fault tolerance and resilience through properties like `ExceptionsAllowedBeforeBreaking`, `DurationOfBreak`, `TimeoutValue`, and `RetryCount`.
2. **Configuration in ocelot.json:** Define QoS policies in the Ocelot configuration file to apply fault-tolerance settings to your Routes.
3. **Programmatic Configuration:** Use code-based configurations if needed to further customize QoS settings beyond what's defined in ocelot.json.

Rate Limits

Rate limiting is a crucial feature in an API Gateway to manage the number of requests that a client can make to an API within a specific time period. It helps protect your backend services from being overwhelmed by too many requests and ensures fair usage across different clients.

Rate Limits Overview

Rate limiting involves setting thresholds on the number of requests allowed from a client in a given time window. Ocelot provides support for rate limiting through its configuration, allowing you to specify limits on requests based on various criteria.

Rate Limits Properties

1. **ClientId** (Optional)

- **Definition:** A unique identifier for the client making the request.
- **Purpose:** Allows rate limits to be applied based on individual client IDs, ensuring that different clients are subject to their own limits.
- **Type:** String
- **Example:** "ClientId": "client-123"

2. **QuotaExceededMessage**

- **Definition:** The message that is returned to the client when the rate limit is exceeded.
- **Purpose:** Provides a custom response to inform the client that they have exceeded their allowed request limit.
- **Type:** String
- **Example:** "QuotaExceededMessage": "Rate limit exceeded. Please try again later."

3. **RateLimitOptions**

- **Definition:** Defines the rate limiting policy for the API route.
- **Purpose:** Specifies how many requests are allowed within a certain time period.
- **Properties:**
 - **Period:** The time period for which the request count is tracked.

- **Type:** TimeSpan (in hh:mm:ss format)
- **Example:** "Period": "00:01:00" (1 minute)
- **Limit:** The maximum number of requests allowed in the defined period.
 - **Type:** Integer
 - **Example:** "Limit": 100 (100 requests per minute)

Example Configuration in ocelot.json

Here's how you can configure rate limiting in the ocelot.json file for a ReRoute:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/products",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5001
        }
      ],
      "UpstreamPathTemplate": "/products",
      "UpstreamHttpMethod": [ "Get" ],
      "RateLimitOptions": {
        "Period": "00:01:00",
        "Limit": 100,
        "QuotaExceededMessage": "Rate limit exceeded. Please try again later."
      }
    }
  ],
  "GlobalConfiguration": {
    "BaseUrl": "http://localhost:5000"
  }
}
```



```
}  
}
```

In this example, the rate limit is set to allow up to 100 requests per minute. If the client exceeds this limit, they will receive a custom quota exceeded message.

Applying Rate Limits Programmatically

To configure rate limits programmatically, you would use the Ocelot configuration API to set up rate limiting policies.

Example Code: Programmatic Rate Limit Configuration

```
using Microsoft.Extensions.DependencyInjection;
```

```
using Ocelot.DependencyInjection;
```

```
using Ocelot.Middleware;
```

```
builder.Services.AddOcelot()
```

```
    .AddRateLimit(); // Ensure rate limiting services are added
```

```
...
```

```
await app.UseOcelot();
```

In this example, rate limiting is enabled for Ocelot. You may need to define additional settings in the `ocelot.json` file or use custom middleware if more complex rate-limiting scenarios are required.

Key Points to Remember (for interview preparation):

1. **Rate Limiting:** Manages the number of requests a client can make to the API within a specific time period.
2. **Configuration in `ocelot.json`:** Define rate limits using properties like `Period`, `Limit`, and `QuotaExceededMessage`.
3. **Programmatic Configuration:** Use Ocelot's configuration API to apply rate limiting if needed.

Response Caching

Response caching is a technique used to improve the performance and scalability of your API by storing and reusing previously generated responses. In an API Gateway context, response caching can significantly reduce the load on your backend services and improve response times for frequently requested data.

Introduction to Response Caching

When a client makes a request, the API Gateway can store the response in a cache. Subsequent requests for the same resource can be served from the cache rather than forwarding the request to the backend service. This reduces the number of calls to the backend and speeds up response times.

Response Caching Properties in Ocelot

Ocelot supports response caching through its configuration. You can specify how long responses should be cached and under what conditions. This involves defining caching policies in the `ocelot.json` file.

1. CacheKey

- **Definition:** A unique key for identifying the cached response.
- **Purpose:** Ensures that each cached response can be uniquely identified and retrieved.
- **Type:** String
- **Example:** "CacheKey": "products-cache-key"

2. CacheDuration

- **Definition:** The duration for which the response should be cached.
- **Purpose:** Controls how long the cached response remains valid before it is considered stale and needs to be refreshed.
- **Type:** TimeSpan (in hh:mm:ss format)
- **Example:** "CacheDuration": "00:10:00" (10 minutes)

3. AllowCachedResponses

- **Definition:** A boolean value indicating whether cached responses are allowed.
- **Purpose:** Enables or disables caching for a specific route.
- **Type:** Boolean
- **Example:** "AllowCachedResponses": true

Example Configuration in ocelot.json

Here's how you can configure response caching in the ocelot.json file for a ReRoute:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/products",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5001
        }
      ],
      "UpstreamPathTemplate": "/products",
      "UpstreamHttpMethod": [ "Get" ],
      "ResponseCachingOptions": {
        "CacheKey": "products-cache-key",
        "CacheDuration": "00:10:00",
        "AllowCachedResponses": true
      }
    }
  ],
  "GlobalConfiguration": {
    "BaseUrl": "http://localhost:5000"
  }
}
```

In this example, responses for the /products endpoint will be cached for 10 minutes. The cache key is set to "products-cache-key", ensuring that responses are uniquely identified.

Applying Response Caching Programmatically

To enable and configure response caching programmatically, you need to add response caching services and middleware in your ASP.NET Core application.

Example Code: Enabling Response Caching in ASP.NET Core

```
using Microsoft.AspNetCore.Builder;  
using Microsoft.Extensions.DependencyInjection;
```

```
builder.Services.AddResponseCaching();  
builder.Services.AddOcelot();
```

```
...
```

```
app.UseResponseCaching();  
await app.UseOcelot();
```

In this example:

- `AddResponseCaching()` registers the response caching services.
- `UseResponseCaching()` adds the response caching middleware to the request pipeline.

Key Points to Remember (for interview preparation):

1. **Response Caching:** Stores and reuses responses to reduce load on backend services and improve performance.
2. **Configuration in ocelot.json:** Define caching policies using properties like `CacheKey`, `CacheDuration`, and `AllowCachedResponses`.
3. **Programmatic Configuration:** Use ASP.NET Core middleware and services to enable and configure response caching.

.NET Microservices – Azure DevOps and AKS

Section 11: Asynchronous Microservice Communication using RabbitMQ- Notes

Introduction to RabbitMQ

RabbitMQ is an open-source message broker that enables applications to communicate with each other by sending messages. It implements the Advanced Message Queuing Protocol (AMQP) for message queuing. RabbitMQ is widely used in microservices architectures to enable asynchronous communication between services.

Key Concepts:

- **Broker:** RabbitMQ server that manages queues and exchanges.
- **Queue:** Storage for messages until they are processed.
- **Exchange:** Routes messages to one or more queues based on routing rules.
- **Binding:** The relationship between an exchange and a queue that determines how messages are routed.
- **Producer:** Application that sends messages to RabbitMQ.
- **Consumer:** Application that receives messages from RabbitMQ.

Types of RabbitMQ Exchanges

RabbitMQ supports several types of exchanges, each serving different routing needs:

1. Direct Exchange:

- **Purpose:** Routes messages to queues based on a routing key.
- **Use Case:** Direct routing of messages to a specific queue. Ideal for scenarios where you want to route messages with a specific routing key to a particular queue.
- **Example:** A logging system where logs of different severity levels (info, error) are routed to different queues.

```
{  
  "exchangeType": "direct",  
  "routingKey": "info",  
  "queueName": "info-logs"  
}
```

2. Fanout Exchange:

- **Purpose:** Routes messages to all queues bound to it, ignoring the routing key.
- **Use Case:** Broadcasting messages to multiple queues. Suitable for scenarios where you want all subscribers to receive the same message.
- **Example:** A notification system where all notifications are broadcasted to multiple subscribers.

```
{  
  "exchangeType": "fanout",  
  "queueName": "notifications"  
}
```

3. Topic Exchange:

- **Purpose:** Routes messages to queues based on matching between routing key patterns and the binding key patterns.
- **Use Case:** Routing messages to multiple queues based on complex routing rules. Ideal for scenarios with multiple consumers interested in specific types of messages.
- **Example:** A news service where messages are routed based on topics like sports, politics, and technology.

```
{  
  "exchangeType": "topic",  
  "routingKeyPattern": "news.sports",  
  "queueName": "sports-news"  
}
```

4. Headers Exchange:

- **Purpose:** Routes messages based on message headers instead of routing keys.
- **Use Case:** Advanced routing based on multiple header attributes. Useful for complex routing scenarios.
- **Example:** A document processing system where documents are routed based on their type and priority.

```
{  
  "exchangeType": "headers",
```

```
"headers": {  
  "type": "invoice",  
  "priority": "high"  
},  
"queueName": "high-priority-invoices"  
}
```

Creating RabbitMQ Container in Docker Compose File

To run RabbitMQ in a Docker container, you can use Docker Compose. Here's a sample docker-compose.yml file to set up RabbitMQ:

```
version: '3'
```

```
services:
```

```
  rabbitmq:
```

```
    image: "rabbitmq:3-management"
```

```
    container_name: rabbitmq
```

```
    ports:
```

```
      - "5672:5672" # RabbitMQ default port
```

```
      - "15672:15672" # Management UI port
```

```
    environment:
```

```
      RABBITMQ_DEFAULT_USER: user
```

```
      RABBITMQ_DEFAULT_PASS: password
```

Explanation:

- **image:** Specifies the RabbitMQ Docker image to use. 3-management includes the RabbitMQ Management Plugin for a web-based UI.
- **ports:** Maps the container ports to host ports. 5672 is the default RabbitMQ port, and 15672 is for the Management UI.
- **environment:** Sets default credentials for RabbitMQ.

Publishing Messages in a Service Class in Business Logic Layer

To publish messages to RabbitMQ from a service class, you can use the RabbitMQ.Client library. Here's a basic example in a .NET Core service class:

```
using RabbitMQ.Client;

using System.Text;

public class MessagePublisher
{
    private readonly IConnection _connection;
    private readonly IModel _channel;

    public MessagePublisher()
    {
        var factory = new ConnectionFactory() { HostName = "localhost" };
        _connection = factory.CreateConnection();
        _channel = _connection.CreateModel();
    }

    public void PublishMessage(string message, string exchangeName, string routingKey)
    {
        _channel.ExchangeDeclare(exchange: exchangeName, type: ExchangeType.Direct);

        var body = Encoding.UTF8.GetBytes(message);

        _channel.BasicPublish(exchange: exchangeName,
                              routingKey: routingKey,
                              basicProperties: null,
                              body: body);
    }
}
```

Explanation:

- **ConnectionFactory:** Creates a connection to RabbitMQ.
- **IConnection:** Represents the RabbitMQ connection.
- **IModel:** Represents a channel within the connection.
- **ExchangeDeclare:** Declares the exchange where messages will be sent.
- **BasicPublish:** Publishes the message to the specified exchange with a routing key.

Consuming Messages in a Service Class in Business Logic Layer

To consume messages from RabbitMQ, use the following example:

```
using RabbitMQ.Client;
using RabbitMQ.Client.Events;
using System;
using System.Text;

public class MessageConsumer
{
    private readonly IConnection _connection;
    private readonly IModel _channel;

    public MessageConsumer()
    {
        var factory = new ConnectionFactory() { HostName = "localhost" };
        _connection = factory.CreateConnection();
        _channel = _connection.CreateModel();
    }

    public void ConsumeMessages(string queueName)
    {
        _channel.QueueDeclare(queue: queueName,
                               durable: false,
                               exclusive: false,
```

```

        autoDelete: false,
        arguments: null);

var consumer = new EventingBasicConsumer(_channel);
consumer.Received += (model, ea) =>
{
    var body = ea.Body.ToArray();
    var message = Encoding.UTF8.GetString(body);
    Console.WriteLine($"Received {message}");
};

_channel.BasicConsume(queue: queueName,
    autoAck: true,
    consumer: consumer);
}
}

```

Explanation:

- **QueueDeclare:** Declares a queue to receive messages.
- **EventingBasicConsumer:** Handles messages received from RabbitMQ.
- **Received:** Event triggered when a message is received.
- **BasicConsume:** Starts consuming messages from the specified queue.

Key Points to Remember (for Interview Preparation)

- **RabbitMQ:** Message broker that supports asynchronous communication between services.
- **Exchanges:** Direct, Fanout, Topic, and Headers exchanges have different routing capabilities.
- **Direct Exchange:** Routes based on exact routing keys.
- **Fanout Exchange:** Routes messages to all bound queues.
- **Topic Exchange:** Routes based on pattern matching of routing keys.
- **Headers Exchange:** Routes based on message headers.
- **Docker Compose:** Used to define and run multi-container Docker applications.

- **Publishing/Consuming:** Use RabbitMQ client libraries to interact with RabbitMQ servers.
- **Management UI:** Accessible via port 15672 for monitoring RabbitMQ.

Hosted Service in ASP.NET Core for Running RabbitMQ Consumers

In ASP.NET Core, you can use a hosted service to run background tasks such as consuming messages from RabbitMQ. A hosted service is a long-running process that can run in the background of your application.

Here's how you can create a hosted service to consume RabbitMQ messages:

1. Implement the Hosted Service:

```
using Microsoft.Extensions.Hosting;
```

```
using RabbitMQ.Client;
```

```
using RabbitMQ.Client.Events;
```

```
using System;
```

```
using System.Text;
```

```
using System.Threading;
```

```
using System.Threading.Tasks;
```

```
public class RabbitMQConsumerHostedService : IHostedService
{
    private readonly IConnection _connection;
    private readonly IModel _channel;
    private readonly string _queueName = "myQueue"; // Name of the queue to consume from

    public RabbitMQConsumerHostedService()
    {
        var factory = new ConnectionFactory() { HostName = "localhost" };
        _connection = factory.CreateConnection();
        _channel = _connection.CreateModel();
    }
}
```

```

        _channel.QueueDeclare(queue: _queueName,
                                durable: false,
                                exclusive: false,
                                autoDelete: false,
                                arguments: null);
    }

    public Task StartAsync(Cancellation_token cancellation_token)
    {
        var consumer = new EventingBasicConsumer(_channel);
        consumer.Received += (model, ea) =>
        {
            var body = ea.Body.ToArray();
            var message = Encoding.UTF8.GetString(body);
            // Process the message here
            Console.WriteLine($"Received message: {message}");
        };

        _channel.BasicConsume(queue: _queueName,
                                autoAck: true,
                                consumer: consumer);

        return Task.CompletedTask;
    }

    public Task StopAsync(Cancellation_token cancellation_token)
    {
        _channel.Close();
        _connection.Close();
        return Task.CompletedTask;
    }

```

```
}  
}
```

Explanation:

- **IHostedService:** An interface that allows you to run background tasks in ASP.NET Core.
- **StartAsync:** Starts the service. Here, it sets up the consumer to listen for messages.
- **StopAsync:** Stops the service and cleans up resources.

2. Register the Hosted Service in Program.cs:

```
builder.Services.AddHostedService<RabbitMQConsumerHostedService>();
```

Explanation:

- **AddHostedService:** Registers the RabbitMQConsumerHostedService to run as a background service.

Fanout Exchange

Fanout Exchange routes messages to all queues bound to it, without considering the routing key. This is useful for scenarios where you need to broadcast messages to multiple consumers.

Example Scenario:

- A notification service that sends a notification to all registered clients.

Setup Example in .NET:

```
_channel.ExchangeDeclare(exchange: "fanout-exchange", type: ExchangeType.Fanout);  
_channel.BasicPublish(exchange: "fanout-exchange",  
    routingKey: "",  
    basicProperties: null,  
    body: Encoding.UTF8.GetBytes("Broadcast message"));
```

Topic Exchange

Topic Exchange allows routing messages to queues based on patterns in the routing key. It is useful for complex routing scenarios where you need to route messages to different queues based on multiple criteria.

Example Scenario:

- A news aggregator service where different news topics (e.g., sports, politics) are routed to different queues.

Setup Example in .NET:

```
_channel.ExchangeDeclare(exchange: "topic-exchange", type: ExchangeType.Topic);  
  
_channel.BasicPublish(exchange: "topic-exchange",  
    routingKey: "news.sports",  
    basicProperties: null,  
    body: Encoding.UTF8.GetBytes("Sports news update"));
```

Headers Exchange

Headers Exchange routes messages based on message headers instead of the routing key. This type of exchange provides advanced routing capabilities.

Example Scenario:

- An order processing system where messages are routed based on header attributes like order priority and type.

Setup Example in .NET:

```
_channel.ExchangeDeclare(exchange: "headers-exchange", type: ExchangeType.Headers);  
  
var properties = _channel.CreateBasicProperties();  
properties.Headers = new Dictionary<string, object>  
{  
    { "order-type", "urgent" },  
    { "order-priority", "high" }  
};  
  
_channel.BasicPublish(exchange: "headers-exchange",
```

```
routingKey: "",  
basicProperties: properties,  
body: Encoding.UTF8.GetBytes("Urgent order message"));
```

Key Points to Remember (for Interview Preparation)

- **RabbitMQ:** A message broker that supports asynchronous communication between services.
- **Exchanges:**
 - **Direct Exchange:** Routes messages based on exact routing keys.
 - **Fanout Exchange:** Routes messages to all bound queues.
 - **Topic Exchange:** Routes messages based on patterns in routing keys.
 - **Headers Exchange:** Routes messages based on message headers.
- **Hosted Services:** Used to run background tasks in ASP.NET Core applications, such as consuming messages from RabbitMQ.
- **Consumer Setup:** Implement `IHostedService` to manage RabbitMQ message consumption.

.NET Microservices – Azure DevOps and AKS

Section 12: Azure - Notes

Introduction to Cloud Computing

Cloud Computing is a technology that allows users to access and use computing resources over the internet. Instead of owning and maintaining physical hardware and software, users can leverage cloud services provided by cloud service providers (CSPs). This model offers flexibility, scalability, and cost efficiency.

Key Concepts:

1. **On-Demand Self-Service:**

- Users can provision computing capabilities like server time and network storage as needed without requiring human interaction with each service provider.

2. **Broad Network Access:**

- Cloud services are accessible over the network from various devices (e.g., smartphones, tablets, laptops) using standard protocols.

3. **Resource Pooling:**

- Cloud providers pool resources to serve multiple customers, using a multi-tenant model where physical and virtual resources are assigned and reassigned according to demand.

4. **Rapid Elasticity:**

- Cloud resources can be scaled up or down quickly based on demand. This elasticity allows for handling varying workloads efficiently.

5. **Measured Service:**

- Cloud systems automatically control and optimize resource usage by leveraging metering capabilities. This ensures resources are paid for based on usage.

Types of Cloud Computing:

1. **Public Cloud:**

- Services offered over the public internet. Examples: AWS, Azure, Google Cloud Platform.
- Benefits: Cost-effective, scalable, and accessible from anywhere.

2. **Private Cloud:**

- Cloud infrastructure is operated solely for one organization. Can be managed internally or by a third party.
- Benefits: Enhanced security and control, customizable to organizational needs.

3. **Hybrid Cloud:**

- Combines public and private clouds, allowing data and applications to be shared between them.
- Benefits: Flexibility, cost efficiency, and enhanced security.

4. **Community Cloud:**

- Shared infrastructure for a specific community of organizations with common concerns (e.g., security, compliance).
- Benefits: Cost savings through shared resources.

Cloud Service Models

1. **Infrastructure as a Service (IaaS):**

- Provides virtualized computing resources over the internet.
- Example: Azure Virtual Machines, AWS EC2.
- Benefits: Scalability, flexible pricing, control over operating systems and applications.

2. **Platform as a Service (PaaS):**

- Provides a platform allowing customers to develop, run, and manage applications without dealing with the underlying infrastructure.
- Example: Azure App Services, Google App Engine.
- Benefits: Focus on application development, automatic updates, and scaling.

3. **Software as a Service (SaaS):**

- Delivers software applications over the internet, on a subscription basis.
- Example: Office 365, Salesforce.
- Benefits: No need for installation or maintenance, accessible from any device, automatic updates.

Introduction to Azure

Azure is Microsoft's cloud computing platform that provides a range of services, including those for computing, analytics, storage, and networking. It offers a robust set of tools and services for building, deploying, and managing applications through a global network of data centers.

Key Features of Azure:

1. **Compute Services:**
 - Virtual Machines, Azure App Services, Azure Kubernetes Service (AKS).
2. **Storage Services:**
 - Blob Storage, Table Storage, Disk Storage.
3. **Networking Services:**
 - Virtual Networks, Load Balancer, VPN Gateway.
4. **Database Services:**
 - Azure SQL Database, Cosmos DB, Azure Database for MySQL/PostgreSQL.
5. **Analytics and AI:**
 - Azure Machine Learning, Azure Synapse Analytics, Azure Data Lake.
6. **Management and Security:**
 - Azure Security Center, Azure Monitor, Azure Policy.

Problems / Limitations with Azure

1. **Complexity:**
 - The vast array of services and features can be overwhelming. It requires a steep learning curve to master and optimize usage.
2. **Cost Management:**
 - Pricing models can be complex, and costs can escalate if resources are not managed properly. Requires careful monitoring and budgeting.
3. **Service Limits:**
 - Some services have limits on resources or performance that can impact large-scale applications. Understanding these limits is crucial.
4. **Downtime and Reliability:**
 - Although Azure offers high availability, there can be occasional outages or disruptions. Relying on multiple regions and services can mitigate this.

5. Compliance and Security:

- Ensuring compliance with regulations and managing security in a cloud environment requires diligent configuration and monitoring.

Azure CLI

Azure Command-Line Interface (CLI) is a cross-platform command-line tool that allows you to manage Azure resources from the command line. It provides a powerful and flexible way to interact with Azure services.

Key Commands:

1. Login to Azure:

`az login`

- Opens a browser window for authentication.

2. List Resource Groups:

`az group list`

- Displays all resource groups in the subscription.

3. Create a Resource Group:

`az group create --name MyResourceGroup --location eastus`

- Creates a new resource group named "MyResourceGroup" in the East US region.

4. Deploy a Resource:

`az deployment group create --resource-group MyResourceGroup --template-file myTemplate.json`

- Deploys resources defined in the specified template file to a resource group.

5. List Containers in Azure Container Registry:

`az acr repository list --name MyRegistry --output table`

- Lists all repositories in the specified Azure Container Registry.

Azure Resource Groups (using CLI)

Resource Groups in Azure are containers that hold related resources for an application. They allow for resource management, access control, and billing.

Common CLI Commands:

1. Create a Resource Group:

```
az group create --name MyResourceGroup --location eastus
```

- Creates a resource group named "MyResourceGroup" in the East US region.

2. Delete a Resource Group:

```
az group delete --name MyResourceGroup --yes --no-wait
```

- Deletes the resource group "MyResourceGroup" without confirmation.

3. List Resource Groups:

```
az group list --output table
```

- Lists all resource groups in the subscription in a tabular format.

4. Show Resource Group Details:

```
az group show --name MyResourceGroup
```

- Displays details of the specified resource group.

5. Update Resource Group Tags:

```
az group update --name MyResourceGroup --tags env=production
```

- Adds or updates tags for the specified resource group.

Azure Container Registry (using CLI)

Azure Container Registry (ACR) is a private Docker registry service in Azure that allows you to store and manage Docker container images and artifacts.

Common CLI Commands:

1. Create a Container Registry:

```
az acr create --resource-group MyResourceGroup --name MyRegistry --sku Basic
```

- Creates a new container registry named "MyRegistry" with Basic SKU in the specified resource group.

2. List Container Registries:

```
az acr list --resource-group MyResourceGroup --output table
```

- Lists all container registries in the specified resource group in a tabular format.

3. Show Container Registry Details:

```
az acr show --name MyRegistry
```

- Displays details of the specified container registry.

4. Delete a Container Registry:

```
az acr delete --name MyRegistry --resource-group MyResourceGroup --yes
```

- Deletes the specified container registry from the resource group.

5. Login to Container Registry:

```
az acr login --name MyRegistry
```

- Logs in to the specified container registry to push or pull images.

Limitations of AppService and Azure Container Apps to Host Complex Microservices

Azure App Service and **Azure Container Apps** are effective for many scenarios but may not be ideal for complex microservices applications. Here's why:

1. Scaling Limitations:

- App Services may struggle with scaling to handle very high loads or complex workloads compared to AKS.

2. **Limited Customization:**

- App Services offer limited control over the underlying infrastructure and configurations, which might be restrictive for complex applications.

3. **Performance Constraints:**

- App Services and Azure Container Apps may not provide the performance and fine-grained control required for high-performance microservices architectures.

4. **Resource Constraints:**

- Resource limits on App Services and Azure Container Apps might not be sufficient for large-scale or resource-intensive applications.

5. **Complex Orchestration Needs:**

- For applications requiring sophisticated orchestration, service discovery, and distributed data management, Azure Kubernetes Service (AKS) offers more robust solutions.

AppService (using CLI - with Docker Container)

Azure App Service is a fully managed platform for building, deploying, and scaling web apps. It supports Docker containers and integrates with various deployment options.

Common CLI Commands:

1. **Create an App Service Plan:**

```
az appservice plan create --name MyAppServicePlan --resource-group MyResourceGroup --sku B1 --is-linux
```

- Creates a new App Service Plan named "MyAppServicePlan" with Basic pricing tier and Linux environment.

2. **Create a Web App:**

```
az webapp create --resource-group MyResourceGroup --plan MyAppServicePlan --name MyWebApp --deployment-container
```

Azure Container Apps (using CLI)

Azure Container Apps is a serverless container service that enables you to deploy and manage containerized applications in a fully managed environment. It is designed for ease of use, providing automatic scaling and built-in support for microservices architectures.

Common CLI Commands:

1. Create an Azure Container App Environment:

```
az containerapp env create --name MyContainerAppEnv --resource-group MyResourceGroup --location eastus
```

- Creates a new environment for Azure Container Apps named "MyContainerAppEnv" in the specified resource group and region.

2. Create an Azure Container App:

```
az containerapp create --name MyContainerApp --resource-group MyResourceGroup --environment MyContainerAppEnv --image mycontainerimage:v1 --cpu 0.5 --memory 1.0Gi --target-port 80
```

- Deploys a container app named "MyContainerApp" using the specified container image, with allocated CPU and memory resources.

3. Update an Azure Container App:

```
az containerapp update --name MyContainerApp --resource-group MyResourceGroup --image mycontainerimage:v2
```

- Updates the container image for an existing container app.

4. List Azure Container Apps:

```
az containerapp list --resource-group MyResourceGroup --output table
```

- Lists all container apps in the specified resource group in a tabular format.

5. Show Azure Container App Details:

```
az containerapp show --name MyContainerApp --resource-group MyResourceGroup
```

- Displays details of the specified container app.

6. Delete an Azure Container App:

```
az containerapp delete --name MyContainerApp --resource-group MyResourceGroup --yes
```

- Deletes the specified container app from the resource group.

Key Points to Remember (for Interview Preparation)

1. Cloud Computing Concepts:

- Understand the fundamental characteristics of cloud computing (on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service).

2. Cloud Service Models:

- Know the differences between IaaS, PaaS, and SaaS, and when to use each model based on project requirements.

3. Azure Features and Benefits:

- Familiarize yourself with core Azure services like Virtual Machines, App Services, AKS, and Azure Container Registry. Understand their benefits and use cases.

4. Azure CLI Basics:

- Be comfortable with basic Azure CLI commands for resource management, including creating, listing, and deleting resources.

5. Azure Resource Management:

- Understand how to work with Azure Resource Groups for organizing and managing resources.

6. Container Management:

- Know the basic commands for managing Azure Container Registry and deploying containers using Azure App Service and Azure Container Apps.

7. App Service vs. AKS:

- Be able to articulate the limitations of Azure App Service and Azure Container Apps in hosting complex microservices and why AKS might be a better choice for large-scale projects.

8. Limitations and Challenges:

- Be aware of common problems and limitations associated with Azure services, such as complexity, cost management, and resource constraints.

.NET Microservices – Azure DevOps and AKS

Section 13: AKS - Notes

Introduction to Kubernetes

Kubernetes (K8s) is an open-source platform designed to automate the deployment, scaling, and operation of containerized applications. It provides a robust framework for managing microservices architectures and large-scale applications.

What is Kubernetes?

- **Definition:** Kubernetes is a container orchestration platform that enables developers to automate the management of containerized applications. Containers encapsulate the application and its dependencies, making them portable and consistent across environments. Kubernetes helps to manage these containers in a clustered environment.
- **Goals:**
 - **Automate Deployment:** Kubernetes automates the deployment of containers to ensure that applications are distributed across the cluster as intended.
 - **Scaling:** Automatically scale applications based on demand, adding or removing instances of services as needed.
 - **Maintain Availability:** Ensure applications are highly available and resilient to failures.

Key Concepts

- **Cluster:**
 - A Kubernetes cluster consists of a set of nodes that run containerized applications. The cluster is managed by a master node (or control plane) which coordinates the worker nodes that run the application containers.
 - **Components:**
 - **Master Node:** Controls the Kubernetes cluster. It includes:
 - **API Server:** Manages API requests, validates, and processes them.
 - **Controller Manager:** Manages controllers that handle routine tasks, such as scaling deployments or managing replicas.
 - **Scheduler:** Assigns Pods to nodes based on resource availability.
 - **etcd:** A distributed key-value store used for storing cluster state and configuration.

- **Worker Nodes:** Run application containers and include:
 - **Kubelet:** An agent that ensures containers are running as expected.
 - **Kube Proxy:** Handles network routing and load balancing for Pods.
 - **Container Runtime:** Software (like Docker) that runs containers.
- **Pod:**
 - The smallest deployable unit in Kubernetes. A Pod can contain one or more containers that share the same network namespace and storage. Pods are designed to run a single instance of a specific application.
- **Service:**
 - An abstraction that defines a logical set of Pods and a policy for accessing them. Services provide a stable endpoint for accessing Pods, which might be dynamic and change over time.
- **Deployment:**
 - Manages the deployment and scaling of Pods. It ensures the desired number of Pods are running and can perform rolling updates and rollbacks to maintain application consistency.
- **ConfigMap and Secret:**
 - **ConfigMap:** Manages non-sensitive configuration data. It allows you to decouple configuration artifacts from container images.
 - **Secret:** Manages sensitive data such as passwords, OAuth tokens, and SSH keys. Secrets are encoded in Base64 to prevent casual inspection.

Benefits of Kubernetes

- **High Availability:**
 - Kubernetes ensures that applications are always available by automatically distributing the load and replicating Pods across multiple nodes. If a node fails, Kubernetes reschedules Pods to other healthy nodes.
- **Self-Healing:**
 - Automatically replaces or restarts failed containers. It ensures that the desired state of the application is maintained even if failures occur.
- **Automated Rollouts and Rollbacks:**
 - Provides mechanisms for deploying application updates gradually and rolling back changes if something goes wrong. This helps in managing application updates with minimal disruption.

- **Scalability:**
 - Kubernetes allows horizontal scaling of applications by adding or removing Pods based on load. It supports both manual and automatic scaling based on predefined metrics.
- **Declarative Configuration:**
 - Allows you to describe the desired state of the system using YAML or JSON files. Kubernetes ensures that the current state matches the desired state specified in these configuration files.
- **Service Discovery and Load Balancing:**
 - Kubernetes automatically assigns a unique IP address and DNS name to each Service. It handles load balancing between Pods and provides service discovery mechanisms.

Use Cases

- **Microservices Architecture:**
 - Kubernetes is particularly suited for managing microservices-based applications where services are decoupled and interact through APIs. It helps in managing the deployment, scaling, and monitoring of these services.
- **CI/CD Pipelines:**
 - Integrates well with continuous integration and continuous deployment (CI/CD) systems. Kubernetes can automate the deployment of applications as part of the CI/CD pipeline.
- **Hybrid Cloud Deployments:**
 - Supports hybrid cloud environments where applications are deployed across on-premises and cloud-based infrastructure.

Key Points to Remember (For Interview Preparation)

1. **Kubernetes Overview:** Understand the role of Kubernetes in container orchestration and its ability to automate deployment, scaling, and operation of applications.
2. **Cluster Architecture:** Be familiar with the components of a Kubernetes cluster, including the master node, worker nodes, and their roles.
3. **Core Concepts:** Know the basic Kubernetes objects such as Pods, Services, and Deployments, and how they interact.
4. **Benefits:** Recognize the advantages of using Kubernetes, including high availability, self-healing, scalability, and automated updates.
5. **Use Cases:** Be able to explain typical scenarios where Kubernetes is beneficial, such as microservices architecture and CI/CD pipelines.

Problems with Kubernetes

While Kubernetes provides powerful capabilities for managing containerized applications, it comes with its own set of challenges and limitations. Understanding these issues is crucial for effectively implementing and operating Kubernetes in a real-world environment.

1. Complexity and Learning Curve

- **Steep Learning Curve:**
 - Kubernetes introduces many new concepts and components (like Pods, Services, Deployments, etc.), which can be overwhelming for newcomers. The complexity of Kubernetes architecture demands significant time and effort to understand and operate effectively.
- **Complex Configuration:**
 - Kubernetes configurations (typically defined in YAML files) can be intricate and prone to human error. Debugging issues related to configuration can be time-consuming.
- **Diverse Ecosystem:**
 - The Kubernetes ecosystem includes various tools and add-ons (e.g., Helm for package management, Prometheus for monitoring). Managing and integrating these tools can add to the overall complexity.

2. Resource Consumption

- **High Resource Overhead:**
 - Kubernetes itself requires resources to run, including the master components and agents on each node. This overhead can be substantial, particularly in smaller clusters or on resource-constrained environments.
- **Efficient Resource Utilization:**
 - Ensuring efficient resource utilization (CPU, memory) within the cluster requires careful configuration of resource requests and limits. Poorly configured resource management can lead to inefficient use of cluster resources.

3. Operational Overhead

- **Cluster Management:**
 - Managing and maintaining a Kubernetes cluster involves regular updates, security patches, and backups. This operational overhead can be significant, especially for self-managed clusters.

- **Troubleshooting:**

- Diagnosing issues in a Kubernetes cluster can be challenging due to the distributed nature of the system. Problems with Pods, Services, or networking can be hard to trace and resolve.

4. Security Concerns

- **Access Control:**

- Kubernetes security requires proper configuration of Role-Based Access Control (RBAC) and Network Policies. Misconfigured access control can lead to security vulnerabilities.

- **Secrets Management:**

- While Kubernetes provides mechanisms for managing sensitive information (Secrets), ensuring that secrets are handled securely and appropriately can be complex.

- **Pod Security:**

- Securing Pods and containerized applications involves configuring security contexts and Pod Security Policies. Inadequate security configurations can expose applications to risks.

5. Networking Challenges

- **Complex Networking:**

- Kubernetes networking involves managing internal and external traffic, Service discovery, and load balancing. Configuring and troubleshooting network issues can be complex.

- **Service Discovery and Load Balancing:**

- Ensuring that Services are discovered correctly and load-balanced effectively across Pods can be challenging, especially in large or dynamic environments.

6. Persistent Storage

- **Stateful Applications:**

- Managing persistent storage for stateful applications in Kubernetes can be complex. Kubernetes supports Persistent Volumes (PVs) and Persistent Volume Claims (PVCs), but configuring and managing storage can require additional effort.

- **Data Migration and Backup:**

- Handling data migration and backup for applications running in Kubernetes needs to be carefully planned. Ensuring data durability and availability across cluster operations is crucial.

7. Integration with Existing Systems

- **Legacy Systems:**
 - Integrating Kubernetes with existing legacy systems and infrastructure can be challenging. Migrating applications from traditional environments to Kubernetes may require significant changes.
- **Interoperability:**
 - Ensuring interoperability between Kubernetes and other systems (e.g., databases, message brokers) can involve additional configuration and management.

8. Cost Management

- **Cloud Costs:**
 - Running Kubernetes on cloud platforms can incur substantial costs due to the need for multiple nodes, storage, and network resources. Effective cost management and optimization are essential.
- **Operational Costs:**
 - The operational costs of managing a Kubernetes cluster, including personnel, tools, and maintenance, can add up. Balancing these costs with the benefits is important for overall efficiency.

Key Points to Remember (For Interview Preparation)

1. **Complexity:** Be aware of the complexity of Kubernetes and its steep learning curve. Understand that configuring and managing Kubernetes requires significant expertise.
2. **Resource Overhead:** Recognize that Kubernetes introduces resource overhead and requires careful resource management to avoid inefficiencies.
3. **Operational Overhead:** Acknowledge the operational overhead involved in managing and maintaining a Kubernetes cluster, including troubleshooting and security.
4. **Security:** Understand the security concerns associated with Kubernetes, including access control, secrets management, and Pod security.
5. **Networking:** Be familiar with the challenges related to Kubernetes networking, Service discovery, and load balancing.
6. **Persistent Storage:** Know the complexities of managing persistent storage for stateful applications and data migration in Kubernetes.
7. **Integration and Costs:** Be aware of the challenges in integrating Kubernetes with existing systems and managing costs associated with cloud and operational resources.

Introduction to AKS

Azure Kubernetes Service (AKS) is a managed container orchestration service provided by Microsoft Azure that simplifies the deployment, management, and scaling of containerized applications using Kubernetes. AKS abstracts much of the complexity associated with Kubernetes management, allowing you to focus on developing and running your applications.

1. Overview of AKS

- **Managed Kubernetes Service:**
 - AKS provides a managed Kubernetes environment, handling many of the operational aspects of Kubernetes, such as the control plane and cluster maintenance, which are managed by Azure.
- **Integration with Azure Ecosystem:**
 - AKS integrates seamlessly with other Azure services, such as Azure Container Registry (ACR) for container image storage, Azure Monitor for logging and monitoring, and Azure Active Directory (AAD) for identity and access management.
- **Simplified Cluster Management:**
 - AKS reduces the operational burden of managing Kubernetes clusters by automating tasks like upgrades, scaling, and security patching.

2. Key Features of AKS

- **Automated Cluster Management:**
 - **Automatic Upgrades:** AKS automates Kubernetes version upgrades, ensuring that your cluster is always up to date with the latest features and security patches.
 - **Scaling:** AKS supports both manual and automatic scaling of nodes and pods to accommodate varying workloads.
- **Integrated Monitoring and Diagnostics:**
 - **Azure Monitor:** Provides comprehensive monitoring and logging capabilities, allowing you to gain insights into cluster performance and diagnose issues.
 - **Azure Log Analytics:** Collects and analyzes log data from your AKS cluster to help with troubleshooting and performance optimization.
- **Security and Compliance:**
 - **Azure Active Directory Integration:** Allows you to manage Kubernetes access and permissions using Azure AD identities and roles.
 - **Network Policies:** AKS supports Kubernetes Network Policies to control traffic between pods and enhance security.

- **Container Registry Integration:**
 - **Azure Container Registry (ACR):** AKS integrates with ACR, allowing you to store and manage container images securely and efficiently.
- **Developer Productivity:**
 - **DevOps Integration:** AKS integrates with Azure DevOps and GitHub Actions for continuous integration and deployment (CI/CD) pipelines, enabling automated application deployments.
- **Cost Management:**
 - **Pay-as-You-Go:** AKS is billed based on the resources used, including virtual machines and storage. You only pay for the compute resources you consume, with no charge for the Kubernetes control plane.

3. Common Use Cases

- **Microservices Architectures:**
 - AKS is well-suited for deploying and managing microservices applications, providing scalability and isolation for different service components.
- **Dev/Test Environments:**
 - Developers and testers can use AKS to quickly spin up Kubernetes clusters for development and testing purposes, leveraging the managed nature of AKS to reduce setup and maintenance overhead.
- **Production Workloads:**
 - AKS supports running production-grade workloads with high availability and reliability, thanks to its integration with Azure's infrastructure and services.
- **Hybrid and Multi-Cloud Scenarios:**
 - AKS can be part of a hybrid cloud or multi-cloud strategy, allowing you to manage workloads across on-premises, Azure, and other cloud environments.

4. Getting Started with AKS

- **Creating an AKS Cluster:**
 - You can create an AKS cluster using the Azure Portal, Azure CLI, or ARM templates. The process involves specifying cluster configuration parameters such as node size, number of nodes, and Kubernetes version.
- **Accessing the Cluster:**
 - Once created, you can access the AKS cluster using kubectl, the Kubernetes command-line tool. You can configure kubectl to communicate with your AKS cluster using the Azure CLI or the Azure Portal.

- **Deploying Applications:**

- Deploying applications to AKS involves creating Kubernetes manifests (Deployment, Service, ConfigMap, etc.) and applying them using kubectl. AKS provides a scalable and resilient environment for running containerized applications.

Key Points to Remember (For Interview Preparation)

1. **Managed Service:** AKS is a managed Kubernetes service that simplifies cluster management by handling operational aspects such as upgrades and scaling.
2. **Integration:** AKS integrates with other Azure services, including Azure Container Registry, Azure Monitor, and Azure Active Directory, enhancing its functionality and security.
3. **Automated Management:** Features like automatic upgrades, scaling, and integrated monitoring reduce operational overhead and improve cluster management.
4. **Cost-Effective:** AKS follows a pay-as-you-go pricing model, charging only for the resources you use without additional costs for the Kubernetes control plane.
5. **Use Cases:** AKS is ideal for microservices architectures, development and testing environments, production workloads, and hybrid/multi-cloud scenarios.
6. **Getting Started:** Creating and managing an AKS cluster involves using the Azure Portal, Azure CLI, or ARM templates, and deploying applications using Kubernetes manifests and kubectl.

Kubernetes Architecture

Understanding Kubernetes architecture is essential to grasp how AKS works, as AKS leverages this architecture to manage containerized applications at scale. Kubernetes architecture consists of several core components that enable it to schedule, manage, and orchestrate containers efficiently.

1. Key Components of Kubernetes

The Kubernetes architecture is divided into two primary sections: the **Control Plane** and the **Worker Nodes**. Let's explore each of these in detail.

1. Control Plane:

The control plane is responsible for managing the overall cluster and orchestrating the worker nodes. It consists of several components, each with a specific role in managing the Kubernetes cluster.

- **API Server (kube-apiserver):**
 - The API Server is the entry point for all Kubernetes-related operations. It serves as the primary interface for users, applications, and administrators to interact with the Kubernetes cluster. All requests to the cluster (e.g., to create or delete resources like pods and services) go through the API server.
- **Etcd:**
 - Etcd is the key-value store used by Kubernetes to store all cluster data. This includes information about nodes, pods, services, configuration data, secrets, and more. Since Etcd stores the cluster's entire state, it plays a crucial role in ensuring the reliability and availability of the cluster.
- **Controller Manager (kube-controller-manager):**
 - The controller manager runs multiple controllers that are responsible for monitoring the state of the cluster and making necessary adjustments to reach the desired state. For example, the ReplicaSet controller ensures that the correct number of pod replicas are running at any given time.
- **Scheduler (kube-scheduler):**
 - The Kubernetes scheduler is responsible for assigning pods to nodes based on resource availability, policies, and constraints. It evaluates each pod's resource requirements and schedules them on the appropriate worker nodes.

2. Worker Nodes:

Worker nodes are the machines (virtual or physical) where containers are deployed. Each worker node runs several components that allow it to communicate with the control plane and manage the running containers.

- **Kubelet:**
 - The kubelet is the primary agent running on each worker node. It is responsible for communicating with the API server, receiving instructions, and ensuring that the containers specified in the pod definitions are running as expected on the node.
- **Kube-Proxy:**
 - The kube-proxy is a network proxy that maintains network rules on each worker node. It enables communication between different services and pods, both within the node and across nodes. It also ensures that the networking environment supports Kubernetes services.
- **Container Runtime:**
 - The container runtime (e.g., Docker or containerd) is the software that actually runs containers on the worker node. It handles the downloading of container images and manages the lifecycle of containers.

2. Interaction Between Components

The control plane components work in tandem with the worker node components to ensure that the desired state of the cluster is always achieved and maintained. Here's how these interactions typically work:

- **Cluster State:**
 - The desired state of the cluster (e.g., the number of replicas, services, and configurations) is stored in the etcd database.
- **API Requests:**
 - Administrators and developers interact with the Kubernetes API server to submit requests to the cluster, such as deploying new applications or scaling existing ones. The API server processes these requests and updates the etcd database.
- **Controllers:**
 - The controller manager continuously monitors the state stored in etcd. If the actual state of the cluster does not match the desired state (e.g., a pod is down or missing), the appropriate controller will take corrective actions, such as creating a new pod.
- **Scheduling:**
 - The scheduler monitors resource availability on the worker nodes and assigns pods to nodes that have the necessary resources to run the application. The kubelet on each worker node ensures that the assigned pods are running and communicates their status back to the API server.
- **Networking:**
 - Kube-proxy manages the network rules and ensures that services and pods can communicate with each other across nodes. It supports service discovery and load balancing.

3. Benefits of Kubernetes Architecture in AKS

- **Scalability:** Kubernetes can easily scale both horizontally (by adding more nodes) and vertically (by increasing node resources). AKS leverages this architecture to scale applications dynamically, ensuring they can handle varying loads.
- **Resilience:** By automatically rescheduling failed pods, AKS ensures high availability and resilience. The controller manager and scheduler work together to maintain the desired state of the application.
- **Modularity:** Kubernetes' modular architecture allows you to use different components interchangeably. For example, you can switch between container runtimes or networking plugins as needed, which makes AKS adaptable to different use cases.
- **Automation:** The control plane components in Kubernetes automate most of the operational tasks, such as load balancing, scaling, and self-healing, which AKS extends to its managed services, reducing the administrative overhead.

4. AKS and Kubernetes Architecture

In AKS, the control plane is managed by Azure, meaning that Azure takes responsibility for maintaining the API server, etcd, scheduler, and controller manager. The worker nodes, however, are managed by you, allowing you to configure and scale them as necessary.

- **Managed Control Plane:**
 - AKS abstracts the complexity of managing the control plane, automating tasks like patching, upgrading, and scaling the control plane components.
- **Customizable Worker Nodes:**
 - You have full control over the worker nodes in AKS. You can specify the size, number, and configuration of these nodes based on your application requirements. You can also scale the nodes manually or use autoscaling features provided by AKS.

Key Points to Remember (For Interview Preparation)

1. **Control Plane vs Worker Nodes:** Kubernetes architecture is divided into the control plane (which manages the cluster) and worker nodes (where containers are deployed).
2. **Control Plane Components:** Key components like the API server, etcd, controller manager, and scheduler play crucial roles in ensuring the cluster's desired state is maintained.
3. **Worker Node Components:** The kubelet, kube-proxy, and container runtime (e.g., Docker) ensure that containers are running as expected on the worker nodes.
4. **AKS Managed Control Plane:** AKS simplifies Kubernetes management by offering a fully managed control plane while giving you control over worker nodes.
5. **Resilience and Automation:** Kubernetes' architecture enables self-healing, scalability, and automation, reducing the need for manual intervention in operational tasks.

5. Creating AKS Cluster (using CLI)

Creating an AKS (Azure Kubernetes Service) cluster using the Azure CLI involves a series of steps to set up and configure a Kubernetes cluster that is managed by Azure. This process allows you to deploy and manage containerized applications efficiently.

1. Prerequisites

Before creating an AKS cluster, ensure that you have the following:

- **Azure CLI:** Make sure the Azure CLI (az) is installed and up-to-date on your machine.
- **Azure Subscription:** An active Azure subscription to create and manage resources.
- **Resource Group:** An existing or new Azure resource group where your AKS cluster will be created.

Install Azure CLI:

For Windows:

```
msiexec /i https://aka.ms/installazurecliwindows
```

For macOS:

```
brew install azure-cli
```

For Linux:

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

2. Authenticate to Azure

Before creating resources, log in to your Azure account using the Azure CLI:

```
az login
```

This command opens a web browser for authentication. Follow the prompts to log in.

3. Create a Resource Group

A resource group is a logical container for Azure resources. If you don't already have a resource group, create one using the Azure CLI:

```
az group create --name myResourceGroup --location eastus
```

Replace myResourceGroup with your desired resource group name and eastus with your preferred Azure region.

4. Create an AKS Cluster

To create an AKS cluster, use the `az aks create` command. This command allows you to specify various parameters for the cluster, including the name, resource group, and node configurations.

```
az aks create \  
  --resource-group myResourceGroup \  
  --name myAKSCluster \  
  --node-count 3 \  
  --enable-addons monitoring \  
  --generate-ssh-keys
```

Parameters:

- `--resource-group`: The resource group where the AKS cluster will be created.
- `--name`: The name of the AKS cluster.
- `--node-count`: The number of nodes in the cluster.
- `--enable-addons`: Optional. Enables specific add-ons like monitoring.
- `--generate-ssh-keys`: Generates SSH keys for node access.

5. Verify Cluster Creation

After creating the AKS cluster, you can verify its status using:

```
az aks show --resource-group myResourceGroup --name myAKSCluster
```

To get detailed information about your cluster, including its version and configuration, use this command.

6. Connect to the AKS Cluster

To manage your AKS cluster, configure `kubectl` to use the cluster:

```
az aks get-credentials --resource-group myResourceGroup --name myAKSCluster
```

This command downloads the cluster credentials and configures `kubectl` to interact with your AKS cluster.

7. Verify kubectl Configuration

Ensure that `kubectl` is correctly configured to interact with your AKS cluster:

```
kubectl get nodes
```

This command lists all nodes in the cluster, confirming that your `kubectl` configuration is working.

Key Points to Remember (For Interview Preparation)

1. **Azure CLI Commands:** Familiarize yourself with the `az aks create` command and its parameters for creating and configuring AKS clusters.
2. **Resource Group:** Understand the role of resource groups in organizing Azure resources and how to create them.
3. **Cluster Configuration:** Be aware of key parameters like node count and add-ons when creating an AKS cluster.
4. **Connecting to AKS:** Know how to use `az aks get-credentials` to configure `kubectl` for managing your AKS cluster.
5. **Verification:** Use commands like `az aks show` and `kubectl get nodes` to verify the cluster creation and connectivity.

All Important Commands of kubectl

`kubectl` is the command-line tool for interacting with Kubernetes clusters. It allows you to manage your applications and clusters efficiently. Below is a detailed overview of some essential `kubectl` commands, categorized by their use cases:

1. Cluster Management

- **Check Cluster Info**

`kubectl cluster-info`

Displays information about the Kubernetes cluster, including the master and services URLs.

- **Get Cluster Nodes**

`kubectl get nodes`

Lists all nodes in the cluster, providing their status and roles.

- **Get Cluster Version**

`kubectl version`

Shows the Kubernetes client and server versions.

2. Pod Management

- **List Pods**

`kubectl get pods`

Lists all pods in the current namespace. Use `--all-namespaces` to list pods across all namespaces.

- **Describe Pod**

`kubectl describe pod <pod-name>`

Provides detailed information about a specific pod, including events and status.

- **Get Pod Logs**

`kubectl logs <pod-name>`

Retrieves logs from a specific pod. Use `--container <container-name>` if the pod has multiple containers.

- **Execute Command in Pod**

`kubectl exec -it <pod-name> -- <command>`

Executes a command inside a specific pod. The `-it` flag is used for interactive commands.

- **Delete Pod**

`kubectl delete pod <pod-name>`

Deletes a specific pod. This is useful for restarting a pod by forcefully removing it.

3. Deployment Management

- **Create Deployment**

`kubectl create deployment <deployment-name> --image=<image-name>`

Creates a new deployment with the specified name and container image.

- **List Deployments**

`kubectl get deployments`

Lists all deployments in the current namespace.

- **Scale Deployment**

`kubectl scale deployment <deployment-name> --replicas=<number>`

Scales the number of replicas for a deployment to the specified number.

- **Update Deployment**

`kubectl apply -f <deployment-file>.yaml`

Applies changes to a deployment based on a YAML configuration file.

- **Delete Deployment**

kubectl delete deployment <deployment-name>

Deletes a specific deployment along with its pods.

4. Service Management

- **List Services**

kubectl get services

Lists all services in the current namespace.

- **Describe Service**

kubectl describe service <service-name>

Provides detailed information about a specific service, including its endpoints.

- **Create Service**

kubectl expose deployment <deployment-name> --port=<port> --target-port=<target-port> --name=<service-name>

Creates a service that exposes a deployment on the specified ports.

- **Delete Service**

kubectl delete service <service-name>

Deletes a specific service.

5. Namespace Management

- **List Namespaces**

kubectl get namespaces

Lists all namespaces in the cluster.

- **Create Namespace**

kubectl create namespace <namespace-name>

Creates a new namespace.

- **Delete Namespace**

kubectl delete namespace <namespace-name>

Deletes a specific namespace and its associated resources.

6. Config Management

- **View Config Maps**

kubectl get configmaps

Lists all config maps in the current namespace.

- **Describe Config Map**

kubectl describe configmap <configmap-name>

Provides detailed information about a specific config map.

- **Create Config Map**

kubectl create configmap <configmap-name> --from-literal=<key>=<value>

Creates a config map with the specified key-value pair.

- **Delete Config Map**

kubectl delete configmap <configmap-name>

Deletes a specific config map.

7. Secret Management

- **List Secrets**

kubectl get secrets

Lists all secrets in the current namespace.

- **Create Secret**

kubectl create secret generic <secret-name> --from-literal=<key>=<value>

Creates a new secret with the specified key-value pair.

- **Describe Secret**

kubectl describe secret <secret-name>

Provides detailed information about a specific secret.

- **Delete Secret**

kubectl delete secret <secret-name>

Deletes a specific secret.

8. Troubleshooting and Debugging

- **Get Pod Events**

kubectl get events

Lists recent events in the cluster that can help in debugging issues.

- **Get Pod Metrics**

`kubectl top pod`

Shows resource usage (CPU and memory) of pods. Requires metrics-server to be installed.

- **Describe Node**

`kubectl describe node <node-name>`

Provides detailed information about a specific node, including resource usage and status.

Key Points to Remember (For Interview Preparation)

1. **Basic Commands:** Understand the basic kubectl commands for managing pods, deployments, and services.
2. **Pod Management:** Be able to list, describe, and interact with pods, and retrieve logs.
3. **Deployment Management:** Familiarize yourself with creating, scaling, and deleting deployments.
4. **Service Management:** Know how to expose deployments via services and manage them.
5. **Namespace and Config Management:** Understand how to manage namespaces and config maps/secrets.
6. **Troubleshooting:** Be aware of commands for troubleshooting issues in the cluster, including viewing events and metrics.

Deployment Manifests

A deployment manifest is a YAML file that defines the desired state of your application and how Kubernetes should manage it. Here's a detailed breakdown of key components and how to create a deployment manifest:

1. Basic Structure of a Deployment Manifest

A deployment manifest typically includes the following sections:

1. **apiVersion:** Specifies the Kubernetes API version to use for the deployment. For deployments, this is usually `apps/v1`.
2. **kind:** Defines the type of Kubernetes resource being created. For deployments, this is `Deployment`.
3. **metadata:** Contains metadata about the deployment, such as its name, labels, and namespace.

4. **spec:** Defines the desired state of the deployment, including the pod template and strategy for managing updates.

Example:

apiVersion: apps/v1

kind: Deployment

metadata:

name: my-app-deployment

labels:

app: my-app

spec:

replicas: 3

selector:

matchLabels:

app: my-app

template:

metadata:

labels:

app: my-app

spec:

containers:

- name: my-app-container

image: my-app-image:latest

ports:

- containerPort: 80

2. Key Sections Explained

- **apiVersion:**
 - Specifies the version of the Kubernetes API you're using. For deployments, use apps/v1.
- **kind:**
 - Indicates the type of resource. For deploying applications, use Deployment.

- **metadata:**
 - **name:** Name of the deployment.
 - **labels:** Labels used to identify and select resources. These are crucial for service selectors and pod management.
- **spec:**
 - **replicas:** The number of pod replicas to maintain. Kubernetes ensures that this number of pods is always running.
 - **selector:** A label selector used to identify the pods managed by this deployment. It must match the labels defined in the pod template.
 - **template:**
 - **metadata:**
 - **labels:** Labels for the pods created by this deployment. They must match the selector.
 - **spec:**
 - **containers:** Defines the containers within each pod.
 - **name:** The name of the container.
 - **image:** The Docker image to use for the container.
 - **ports:** Ports exposed by the container.

3. Example Deployment Manifest

Here is a complete example of a deployment manifest:

apiVersion: apps/v1

kind: Deployment

metadata:

name: my-web-app

labels:

app: my-web-app

spec:

replicas: 3

selector:

matchLabels:

app: my-web-app

```
template:
  metadata:
    labels:
      app: my-web-app
  spec:
    containers:
      - name: web
        image: my-web-app:1.0
        ports:
          - containerPort: 80
        env:
          - name: ASPNETCORE_ENVIRONMENT
            value: "Production"
```

Explanation:

- **metadata.name:** my-web-app - The name of the deployment.
- **spec.replicas:** 3 - Three replicas of the pod will be maintained.
- **spec.selector.matchLabels:** app: my-web-app - This selector will match the pods with the label app: my-web-app.
- **spec.template.metadata.labels:** app: my-web-app - Labels for the pods to match the selector.
- **spec.template.spec.containers:**
 - **name:** web - Name of the container.
 - **image:** my-web-app:1.0 - Docker image used.
 - **ports:** Exposes port 80 on the container.
 - **env:** Defines an environment variable ASPNETCORE_ENVIRONMENT with value "Production".

4. Managing Deployments

- **Apply Deployment Manifest**

```
kubectl apply -f deployment.yaml
```

Applies the deployment manifest to create or update resources in the cluster.

- **View Deployment Status**

`kubectl rollout status deployment/my-web-app`

Checks the rollout status of a deployment.

- **Update Deployment** Modify the deployment manifest and reapply it with `kubectl apply -f deployment.yaml` to update the deployment.
- **Rollback Deployment**

`kubectl rollout undo deployment/my-web-app`

Rolls back to the previous version of the deployment if there is an issue with the new version.

5. Best Practices

- **Version Tags:** Always use version tags for images to ensure that your deployments are predictable and reproducible.
- **Health Checks:** Consider adding `livenessProbe` and `readinessProbe` to ensure your application is running correctly and is ready to serve traffic.
- **Resource Limits:** Define resource limits and requests to ensure that your containers don't consume more resources than available.

Key Points to Remember (For Interview Preparation)

1. **Manifest Structure:** Understand the key sections of a deployment manifest (`apiVersion`, `kind`, `metadata`, `spec`).
2. **Pod Management:** Know how to configure replicas and manage pod templates.
3. **Updating Deployments:** Be familiar with updating and rolling back deployments.
4. **Best Practices:** Be aware of best practices for defining and managing deployments.

Service Manifests

A service manifest defines how to expose and access your applications running in Kubernetes. Services provide a stable IP address and DNS name to access your application, enabling communication between different parts of your application.

1. Basic Structure of a Service Manifest

A service manifest typically includes the following sections:

1. **apiVersion:** Specifies the Kubernetes API version to use for the service. For services, this is usually v1.
2. **kind:** Defines the type of Kubernetes resource being created. For services, this is Service.
3. **metadata:** Contains metadata about the service, such as its name and labels.
4. **spec:** Defines the desired state of the service, including type, selector, and ports.

Example:

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
  labels:
    app: my-app
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: LoadBalancer
```

2. Key Sections Explained

- **apiVersion:**
 - Specifies the version of the Kubernetes API you're using. For services, use v1.
- **kind:**
 - Indicates the type of resource. For exposing applications, use Service.
- **metadata:**
 - **name:** Name of the service.
 - **labels:** Labels used to identify and manage the service.

- **spec:**
 - **selector:** A label selector used to identify the pods that the service will route traffic to. It must match the labels defined in the pod template.
 - **ports:**
 - **protocol:** Protocol used by the service, typically TCP.
 - **port:** The port on which the service is exposed.
 - **targetPort:** The port on the container to which traffic is forwarded.
 - **type:** Type of service. Common types include:
 - **ClusterIP** (default): Exposes the service on a cluster-internal IP. Only accessible within the cluster.
 - **NodePort:** Exposes the service on each node's IP at a static port. Accessible externally via <NodeIP>:<NodePort>.
 - **LoadBalancer:** Exposes the service externally using a cloud provider's load balancer. Provides a stable external IP address.
 - **ExternalName:** Maps the service to an external DNS name.

3. Example Service Manifest

Here's a more detailed example of a service manifest:

apiVersion: v1

kind: Service

metadata:

name: my-web-service

labels:

app: my-web-app

spec:

selector:

app: my-web-app

ports:

- protocol: TCP

port: 80

targetPort: 8080

type: NodePort

Explanation:

- **metadata.name:** my-web-service - The name of the service.
- **spec.selector:** app: my-web-app - The service routes traffic to pods with the label app: my-web-app.
- **spec.ports:**
 - **port:** 80 - The port exposed by the service.
 - **targetPort:** 8080 - The port on the container where the service directs traffic.
- **spec.type:** NodePort - The service is exposed on a static port on each node.

4. Managing Services

- **Apply Service Manifest**

`kubectl apply -f service.yaml`

Applies the service manifest to create or update services in the cluster.

- **View Service Details**
- `kubectl get services`

`kubectl describe service/my-web-service`

Lists all services and describes a specific service to show details.

- **Expose Deployment as a Service**

`kubectl expose deployment my-web-app --type=LoadBalancer --port=80 --target-port=8080`

Creates a service for an existing deployment.

- **Delete Service**

`kubectl delete service/my-web-service`

Deletes the specified service.

5. Best Practices

- **Use Labels and Selectors:** Ensure the selector in your service manifest matches the labels of the pods to route traffic correctly.
- **Choose Service Type Wisely:** Select the service type based on your needs (e.g., ClusterIP for internal services, LoadBalancer for external access).
- **Security Considerations:** Be aware of the security implications of exposing services, especially with types like NodePort and LoadBalancer.

Key Points to Remember (For Interview Preparation)

1. **Service Structure:** Understand the key sections of a service manifest (apiVersion, kind, metadata, spec).
2. **Service Types:** Know the different service types (ClusterIP, NodePort, LoadBalancer, ExternalName) and when to use each.
3. **Service Management:** Be familiar with creating, updating, and managing services.
4. **Selector and Labels:** Understand how selectors are used to route traffic to the appropriate pods.

Troubleshooting Pods

Troubleshooting Kubernetes pods involves diagnosing and resolving issues related to pod creation, execution, and functionality. Here's a comprehensive guide to help you troubleshoot pods effectively.

1. Checking Pod Status

- **List Pods:**

```
kubectl get pods
```

This command shows the list of all pods and their statuses. Look for pods that are not in the Running state.

- **Describe Pod:**

```
kubectl describe pod <pod-name>
```

Provides detailed information about a specific pod, including events and error messages.

- **View Pod Logs:**

```
kubectl logs <pod-name>
```

Displays the logs from a container within the pod. Useful for debugging application-level issues.

For a specific container in a pod:

```
kubectl logs <pod-name> -c <container-name>
```

2. Common Pod Issues and Solutions

- **Pod Statuses:**

- **Pending:** The pod is waiting for resources to be allocated.
 - **Check Events:** Use `kubectl describe pod <pod-name>` to view events and see why the pod is pending.
 - **Check Resource Requests:** Ensure that your cluster has enough resources to fulfill the pod's requests.
- **CrashLoopBackOff:** The pod is repeatedly crashing and restarting.
 - **Examine Logs:** Use `kubectl logs <pod-name>` to check the logs for errors.
 - **Check Liveness and Readiness Probes:** Misconfigured probes can cause restarts.
- **Error:** The pod is unable to start due to errors.
 - **Check Configuration:** Review the pod's YAML configuration for mistakes.
 - **Examine Logs:** Logs can provide clues about the issue.

- **Inspecting Container State:**

```
kubectl get pod <pod-name> -o=jsonpath='{.status.containerStatuses[*].state}'
```

This command provides the state of all containers within the pod.

- **Checking Resource Utilization:**

```
kubectl top pod <pod-name>
```

Shows the CPU and memory usage of a pod. High resource usage can cause performance issues.

3. Debugging Tools

- **kubectl exec:**

```
kubectl exec -it <pod-name> -- /bin/sh
```

Opens a shell inside a running pod, allowing you to run commands and inspect the file system.

- **kubectl port-forward:**

```
kubectl port-forward <pod-name> <local-port>:<container-port>
```

Forwards a port from your local machine to a pod's port, useful for accessing applications running inside the pod.

- **kubectl cp:**

```
kubectl cp <local-file> <pod-name>:<container-path>
```

Copies files between your local machine and a container in a pod.

4. Troubleshooting Techniques

- **Logs Analysis:** Start by checking the logs for error messages or stack traces that can provide clues about the problem.
- **Events Inspection:** Use `kubectl describe` to view events related to the pod, which can help identify issues like scheduling problems or resource constraints.
- **Configuration Review:** Ensure that the pod's configuration (e.g., resource requests, environment variables, volumes) is correct and matches your expectations.
- **Resource Monitoring:** Check resource usage to ensure that the pod has enough CPU and memory. Use `kubectl top` and monitor metrics from the cluster.

5. Example Scenarios

- **Pod Not Starting:**
 - **Problem:** Pod is stuck in Pending state.
 - **Solution:** Check for resource availability. Ensure there are no unsatisfied resource requests. Review events with `kubectl describe pod`.
- **Application Crashing:**
 - **Problem:** Container is repeatedly crashing (CrashLoopBackOff).
 - **Solution:** Examine application logs with `kubectl logs`. Check for application errors or misconfigurations. Adjust liveness and readiness probes if necessary.
- **Network Issues:**
 - **Problem:** Application is not accessible.
 - **Solution:** Verify service and endpoint configuration. Use `kubectl port-forward` to test connectivity. Check network policies and firewall rules.

Key Points to Remember (For Interview Preparation)

1. **Pod Statuses:** Understand the meaning of different pod statuses (Pending, Running, CrashLoopBackOff, Error).
2. **Troubleshooting Commands:** Be familiar with `kubectl get pods`, `kubectl describe pod`, `kubectl logs`, and `kubectl exec`.
3. **Common Issues:** Know common pod issues and how to diagnose them (e.g., pending pods, crashing containers).
4. **Resource Management:** Monitor and manage pod resources to prevent issues related to resource constraints.

Zero Downtime Rollout

Zero Downtime Rollout refers to deploying new versions of an application without interrupting the service or causing downtime for users. This approach ensures continuous availability and minimizes disruption.

1. What is Zero Downtime Rollout?

Zero Downtime Rollout is a deployment strategy where new versions of applications are rolled out gradually, ensuring that the system remains available and operational during the update. This technique is crucial for high-availability systems where even short downtimes can affect user experience and business operations.

2. Strategies for Zero Downtime Rollout

1. Blue-Green Deployment

- **Concept:** Involves running two identical production environments (Blue and Green). At any time, only one environment serves the live traffic.
- **Process:**
 - Deploy the new version of the application to the idle environment (e.g., Green).
 - Test the new version in the Green environment.
 - Switch traffic from the Blue environment to the Green environment.
 - The Blue environment becomes idle and can be used for the next deployment.
- **Advantages:**
 - Easy rollback: Switch back to the previous environment if issues arise.
 - Minimal disruption: Users experience no downtime during the switch.

2. Rolling Updates

- **Concept:** Gradually updates instances of an application one by one.
- **Process:**
 - Update a small subset of instances (e.g., one pod) at a time.
 - Ensure that the new version is healthy before updating the next subset.
 - Continue this process until all instances are updated.

- **Advantages:**
 - Gradual deployment reduces risk by isolating issues to a small number of instances.
 - Continuous availability of the service.

3. Canary Releases

- **Concept:** Deploy the new version to a small subset of users or instances before a full rollout.
- **Process:**
 - Deploy the new version to a small portion of the production environment (e.g., 5% of instances).
 - Monitor the performance and user feedback.
 - Gradually increase the rollout percentage if the new version is stable.
- **Advantages:**
 - Limited exposure of potential issues.
 - Allows for real-world testing with a controlled group.

4. Feature Toggles (Flags)

- **Concept:** Use feature flags to control the visibility of new features in the application.
- **Process:**
 - Deploy the new version with feature flags turned off.
 - Gradually enable the new features by toggling flags on.
- **Advantages:**
 - Enables feature-level rollouts without deploying new code.
 - Provides the ability to roll back specific features without affecting the entire deployment.

3. Implementing Zero Downtime in Kubernetes

- **Rolling Updates in Kubernetes:**
 - Kubernetes supports rolling updates natively through its deployment objects.
 - **Deployment Manifest Example:**

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app-container
          image: my-app:latest
          ports:
```

- containerPort: 80

- Kubernetes updates pods one at a time to ensure that at least some instances are always available.

- **Readiness Probes:**

- Use readiness probes to ensure that new pods are only added to the service once they are ready.

- **Readiness Probe Example:**

```
readinessProbe:
  httpGet:
    path: /health
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 10
```

- **Rolling Update Strategy:**

- Define the rolling update strategy in the deployment manifest.
- **Strategy Example:**


```
strategy:  
  type: RollingUpdate  
rollingUpdate:  
  maxSurge: 1  
  maxUnavailable: 1
```

4. Monitoring and Rollback

- **Monitoring:**
 - Monitor application metrics and logs during the rollout to detect issues early.
 - Use tools like Prometheus and Grafana for real-time monitoring.
- **Rollback:**
 - Ensure that rollback procedures are in place to revert to the previous version if issues are detected.
 - Kubernetes supports rollback commands:

```
kubectl rollout undo deployment/my-app
```

5. Best Practices for Zero Downtime Rollout

- **Automated Testing:** Perform automated tests to verify that the new version works correctly before deploying.
- **Monitoring and Alerts:** Set up monitoring and alerts to detect and respond to issues quickly.
- **Incremental Rollouts:** Use canary releases or rolling updates to deploy changes incrementally.
- **Backup and Recovery:** Ensure that you have backup and recovery procedures in place in case of deployment failures.

Key Points to Remember (For Interview Preparation)

1. **Deployment Strategies:** Understand different deployment strategies like Blue-Green Deployment, Rolling Updates, Canary Releases, and Feature Toggles.
2. **Kubernetes Rollouts:** Familiarize yourself with how Kubernetes handles rolling updates and readiness probes.
3. **Monitoring and Rollback:** Be aware of tools and practices for monitoring deployments and rolling back if necessary.

Kubernetes Secrets

Kubernetes Secrets are used to store and manage sensitive information, such as passwords, OAuth tokens, SSH keys, and other confidential data. Using Secrets helps keep sensitive information separate from application code and configuration.

1. What are Kubernetes Secrets?

Kubernetes Secrets allow you to securely store and manage sensitive data within your Kubernetes cluster. Secrets are encoded and can be referenced in your pods and deployments without exposing the actual values in plaintext.

2. Types of Secrets

1. Opaque Secrets

- General-purpose secrets that contain arbitrary user-defined data.
- Example use case: storing database passwords.

2. Docker Registry Secrets

- Used for authenticating with private Docker registries.
- Example use case: pulling private Docker images.

3. TLS Secrets

- Used to store TLS certificates and private keys.
- Example use case: securing communication between services.

4. Service Account Tokens

- Automatically created for service accounts.
- Example use case: providing credentials for API access.

3. Creating Secrets

Secrets can be created using `kubectl` commands or YAML manifests.

1. Creating Secrets with `kubectl`

- **Command Example:**

```
kubectl create secret generic my-secret --from-literal=username=myuser --from-literal=password=mypassword
```

- my-secret: Name of the Secret.
- --from-literal: Adds literal values to the Secret.
- **Creating a Secret from a File:**

`kubectl create secret generic my-secret --from-file=ssh-privatekey=/path/to/private.key`

2. Creating Secrets using YAML

- **YAML Example:**
- apiVersion: v1
- kind: Secret
- metadata:
- name: my-secret
- type: Opaque
- data:
- username: bXl1c2Vy

password: bXlwYXNzd29yZA==

- data: The Secret data in base64 encoding. username and password are encoded base64 values.

4. Using Secrets in Pods

Secrets can be used in Pods as environment variables or mounted as files.

1. Using Secrets as Environment Variables

- **YAML Example:**
- apiVersion: v1
- kind: Pod
- metadata:
- name: my-pod
- spec:
- containers:
- name: my-container
- image: my-image
- env:

- name: DB_USERNAME

valueFrom:

secretKeyRef:

name: my-secret

key: username

- name: DB_PASSWORD

valueFrom:

secretKeyRef:

name: my-secret

key: password

2. Mounting Secrets as Files

- **YAML Example:**

apiVersion: v1

kind: Pod

metadata:

name: my-pod

spec:

containers:

- name: my-container

image: my-image

volumeMounts:

- name: secret-volume

mountPath: /etc/secret

readOnly: true

volumes:

- name: secret-volume

secret:

secretName: my-secret

5. Managing Secrets

1. Viewing Secrets

- **Command Example:**

```
kubectl get secrets
```

- **Viewing Secret Details:**

```
kubectl describe secret my-secret
```

- **Decoding Secret Data:**

```
kubectl get secret my-secret -o jsonpath='{.data.username}' | base64 --decode
```

2. Updating Secrets

- **Updating with kubectl:**

```
kubectl create secret generic my-secret --from-literal=username=newuser --from-literal=password=newpassword --dry-run=client -o yaml | kubectl apply -f -
```

- **Updating with YAML:** Modify the existing YAML file and apply the changes:

```
kubectl apply -f secret.yaml
```

3. Deleting Secrets

- **Command Example:**

```
kubectl delete secret my-secret
```

6. Best Practices for Managing Secrets

- **Avoid Hardcoding Secrets:** Use Kubernetes Secrets to manage sensitive data instead of hardcoding them in your application code.
- **Use RBAC:** Implement Role-Based Access Control (RBAC) to restrict access to secrets.
- **Limit Secret Scope:** Ensure that secrets are only accessible to the necessary components.
- **Regularly Rotate Secrets:** Regularly update and rotate secrets to minimize security risks.

Key Points to Remember (For Interview Preparation)

1. **Types of Secrets:** Understand the different types of Kubernetes Secrets and their use cases.
2. **Creating and Managing Secrets:** Familiarize yourself with creating, viewing, updating, and deleting secrets.
3. **Using Secrets in Pods:** Know how to use secrets as environment variables and mounted files.
4. **Best Practices:** Be aware of best practices for managing secrets securely in Kubernetes.

.NET Microservices – Azure DevOps and AKS

Section 14: Azure DevOps - Notes

Introduction to Azure DevOps

Azure DevOps is a suite of development tools offered by Microsoft that provides a set of services for software development teams. It facilitates the entire DevOps lifecycle from planning to deployment, helping teams to plan, develop, deliver, and monitor applications.

Key Components of Azure DevOps:

1. **Azure Boards:** A project management tool that supports agile methodologies like Scrum and Kanban. It helps in tracking work with features such as backlogs, boards, and sprints.
2. **Azure Repos:** Provides Git repositories or Team Foundation Version Control (TFVC) for source control. It allows teams to collaborate on code development.
3. **Azure Pipelines:** A service that supports Continuous Integration (CI) and Continuous Deployment (CD) of applications. It automates the build, test, and deployment processes.
4. **Azure Test Plans:** Offers a set of tools for managing your testing efforts. It includes manual testing, exploratory testing, and user acceptance testing.
5. **Azure Artifacts:** Allows teams to share packages, manage dependencies, and integrate with CI/CD pipelines.

Benefits of Azure DevOps

- **Integrated Platform:** Combines several DevOps tools in one platform, providing seamless integration and communication between different stages of the development lifecycle.
- **Scalability:** Can handle projects of all sizes, from small teams to large enterprises, with its robust and scalable infrastructure.
- **Flexibility:** Supports various workflows and can integrate with other tools and services, offering flexibility in how teams manage their development process.
- **Automation:** Automates repetitive tasks like building, testing, and deployment, which improves efficiency and reduces the risk of human error.
- **Collaboration:** Facilitates better collaboration among team members through integrated tools like issue tracking, code review, and chat.

Key Points for Interview Preparation

- **Azure DevOps Services:** Familiarize yourself with the core services offered by Azure DevOps (Azure Boards, Repos, Pipelines, Test Plans, Artifacts).
- **DevOps Lifecycle:** Understand how Azure DevOps supports the entire software development lifecycle from planning to deployment.
- **Benefits:** Be able to explain the benefits of using Azure DevOps, including its integration, scalability, flexibility, and automation features.

Azure DevOps Organization

An Azure DevOps organization is a fundamental unit for managing your projects and users. It acts as a container for multiple projects and offers a centralized place for managing resources and settings.

- **Creating an Organization:** When you first sign up for Azure DevOps, you'll create an organization. This organization serves as a namespace for your projects and teams.
- **Organization Settings:** You can configure settings like security, billing, and policies at the organization level. This includes managing user access, setting up policies for code repositories, and configuring integrations with other services.
- **Permissions:** Organizations allow you to define and manage permissions at various levels. You can control who has access to what within your organization, including who can create projects, manage repositories, and access pipelines.

Projects in Azure DevOps

Projects are containers within an Azure DevOps organization that hold all the assets related to a particular software project.

- **Creating a Project:** You can create multiple projects within an organization. Each project can contain its own set of boards, repos, pipelines, test plans, and artifacts.
- **Project Settings:** Within a project, you can configure settings like boards, repositories, pipelines, and security policies. Each project can have its own unique settings and configurations.
- **Project Features:** Azure DevOps projects come with features like work item tracking, source control, build and release pipelines, and test management.
- **Project Access:** Access to projects can be controlled via security groups and permissions. You can specify who can view, edit, and manage different aspects of the project.
- **Project Templates:** When creating a new project, you can select from various templates that help set up your project with predefined settings and configurations based on your project type (e.g., Agile, Scrum).

Managing Multiple Projects

- **Project Collection:** An organization can have multiple projects, and these projects are grouped together under a project collection. This allows for better organization and management of related projects.
- **Cross-Project Collaboration:** Azure DevOps enables collaboration across different projects within the same organization. Teams can share resources, track work items, and manage dependencies across projects.

Key Points for Interview Preparation

- **Organization Structure:** Understand the concept of Azure DevOps organization and how it relates to projects. Be able to explain the purpose and benefits of creating multiple projects within an organization.
- **Permissions and Access:** Be familiar with how permissions are managed at the organization and project levels. Know how to configure user access and control permissions.
- **Project Features:** Have a good grasp of the features available within Azure DevOps projects and how they support various stages of the software development lifecycle.
- **Managing Projects:** Be able to discuss how to manage multiple projects and the benefits of organizing related projects within a single Azure DevOps organization.

Parallelism Request for Pipelines

Parallelism in Azure DevOps pipelines refers to the ability to run multiple jobs or stages simultaneously. This helps in speeding up the overall build and deployment process by utilizing concurrent execution.

Pipeline Parallelism

1. **Jobs and Stages:** In a pipeline, you can define multiple jobs that can run in parallel. Jobs within a stage can run concurrently, and different stages can also run in parallel if they are not dependent on each other.
2. **Concurrency Limits:** Azure DevOps provides a default level of parallelism, which can be increased by requesting additional parallel jobs if needed. Each organization has a predefined number of parallel jobs available based on the pricing tier.

3. Parallel Jobs in Azure DevOps:

- **Free Tier:** Includes a limited number of parallel jobs (e.g., 1 parallel job for public projects and 1 parallel job for private projects).
- **Paid Plans:** Additional parallel jobs can be purchased if the default number is insufficient for your needs.

Requesting Additional Parallel Jobs

1. **Pricing Tiers:** Understand the different pricing tiers offered by Azure DevOps for parallel jobs. Upgrading your plan may provide additional parallel jobs and other benefits.
2. **How to Request More Jobs:**
 - **Navigate to Organization Settings:** Go to your Azure DevOps organization and find the settings for parallel jobs.
 - **Select the Appropriate Plan:** Choose the plan that meets your needs for parallel job execution.
 - **Request Additional Parallel Jobs:** Follow the process to request or purchase additional parallel jobs if necessary.
3. **Usage Considerations:**
 - **Cost Management:** Keep an eye on the cost associated with additional parallel jobs. Efficient use of parallelism can help reduce the time taken for builds and deployments.
 - **Resource Utilization:** Ensure that your build agents are capable of handling the increased parallelism without performance degradation.

Configuring Parallelism in Pipelines

1. **Pipeline YAML Configuration:** Configure parallel jobs in your pipeline YAML file by defining multiple jobs within stages. Azure DevOps will automatically handle the parallel execution based on the available agents and configured limits.

Example YAML configuration:

jobs:

- job: Job1

pool:

vmImage: 'ubuntu-latest'

steps:

- script: echo This is Job 1

```
- job: Job2  
pool:  
  vmImage: 'ubuntu-latest'  
steps:  
  - script: echo This is Job 2
```

```
- job: Job3  
dependsOn: []  
pool:  
  vmImage: 'ubuntu-latest'  
steps:  
  - script: echo This is Job 3
```

In this example, Job1, Job2, and Job3 will run in parallel because they are not dependent on each other.

2. **Managing Dependencies:** Use `dependsOn` to specify dependencies between jobs and control the order of execution while still allowing parallelism where possible.

Key Points for Interview Preparation

- **Parallel Jobs Concept:** Be able to explain what parallelism is in the context of Azure DevOps pipelines and how it benefits the build and deployment process.
- **Configuring Parallelism:** Know how to configure parallel jobs in Azure DevOps pipelines, including YAML configuration and setting up parallel jobs in the pipeline definition.
- **Managing Parallelism:** Understand the limitations and costs associated with parallel jobs and how to request or purchase additional parallel jobs if needed.
- **Resource Utilization:** Be familiar with the implications of parallelism on resource utilization and performance.

Introduction to Azure Boards (Scrum)

Azure Boards is a service provided by Azure DevOps that helps manage and track work in software development projects. It supports various methodologies, including Scrum, Kanban, and Agile, allowing teams to plan, track, and discuss work in a collaborative environment.

Key Features of Azure Boards

1. **Work Items:** Azure Boards uses work items to track tasks, bugs, features, user stories, and other work-related activities. Each work item has a unique ID and can be customized with fields, states, and workflows.
2. **Boards:** The boards provide a visual representation of work items and their states. For Scrum, this includes Sprint Boards and Backlogs.
3. **Backlogs:** The backlog is a prioritized list of work items that need to be completed. It can include product backlog items (PBIs), user stories, tasks, and bugs.
4. **Sprints:** In Scrum, work is organized into sprints, which are time-boxed iterations. Azure Boards supports sprint planning, tracking, and reviewing.
5. **Queries:** Queries help filter and search for work items based on specific criteria. They can be saved and shared with team members.
6. **Dashboards:** Customizable dashboards provide insights and visualizations of project metrics, such as work item progress, sprint burndown charts, and team velocity.

Using Azure Boards with Scrum

1. **Creating and Managing Teams:**
 - **Teams:** Create and manage teams within Azure Boards to organize work and manage sprints. Each team can have its own backlog, board, and sprints.
 - **Team Settings:** Configure team settings, including iteration paths, areas, and team capacity.
2. **Creating a Scrum Project:**
 - **New Project:** When creating a new project, select the "Scrum" process template. This template includes predefined work item types and workflows suited for Scrum methodology.
 - **Configuring Backlogs and Sprints:** Set up and configure product and sprint backlogs. Define iteration paths to represent different sprints.
3. **Work Item Types:**
 - **User Stories:** Represent features or functionality from the user's perspective.
 - **Tasks:** Break down user stories or features into actionable items.
 - **Bugs:** Track defects or issues found during development or testing.
 - **Epics and Features:** Higher-level work items that represent significant work or project milestones.
4. **Planning and Tracking Sprints:**
 - **Sprint Planning:** Plan and prioritize work items for each sprint. Add user stories and tasks to the sprint backlog.

- **Sprint Execution:** Track progress during the sprint using the sprint board. Update work items as they move through different states.
- **Sprint Review and Retrospective:** Review completed work at the end of the sprint and conduct a retrospective to discuss improvements.

5. Reporting and Metrics:

- **Burndown Charts:** Visualize the progress of work completed versus work remaining during a sprint.
- **Velocity Charts:** Track the amount of work completed in each sprint to measure team performance and capacity.

6. Customizing Boards and Backlogs:

- **Columns and States:** Customize board columns and work item states to fit your team's workflow.
- **Work Item Types and Fields:** Modify work item types and add custom fields as needed for your project.

Configuring Azure Boards with Scrum

1. Setting Up Boards:

- Navigate to Azure Boards and select the Scrum process template.
- Configure boards for each team, ensuring they reflect the Scrum workflow.

2. Managing Backlogs:

- Access the product backlog and add user stories, bugs, and tasks.
- Prioritize backlog items based on business value and project requirements.

3. Planning Sprints:

- Define sprint iterations and assign backlog items to each sprint.
- Adjust sprint goals and capacity based on team availability and workload.

Key Points for Interview Preparation

- **Understanding Azure Boards:** Be able to explain the key features and benefits of Azure Boards, including work items, boards, backlogs, and sprints.
- **Scrum Methodology:** Understand how Azure Boards supports Scrum methodology, including sprint planning, tracking, and review.
- **Configuring Azure Boards:** Know how to create and configure Scrum projects, manage teams, and customize boards and backlogs.

- **Reporting and Metrics:** Be familiar with the types of reports and metrics available in Azure Boards, such as burndown charts and velocity charts.
- **Customizing Work Items:** Understand how to customize work items, fields, and workflows to fit the needs of your project.

Azure Repos

Azure Repos is a set of version control tools that you can use to manage your code. It offers Git repositories or Team Foundation Version Control (TFVC) to help manage your source code and track changes over time. Azure Repos is integrated with Azure DevOps and supports collaborative development, code reviews, and versioning.

Key Features of Azure Repos

1. **Git Repositories:** Git is a distributed version control system that allows multiple developers to work on code simultaneously. Azure Repos provides Git-based repositories that offer powerful branching and merging capabilities.
2. **TFVC (Team Foundation Version Control):** TFVC is a centralized version control system where changes are tracked in a central repository. It is less commonly used compared to Git but is still supported by Azure Repos.
3. **Branching and Merging:** Azure Repos supports branching strategies that help manage code changes and feature development. You can create branches for new features, bug fixes, or experiments and merge them into the main branch when ready.
4. **Pull Requests:** Pull requests are a way to review and merge changes from one branch into another. They facilitate code reviews and discussions among team members before changes are integrated.
5. **Code Reviews:** Code reviews are integrated into the pull request process. Team members can comment on code changes, request modifications, and approve or reject pull requests.
6. **Branch Policies:** Branch policies enforce rules for code quality and process. For example, you can require that code passes automated tests before it can be merged into a branch.
7. **Commit History:** Azure Repos tracks the history of all changes made to the codebase. You can view the commit history, compare changes between commits, and revert changes if necessary.

Using Azure Repos

1. Creating a Repository:

- **Create a New Repository:** Navigate to Azure Repos in Azure DevOps, and create a new Git repository. You can choose to initialize it with a README file or import an existing repository.
- **Clone a Repository:** Clone the repository to your local machine using Git commands or Git clients. For example:

```
git clone https://dev.azure.com/{organization}/{project}/_git/{repository}
```

2. Branching:

- **Create a Branch:** Create a new branch to work on a specific feature or bug fix. For example:

```
git checkout -b feature/new-feature
```

- **Merge a Branch:** Once your changes are complete, merge the branch back into the main branch. For example:
- `git checkout main`

```
git merge feature/new-feature
```

3. Pull Requests:

- **Create a Pull Request:** Open a pull request to propose changes from one branch to another. Add a title, description, and reviewers.
- **Review and Merge:** Review the code changes, discuss any issues, and address feedback. Approve the pull request and merge it into the target branch.

4. Branch Policies:

- **Configure Policies:** Set up branch policies to enforce code quality standards. For example, require successful build validation and code reviews before merging.

5. Commit History:

- **View Commit History:** Access the commit history to view changes, compare versions, and find specific commits.

6. Code Reviews:

- **Request a Code Review:** Request a code review from team members as part of the pull request process. Provide feedback and make changes as needed.

Branching Strategies

1. Feature Branching:

- **Create a Feature Branch:** Develop new features in separate branches. This isolates feature development from the main branch.

2. Release Branching:

- **Create Release Branches:** Manage releases by creating branches for different versions or stages of your application.

3. Hotfix Branching:

- **Create Hotfix Branches:** Address critical issues in production by creating hotfix branches that can be quickly merged into the main branch.

4. Main Branch:

- **Main Branch:** The main branch (often called main or master) represents the stable version of your codebase. Only tested and approved changes should be merged into this branch.

Key Points for Interview Preparation

- **Understanding Azure Repos:** Be able to explain the key features and benefits of Azure Repos, including Git repositories, TFVC, and branching strategies.
- **Branching and Merging:** Understand how to create and manage branches, merge changes, and resolve conflicts.
- **Pull Requests:** Know how to create, review, and merge pull requests. Understand the role of code reviews in maintaining code quality.
- **Branch Policies:** Be familiar with configuring branch policies to enforce code quality standards and automated builds.
- **Commit History:** Understand how to view and manage commit history, and use it to track changes and revert if necessary.

Git Branches

Branches in Git allow you to work on different versions of a project simultaneously. They are essential for managing features, bug fixes, and experiments in isolation from the main codebase.

Key Concepts:

1. **Main Branch:** The default branch in a Git repository, usually named main or master. This branch should contain the stable code that is ready for production.
2. **Feature Branch:** Created for developing new features. Named descriptively to indicate the feature being developed, e.g., feature/user-authentication.
3. **Bugfix Branch:** Used to fix bugs or issues in the code. Named to reflect the bug or issue being addressed, e.g., bugfix/fix-login-error.
4. **Release Branch:** Used to prepare for a new release. Allows for final testing and minor adjustments before merging into the main branch, e.g., release/v1.0.

5. **Hotfix Branch:** Used to address critical issues in the production code quickly. Created from the main branch, e.g., hotfix/fix-critical-bug.

Creating and Managing Branches:

1. **Create a Branch:** Use the following command to create a new branch and switch to it:

```
git checkout -b branch-name
```

Example:

```
git checkout -b feature/new-login
```

2. **List Branches:** View all branches in your repository with:

```
git branch
```

3. **Switch Branches:** Change to a different branch using:

```
git checkout branch-name
```

Example:

```
git checkout main
```

4. **Merge Branches:** Integrate changes from one branch into another. Switch to the target branch and use:

```
git merge branch-name
```

Example:

```
git checkout main
```

```
git merge feature/new-login
```

5. **Delete a Branch:** Remove a branch that is no longer needed:

```
git branch -d branch-name
```

Example:

```
git branch -d feature/new-login
```


Pull Requests

Pull Requests (PRs) are a way to propose changes to the codebase and facilitate code review before merging changes into the main branch.

Creating a Pull Request:

1. **Create a Pull Request:** Go to the Azure Repos or GitHub web interface and navigate to the repository. Choose the branch you want to merge from and the target branch (usually main or master).
2. **Add a Title and Description:** Provide a clear title and description of the changes made. This helps reviewers understand the purpose of the PR.
3. **Add Reviewers:** Assign team members to review the PR. Reviewers can comment, suggest changes, and approve or reject the PR.
4. **Review Process:** Reviewers examine the code changes, provide feedback, and request modifications if necessary. This process ensures code quality and consistency.
5. **Merge the Pull Request:** Once the PR is approved, it can be merged into the target branch. This is usually done via the web interface, but can also be done using Git commands:
6. `git checkout main`

`git merge feature/new-login`

7. **Close the Pull Request:** After merging, the PR is closed. This can also be done manually if the PR is not to be merged.

Best Practices for Pull Requests:

1. **Small, Focused PRs:** Keep pull requests small and focused on a single change or feature to make them easier to review.
2. **Descriptive Titles and Descriptions:** Use clear and descriptive titles and descriptions for pull requests to provide context.
3. **Automated Checks:** Integrate automated checks (e.g., CI/CD pipelines) to validate code changes before review.
4. **Address Feedback Promptly:** Respond to reviewer feedback promptly and make necessary changes to facilitate smooth merging.
5. **Ensure Code Quality:** Review code for adherence to coding standards, best practices, and potential issues before merging.

Key Points for Interview Preparation

- **Understanding Git Branches:** Be able to explain the different types of branches (main, feature, bugfix, release, hotfix) and their purposes.

- **Branch Management:** Understand how to create, switch, list, and delete branches. Be familiar with the commands used for these operations.
- **Pull Requests:** Know the process for creating, reviewing, and merging pull requests. Understand the importance of pull requests in code quality and collaboration.
- **Review Practices:** Be aware of best practices for managing pull requests, including maintaining small, focused PRs, using descriptive titles, and addressing feedback promptly.
- **Branch Policies:** Understand how branch policies can enforce code quality and ensure that code meets certain standards before being merged.

Introduction to Azure Pipelines

Azure Pipelines is a cloud-based service within Azure DevOps that provides Continuous Integration (CI) and Continuous Deployment (CD) capabilities. It automates the process of building, testing, and deploying code, allowing teams to deliver software faster and more reliably.

Key Concepts of Azure Pipelines

1. CI/CD Pipelines:

- **Continuous Integration (CI):** Automates the process of building and testing code changes to ensure they integrate well into the shared codebase.
- **Continuous Deployment (CD):** Automates the deployment of code changes to various environments (e.g., staging, production) after passing the CI stage.

2. Pipeline Components:

- **Pipeline:** Defines the workflow for building, testing, and deploying code. It consists of stages, jobs, and steps.
- **Stage:** A phase in the pipeline that groups jobs. Stages are used to represent different stages in the deployment process, such as build, test, and deploy.
- **Job:** A collection of steps that are executed sequentially. Jobs run on agents and can be parallelized across multiple agents.
- **Step:** The smallest unit of work in a pipeline. Steps are individual tasks like running a script or executing a command.

3. Pipeline YAML vs. Classic Editor:

- **YAML Pipelines:** Written in YAML syntax, allowing you to define the pipeline configuration as code. This is more flexible and version-controlled.

- **Classic Editor:** A graphical interface to create pipelines, which provides a drag-and-drop experience for configuring pipeline tasks.

Creating Azure Pipelines

1. Setting Up a Pipeline:

- **Navigate to Azure Pipelines:** In your Azure DevOps project, go to Pipelines > New Pipeline.
- **Choose a Repository:** Select the source repository for the code. This can be Azure Repos, GitHub, or another Git repository.
- **Configure Pipeline:** Choose to use the YAML file or the Classic Editor to configure the pipeline.

2. YAML Pipeline Configuration:

- **Define Pipeline:** Create a YAML file (e.g., azure-pipelines.yml) in the root of your repository to define the pipeline.
- **Basic YAML Structure:**

trigger:

branches:

include:

- main

pool:

vmImage: 'ubuntu-latest'

steps:

- script: echo Hello, world!

displayName: 'Run a one-line script'

- trigger: Specifies which branches trigger the pipeline.
- pool: Defines the VM image to use for the build agent.
- steps: Lists the tasks to run, such as scripts or commands.

3. Classic Pipeline Configuration:

- **Add Tasks:** Use the Classic Editor to add tasks such as build, test, and deploy. Drag and drop tasks onto the pipeline canvas and configure them as needed.

- **Define Pipeline Variables:** Set pipeline variables that can be used across the pipeline to manage configuration settings and secrets.

Adding Stages for Deploying ASP.NET Core Microservice Apps to AKS Cluster

1. Defining Stages:

- **Build Stage:** Compiles the code and creates a Docker image.
- **Test Stage:** Runs unit tests and integration tests.
- **Deploy Stage:** Deploys the Docker image to an Azure Kubernetes Service (AKS) cluster.

2. Sample YAML Configuration:

trigger:

branches:

include:

- main

pool:

vmImage: 'ubuntu-latest'

stages:

- stage: Build

jobs:

- job: Build

steps:

- task: Docker@2

inputs:

command: 'build'

dockerfile: '**/Dockerfile'

tags: '\$(Build.BuildId)'

- stage: Test

dependsOn: Build

jobs:

```
- job: Test
  steps:
    - script: dotnet test
      displayName: 'Run unit tests'
```

```
- stage: Deploy
  dependsOn: Test
```

```
jobs:
```

```
- job: Deploy
```

```
  steps:
```

```
    - task: AzureCLI@2
```

```
      inputs:
```

```
        azureSubscription: 'YourAzureSubscription'
```

```
        scriptType: 'bash'
```

```
        scriptPath: 'deploy-to-aks.sh'
```

- **Build Stage:** Uses Docker to build the image.
- **Test Stage:** Runs tests to ensure code quality.
- **Deploy Stage:** Uses Azure CLI to deploy to AKS using a script.

3. **Deployment Script (deploy-to-aks.sh):**

4. `kubectl apply -f k8s/deployment.yaml`

`kubectl apply -f k8s/service.yaml`

- **deployment.yaml:** Defines the Kubernetes deployment.
- **service.yaml:** Defines the Kubernetes service.

Working with Environments in Azure DevOps

Environments in Azure DevOps help manage deployments to different environments like development, staging, and production.

1. Creating Environments:

- Navigate to Pipelines > Environments.
- Create a new environment and define the deployment targets, such as Kubernetes clusters or virtual machines.

2. Using Environments in Pipelines:

- Add environment references in the pipeline YAML to deploy to specific environments.

jobs:

- deployment: DeployToProduction

environment: 'production'

strategy:

runOnce:

deploy:

steps:

- task: AzureCLI@2

inputs:

azureSubscription: 'YourAzureSubscription'

scriptType: 'bash'

scriptPath: 'deploy-to-production.sh'

Variable Groups

Variable Groups allow you to manage and share variables across multiple pipelines. They are useful for managing configuration settings and secrets.

1. Creating Variable Groups:

- Navigate to Pipelines > Library > Variable groups.
- Create a new variable group and add variables.

2. Using Variable Groups in Pipelines:

- Reference variable groups in your pipeline YAML.

3. variables:

- group: my-variable-group

Azure Key Vault Integration

Azure Key Vault is a service that securely stores and manages sensitive information like secrets, keys, and certificates.

1. Setting Up Azure Key Vault:

- Create a Key Vault in the Azure portal.
- Add secrets to the Key Vault.

2. Accessing Key Vault Secrets in Pipelines:

- Use the Azure Key Vault task to fetch secrets.

- task: AzureKeyVault@2

inputs:

azureSubscription: 'YourAzureSubscription'

KeyVaultName: 'YourKeyVaultName'

SecretsFilter: '*'

RunAsPreJob: false

- Secrets can then be accessed in subsequent pipeline steps using the variable names defined in the Key Vault.

Key Points for Interview Preparation

- **Azure Pipelines Overview:** Understand the role of Azure Pipelines in CI/CD and its components (stages, jobs, steps).
- **YAML vs. Classic Pipelines:** Be able to explain the difference between YAML and Classic pipelines and their use cases.

- **Pipeline Stages:** Know how to configure stages for building, testing, and deploying applications, including examples of YAML configurations.
- **Environments:** Understand how to create and use environments in Azure DevOps for managing different deployment targets.
- **Variable Groups:** Be familiar with creating and using variable groups to manage configuration settings across pipelines.
- **Azure Key Vault:** Know how to integrate Azure Key Vault with Azure Pipelines to manage secrets securely.

Adding Stages for Deploying ASP.NET Core Microservice Apps to AKS Cluster

In this part, we'll dive into how to add stages to your Azure Pipelines YAML configuration to deploy ASP.NET Core microservice applications to an Azure Kubernetes Service (AKS) cluster.

Overview of Pipeline Stages for AKS Deployment

A typical CI/CD pipeline for deploying to AKS involves several stages:

1. **Build:** Compile the code and package it into Docker images.
2. **Test:** Run unit tests and integration tests to ensure code quality.
3. **Deploy:** Deploy the Docker images to an AKS cluster.

Detailed Pipeline Configuration

1. Define Your Pipeline

Start by creating a `azure-pipelines.yml` file in your repository. This YAML file defines your CI/CD pipeline.

trigger:

branches:

include:

- main

- **trigger:** Specifies the branch that triggers the pipeline. Here, it is set to trigger on changes to the main branch.

2. Define Pipeline Stages

The pipeline is divided into multiple stages: Build, Test, and Deploy.

Build Stage

stages:

- stage: Build

displayName: 'Build Docker Image'

jobs:

- job: Build

pool:

vmImage: 'ubuntu-latest'

steps:

- task: Docker@2

inputs:

command: 'build'

dockerfile: '**/Dockerfile'

tags: '\$(Build.BuildId)'

containerRegistry: 'myContainerRegistry'

- **stage: Build:** Defines the build stage of the pipeline.
- **displayName:** Provides a human-readable name for the stage.
- **jobs:** Contains the jobs to be executed in this stage.
- **job: Build:** Defines a job named Build.
- **pool:** Specifies the VM image to use for the build agent.
- **steps:** Lists the tasks to perform. In this case, the Docker task builds the Docker image from the Dockerfile.

Test Stage

- stage: Test

displayName: 'Run Unit Tests'

dependsOn: Build

jobs:

- job: Test

pool:

vmImage: 'ubuntu-latest'

steps:

- script: dotnet test

displayName: 'Run unit tests'

- **stage: Test:** Defines the test stage.
- **dependsOn: Build:** Specifies that this stage depends on the successful completion of the Build stage.
- **steps:** Runs the dotnet test command to execute unit tests.

Deploy Stage

- stage: Deploy

displayName: 'Deploy to AKS'

dependsOn: Test

jobs:

- job: Deploy

pool:

vmImage: 'ubuntu-latest'

steps:

- task: AzureCLI@2

inputs:

azureSubscription: 'YourAzureSubscription'

scriptType: 'bash'

scriptPath: 'scripts/deploy-to-aks.sh'

- **stage: Deploy:** Defines the deployment stage.
- **dependsOn: Test:** Specifies that this stage depends on the successful completion of the Test stage.
- **steps:** Executes a script using Azure CLI to deploy to AKS.

3. Deployment Script (deploy-to-aks.sh)

Create a script named deploy-to-aks.sh in the scripts directory. This script contains the commands to deploy your application to AKS.

```
#!/bin/bash
```

```
# Set variables
```

```
IMAGE_NAME="myapp"
```

```
IMAGE_TAG="${Build.BuildId}"
```

```
REGISTRY_NAME="myContainerRegistry"
```

```
AKS_CLUSTER_NAME="myAKSCluster"
```

```
RESOURCE_GROUP="myResourceGroup"
```

```
# Log in to Azure
```

```
az login --service-principal -u $SP_APP_ID -p $SP_PASSWORD --tenant $TENANT_ID
```

```
# Set the current subscription
```

```
az account set --subscription $AZURE_SUBSCRIPTION_ID
```

```
# Get AKS credentials
```

```
az aks get-credentials --resource-group $RESOURCE_GROUP --name $AKS_CLUSTER_NAME
```

```
# Deploy the Docker image
```

```
kubectl apply -f k8s/deployment.yaml
```

```
kubectl apply -f k8s/service.yaml
```

- **az login:** Authenticates with Azure using service principal credentials.
- **az aks get-credentials:** Configures kubectl to use the AKS cluster credentials.
- **kubectl apply:** Deploys the Docker image to the AKS cluster using Kubernetes manifests.

4. Kubernetes Manifests

Deployment Manifest (k8s/deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myContainerRegistry.azurecr.io/myapp:$(Build.BuildId)
          ports:
            - containerPort: 80
```

- **apiVersion:** Specifies the Kubernetes API version.
- **kind:** Defines the resource type (Deployment).
- **replicas:** Specifies the number of replicas (instances) of the application.
- **containers:** Defines the Docker container image and port configuration.

Service Manifest (k8s/service.yaml)

```
apiVersion: v1
kind: Service
metadata:
```

name: myapp-service

spec:

type: LoadBalancer

selector:

app: myapp

ports:

- protocol: TCP

port: 80

targetPort: 80

- **kind:** Defines the resource type (Service).
- **type:** Specifies the service type (LoadBalancer to expose the service externally).
- **selector:** Matches the service to the deployment using labels.

Key Points for Interview Preparation

- **Pipeline Stages:** Understand how to define and configure stages for build, test, and deploy operations in Azure Pipelines.
- **YAML Configuration:** Be familiar with YAML syntax and structure for creating pipeline configurations.
- **Deployment Scripts:** Know how to write and use deployment scripts to deploy applications to AKS.
- **Kubernetes Manifests:** Understand the purpose of deployment and service manifests in Kubernetes and how to configure them.

Working with Environments in Azure DevOps

In Azure DevOps, **Environments** provide a way to manage and control the deployment of applications to various stages or environments such as Development, Testing, Staging, and Production. They offer a more structured approach to manage deployments and enforce deployment policies.

Overview of Environments

Environments in Azure DevOps help manage the deployment of your applications by providing:

1. **Deployment Resources:** Track and manage resources like Kubernetes clusters or virtual machines.
2. **Approval Gates:** Implement pre-deployment and post-deployment approvals.
3. **Checks:** Enforce security and compliance checks before deployment.

Creating and Configuring Environments

1. Creating an Environment

To create an environment in Azure DevOps:

- Go to your Azure DevOps project.
- Navigate to **Pipelines > Environments**.
- Click on **New environment**.
- Provide a name for the environment (e.g., Staging or Production).
- Optionally, add resources like Kubernetes clusters or virtual machines.

2. Configuring Environment Approvals and Checks

Approvals and Checks are set up to control when and how deployments happen. This is crucial for maintaining quality and compliance.

- **Approvals:** Configure who must approve deployments to the environment. You can add multiple approvers and set conditions for approval.
- **Checks:** Set up checks for security, compliance, and quality. Examples include checking for the latest security patches or verifying that all tests pass.

To configure approvals and checks:

- Go to the environment you created.
- Click on **Approvals and Checks**.
- Add new checks or approvals as needed.

Example Configuration:

- **Approvals:** Require approval from a specific user or group before proceeding with the deployment.
- **Checks:** Ensure that a security scan is completed or a specific compliance requirement is met before deployment.

3. Using Environments in Pipelines

To use environments in your Azure Pipelines YAML file, specify the environment in your deployment steps:

stages:

- stage: Deploy

displayName: 'Deploy to AKS'

jobs:

- deployment: DeployToAKS

environment: 'Staging'

strategy:

runOnce:

deploy:

steps:

- task: AzureCLI@2

inputs:

azureSubscription: 'YourAzureSubscription'

scriptType: 'bash'

scriptPath: 'scripts/deploy-to-aks.sh'

4. **environment:** Specifies the environment to which the application will be deployed (e.g., Staging).

The pipeline will use the configuration and policies associated with the specified environment during the deployment process.

5. Monitoring and Managing Deployments

- **Deployment History:** View the history of deployments to the environment. This helps track successful and failed deployments.
- **Resource Health:** Monitor the health of resources associated with the environment. Ensure that all deployed applications are running smoothly.

Key Points for Interview Preparation

- **Environments:** Understand the concept and purpose of environments in Azure DevOps.
- **Approvals and Checks:** Know how to configure approvals and checks to control and enforce deployment policies.
- **Pipeline Integration:** Be familiar with how to reference environments in Azure Pipelines YAML configurations.
- **Resource Management:** Be aware of how to manage and monitor resources and deployments within an environment.

Variable Groups in Azure DevOps

Variable Groups in Azure DevOps are a way to manage and share values across multiple pipelines. They are useful for storing and reusing common configuration settings, secrets, and environment-specific values.

Overview of Variable Groups

- **Purpose:** To centralize the management of variables used in different pipelines, avoiding duplication and easing management.
- **Scope:** Variables can be scoped to different pipelines, stages, or environments. They can also be used across different projects.

Creating and Configuring Variable Groups

1. Creating a Variable Group

To create a Variable Group:

- Go to your Azure DevOps project.
- Navigate to **Pipelines > Library**.
- Click on **Variable groups**.
- Click on **+ Variable group** to create a new group.
- Provide a name for the variable group (e.g., ProductionSecrets).

Example:

- **Name:** ProductionSecrets
- **Description:** Variables used in production deployments

2. Adding Variables to the Group

- After creating the variable group, click on it to open.
- Click on **Add** to add new variables.
- Enter the name and value for each variable. For sensitive data, you can mark the variable as **secret**, which will encrypt the value and hide it in logs.

Example:

- **Variable Name:** ConnectionString
- **Value:** Server=myServerAddress;Database=myDataBase;UserId=myUsername;Password=myPassword;
- **Secret:** Checked

3. Linking Variable Groups to Pipelines

To use the variable group in your pipelines:

- Edit your pipeline YAML file.
- Add a reference to the variable group under the variables section.

Example YAML Configuration:

variables:

- group: ProductionSecrets

- This includes all variables defined in the ProductionSecrets group into your pipeline, making them available for use.

4. Using Variables in Pipelines

Once linked, you can use these variables in your pipeline tasks, scripts, and other steps.

Example Usage in a Pipeline:

jobs:

- job: Deploy

steps:

- script: |

```
echo $(ConnectionString)
```

```
displayName: 'Print connection string'
```

- **\$(ConnectionString)**: This references the variable ConnectionString from the ProductionSecrets variable group.

5. Managing Variable Groups

- **Editing Variables**: Update the value of variables if needed. Changes are automatically reflected in all pipelines using the group.
- **Deleting Variable Groups**: Remove a variable group when it is no longer needed. Ensure that no pipeline relies on the variables before deletion.

Key Points for Interview Preparation

- **Variable Groups**: Understand what variable groups are and why they are used.
- **Creating and Managing Groups**: Know how to create, configure, and manage variable groups.
- **Linking to Pipelines**: Be familiar with how to link variable groups to pipelines and use variables in pipeline configurations.
- **Secrets Management**: Understand how to manage sensitive data using secret variables in Azure DevOps.

Azure Key Vault and Accessing Secrets in Pipelines

Azure Key Vault is a cloud service that helps securely store and access secrets, such as API keys, passwords, and certificates. It provides a centralized place to manage sensitive information and integrates well with Azure DevOps for secure secret management in CI/CD pipelines.

Overview of Azure Key Vault

- **Purpose**: To safeguard secrets, keys, and certificates used by cloud applications and services.
- **Features**: Secure storage, access policies, logging, and integration with other Azure services.

Creating and Configuring an Azure Key Vault

1. Creating a Key Vault

- **Azure Portal:**
 - Navigate to the **Azure Portal**.
 - Search for **Key Vault** and select **Create**.
 - Enter the **Name**, **Subscription**, **Resource Group**, and **Location**.
 - Click **Review + create** and then **Create**.

Example:

- **Name:** MyKeyVault
- **Resource Group:** MyResourceGroup
- **Location:** East US

2. Adding Secrets to the Key Vault

- Go to your newly created Key Vault in the Azure Portal.
- Navigate to **Secrets** under the **Settings** section.
- Click on **+ Generate/Import** to add a new secret.
- Enter a **Name** and **Value** for the secret.

Example:

- **Secret Name:** MyDatabasePassword
- **Value:** P@ssw0rd1234

3. Configuring Access Policies

- Go to **Access policies** in the Key Vault settings.
- Click **+ Add Access Policy**.
- Select the appropriate **Secret permissions** (e.g., **Get**, **List**).
- Assign the policy to a **Service Principal** or **Azure DevOps** to allow access from your pipeline.

Example:

- **Secret permissions:** Get, List
- **Principal:** MyAzureDevOpsServicePrincipal

Accessing Azure Key Vault Secrets in Azure DevOps Pipelines

1. Setting Up the Service Connection

- **Azure DevOps Portal:**
 - Navigate to your Azure DevOps project.
 - Go to **Project settings > Service connections**.
 - Click **New service connection** and select **Azure Resource Manager**.
 - Authenticate using the appropriate method (e.g., Service Principal).
 - Select the subscription and the Key Vault to which you want to grant access.

2. Using Key Vault Secrets in Pipeline YAML

- To use secrets from Azure Key Vault, update your pipeline YAML file to reference the Key Vault.

Example YAML Configuration:

variables:

- group: ProductionSecrets

stages:

- stage: Build

jobs:

- job: BuildJob

steps:

- task: AzureKeyVault@2

inputs:

azureSubscription: 'MyAzureServiceConnection'

KeyVaultName: 'MyKeyVault'

SecretsFilter: '*'

- script: |

echo \$(MyDatabasePassword)

displayName: 'Print secret'

- **AzureKeyVault@2:** This task fetches secrets from the specified Key Vault.

- **azureSubscription:** Service connection name for Azure.
- **KeyVaultName:** The name of your Key Vault.
- **SecretsFilter:** Filters secrets to be fetched (use '*' to fetch all).

3. Accessing Secrets in Pipeline Tasks

- Once fetched, secrets can be accessed as pipeline variables.
- They can be used directly in scripts and tasks.

Example Usage in a Script:

- script: |

```
echo $(MyDatabasePassword)
```

```
displayName: 'Display database password'
```

- **\$(MyDatabasePassword):** Accesses the secret value from the Key Vault.

Key Points for Interview Preparation

- **Azure Key Vault:** Understand the purpose, features, and configuration of Azure Key Vault.
- **Creating Secrets:** Know how to add and manage secrets in Key Vault.
- **Access Policies:** Be familiar with configuring access policies for secure access.
- **Service Connections:** Understand how to set up Azure DevOps service connections to Azure.
- **Pipeline Integration:** Know how to access and use Key Vault secrets in Azure DevOps pipelines.

.NET Microservices – Azure DevOps and AKS

Section 15: Azure API Management - Notes

1. Introduction to Azure API Management (APIM)

Azure APIM enables secure, reliable, and scalable access to services by providing features such as:

- **Traffic Management:** Routes and manages API traffic, balancing requests among services and scaling automatically.
- **Security:** Offers built-in policies to enforce security, including rate limiting, IP filtering, and CORS. APIM also supports authentication via OAuth2, Azure AD, and JWT token validation.
- **Analytics:** Provides insights into API usage, errors, and response times, essential for monitoring and improving API performance.
- **Developer Portal:** A customizable portal for developers to discover, subscribe to, and access APIs with documentation and testing tools.

2. How APIM Works Internally with AKS as Backend

In a setup where APIM is acting as a frontend for services running on Azure Kubernetes Service (AKS), the internal workflow looks like this:

1. **Request Routing:** Client applications (such as mobile apps or web clients) send requests to the APIM endpoint.
2. **Request Processing:**
 - APIM verifies the request using applied policies, such as validating tokens or checking IP restrictions.
 - It may rewrite, transform, or rate-limit the request based on configured rules.
3. **Backend Communication:**
 - After verification, APIM routes the request to the appropriate AKS service. APIM uses a **backend configuration** that defines which AKS service should handle the request based on rules.
 - For this, the AKS service's IP address and port are typically registered in APIM as a backend service, often with DNS or a Virtual Network (VNet) for secure communication.
4. **Response Processing:**
 - APIM receives the response from the AKS service.
 - Any post-processing policies (e.g., transforming the response format or stripping sensitive information) are applied before sending the response back to the client.

This setup provides an added layer of abstraction and control over how client applications interact with backend services on AKS, creating a structured API ecosystem with policies that enforce security, logging, and monitoring.

3. Benefits of Using APIM as a Gateway for AKS Services

- **Security Control:** Acts as a security gateway for AKS services, enforcing authentication and access control.
- **Unified Interface:** Provides a single entry point for multiple services running in the AKS cluster, simplifying client access.
- **Resilience and Scalability:** APIM scales requests across backend AKS services, handling load management and retry policies.
- **Rate Limiting and Throttling:** Manages traffic spikes and prevents backend overload by controlling request limits.
- **Analytics and Logging:** Tracks API usage and performance metrics, helping diagnose issues and optimize services.

Key Terminologies in APIM

- **Product:** Grouping of APIs that can be exposed to developers with specific policies.
- **Policy:** A set of rules applied to API requests and responses, like caching, authorization, or logging.
- **Backend:** A configuration in APIM representing the backend service, like an AKS microservice, with settings for routing and authentication.
- **Operations:** Defines the various methods (GET, POST, etc.) for each endpoint in an API.

.NET Microservices – Azure DevOps and AKS

Section 16: Microsoft EntraID - B2C Authentication - Notes

Introduction to Azure Entra ID (Azure AD)

Azure Active Directory (Azure AD), now part of the **Microsoft Entra** family, is Microsoft's cloud-based identity and access management (IAM) service. It enables organizations to manage identities, access, and security in a unified, centralized way for both internal and external resources. It forms the backbone of Microsoft's identity services for cloud-based applications and integrates with Microsoft 365, Azure, and thousands of third-party services.

Key Concepts of Azure AD

1. Identity as a Service (IDaaS):

- Azure AD is a cloud-based IDaaS solution, providing user authentication and authorization as a service, which means it handles authentication requests and enables secure access to applications.

2. Multi-Factor Authentication (MFA):

- Azure AD supports MFA, which adds an additional layer of security. This requires users to provide two or more verification methods, ensuring more robust protection.

3. Single Sign-On (SSO):

- Azure AD offers Single Sign-On capabilities, allowing users to authenticate once and gain access to multiple applications, both on-premises and in the cloud, without needing to log in repeatedly.

4. Conditional Access:

- Conditional Access is a powerful security feature in Azure AD, allowing organizations to define access policies based on specific conditions (e.g., user location, device state). Policies can limit access to sensitive applications based on these rules, enhancing security.

5. Role-Based Access Control (RBAC):

- With RBAC, Azure AD enables organizations to manage who has access to resources and what they can do with those resources. Permissions can be assigned to roles, which are then applied to users or groups.

6. Self-Service Capabilities:

- Self-service password reset, account recovery, and access requests can be enabled for users in Azure AD, reducing administrative overhead and enhancing user experience.

How Azure AD is Different from Traditional Active Directory

Traditional Active Directory (AD) is an on-premises identity solution primarily used to manage Windows domains. Azure AD, in contrast, is designed for the cloud, supporting a wider range of applications and services, and provides more advanced features such as MFA, SSO, and Conditional Access that aren't as easily implemented in traditional AD environments.

Key Components in Azure AD

1. Users and Groups:

- Users represent individual identities within an organization. Groups allow for efficient management of user permissions, as access to resources can be granted at the group level.

2. Applications:

- Azure AD provides seamless integration with applications, allowing them to use Azure AD for authentication. Apps can be internal (for organization use only) or external (for customers and partners).

3. Devices:

- Devices registered with Azure AD allow users to access organizational resources more securely. Device management in Azure AD includes compliance and security policies.

4. Domains:

- Azure AD allows organizations to add custom domains to manage users and resources under specific company names.

Authentication Protocols in Azure AD

Azure AD supports multiple industry-standard protocols for secure authentication:

1. OAuth 2.0:

- A protocol allowing secure token-based access to resources on behalf of a user. Widely used in web and mobile applications.

2. **OpenID Connect (OIDC):**

- An authentication layer on top of OAuth 2.0, allowing applications to verify the identity of users and obtain basic profile information.

3. **SAML 2.0:**

- Security Assertion Markup Language (SAML) is used primarily for enterprise-level single sign-on (SSO) scenarios. It's widely supported by cloud applications.

4. **WS-Federation:**

- Primarily used for SSO in older enterprise applications.

Azure AD Editions

Azure AD comes in four main editions to meet the needs of different organizations:

1. **Free:** Basic identity management and access management.
2. **Microsoft 365 Apps:** Additional SSO for Office 365 apps, self-service password change, and multi-factor authentication for privileged accounts.
3. **Premium P1:** Adds Conditional Access, advanced group management, and more control over access.
4. **Premium P2:** Includes all P1 features with Identity Protection, Privileged Identity Management, and advanced security reports and alerts.

Azure AD Use Cases

1. **Employee Access Management:**

- Manage user identities, enforce security policies, and provide employees with secure access to apps and data.

2. **B2B Collaboration:**

- Azure AD allows external users (partners, contractors) to access applications and resources securely through **Azure AD B2B** (Business-to-Business).

3. **B2C Scenarios:**

- Azure AD B2C (Business-to-Consumer) offers customizable and secure authentication options for customer-facing applications.

4. **Application Access Control:**

- Organizations can secure access to thousands of SaaS applications, including Microsoft 365, Salesforce, and more.

5. API Security:

- APIs can use Azure AD for token-based authentication, protecting API endpoints and ensuring secure access.

Key Benefits of Azure AD

- **Scalability and Flexibility:** Easily integrates with multiple applications, platforms, and cloud environments.
- **Enhanced Security:** Offers robust security features like Conditional Access, MFA, and Identity Protection.
- **Centralized Identity Management:** Simplifies the management of user access to multiple applications.
- **Improved User Experience:** SSO and self-service options make it easier for users to access resources.

Commonly Used Terminologies

1. **Tenant:** The organization within Azure AD, representing a unique entity and its directory.
2. **Subscription:** An agreement to use Azure resources, typically tied to billing and administrative boundaries.
3. **Enterprise Applications:** Applications that use Azure AD for authentication, often listed in the Azure AD Enterprise applications section.
4. **App Registration:** The process of creating an application in Azure AD, allowing it to use Azure AD for authentication.

Key Points to Remember (for Interview Preparation)

1. **Azure AD** is Microsoft's cloud-based identity and access management service, part of the Microsoft Entra suite.
2. It supports **SSO, MFA, and Conditional Access** for secure, seamless access to resources.
3. **OAuth 2.0, OpenID Connect, and SAML** are among the primary protocols it supports for authentication.
4. Azure AD enables **B2B and B2C scenarios**, facilitating secure external collaboration and customer authentication.

5. **Role-Based Access Control (RBAC)** and **Conditional Access** are crucial features for secure, role-based resource access.
6. Key editions of Azure AD include **Free, Microsoft 365 Apps, Premium P1, and Premium P2** with increasing features at each level.
7. **App registrations** in Azure AD are necessary to allow applications to authenticate with Azure AD.